JOPE DIME / PRIVATE MAINTAINER EDITION

Algorithm Synthesis Model

Technology, operations and the evidence behind the claims

Learns to generate and optimize executable algorithms from data.

User & maintainer manual
Edition 3 · 2026-09-24 · implementation 7f2b2bf860e4
43 primary acceptance items
Qualification observations and pending acceptance are recorded separately.
See ASM at a glance in chapter 1.
Historical rankings and current measurements are identified separately.

How to use this book

Start with the status chapter to distinguish what has shipped, what has been demonstrated, and what remains pending. Operators can then use installation and training procedures; maintainers can follow the architecture, language matrix, benchmark contract, detailed roadmap and evidence index.

For actual learning progress, start with chapters 7–8 and 13, then appendices B–D in Volume 2: historical optimizer and target trajectories, binary packing, model versions, measurable KPIs, continuation commands and the remaining flywheel gaps. The plots distinguish real step logs, sampled endpoints and changing evaluation populations.

§A All thirty datasets and all 522 feature records gives development statistics for all thirty frozen datasets. The adjacent data directory supplies complete machine-readable statistics, historical ranking inputs, metadata inventories and source pins. The property list supplies the primary acceptance inventory and separately marked coordinated proposals; §E.1.1.1 LIFE-01 — coordination prints them as acceptance cards.

The publication record below identifies the implementation snapshot, dated qualification observations and document build independently. The core chapters and roadmap preserve the PR83 application and stopped-overlay observations; chapter 1 separately records the later PR88 integration. A later build date does not turn historical pending labels into a current-main inventory.

This private manual explains internal methods at the user's request. Public inference bundle allowlists remain separate. No sealed label bodies, model weights or credentials are embedded in this book.

Evidence vocabulary

Implemented means source exists. **Qualified** means a named execution was measured. **Pending** means acceptance remains. **Unknown** is not zero. Historical results do not establish current capability. §1.2 Reading this manual: audience, evidence levels and figures gives the full evidence ladder.

Publication record

Three identities describe this book. They change independently, and none of them implies the others.

Implementation snapshot

Source `bce4791a8b2a67431da62f27da367b53bb9c48f9`, merged to main as `7f2b2bf860e4b1ac8815c8da55e507143570a905` at 2026-09-24T17:10:23Z; Git tree `870fa7d594997d21dc3dd5254ca5b1bc03f3afa6`. Current core-technology and transfer-reporting additions bind this merged source. Preserved historical chapters retain their explicitly dated PR83 snapshot; source and CPU fixture support do not establish live learning qualification.

Qualification observations

Dated observations; a configured command, implemented source and live qualification are separate states.

Observation	Observed (UTC)	Receipt
PR86 exact-head manual CI: eight lanes and aggregate passed.	2026-09-24T00:02:23Z	`https://github.com/veox-ai/JopeDime/actions/runs/35930124993`
Full-main diagnostic failed: eight families passed, two failed, one unavailable; retained separately from fast CI and learning qualification.	2026-09-24T00:31:10Z	`https://github.com/veox-ai/JopeDime/actions/runs/35936673628`
Edition 2 published through PR87 with exact-head CI and independent native approval; its main-guard passed.	2026-09-24T02:58:17Z	`https://github.com/veox-ai/JopeDime/pull/87`
Later PR88 source integration: all eight exact-head manual CI lanes and aggregate passed; independent native approval and main-guard passed. This is outside the retained PR83 application snapshot and does not establish live learning qualification.	2026-09-24T05:12:05Z	`https://github.com/veox-ai/JopeDime/actions/runs/35957350158`
PR90 zero-fit transfer planning/ reporting merged after all eight exact-head manual CI lanes and independent native approval. Global transferable lift remains not yet measured.	2026-09-24T17:10:23Z	`https://github.com/veox-ai/JopeDime/pull/90`
Latest retained completed full-main diagnostic at 5de90b1cc failed; preserved separately from exact-head fast CI. This rebuild does not relabel it passed.	2026-09-24T06:23:44Z	`https://github.com/veox-ai/JopeDime/actions/runs/35962317775`

Document build

Edition 3, 2026-09-24. Rendered by `docs/internal/asm-manual/tools/build_manual.py` with SOURCE_DATE_EPOCH 1790270756 from 192 pinned inputs (digest of their path and SHA-256 list: `4c3725f1002cf74dc3d215a21f34b1fddbb6797d46b38ecda22a5fef9f44ec8b`). Each volume's PDF hash, size and page count are recorded in `render-receipt.json`, because a PDF cannot contain its own hash. Build time is fixed by this manifest; render receipts bind all publication inputs and both output PDFs.

Contents

1 Current status, versions, north star and measured limitations 7

1.1 ASM at a glance 7
1.2 Reading this manual: audience, evidence levels and figures 9
1.3 North star: better algorithms from accumulated experience 10
1.4 What works now 12
1.5 Latest measured operational proof 13
1.6 Failure records are retained 17
1.7 Pending acceptance 17
1.8 What version are we running? 18

2 Architecture, encodings, GP representation and model lineages 22

2.1 Explain it like I'm 5: learning to suggest better recipes 22
2.2 Current executable pipeline and the historical snapshot 23
2.3 Preserved historical chapter snapshot 26
2.4 System at a glance: flow diagram and component map 27
2.5 The learning cycle: what is proven and what remains 28
2.6 GP representation: recipes, fitted predictors and the four artifact types 30
2.7 Encodings and the proposal context 31
2.8 The accepted regression model: lineage B, step 60,000 33
2.9 Model lineages 35

3 How ASM works: data, encoding, model, GP, code, training and learning 38

3.1 Explain it like I'm 5 39
3.2 The stack at a glance 40
3.3 Deep technical walkthrough 41
3.4 Training runs and global metrics 48
3.5 Measurement status and figure provenance 52

4 Installation, preparation, assets, workers and doctor 53

4.1 Scope and command surface 53
4.2 Physical layout and identities 55
4.3 Build and install 55
4.4 Prepare the panel and external runtimes 56
4.5 Stored model assets and the active encoder distinction 57
4.6 Bounded workers: the context worker and the policy producer 59
4.7 Doctor checks and troubleshooting 60

5 Data, splits, groups, families and permitted input boundaries 62

5.1 The frozen thirty-dataset panel: identity, totals, groups and immutability 62
5.2 Local preparation and panel storage 63
5.3 The broader S3 corpus catalog 63
5.4 The historical input closure and the meta-test question 65
5.5 Family-level domains and the permitted input closure 65

6 Regression search, fitting, prediction and focused optimization 68

6.1 Proposal generation and ranking 68
6.2 Paired arms, generation zero and evolution 69
6.3 Dense-width refusals and the proposed feasibility preflight 69
6.4 Warm fitted demonstration after integration 70
6.5 Focused single-dataset search 72

7 Teacher admission, HWM identities and corpus updates 77

7.1 A dataset is not one training example 77
7.2 How a fitted recipe becomes an eligible teacher 79
7.3 From optimization to reusable knowledge 80
7.4 Present teacher readiness and the smallest real path 81
7.5 Prepare a teaching campaign before it starts 81
7.6 Corpus updates: reuse, device caches and loading 84
7.7 Concrete corpus improvements, with acceptance criteria 86

8 Training, continuation, export and compatibility 88

8.1 What can be claimed now 88
8.2 The four continuation names and their commands 89
8.3 Evidence and identity before a bounded campaign 96
8.4 The bounded training campaign chain 97
8.5 Consuming a newly trained checkpoint 101
8.6 Remaining acceptance queue and maintenance risks 104

9 Measuring repeated learning and deciding improvement 105

9.1 Observed transferable lift: reporting contract, not yet measured 105
9.2 Preserved historical chapter snapshot 107
9.3 What “better and better” must mean 107
9.4 What the historical training and target evidence establishes 107
9.5 LOOP-01: the central regression milestone 108
9.6 The closure ledger 110
9.7 A minimal experiment that can establish learning 111
9.8 Distinguish routing, ranking and search recovery 113
9.9 Conditioning diagnostics before the first causal experiment 114
9.10 Specialization without starving the general learner 115
9.11 A KPI ledger with defined denominators 115
9.12 What is still needed to demonstrate the learning flywheel 118

10 Complete fitted inference and language support 120

10.1 Deployment support: definition, priority and status 120
10.2 Language interfaces at the frozen source 121
10.3 Complete registered-recipe matrix 122
10.4 Private worked examples and their measured execution 125
10.5 Example C: exact fitted winner export after pending integration 128
10.6 Future work and acceptance boundaries 131

11 Benchmark execution, ranking, Elo and interpretation 132

11.1 Three evidence collections with different purposes 132
11.2 Frozen-panel execution contract 133
11.3 Running the frozen-panel campaign: plan, execute and report 134
11.4 Metrics and class order 137
11.5 What Elo means here 137
11.6 Minimum complete future campaign report 138

12 Scheduling, cancellation, recovery and accounting 140

12.1 Arm order and clocks 140
12.2 Resource limits, shared compute and supervision 140
12.3 Cancellation, resume and recovery 142
12.4 Artifacts: what evidence to retain 143
12.5 Cost accounting 144

13 Maintenance, privacy, release and preservation 147
13.1 Questions that should remain answerable after a maintainer handoff 147
13.2 Repository authority and coordination 147
13.3 Worktrees and builds 148
13.4 Coverage and reviewed lifecycle 148
13.5 Privacy boundaries: the private manual and public fitted bundles 149
13.6 Release and installation are independent from model promotion 150
13.7 Preservation and lane closure 150
13.8 Rebuilding and updating this manual 150

14 Prioritized implementation plan and extension interfaces 155
14.1 Prioritized implementation plan 155
14.2 Extension interfaces 162

Volume 2 · Evidence reference

A All thirty datasets and all 522 feature records 165

B Historical training, checkpoint lineage and original loss trajectories 165

C Historical GP targets, rankings, Elo and attribution limits 165

D Dated data estate, recovery evidence and binary formats 165

E Full task catalogue, source/schema index, receipts and glossary 166

CHAPTER 1

Current status, versions, north star and measured limitations

IN THIS CHAPTER

1.1 ASM at a glance 7
1.2 Reading this manual: audience, evidence levels and figures 9
1.3 North star: better algorithms from accumulated experience 10
1.3.1 The proposed north star and matched comparisons 10
1.3.2 Where the current evidence stands 11
1.4 What works now 12
1.5 Latest measured operational proof 13
1.5.1 Summary of the one-dataset proof 13
1.5.2 Canonical proof record and per-arm facts 14
1.5.3 The outcome figure and its limits 16
1.5.4 Failures: dense-width refusals and the earlier failed reproducer 17
1.6 Failure records are retained 17
1.7 Pending acceptance 17
1.8 What version are we running? 18
1.8.1 Implementation snapshot and this manual's own identity 18
1.8.2 The identity tuple 19

An **Algorithm Synthesis Model (ASM) learns to generate and optimize executable algorithms from data.** JopeDime turns that objective into a bounded, inspectable workflow: validate original data and row assignments, encode a fitting-only context, propose executable recipes, fit them, measure them under a fixed protocol, evolve measured recipes, retain fitted artifacts, and eventually admit qualified examples into another round of model training. This chapter states the position on 23 September 2026 at about 22:00 UTC, at merged main `a75e543d2166` (reviewed head `68f56404c3e1`, same source tree).

Later implementation observation — 2026-09-24. PR88 integrated the training64, product28, warm-demonstration and winner-export packets, the mandatory regression-consumption gate, and preparation, input-closure, strict-resume and controlled-comparison changes. Reviewed source `8fcf5470cf90f8facb4fac598b67725660448864` merged as main `0fa1841834d1fc67535f021a91a1911a321e6363` at 05:11:50 UTC; exact-head CI 35957350158 attempt 1, independent native approval 5300062934 and main-guard 35958877242 passed. A source-bound runtime build and small CPU fixtures passed. No new real-teacher campaign, GPU window or serving promotion was admitted. The operational learning outcome remains inconclusive. The chapter and roadmap status labels below describe their named PR83/stopped-overlay snapshot, not later main. Use the later implementation runbook for the integrated interfaces; historical commands and evidence are retained here.

1.1 ASM at a glance

This table is the bottom line on 23 September 2026 at about 22:00 UTC. Each row links to the sections that hold its evidence. Development scores are adaptive, and historical results do not establish current capability.

Area	Current state	Evidence
Works	One bounded regression proof on one frozen-panel dataset, `230_machine_cpu`: verified context → 64 policy proposals and 16 exploration proposals → five-fold fits on 20 bounded workers → 16 offspring (8 mutation, 8 crossover) → an independently recovered report. Genuine neural-origin fits and fitted descendants are verified.	§1.5.1 Summary of the one-dataset proof, §1.5.2 Canonical proof record and per-arm facts
Not better yet	On that dataset the neural arm (policy-seeded search) had 1.071 times the development-hold RMSE of the generic arm (grammar-uniform search): 166.3 against 155.3 on 34 held rows, under equal declared allowances but unequal fit time. In retained historical final searches, whose arms shared their seed pools, the two arms tied exactly on 3,227 of 3,268 paired comparisons (98.7%). Parity of the product's CPU policy port with the trained model is not established.	§12.1 Arm order and clocks, §C.1.4 Paired final-search outcomes within the admitted development inventory, §1.4 What works now
Not run	The frozen thirty-dataset panel (0 of 30 datasets); teacher admission under the new contract (0 teachers); new training (no newly trained checkpoint); consumption of a newly trained checkpoint; exact fitted-winner export; installer and warm demonstration.	§1.7 Pending acceptance, §8.1 What can be claimed now
Historical standing	On the corrected, benchmark-valid historical SOTArena ladder the fixed-seed `jope-dime-regressor` entrant ranks 31 of 304 methods (Elo 1687.0; rank 1 is TabFM Ensemble at 1934.9). The published ladder's rank 1 for `jope-dime` rests on test-selected rows and is not a held-out claim.	§C.2.3.2 Why the two historical ladders differ
Next	The critical order, in short: integrate the guarded follow-on source; run the actual tests, host qualification and packaging; produce fresh qualified contexts and admit preregistered real teachers; build the all-eligible family-disjoint corpus; one GPU-hour of training with continuation; consume the new checkpoint; run the real fitted warm demonstration; then run the unchanged thirty-dataset campaign. Under the current coordination rule, source integration waits until the manual's pull-request lifecycle closes.	§14.1.2 Critical order, §14.1.4 Proposed owner decisions and external dependencies

Area	Current state	Evidence
Risks	Cross-region protection of the current learning objects is not established: the surveyed learning replica is empty and the `staging/` prefix is not replicated. Several large data classes have no established remote copy, and the disk was 87% used at the dated readback.	§D.4 Recorded copy and retention gaps

1.2 Reading this manual: audience, evidence levels and figures

This manual serves operators, scientists, implementation maintainers and reviewers. Operators need installation, preparation, execution, reporting and recovery procedures. Scientists need dataset identities, split boundaries, metrics, model lineage, comparison budgets and limits of the evidence. Maintainers need the workspace map, command contracts, fitted-state formats, feature boundaries and tests. Reviewers need to distinguish code that exists, behavior that has been demonstrated, changes that have merged, and claims still requiring evidence.

Evidence level	Establishes	Does not establish
Source implemented	The named source and command path exist at that revision	Successful execution on this host
Static or fixture verification	Parsing, linting, schema checks or named synthetic tests passed	Real corpus behavior, GPU execution or competitive performance
Bounded local qualification	Named real processes, artifacts, limits and receipts were observed	Other datasets, backends, checkpoints or protocols
Exact-head CI	Required current-attempt jobs and aggregate passed	Skipped tests, all optional features, GPU qualification or promotion
Reviewed merge	Eligible independent native review and guarded merge were verified	Installation qualification or serving promotion
Measured campaign	Admitted and unadmitted outcomes are accounted for under one window	Promised win rate or unbiased sealed-test performance
Release qualification	Exact installation, artifact and backend passed acceptance	Other runtimes, artifacts or numerical policies

A checkpoint file, a nonempty journal, a successful exit and a genuine optimizer update are different observations. A skipped test is not a pass. A missing measurement is unknown, not zero. Cancelled, failed and unavailable fits remain outcomes rather than scores.

The corpus-size figures are metadata/development statistics, not performance. The language coverage figure counts admitted source recipes under additional state restrictions; it is not a backend success rate. The architecture diagram is explanatory, and the completion matrix is an evidence inventory; neither is an experimental result.

1.3 North star: better algorithms from accumulated experience

1.3.1 The proposed north star and matched comparisons

The product goal is a repeatable learning cycle: verified dataset-and-split inputs become contexts; the policy proposes executable programs; bounded genetic-programming (GP) search fits and improves them; eligible measured programs become teachers; training produces a new checkpoint; and that checkpoint improves a subsequent bounded search. The north star (*proposed; owner ratification pending*) is **better predictive algorithms on previously unseen dataset families at the same declared compute budget**, with reproducible fitted artifacts; §1.3.1 The proposed north star and matched comparisons states it in full. More optimizer steps, a larger corpus, lower training loss or a higher historical Elo are supporting observations; each answers a different question.

Measure progress against both the generic search control and the previous accepted checkpoint, using the same data identities, folds, method set and resource envelope. Report task-specific loss, wins/ties/losses, failure and unavailable rates, and cost per unique fitted predictor. A trained checkpoint earns a promotion claim only through its bound qualification procedure. The frozen thirty-dataset panel is a development comparison; it cannot establish unseen-family performance when prior checkpoint exposure is unknown. Sealed evaluation remains untouched.

ASM learns to generate and optimize executable algorithms from data. Its north star (proposed; owner ratification pending) is better downstream predictive loss from generated, actually fitted algorithms on previously unseen dataset families, under the same compute allowance, with reproducible predictions and a usable fitted artifact. This is an operational definition for the next evaluation, not a ratified objective or a claim that the current model has already met it. The frozen-panel win count PANEL-WINS (proposed definition; owner ratification pending; §9.11.1 KPI definitions) is a separate development KPI, not the north star: the panel is adaptive development data whose prior checkpoint exposure is unknown, so it cannot establish unseen-family performance.

Compare each challenger against both a matched generic search and the previous checkpoint, with the same grammar, task, rows, class ordering, preparation, seeds, resources, scoring policy and exposure rules. When the challenger belongs to a different model family, compare it with the previous checkpoint only through such matched downstream fits, never through loss values: the pending bounded campaign trains a width-256, two-layer Python `JopePolicyModelV3` (§8.4.1 What the pending campaign trains), not a continuation of the accepted width-512, six-layer step-60,000 model (§2.8 The accepted regression model: lineage B, step 60,000). A lower imitation loss alone is insufficient: the generated programs must compile or interpret, fit, survive the resource limits, and improve predictions. Conversely, an honestly measured optimizer update is training progress even when the downstream candidate does not win.

The frozen thirty-dataset panel (asm-private-30-v1; not yet executed) defines an adaptive **development** comparison whose protocol records selection and confirmation exposure. It cannot establish the previously-unseen-family part of the north star merely because a dataset appears in that panel. That requires independently frozen family/exposure evidence and a separately authorized evaluation; this manual grants no access to sealed labels. Historical exposure that cannot be reconstructed stays unknown. See §9.4 What the historical training and target evidence establishes, §C.1 Did GP targets get better? and §9.6 The closure ledger.

1.3.2 Where the current evidence stands

The latest retained origin proof at source `68f56404` is a useful concrete boundary: it executed actual neural proposals, fitted generation zero, and fitted neural descendants, with an authenticated recovered report. Its neural development-hold RMSE (n = 34) was 166.3181717454848 versus generic 155.25909998168862 on one regression dataset, under equal declared arm allowances but unequal fit time; the neural arm's selected recipe descends only from random-exploration candidates, not from a `neural_policy` proposal. This demonstrates consumption, not a quality win, a quality loss or a thirty-dataset result. The fourteen private language-example commands establish recipe-refit prediction parity across five language configurations; they run no GP evolution and are not exact-state fitted-winner export. Historical trained corpora also must not be confused with the zero newly demonstrated teachers under the pending ASM admission contract. The measured origin report SHA is `6e0c20720256b0735a6969c45032f44082c92abacadf2c4eb21ef5c390f249dd`; see §1.5.2 Canonical proof record and per-arm facts and §10.4.3 Measured execution of the private examples.

There **has been real historical policy training**. The accepted serving model is the recovered lineage-B step-60000 checkpoint. Retained logs cover 63,000 optimizer steps of that lineage (steps 6,001–69,000, in 21 blocks, each on a different corpus). Loss falls inside every block and jumps upward at every corpus change. The descriptive linear fit over steps 6,001–60,000 changes by −0.018284 in total, while the tabulated block losses lie between 0.785951 and 0.855045 (§B.3.3.2 Within-block and boundary patterns). The 78 retained checkpoint observations of that lineage use changing evaluation populations. The retained fixed-monitor comparison covers three earlier checkpoints (recovery, cycle 25 and cycle 107), and there sequence and reward-ranking losses did not improve over recovery (§B.3.2 Fixed monitoring and actual promotion evidence). In retained paired final searches, whose arms shared their seed pools, the neural arm tied the generic arm exactly on 3,227 of 3,268 comparisons (§C.1.4 Paired final-search outcomes within the admitted development inventory). Across all 26,357 historical search receipts, an external seed audit reports that 98.84% of reported winners were already in their search's seed population (§C.1.2 First-to-last comparison at 1,000 evaluations). The optimizer fitted its objective; better proposals have not been demonstrated.

Historical optimizer evidence for the serving lineage: 21 original A100-recovery logs retain all 63,000 steps from 6,001 through 69,000, with 378,000 selected loss/component/gradient values preserved exactly. All 21 blocks show declining within-block composite loss, while each uses a different corpus. This is historical fitting evidence only, separate from better downstream GP targets or proposal quality, and it implies no new training or promotion. Accepted serving remains step 60,000; later A100 step 69,000 is explicitly refused, and the separately retained world-1 step 60,100 is hardware qualification. The source-bound CSV, corpus boundaries and explicit averaging are in §B.3.3 The complete A100 loss series: steps 6,001–69,000.

The new ASM training path has not completed a real bounded acceptance. No eligible real corpus exists for it yet: zero teachers have been admitted under the new contract, and its two prospective teaching families, `230_machine_cpu` and `712_chscase_geyser1`, are both frozen-panel regression datasets (§7.4 Present teacher readiness and the smallest real path). Its pending campaign would train a new, smaller Python model (width 256, two layers), not a continuation of step 60,000 (§8.4.1 What the pending campaign trains). The native command that could continue step 60,000 is held: its GPU-grant check always refuses (`crates/jope-engine/src/task_contract.rs:88-92`; §8.2.4.2 The dispatch hold, resource incompatibility and valid topologies). Zero newly admitted teachers must not be read as an empty historical corpus or as evidence that JopeDime never trained: the accepted checkpoint's retained closure corpus, `snapshot-000097`, holds 10,214,332 target records labelled under the historical `reference_v1` policy (§D.6.1 Six retained corpora).

1.4 What works now

The frozen benchmark foundation consumes actual row assignments, original labels, declared categorical columns, missing values, groups and class order. It uses regression RMSE, binary AUROC and multiclass log loss from a complete probability matrix. §10.3 Complete registered-recipe matrix inventories the executable native recipes, and §4.4.2 Native roster and external comparator runtimes the four external adapter families, which run at their default configurations.

The shared execution layer has twenty bounded fit slots, resource accounting, independent deadline enforcement, process-tree cleanup, durable journals and original-window resume. Its ten real Linux process/cgroup tests use synthetic workloads to exercise completion, stalled controllers, cancellation, forced process death, memory failure, concurrency and setup failure.

The regression neural path has a qualified native context bridge and a measured proposal-to-fit-to-evolution proof. Policy-origin metadata is checked against pinned producer output, completed inner-fold measurements and measured ancestry. A candidate labelled neural without those links is insufficient. Canonical duplicates retain proposal slots and origins but do not justify fitting the same executable twice.

Numerical parity of the product's policy implementation with the trained model is not established. The product runs a Candle port of the policy on the CPU in float32. The crate states that no numeric parity with the trained tch forward pass is claimed, that its logits and rewards are unverified until reference fixtures exist, and that its `cuda` feature is declared but has never been compiled (`crates/jope-policy/src/lib.rs:84-95`; `crates/jope-policy/src/model.rs:18`). Until such fixtures establish parity, a neural-versus-generic result measures this port, not necessarily the trained model.

Observed generations survive cancellation at the arm cutoff and interruption before the final receipt. Resume reauthenticates completed folds, including completed work inside a partially failed candidate. Only jobs positively proven never admitted can acquire a different expiry status. That exception cannot create a new fit, replace fitted state or extend the clock.

Legacy fit, emit and pipeline surfaces also exist. Their recipe emitters write a program that trains the recipe again; they do not export a fitted predictor; that is the separate, pending portable fitted-state export work. At `a75e543d2` they also cannot emit any registered frozen-panel recipe or executable synthesized ASM recipe: every such recipe carries the raw-table `(input …)` stage, and the legacy planner has no case for it (`crates/jope-bench/src/panel/methods.rs:20-21, :50`; `crates/jope-pipeline/src/synthesized_recipe.rs:34-46`; `crates/feat-math/src/spec.rs:383, :443, :592`). §10.2 Language interfaces at the frozen source gives the detail and §10.4.3.1 What passed and what was measured the worked-example result. In short:

Language	Recipe emit and refit (legacy emitters)	Exact fitted-state export
Python, Rust	The emitter's legacy program set, not panel or ASM recipes; in the private worked example the emitted programs refitted `(pipe X (standardize) (ridge))` with bit-identical predictions	Pending (INFER-02); the public fitted Python/Rust packet is unmerged
C++17, C++20	One C++ emitter compiled under both standards; same worked-example result	Pending (INFER-02); the pending public bundle still needs Python `ctypes` at inference
JavaScript	ES2020 emitter; same worked-example result	Pending (INFER-02)

Language	Recipe emit and refit (legacy emitters)	Exact fitted-state export
CUDA	None	Deferred (FUTURE-01)

1.5 Latest measured operational proof

The current source (reviewed `68f56404`, merged as main `a75e543d2` with the same tree) has a successful operational proof on `230_machine_cpu`: the verified encoder produced context; the neural producer supplied actual generation-zero fits; a fitted descendant followed; two fitted refits each returned 34 development-hold predictions. The neural refit did not beat the generic arm, which had the same declared allowance but more fit time; this is one budget-mismatched sample, not a matched quality comparison. This proof uses the accepted checkpoint, not a newly trained experimental export. As of 2026-09-23, new training, final installed warm demonstrations and the frozen thirty-dataset panel (asm-private-30-v1) had not started, and no current thirty-dataset ranking or Elo exists.

1.5.1 Summary of the one-dataset proof

This is **one preselected regression-dataset operational qualification**, not the unchanged thirty-dataset campaign, which has not run. Its scores are development-hold scores: each final refit is scored on the 34 held rows of development fold 0. This is a short summary; the canonical proof record is §1.5.2 Canonical proof record and per-arm facts.

Item	Observation
Dataset	`230_machine_cpu`
Initial neural arm	64 actual `neural_policy` slots, zero fallback slots, plus 16 `random_exploration` slots in this proof
Initial generic arm	80 generic slots
Declared arm / fit allowance	120 seconds and at most 96 candidate slots per arm; 30 seconds per worker fit. Equal declared allowances are not equal fit time: context and policy production ran inside the neural arm's clock, and the policy producer's exit was observed 41.581 seconds after the campaign start
Arm stop reasons	Neural `original_deadline`; generic `evaluation_limit`
Global proof window	600 seconds, including 30 seconds for reporting
Host limits	20 CPUs/workers, 64 GiB, zero swap, no GPU tasks
Started and finished worker commands	964
Completed / failed commands	834 / 130
Unique fitted predictors / reusable recipes	832 / 164
Final refits / predictions per refit	2 / 34
Run-command wall time	239.1 seconds
Independent report replay	18.7 seconds
Neural development-hold RMSE (n = 34)	166.3
Generic development-hold RMSE (n = 34)	155.3

Item	Observation
Neural ÷ generic RMSE	1.071 (neural worse)
Score ordering	Generic lower RMSE in one budget-mismatched, one-dataset sample; not a matched quality comparison
Interval, baseline and winner origin	Not computed: no confidence interval and no mean-predictor RMSE on the same 34 rows. The neural arm's selected recipe (`ad665e5b…`) is a generation-1 crossover of two generation-0 `random_exploration` candidates, with no `neural_policy` ancestor (§1.5.2 Canonical proof record and per-arm facts)
Genuine neural initial fits and fitted descendants	Verified
Teachers admitted from this proof	None
Full frozen-panel conclusion	Not established

Table values are rounded; the exact values are in §1.5.2 Canonical proof record and per-arm facts, and the exact losses and run wall time are also in `data/performance-evidence.json.gz`. The recovered report SHA256 is `6e0c20720256b0735a6969c45032f44082c92abacadf2c4eb21ef5c390f249dd`. The experiment identity is `646095badab6706168c7e869118d62fddd399faa2a9d6bc7f484c95a021a3944`; its original window ran from 2026-09-23T16:16:50.613Z to 16:26:50.613Z (Unix milliseconds `1790180210613` and `1790180810613`). Plan identity: `a7befb386344bd0cbffa866ad8793077dead185f64e2ae0fb4035a4ed3dcfcc2`.

This proves execution and evidence continuity. It does not prove ASM superiority, ten wins per task, learned improvement after training, an under-sixty-second warm demo, installation speed or portable inference for every winner. The arm clocks are explained in §12.1 Arm order and clocks; the dense-width refusals (16 of 64 genuine neural-policy candidates against 4 of 80 generic candidates) are in §1.5.4 Failures: dense-width refusals and the earlier failed reproducer.

1.5.2 Canonical proof record and per-arm facts

Quantity	Measured result
Source	`68f56404c3e199e06413bfd645d3fe0803d7b96f`
Dataset	`230_machine_cpu`, regression
Immutable build wall seconds (preparation; not a cold-install measurement)	170.4488760890672
Eight-case native context qualification wall seconds (separate preparation)	29.248627813067287
Run wall seconds	239.14312197000254
Commands started / finished	964 / 964
Completed / failed command outcomes	834 / 130
Train OOF / diagnostic / refit-screen commands	960 / 2 / 2
Unique complete fitted predictor artifacts	832
Unique reusable recipes	164
Successful final refits	2, each 34 predictions

Quantity	Measured result
Neural development-hold RMSE (n = 34)	166.3181717454848
Generic development-hold RMSE (n = 34)	155.25909998168862
Generic minus neural loss	−11.059071763796169; neural did not win
Neural ÷ generic RMSE	1.071 (neural 7.1% higher; `neural_win=false`)
Known worker CPU seconds	181.220773
Sum of worker wall seconds	202.266164543
Exact supervisor aggregate CPU seconds	270.889403
Unknown worker CPU / wall counters	0 / 0
Teachers admitted from this proof	0

The two arms had the same declared allowance but not the same work. Per arm, from the recovered report and its audit:

Quantity	Neural arm	Generic arm
Declared arm clock	120 s	120 s
Work inside the arm clock before any fit	Context encoding and policy production; producer exit observed 41.581 s after campaign start, leaving at most 78.613 s to the neural cutoff (§12.1 Arm order and clocks)	No context or proposal production (`proposal: null`)
Evaluation cap	96 slots: 80 generation zero, 8 mutation, 8 crossover (one offspring wave)	96 slots, same split
Stop reason	`original_deadline`	`evaluation_limit`
Completed five-fold unique recipes	77 of 96	89 of 96
Generation-zero completions	62 of 80: `neural_policy` 48 of 64, `random_exploration` 14 of 16	76 of 80 `generic`
Generation-zero dense-width failures	16 of 64 `neural_policy`	4 of 80 `generic`
Selected recipe	`ad665e5b…`: generation-1 crossover of two generation-0 `random_exploration` candidates; no `neural_policy` ancestor	`c806c3be…`: generation-1 mutation of a generation-0 `generic` candidate
Selection score of the selected recipe (mean of five inner-fold RMSEs on the same 135 cut-0 fitting rows and inner split)	61.776	65.362
Refit and scored rows	Refit on the 135 cut-0 fitting rows; scored on the 34 held rows of development fold 0	Same rows
Development-hold RMSE	166.3181717454848	155.25909998168862
Neural ÷ generic RMSE	1.071	—

Generic has the lower development-hold RMSE in one budget-mismatched, one-dataset sample; this is not a matched quality comparison. Because the neural arm's selected recipe has no `neural_policy` ancestor, its refit score measures the arm as a whole, not a program the policy proposed. Not computed or not recorded for this proof: a mean-predictor (trivial-baseline) RMSE on the same 34 rows, a paired bootstrap interval for the RMSE difference, and a comparison at equal completed evaluations. Numeric parity of the Candle CPU policy port that produced the proposals with the trained tch forward pass is not established, and the port's `cuda` feature has never been compiled (`crates/jope-policy/src/lib.rs:84-95` at `a75e543d2`); the report records `ranking_backend: candle_cpu_float32` and `native_cuda_parity_qualified: false`.

1.5.3 The outcome figure and its limits

This is the current bounded operational proof, with one preselected regression dataset and equal declared search-arm allowances but unequal fit time. It demonstrates authenticated neural-origin fits and fitted descendants. The complete thirty-dataset ASM panel has not run, so this figure cannot establish its wins, ranking or Elo.

The current operational figure contains only the two development-hold RMSE values (n = 34) from the preselected `230_machine_cpu` proof: neural 166.3181717454848 and generic 155.25909998168862. Lower is better. Generic has the lower RMSE in this one budget-mismatched, one-dataset sample; it is not a matched quality comparison (§1.5.2 Canonical proof record and per-arm facts). It would be misleading to compute a new thirty-dataset Elo from these two values or describe the 832 fitted predictors as 832 independent datasets.

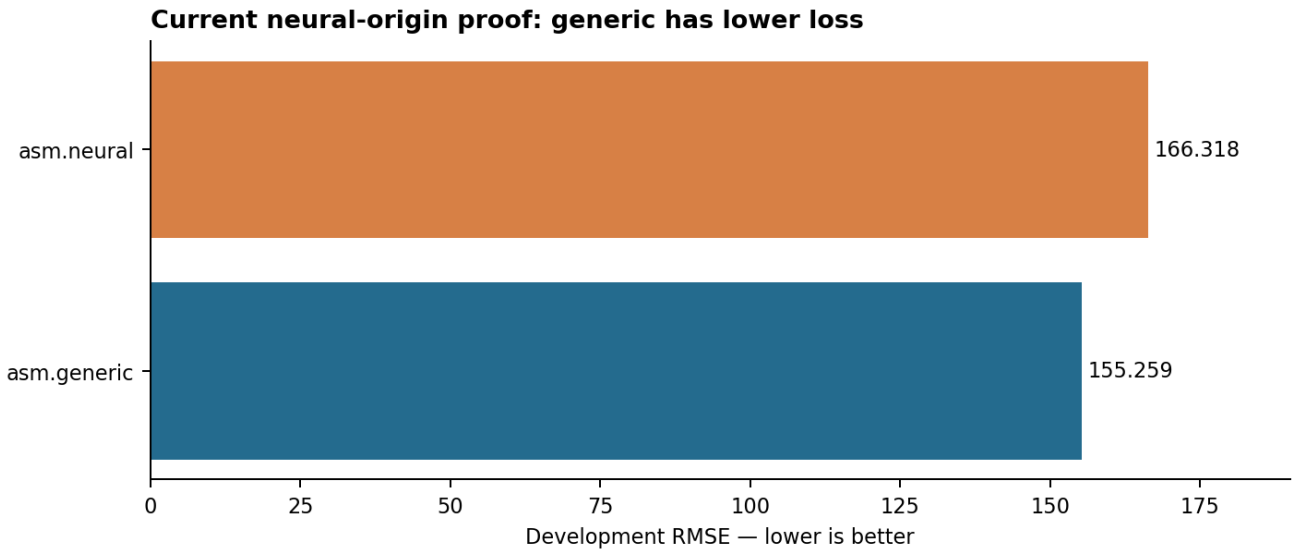


Figure 1.1 Current operational proof: neural and generic development-hold RMSE (n = 34)

The two underlying records are in `data/plot-current-origin.csv`; exact evidence and denominators are in `data/performance-evidence.json.gz`. There are 34 predictions per final refit. The proof's run wall time is 239.143 seconds; its independent recovered report takes another 18.719 seconds. CPU, wall-time and per-predictor accounting follows in §12.5.2 Worked example: CPU, wall time and per-predictor cost of the one-dataset proof; the status summary is §1.5 Latest measured operational proof.

1.5.4 Failures: dense-width refusals and the earlier failed reproducer

Every one of those 130 failed commands exited 1 with a verified stderr message `generated dense width N exceeds 512`. They represent 26 distinct candidate recipes with five failed folds each. Generation zero included 16 of 64 genuine neural-policy candidates failing this way (25%), compared with 4 of 80 generic candidates (5%); the remaining failures came from exploration or offspring. These jobs reached their process gates, but the failed candidates lack complete physical-fit counters, so the corresponding per-origin total remains unknown rather than inferred as zero or five. The 512-column check occurs inside fit execution; a verified prelaunch width rejection remains an optimization opportunity. Sources: `crates/feat-math/src/advanced_v3/feature_block.rs:109`, `crates/feat-math/src/advanced_v3/grammar.rs:2299-2300`; retained 130 finished/stderr hashes and report origin counts.

The earlier failed reproducer at source `d0f5748b7db1` failed after 194.789382994 seconds at scorer replay. All 964 commands had terminal receipts, with 834 completed and 130 failed. Its exact supervisor CPU was 252.614611 seconds and worker CPU 181.863951 seconds. It remains a failed reproducer; the old journal and deadline were not repaired or reopened.

1.6 Failure records are retained

Two failed cancellation/crash reproductions preceded the resume correction. A partial-fold fixture initially failed on a noncanonical path, then passed after fixing the fixture without weakening verification. A live run at `d0f5748b7db1ac419110289f912c7eface0022bc` completed its worker inventory but failed report verification because live and recovered scoring populated different fold-evidence fields. Its journal remains unchanged. The shared scorer correction and the successful proof at the corrected PR head commit `68f56404c3e1` are separate evidence.

Final local suites at the snapshot passed **53 benchmark and 106 pipeline tests**. One privileged test was ignored in that invocation and qualified separately through **ten actual host cases**. The numerical encoder proof passed **eight fixed cases** under unchanged tolerances and thirty-second per-job limits. §E.6.2 The PR83 lifecycle record records CI and merge state independently.

1.7 Pending acceptance

Workstream	Current boundary	Remaining acceptance
Regression neural consumption	Qualified contexts, proposals, actual fits and descendants on one dataset; parity of the CPU policy port with the trained model not established	Reference-fixture parity with the trained model, full panel and newly trained checkpoint consumption
Classification neural consumption	No classification neural arm at the current source: the accepted policy is regression-only, and classification `asm.generic` searches ridge heads only (§2.7.2 Experimental classification context). Experimental context/invariance support; follow-on producer prepared	Task-correct live consumption and clearly labelled fixture scope
High-water mark (HWM) and teaching eligibility	Follow-on source prepared; zero teachers admitted under the new contract	Full identity, finite out-of-fold (OOF) improvement, four-of-five-fold gate and no-signal refusal

Workstream	Current boundary	Remaining acceptance
New ASM training path	Historical trained model exists; new guarded source and test plans prepared. The pending campaign trains a new, smaller Python model, not a continuation of step 60,000	Positive updates, changed parameters, reload, continuation, resume compatibility and bounded GPU proof for this new path
Trained regression export use	Explicit experimental consumer prepared	Compile and execute exported-checkpoint-driven fits
Portable fitted inference	Native fitted artifacts; export packet prepared; legacy emitters cannot emit panel or ASM recipes	Full state parity and actual selected-winner backend qualification
Installation and warm demo	Source packets prepared	Checksum install, CLI-only build, preparation, fitted artifact/predictions and separate cold/warm timing
Thirty-dataset comparison	Frozen panel and protocol; not yet run (0 of 30 datasets)	Unchanged two-hour campaign, every loss/refusal/failure and gap
Motifs	Separate typed library branch	Eligible-state reader, mine command, umbrella wiring and validated reuse
Broad full-main tests	Historical failures retained; full-main runs 35929010741 and 35936673628 failed, each with eight families passed, two failed and one unavailable. The earlier PR83 record remains historical (§E.6.2 The PR83 lifecycle record)	Reproduce and repair; fast CI is not full qualification
New serving promotion	Recovered accepted step-60000 remains the model binding; the native command that could continue it is held	Exact release/backend/model acceptance and reviewed promotion for a new checkpoint
Manual publication	Edition 1 landed through PR85 at `bdfc95c63`; DOCFIX-01 landed through PR86 at `2b375ef11`. PR84 was superseded because its intermediate history exceeded the 5 MiB file cap. Edition 2 landed through PR87 at `02c0bef6c` after exact-head CI and independent native approval; main-guard passed	Each later revision needs fresh exact-head CI, independent native approval and reviewed merge

§14.1.3 Compact task index and the acceptance cards in §E.1 Primary acceptance items by stage expand these rows into actionable items; the property list carries the same items. A prepared patch or planned command is not an installed or executed capability.

1.8 What version are we running?

1.8.1 Implementation snapshot and this manual's own identity

The implementation snapshot is **68f56404c3e199e06413bfd645d3fe0803d7b96f**, dated 23 September 2026. §E.6.2 The PR83 lifecycle record records the separately refreshed reviewed-main state. A rebase merge can produce a different commit with the same reviewed tree. Older plans and historical PASS entries do not establish current capability.

This edition describes the implementation reviewed at `68f56404c3e199e06413bfd645d3fe0803d7b96f`. PR83 merged on 23 September 2026 at 16:34:47 UTC. The resulting remote main is `a75e543d2166017cb0139e0a251d51d9a0c98e08`. Both revisions have Git tree `176b75bc36d76c7ce4eaeb7234677ec013468889`. Rebase merge changed commit identity; it did not change the reviewed source tree.

The manual itself is a later documentation change. Its publication identity must be read from its own Git history and CI receipt, not confused with the implementation snapshot. Follow-on patches described as pending are outside the implementation snapshot even if they are preserved on this host.

Publication history, refreshed after the PR87 merge; the application snapshot above stays pinned to PR83:

- **PR84 was closed as superseded** (coordination record of 22:22:38 UTC). Its head is `e06c0551e3ec5ee5260debb3d953c17f34d15093`. The independent non-contributor review 5295763455 (state COMMENTED, 19:30:12 UTC, at that head) withheld approval for two reasons: a blocking finding that three intermediate commits each contain two files larger than the repository's 5 MiB cap (`deploy/ci/check.sh:46`), so a rebase merge would put those commits on `main`; and exact-head CI that was not yet fully green.
- **PR85, the clean-history successor, merged at 22:33:36 UTC.** Codex, the manual owner, published the single commit `2022017a8c45cc25967ae99ae781ca26276140b8` on `a75e543d2` (branch `docs/asm-maintainer-manual-final`, pushed at 22:21:30 UTC) with a 203-page PDF (SHA-256 `69418c48…`). Exact-head CI run 35927921851 attempt 1 succeeded, and the independent non-contributor review 5297559032 approved that head at 22:32:39 UTC. The merge produced `main` `bdfc95c63d905c647e41175e79dc30cdc8600287`, whose Git tree `fbbed08a…` is identical to that of `2022017a8`.
- **Edition 2** restructures and corrects `2022017a8`. It was prepared in Claude's lane and integrated by the manual owner, preserving the authoring commits. Edition 2 merged through PR87 at 2026-09-24T02:58:00Z as main `02c0bef6c81ab17e034daea308c1a0b5a7940467`, after exact-head CI 35948653955 attempt 1 and independent native approval 5299139743; main-guard 35949456287 passed. Further corrections repeat the reviewed lifecycle.

1.8.2 The identity tuple

There is no single version number that describes all of the following. Record the entire identity tuple whenever comparing results or continuing training.

Dimension	Current evidenced identity	What changing it means
Product/package	Rust workspace and Python package `0.1.0`; immutable CLI reports `jope 0.1.0`	A packaging/API release number; not a checkpoint generation or proof of installed capability.
Implementation	Reviewed source `68f56404c3e199e06413bfd645d3fe0803d7b96f`, merged as `a75e543d2166017cb0139e0a251d51d9a0c98e08` with the same source tree	A later documentation commit does not rebuild the binary. Preserve binary/build/source-manifest hashes.

Dimension	Current evidenced identity	What changing it means
Accepted regression policy	Lineage B, step 60000, `parallel.checkpoint.v1`; state hash `8ac80c269cc47c253b6f53fa3bc1eb75bb2fcda0bdace3ff8554be0175c400d6`	The current accepted consumer hard-pins this identity and its model/configuration/parameters. Pinning does not establish numerical parity: the Candle CPU port claims no parity with the trained tch forward pass, its logits and rewards are unverified, and its `cuda` feature has never been compiled (`crates/jope-policy/src/lib.rs:84-95`).
Hardware-qualification continuation	Step 60100 is retained hardware-qualification continuation evidence, state SHA `745be4b5e998bce85615349a5b03b901cd8ad2cf605e6dce3b86fbf777cc7388`; verify the actual training pointer (the `training-head` key) independently	This is not the accepted serving checkpoint. Do not advance a pointer from a filename or step number.
Rejected policy	Step 69000 is explicitly refused by state or model hash	A numerically higher training step does not imply promotion.
Python model family	Current Python implementation declares `policy_v3.4-split-context`, with legacy `policy_v3.3-fused-heads`; trainer resume format `PrimePolicyTrainerResume.v1`	These identify a different trainer/model serialization surface from the accepted native `model.ot` lineage.
Experimental ASM policy (pending)	The stopped training64 patch would train a new Python `JopePolicyModelV3` (width 256, 2 layers, 4 heads, feed-forward 1024), initialized at most from a Python checkpoint; no real campaign has run with it	Experimental export only: not serving, not lineage B and not a continuation of step 60000, which the Python path cannot load. Compare the two only through matched downstream fits, never through loss values. The native route that would continue step 60000, `jope-parallel`, is held (§8.2.4.2 The dispatch hold, resource incompatibility and valid topologies).
Encoder	Current CPU (Rust/Candle FP32) context schema `jope.cpu-context/v1`, separate from the held native GPU path; produced artifacts require pinned native-fixture replay before qualification. Dataset 1536 (Hyperion 512 from the features only + projected foundation 1024 from the features and training targets) + target 512 = 2048 floats, so “dataset 1536” is a historical label that includes target-conditioned values; legacy frozen encoder descriptor `parallel.frozen_encoder.v1`	Every component weight, transform, source/backend and assembly hash participates in identity. Dimensions alone do not establish compatibility.
Hyperion assets	Current CPU producer pins the v5 arm-B weight hash `88b9e8345e44d3ea609a8286215618a762f14e607fcdada6d27447b1f74668be`; a separately stored bundle is called `hyperion-v7`	A bundle name or Rust `V7Model` type is not permission to substitute newer-looking weights. “v5” here is asset lineage, not a global product schema version.

Dimension	Current evidenced identity	What changing it means
Action vocabulary	Accepted 401-token vocabulary SHA `8d55fee4366acd2c175cf55bfd769cebc904770f5227a6fd50a613723dc85787`; documented product restriction SHA `570b2e5e8d859a73b888f5d0bee27f59da0e2376536babce24c241604f818e81`	The restriction preserves the token table. Adding tokens or changing parameter bins requires a new compatibility and training decision.
Corpus	Native `parallel.teaching_shards.v1` / `parallel.teaching_corpus.v1`, plus immutable index, vocabulary, contexts, records, label policy and exposure identities	A corpus schema does not certify teacher eligibility, data freshness, or unseen-family status.
Dataset/catalog	`jope.dataset-catalog/v1`, `jope.dataset-collection/v1`; individual content and split hashes remain separate	Keeping the schema while changing rows, labels, categories, groups or splits produces a different comparison.
Model packaging	`jope.model-bundle/v2`	Package/license/inventory metadata is separate from accepted model identity and successful inference.
Frozen benchmark	`asm-panel-v1:screen-fold=0:top=5:confirm=all-5:mean-fold-loss`; evaluator `jope-bench.frozen-panel-f64/v1`; manifest SHA `ba7dace4cd535687002cc95538dbc7f7aaf77ec973177c7159b3c6d22471eee0`	Altering a method, split, deadline, evaluator or selection rule requires a new protocol identity. Do not relabel an altered run as the frozen panel.

Source: workspace version, Python package, accepted checkpoint and vocabulary pins, Python model version, resume schema, context assets and identity, legacy encoder descriptor, catalog schemas, bundle schema, panel identity.

CHAPTER 2

Architecture, encodings, GP representation and model lineages

IN THIS CHAPTER

2.1 Explain it like I’m 5: learning to suggest better recipes 22
2.2 Current executable pipeline and the historical snapshot 23
2.2.1 From original rows to a conditional program 24
2.2.2 From program candidates to eligible teachers 25
2.2.3 From teachers to changed parameters and another measured search 26
2.2.4 From a fitted predictor to deployable code 26
2.3 Preserved historical chapter snapshot 26
2.4 System at a glance: flow diagram and component map 27
2.5 The learning cycle: what is proven and what remains 28
2.6 GP representation: recipes, fitted predictors and the four artifact types 30
2.7 Encodings and the proposal context 31
2.7.1 Regression context production 31
2.7.1.1 Original data and preprocessing 31
2.7.1.2 Component dimensions and assembly 31
2.7.1.3 Qualification and admission 32
2.7.2 Experimental classification context 33
2.8 The accepted regression model: lineage B, step 60,000 33
2.8.1 Checkpoint binding and configuration 33
2.8.2 Decoder and reward architecture 34
2.8.3 Accepted identities and the product restriction 35
2.9 Model lineages 35

2.1 Explain it like I’m 5: learning to suggest better recipes

Imagine a child learning to choose recipes for unfamiliar ingredients. The table of data is the ingredients. A small description says what those ingredients look like. The neural model reads that description and suggests recipes: perhaps clean the numbers, combine some columns, then fit a predictor. It suggests instructions; it does not already know the fitted answer for this table.

We cook the recipes by actually fitting them. We test them on examples whose answers were withheld from that fit. Genetic programming (GP) changes promising recipes, or combines parts of them, and cooks the new versions too. A learned reward score helps choose what to try; only the real fitting and scoring tells us how a recipe performed.

Some recipes become teaching examples, but only after the training-side admission rules check them. Neural training changes the model’s numerical weights so it can imitate useful recipes and rank candidates. We then ask the new model to suggest recipes for another measured search. A lower loss on the same objective and examples means a better fit to that lesson; it does not by itself mean it will cook better on a new table.

Think of withheld answers as an exam. Practice answers may guide study. The exam answer key must stay outside every decision that chooses the recipe being tested. If we look at an exam score and change the recipe, that exam has become practice. The nested transfer audit gives each practice-side audit its own full search, then separately tests a full-training search on the outer exam. All predictions must be committed before anyone sees the round's audit or outer scores.

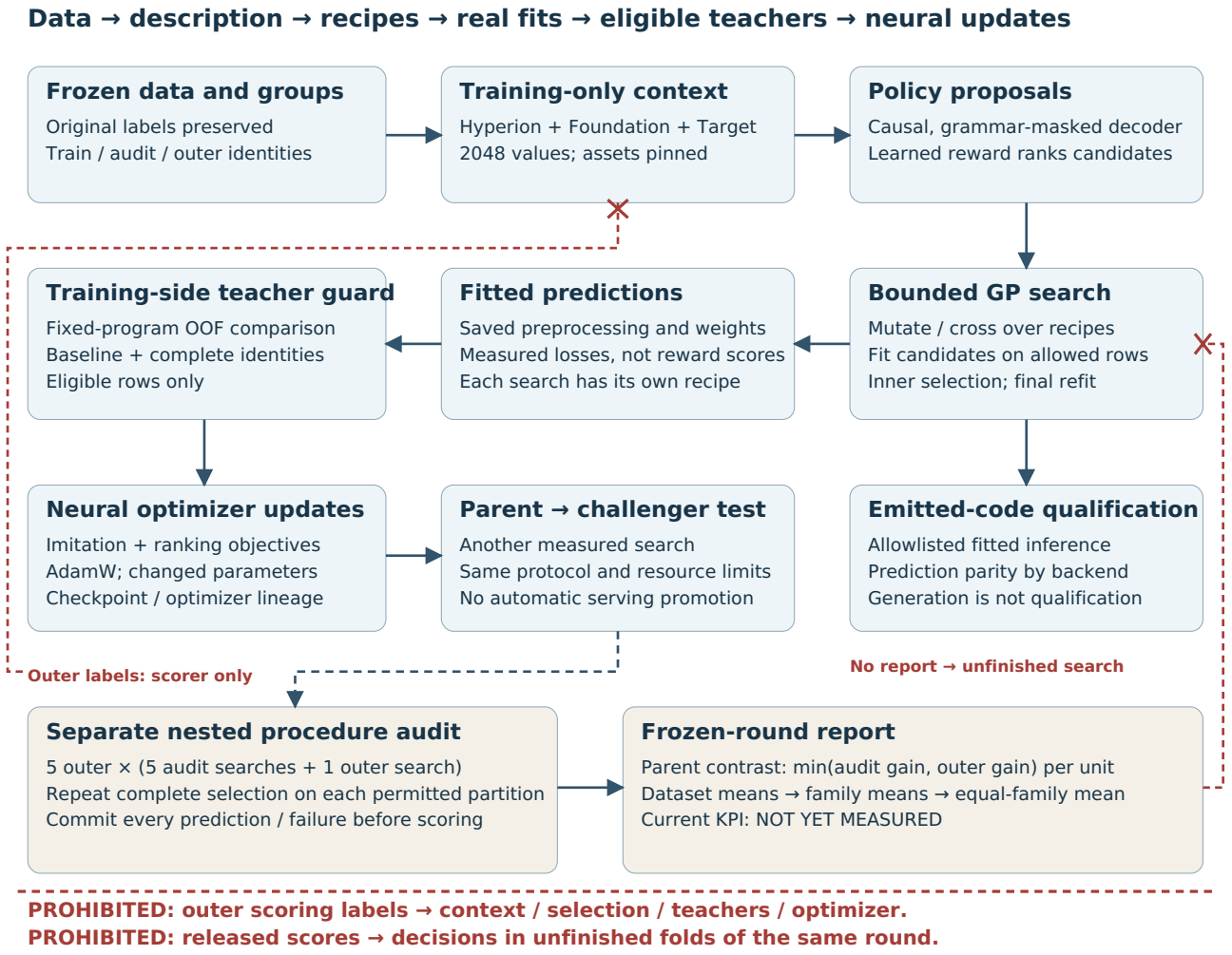


Figure 2.1 Data, search, teaching and measurement boundaries

The diagram describes executable source responsibilities and the required measurement boundary. It does not claim that a new live learning loop has been qualified. Red paths are prohibited dependencies: outer answers must not reach context construction, search, teaching or optimizer updates; released results must not adapt unfinished folds.

2.2 Current executable pipeline and the historical snapshot

This addition is pinned to merged measurement source `7f2b2bf860e4b1ac8815c8da55e507143570a905` (PR90). It refreshes the executable path while preserving the older sections and their anchors. The PR83 source narrative below is a **historical snapshot**: its "pending overlay" language is dated to that snapshot. Several of those source components have since merged, as the following map records. Implemented source, CPU fixtures, historical runs, live qualification and serving promotion remain separate claims.

2.2.1 From original rows to a conditional program

The native regression context producer uses only permitted fitting rows. Hyperion embeds feature rows into 128 values per row, then pools mean, population standard deviation and 10th/90th percentiles into 512 values. Foundation consumes features **and permitted training labels**, producing 2048 values projected to 1024. The target encoder contributes 512. Frozen normalizations and clipping assemble a 2048-vector. Thus the configuration name `dataset_dim=1536` does not mean label-free: its Foundation block is target-conditioned. The source and numerical qualification details remain in §2.7.1.2 Component dimensions and assembly.

The accepted native model has width 512, six causal decoder layers, four attention heads and feed-forward width 2048. Its vocabulary projects to 401 output tokens from 657 embedding rows. Separate context MLPs map 2048→512→512 for decoding and reward. Decoding prepends a context token to program embeddings and adds positions; causal and padding-key masks prevent future-token/padding attention. A grammar mask limits each sampled token to legal continuations. A valid token sequence still needs executable-schema validation and an actual bounded fit.

The reward path has a different representation: it averages non-padding program-token embeddings, without positions. For one context, reordering the same token multiset cannot create a different semantic reward representation. It combines that pooled vector with a context-gated product and an absolute-difference block, giving 3×width features to four reward heads. This learned ranking signal is not a measured RMSE. The runtime head applies softplus to predict a positive cost. The autoregressive decoder remains order-sensitive.

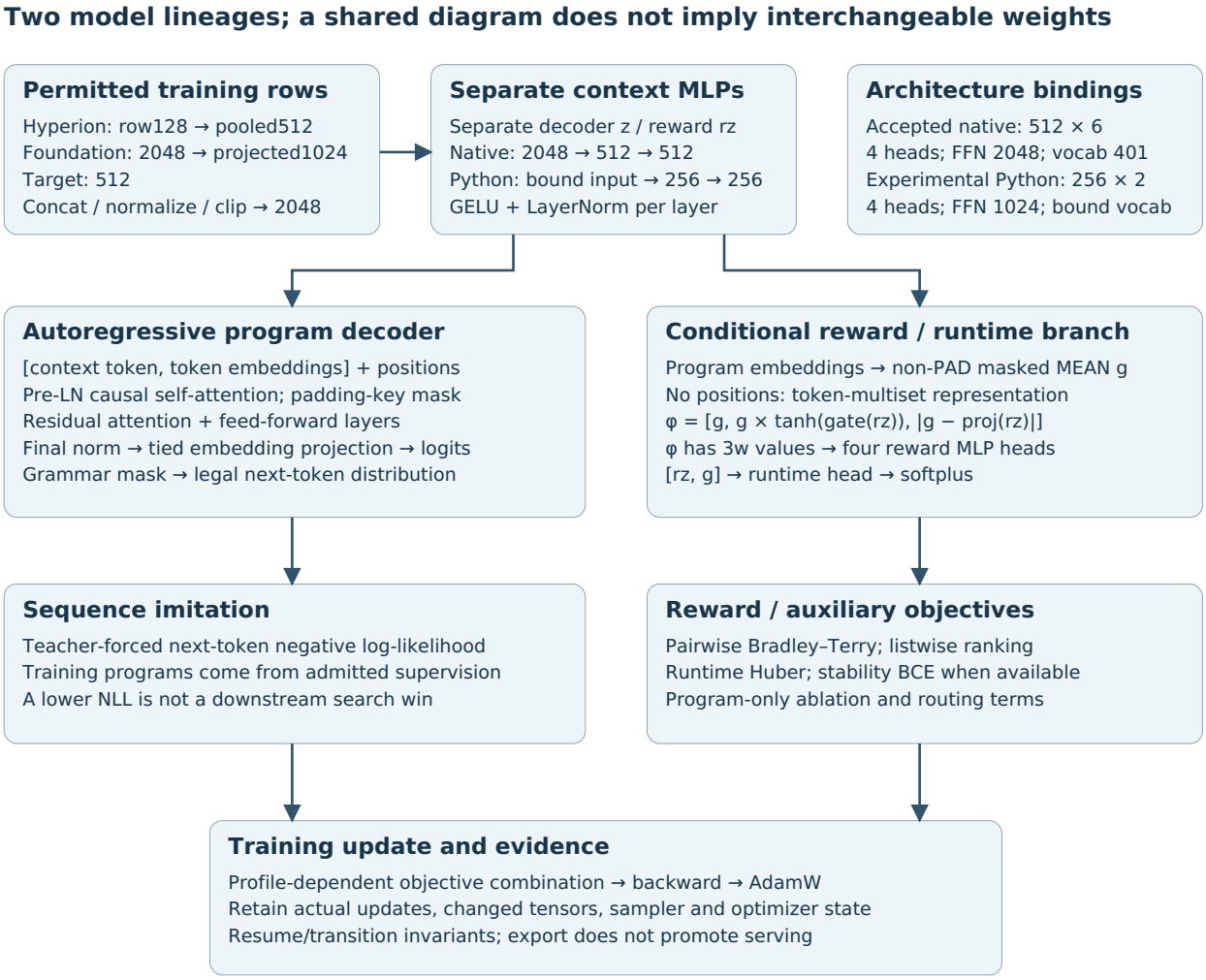


Figure 2.2 Accepted and experimental neural dimensions, attention and objectives

2.2.2 From program candidates to eligible teachers

`crates/jope-policy/src/decode.rs` and `ranking.rs` produce and rank a grammar-constrained candidate pool. `crates/jope-pipeline/src/neural_search.rs` and the `jope-bench::panel` orchestration retain origin, ancestry, selection and resource identities. `feat-math` supplies typed variation and fitted preprocessing/predictors. Mutation and crossover operate on executable recipes; scoring evaluates retained predictions against the appropriate labels. A neural proposal, a mutated descendant and a generic candidate are distinct origins.

`crates/jope-bench/src/panel/teaching.rs` now implements preregistration, admission and verification; `crates/jope-learning/src/eligibility.rs` supplies the guard. Admission uses the fixed candidate's complete train-side OOF comparison, including the trivial baseline and immutable comparison identity. Failed, incomplete or ineligible observations are not quietly relabeled teachers. That fixed-program evidence serves teaching. The new transfer procedure audit instead repeats selection independently inside each fold; these receipts are not interchangeable.

2.2.3 From teachers to changed parameters and another measured search

`crates/jope-cli/src/train.rs` now exposes preparation/materialization, planning, run, resume, comparison and export. `python/src/jope/policy/training_campaign.py` supervises the bounded Python path. Its normal experimental configuration is width 256, two layers, four heads and feed-forward width 1024. Its input width is corpus/checkpoint-bound `dataset_dim + target_keep_dim`, including any permitted zero-tail pruning; it is not universally the native 2048. This is a newly initialized lineage, not a continuation of native 512×6 step 60,000. Tiny 8×1 CPU smoke fixtures are a third configuration, not historical training evidence.

`model_v3.py::loss_components` and `losses.py::kg_combine` implement supervised sequence negative log-likelihood, reward pairwise Bradley–Terry and listwise ranking terms, runtime Huber, stability BCE, a program-only ablation term and routing cross-entropy, subject to the selected profile and available supervision. Diagnostic listwise entropy is a baseline for interpreting cross-entropy, not a new measure of generalization. These are imitation and reward/ranking objectives; the implemented training path is not PPO or GRPO.

`train_v3.py` runs backward passes and AdamW updates, retaining model/optimizer state, sampler/exposure information and proofs of actual update counts and parameter changes. `training_transition.py` binds corpus, optimizer and schedule invariants. The comparison path implements declared fresh/replay siblings and a replay-only control within their original windows; actual sampled exposure must be verified, not inferred from the nominal mixture. Resume is not permission to reset a deadline or silently replace replay supervision.

A checkpoint file alone proves none of this. Verify committed updates, changed parameters, lineage and reload first; then compare the challenger with its parent by actual downstream fitting under one frozen protocol. `training_campaign.py` export verifies the campaign and marks the experimental bundle `serving_eligible: false`. It does not promote the accepted serving pointer. Native Candle-versus-tch policy numerical parity remains limited as recorded in §2.8.2 Decoder and reward architecture; the source map is not a new parity qualification.

2.2.4 From a fitted predictor to deployable code

The private recipe, fitted native state, emitted refitting recipe and fitted inference bundle remain four different artifacts (§2.6 GP representation: recipes, fitted predictors and the four artifact types). `fitted_export.rs` prepares an allowlisted bundle and `fitted_export_run.rs` qualifies emitted predictions under the original limits. Match the raw input boundary, saved preprocessing and native predictions across supported backends. Successful source generation is not numerical qualification, and a qualified export is not evidence that training improved the policy.

2.3 Preserved historical chapter snapshot

The following sections retain the original dated source narrative and evidence. The current-source additions above supersede implementation-status descriptions where explicitly stated.

Algorithm Synthesis Model (ASM) learns to generate and optimize executable algorithms from data. The implementation combines a learned proposal policy with actual fitted evaluation and genetic variation. A generated program is a candidate recipe; a fitted predictor is that recipe plus the state learned from a specific training partition. A checkpoint is the proposal model. Those three artifacts have different identities and different evidence requirements.

This chapter describes source commit `68f56404c3e199e06413bfd645d3fe0803d7b96f`, the reviewed PR83 head. PR83 merged on 2026-09-23 at 16:34:47Z as `main` commit `a75e543d2166017cb0139e0a251d51d9a0c98e08`, which has the same tree

(`176b75bc36d76c7ce4eaeb7234677ec013468889`), so the current-source references below also describe `main`. As of 2026-09-23 about 23:55 UTC, the only later merge to `main` was PR85, edition 1 of this manual, which produced `main bdfc95c63d905c647e41175e79dc30cdc8600287` at 22:33:36 UTC. PR85 changed only files under `docs/` plus `README.md` and `AGENT_CHAT.md`, and no file under `crates/` or `python/`, so the source references below still describe `main`. Changes described as **pending training overlay** refer to the stopped 64-file custody patch `595824ef6eb8e15872be058668503cedad1dd9a5ff98ac4700c6f713251d384c`. That patch includes experimental regression consumption; it is not in this source commit. Current-source references below are repository paths with source line numbers. Overlay references are explicitly marked and use the stopped overlay's line numbers.

2.4 System at a glance: flow diagram and component map

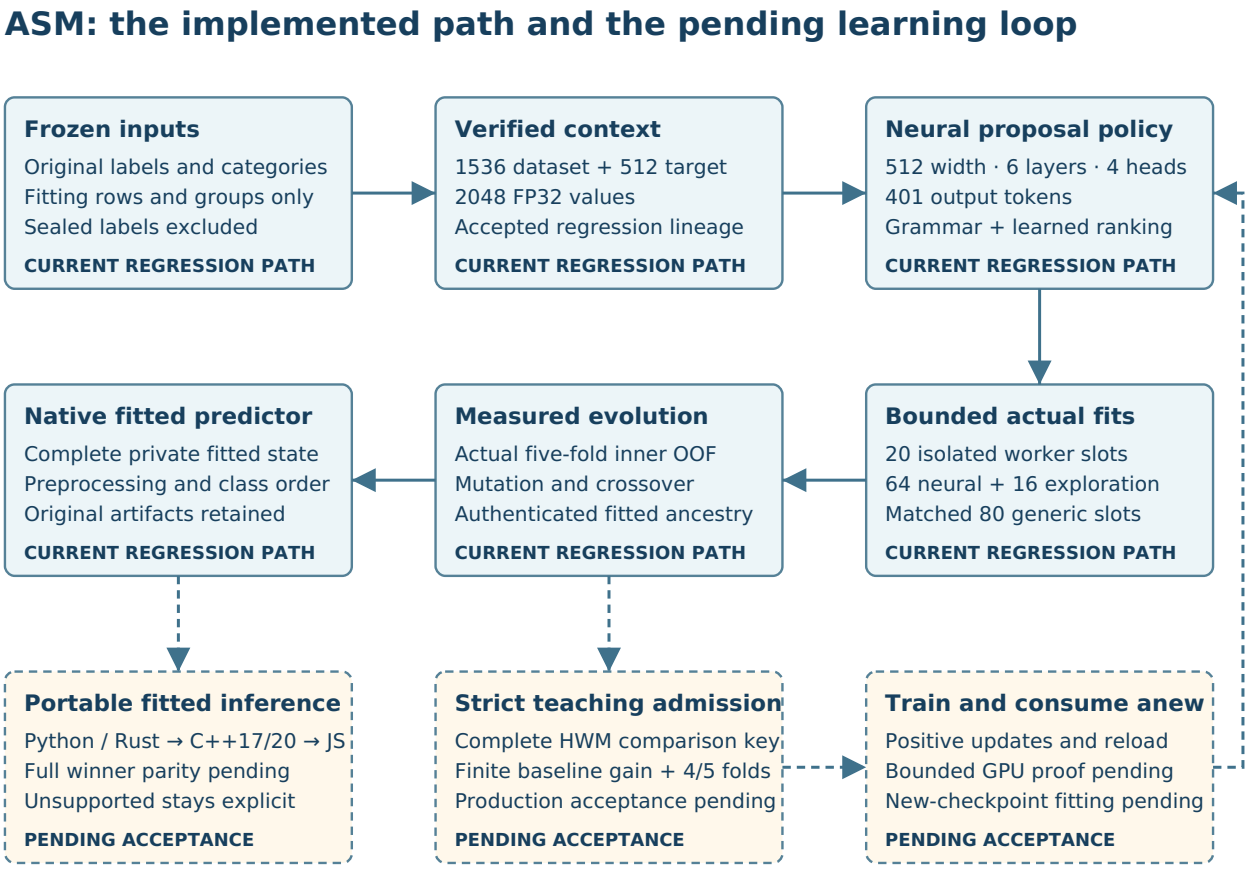


Figure 2.3 ASM implementation and pending learning loop

The solid path has bounded regression execution evidence. Dashed stages require the named runtime acceptance; this diagram does not claim an already qualified self-improving training loop.

Component	Main responsibility	Durable boundary
`jope-datasets`	Frozen dataset metadata, source and split identity	Content-addressed manifests, source hashes and exact row assignments
`jope-bench::panel`	Development-data preparation, methods, metrics, screen/freeze/ confirm protocol, ASM orchestration	Campaign plan, prepared data, fits, frozen selections and reports

Component	Main responsibility	Durable boundary
`jope-pipeline`	Search identity, bounded execution, original deadlines, process and output authentication	Started/finished records, arm windows, observations and receipts
`jope-policy`	Accepted regression checkpoint binding, CPU context producer, grammar-constrained proposals and learned ranking	Checkpoint receipt, vocabulary, qualified context and proposal evidence
`feat-math`	Executable recipe representation, variation and fitting	Canonical recipe identity, fitted state and prediction output
`jope-learning`	HWM and score ledgers; pending stricter teaching admission	Comparison identity, immutable blobs, hash-chained events and frozen teaching manifests
Python policy package	Existing trainable policy model; pending bounded training campaign and experimental proposal consumer	Model/optimizer checkpoints, architecture, task/vocabulary binding and training proof
Native CLI	Current planning/execution front doors; pending `train` orchestration	Pinned requests, source/runtime inventories and supervised execution

The ordinary CLI and `asm` feature graphs in the stopped follow-on were inspected as manifest text: they have no edge to the engine, Torch, CUDA bindings or a local dependency cycle. That is a source graph check, not a compilation result. Python training runs as a supervised external process; native orchestration does not require linking Torch into the CLI.

2.5 The learning cycle: what is proven and what remains

The most recent bounded origin proof at this source completed its run command in 239.143 seconds; independent report replay took 18.719 seconds. It demonstrated neural-origin fitting and evolution on one regression dataset, `230_machine_cpu`. On the same 34 held development rows the neural arm's refit scored an RMSE of 166.32 against 155.26 for the generic arm (`neural_win: false`). The arms had equal declared allowances, not equal fit time: context encoding and policy production ran inside the neural arm's clock, the neural arm stopped at its original deadline and the generic arm at its evaluation limit (§12.1 Arm order and clocks). This one-dataset result is therefore not a matched quality comparison. It did not execute the complete thirty-dataset demonstration, create current eligible teachers, train a new policy, or qualify a newly trained policy for serving. §1.5.2 Canonical proof record and per-arm facts gives its exact receipts and denominators.

1. Preserve the frozen data schema, original target strings, category strings, missingness and row/group assignments.
2. Derive the proposal context using only the allowed fitting rows. Record preprocessing and encoder identities, then verify actual context production and the numerical qualification required by that context lineage.
3. Use the chosen checkpoint to sample in a bounded number of rounds, stopping once 64 distinct valid executable recipes exist; the first 64 in draw order form the neural pool, and any shortfall is filled as described in step 4. Score exactly the 64 pool programs with the mean of the four learned reward heads. That score orders the pool's slots and records `rank_log_weight`; it does not select or reject programs. This stage has no access to held-out scoring labels or fitted GP scores.
4. Place a 64-slot policy pool plus 16 exploration candidates into the neural arm's generation zero; preserve any policy-pool shortfall as `neural_rejection_fallback`, never as a genuine neural sample.

Compare with an 80-candidate generic arm under the same declared budgets and executable grammar policy. Context encoding and policy production run inside the neural arm's clock, so equal declared budgets are not equal fit time.

5. Actually fit candidates on inner training folds and score their out-of-fold predictions. The measured outcomes, not model-predicted quality, determine the search's parents and winner.
6. Generate mutation and crossover descendants with explicit parent identities. Preserve duplicate/cache handling, rejected variations, unavailable methods and every fit failure.
7. Freeze a recipe and evaluate the unchanged development protocol. Its fitted fold artifacts are separate from its reusable recipe.
8. In the pending teaching path, admit a recipe only after its independently preregistered train-side OOF evidence beats a trivial baseline under the frozen numerical rule.
9. Freeze the eligible corpus, train an explicitly experimental policy, prove real updates and continuation, and export a pinned checkpoint. The pending campaign trains a new, smaller Python `JopePolicyModelV3` (width 256, two layers, four heads); it is a different architecture from the accepted native step-60000 model (width 512, six layers), not a continuation of it (§8.4.1 What the pending campaign trains). This manual names that line **experimental regression lineage v1 (256×2, new initialization)**; §2.9 Model lineages keeps it separate from the accepted 512×6 lineage-B model.
10. Consume that exact new checkpoint in a subsequent bounded experiment. A successful export alone is insufficient evidence for this final step.

Steps 1–7 have current bounded regression evidence at the scope described above. The stricter teaching/training/experimental consumption integration is stopped source work awaiting the next reviewed lifecycle and actual runtime acceptance.

Steps 8–10 support four distinct claims, and evidence for one never establishes another (owner rule):

Claim	What must establish it	Status
Eligible teacher	Admission under the preregistered train-side guard of step 8, with the full comparison identity; a hash, a stored score or a corpus row is not eligibility	Pending integration
New discovery	A program absent from the authenticated initial population of the search that found it, retained with its discovering arm and ancestry; a discovery cannot silently seed another arm	Proposed
Actually consumed information	Committed optimizer updates that measurably sampled the new rows, with that exposure recorded	Proposed
Improved policy	A matched downstream fitted comparison of the trained checkpoint with its parent under the same budgets, as in step 10	Proposed

The preregistered train-side guard exists only in the stopped overlay, and the demonstrated teacher count under it remains zero in the inspected evidence (§8.1 What can be claimed now). A source investigation of that overlay found no initial-population or discovery identity and no per-update fresh-exposure record in the pending campaign (`plan-execution-20260923/boundary-resume-siblings.md` under the ASM custody root, §4.2 Physical layout and identities).

The owner's rules, also proposed, fix three family-level domains: teacher/train families, adaptive development families, and evaluation families kept outside training and selection. The frozen thirty-dataset panel remains adaptive development (§11.2.2 Adaptive development exposure), and the exposure of the pretrained encoders to any family remains unknown. The permitted input closure covers records, contexts, transforms and all assets; an unused forbidden descriptor must be rejected before loading, not merely left unsampled (§5.4.2 Audit the entire input closure before continuation). The same investigation found that the stopped overlay's native family partitioning has only train and development domains, and that its Python campaign does not reject extra context-index entries.

Remaining architectural acceptance includes real eligible-teacher collection, actual new-parameter training/reload/continuation, subsequent fitted use of that newly trained checkpoint, classification operational qualification, complete fitted-state parity across supported deployment backends, the unchanged full30 panel, and measured warm/cold installation demonstrations. Reusable motif mining must retain full comparison identity and eligibility; new operators need ablation and compatibility tests. GPU inference portability remains future work. Numerical parity of the Candle CPU policy port with the trained tch forward pass is not established: the source states that logits and rewards from the port are unverified until reference fixtures exist, and its `cuda` feature has never been compiled (`crates/jope-policy/src/lib.rs:84-95`).

2.6 GP representation: recipes, fitted predictors and the four artifact types

The private program `(pipe X (standardize) (ridge))` is a small executable algorithm description. It takes a feature table, fits standardization from training rows, then fits a ridge-regression head. It is an ordered typed recipe, rather than an already fitted set of coefficients. A different transform or head changes the algorithm. GP search works within its admitted grammar and task constraints; the presence of a code emitter does not grant arbitrary-language or arbitrary-operator coverage.

Keep these artifact types distinct:

1. **Private recipe:** the lossless canonical program and its configuration. It can be fitted again on new training data and can carry private discovery provenance.
2. **Native fitted predictor:** preprocessing state, category dictionaries, imputation statistics, transforms, calibration and head parameters learned from one exact fit. It predicts without training again.
3. **Emitted recipe:** a program in another language with `fit` and `predict` behavior. Running it on training data fits a new instance of the algorithm. Agreement between these refits tests recipe/runtime equivalence.
4. **Fitted inference bundle:** an allowlisted implementation of one exact already fitted predictor. It contains the selected inference state and an input boundary; qualification compares its predictions against the original saved native artifact.

2.7 Encodings and the proposal context

2.7.1 Regression context production

2.7.1.1 Original data and preprocessing

The context input binds prepared-data hash, exact training-content hash, ordered fitting-row hash, cut identity and preprocessing hash. It requires at least eight training rows and a nonempty declared feature schema. Numeric versus categorical kinds come from the schema rather than guesses based on strings. Numeric nulls are imputed using training-only medians, with a zero fallback for an entirely missing column; the recorded training transform contains its fitted location/scale. For categories, every present string is a value, including empty text or strings spelling `NA` or `null`. Only an absent value is missing. Original target strings remain separately authenticated before numerical regression parsing.

This context preprocessing is for the proposal encoder. It does not authorize discarding the original schema or replacing a fitted predictor's own training-only preprocessing. Source: `crates/jope-policy/src/context_producer/input.rs:84`.

2.7.1.2 Component dimensions and assembly

The current CPU producer uses three components with fixed pretrained assets and fixed transforms:

Component	Production	Raw output	Contribution to policy context
Hyperion	Training feature columns only (no label), quantized from fitted quantile bounds; three embedding sweeps, scales 4/7/11/15, batch 1024, internal setting `target_chunks_per_sweep=1` (a chunking setting, not the label)	128 values per row	Mean, population standard deviation, 10th and 90th percentile per coordinate: 512
Foundation	Feature columns plus the training targets as in-context labels, prepared with the native deterministic recipe; target normalization computed in f64 after the native f32 cast	2048	Frozen mean-centering, learned 2048→1024 projection, then frozen normalization: 1024
Target	Training targets, already standardized in preparation, then min–max rescaled to [0,1] by the training minimum and maximum (span 1 when constant); two frozen native target windows with seed 17	512	Frozen normalization: 512

Assembly concatenates normalized Hyperion512 + projected Foundation1024 + Target512 and clips the full 2048-vector to [-32,32]. Only the Hyperion block is label-free: Foundation is conditioned on the training targets, so the first 1,536 values, which the policy configuration still calls `dataset_dim`, include

target-conditioned values. It checks all component dimensions and finite values. The source explicitly checks that the Hyperion row width is 128; confusing row width with pooled512 changes the model input meaning. Sources: `crates/jope-policy/src/context_producer.rs:188`, `:278`, `:356`, `:399`, `:422`.

The deterministic foundation preparation selects at most 768 context rows and 256 query rows before `encode_standardized`; that encoder adds no further cap. Hyperion receives every supplied training row and feature column, with no row/column subsampling in this producer. The serialized input is capped at 128 MiB and every context job remains subject to the fixed 30-second worker deadline. On the eight final reference jobs at source `68f56404`, Hyperion-stage elapsed time ranged from 6.143 to 14.287 seconds; whole-job wall time ranged from 11.731 to 20.755 seconds. These small-case measurements do not establish coverage for wider or larger cuts. A timeout leaves the context unavailable; it does not permit a new deadline or silent subsampling.

The frozen numerical qualification contract is unchanged, but the implementation includes both execution and numerical corrections. Hyperion uses an embeddings-only sweep that skips the unused confidence head. Foundation uses the separate `encode_standardized` path: train-side feature moments accumulate in f64 before the retained f32 transform, and p10/p90 use Torch 2.7 FP32 rank and stable-lerp arithmetic. Ordinary `encode` retains its legacy feature-moment and percentile arithmetic. The standardized route consumes all rows supplied by the preceding deterministic foundation preparation without additional caps. Cached-decoder fixtures separately check small random-model operations/full-prefix agreement within 1e-4, row/padding behavior and sampling-token/RNG agreement. They do not establish that the Candle CPU policy port matches the trained tch forward pass: the source states that logits and rewards from the port are unverified until reference fixtures exist (`crates/jope-policy/src/lib.rs:84-92`). Native CUDA policy parity is also not established; the `cuda` feature has never been compiled (`lib.rs:93-95`). Sources: `crates/jope-policy/src/context_producer.rs:238`; `crates/jope-encoders/src/foundation.rs:345`; `crates/jope-encoders/src/foundation/support.rs:123`; `crates/jope-encoders/src/model/tensor_ops.rs:114`; `crates/jope-policy/src/model/cached.rs:270`; `crates/jope-policy/src/decode.rs:485`.

2.7.1.3 Qualification and admission

The CPU regression encoder has a stronger boundary than “the vector has 2048 values.” Eight fixed native-reference cases span two families and cuts 0–3. Qualification checks finite FP32 component and final-context values with maximum absolute difference at most 0.0003, exact asset/source/program identities, and actual bounded executions. Consumption binds those eight jobs plus the actual dataset context job: nine independently authenticated process proofs. The reference registry/index and transform hashes are pinned in source.

The eight retained reference outputs at source `68f56404` have final-context maximum absolute differences from 5.3644180e-5 to 2.8765202e-4. The worst case is `cardata-blackmore`, dataset `55e9dfc7a86361d7`, cut 0: 95.884% of the unchanged 3e-4 tolerance. Across all eight cases, maximum component differences are 4.127622e-6 for Hyperion, 2.908707e-5 for foundation and 3.911555e-7 for target. The frozen projection and projected-scale division can amplify small component differences in the assembled vector. Coverage is limited to two numeric meta-train families: blackmore, 629–630 training rows with 4 features, and `cardata-ces11`, 800 rows with 8 features, each at cuts 0–3. Changes to arithmetic, libraries, build/ISA flags or summation order require requalification; this gate is not evidence for wide or category-heavy contexts. Never relax its tolerance to make a build pass.

`VerifiedCpuContext` has private fields and can only be constructed by the verification path outside explicit test support. A syntactically valid JSON vector cannot create admission authority. Sources: `crates/jope-policy/src/context_producer/admission.rs:4`, `:115`, `:234`, `:297`; `crates/jope-pipeline/src/cpu_context_bridge.rs`.

2.7.2 Experimental classification context

Current source contains the pure class-partition context representation. The pending training overlay adds the bounded feature producer, process admission and operational proposal wiring. Classification does not parse class names as numbers or reuse the regression target encoder by ordinal coding.

The feature blocks remain Hyperion512 and projected Foundation1024. The pending producer explicitly conditions the foundation path on neutral zero targets and labels this an experimental feature-only probe. Each class gets a centroid over the 128-dimensional training row encodings. The target block contains, per coordinate, the mean, standard deviation, minimum and maximum across class centroids, totaling 512. The centroids are unweighted, so class frequencies never enter the block. Classes are visited in first-row order, so a bijective renaming preserves even floating-point summation order. Original labels still appear in authenticated input identity, even though their spelling cannot influence numerical pooling.

The resulting lineage is `asm.experimental-class-partition-pool/v1`; `policy_compatible` is false. The false marker means it cannot enter the accepted regression policy loader. A declared binary input must have exactly two classes; multiclass requires at least three, and the fitting data must contain the declared classes. Missing classes are an explicit refusal. Sources: `crates/jope-pipeline/src/search_context.rs:278`; pending `crates/jope-policy/src/context_producer/classification.rs:1`.

At source `68f56404`, classification has no neural arm: planning records the accepted regression-only policy as unavailable for binary/multiclass data. The available `asm.generic` classification sampler uses `logit_ridge` for binary or `multiclass_ridge` for multiclass, with l2 in [1e-4, 10], preceded by no transform, `standardize`, `winsor_clip`, or both. Advanced-v3 mutation/crossover preserve that head family and allow at most 4 transforms. A classification ASM result at this source would therefore be a ridge-head transform search; the pending experimental neural consumer is a separate capability.

2.8 The accepted regression model: lineage B, step 60,000

2.8.1 Checkpoint binding and configuration

The accepted lineage is the recovered lineage-B step-60000 checkpoint. The binding verifies the state, original model, exported tensors, configuration, parameter digest and vocabulary. The rejected step-69000 identity is explicitly refused. A self-consistent receipt around unrelated weights cannot impersonate the accepted model. The model loader rejects missing, extra, incorrectly shaped or non-FP32 tensors.

Accepted-model property	Value or behavior
Dataset context (historical name of `dataset_dim`)	1536 FP32 values: Hyperion 512 from features only plus Foundation 1024 conditioned on the training targets, so not label-free
Target context	512 FP32 values; all 512 retained by the configuration (`target_keep_dim` 512), although most of them may be constant zero (`python/src/jope/policy/train_v3.py:919-920`)
Combined context	2048 FP32 values; 8192 bytes in little-endian vector form
Decoder width	512
Decoder depth	6 layers

Accepted-model property	Value or behavior
Attention heads	4
Feed-forward width	2048
Token vocabulary	401 output tokens
Embedding rows	657, including the configuration's 256 extra rows
Maximum configured sequence	128 tokens; positional table includes the context position
Training dropout field	0.1; disabled for inference
Reward ensemble	4 heads
Tensor inventory	136 tensors for this configuration, including unused inference heads and scalar loss weights
Current Rust arithmetic	Candle CPU FP32
Native CUDA BF16 policy parity	Not established by the current CPU inference implementation

Sources: `crates/jope-policy/src/config.rs:21`, `config.rs:39`, `crates/jope-policy/src/model.rs:1`, `model.rs:41`, `crates/jope-policy/src/lib.rs:133`.

2.8.2 Decoder and reward architecture

The decoder and reward paths each encode context with a two-layer MLP: 2048→512, GELU, LayerNorm, 512→512, GELU, LayerNorm. The decoder prepends a projected context token to the token embeddings, adds positional embeddings, applies causal self-attention with padding-key masking through pre-normalized layers, normalizes the result and projects through the tied token embedding table narrowed to the output vocabulary.

The reward path averages the program-token embeddings over non-padding positions, with no positional embedding, combines that mean with a separately encoded context, and evaluates four reward MLPs. The combined representation contains the program embedding, its elementwise interaction with a context gate, and an absolute-difference block. A separate runtime head emits a positive runtime estimate through softplus. Learned reward is a proposal-ranking signal; it is not a measured dataset loss.

Reward pooling is order-blind. The mean has no positional input. For one context, two programs whose tokens form the same multiset, for example the same operators or parameter values in a different order, receive the same pooled representation and therefore the same four reward scores, up to floating-point summation order. The decoder still conditions on token order; only the learned ranking signal is affected. At this source that signal orders the neural pool's slots and records `rank_log_weight`; it does not select programs (step 3 of §2.5 The learning cycle: what is proven and what remains). The pooling is the same in the Candle port (`crates/jope-policy/src/model.rs:14`, `:555-567`), in the native tch model it mirrors (`crates/jope-engine/src/neural.rs:496-505`), and in the Python `JopePolicyModelV3` that the pending campaign trains (`python/src/jope/policy/model_v3.py:391-404`; the stopped overlay's copy keeps it). The Python module docstring (`model_v3.py:7-9`) still describes a bidirectional reward encoder with attention pooling; the code discards that encoder, recording that it collapsed (`:258`), and mean-pools instead.

Native source also contains an isolated experimental `OrderAwareRewardEncoder` (`parallel.reward_program_attention_2x256x4.v1`), which adds positional embeddings and two attention layers. Its source comment states that production `Policy` does not construct it and that a separately

qualified reranker must opt in with a new artifact; at this source only its unit tests use it (`crates/jope-engine/src/neural.rs:180-263`; `crates/jope-engine/tests/reward_sequence.rs:7-24`). Status: implemented but unqualified.

Owner rule (proposed): measure before changing the architecture. First measure how often distinct candidate programs collide on one token multiset, and the ranking regret those collisions cause. Defer architecture changes, including adopting an order-aware reward input, so that the first causal learning experiment keeps changes to architecture, objective, corpus and sampler distinguishable.

2.8.3 Accepted identities and the product restriction

The source records these accepted identities:

Identity	SHA256
State	`8ac80c269cc47c253b6f53fa3bc1eb75bb2fcda0bdace3ff8554be0175c400d6`
Original model	`053df4f53b84d2ebe7c14fbf907a149760af3d11de4a57b5ff89be8c2d3e8129`
Canonical parameter values	`9c7057e7f6a9e348f030da73f03522977b8c08915d3a221d7d5ab2e4213fcd92`
Configuration	`2508fc8b8268ea000141c9b119ba3da4e777c506c366168912a88d03482939bf`
Original checkpoint vocabulary	`8d55fee4366acd2c175cf55bfd769cebc904770f5227a6fd50a613723dc85787`
Executable product restriction	`570b2e5e8d859a73b888f5d0bee27f59da0e2376536babce24c241604f818e81`

The product restriction preserves the 401-token table while limiting executable heads. It removes ten head operators (`extra_trees`, `gam_boost`, `logit_ridge`, the four multiclass heads, `portfolio`, `ridge_gcv` and `tab_boost`), and the vocabulary has no operator token for `pls` or `hurdle2`. The regression proof bound both arms to this restricted vocabulary, and mutation replaces a head only with `histgbm`, `ridge`, `ridge_gcv`, `elasticnet`, `histgbm2`, `gpr` or `ksmooth`; crossover keeps a parent's head. Six of the seventeen registered regression recipes (`tab_boost`, `extra_trees`, `gam_boost`, `pls`, `hurdle2` and `portfolio`) are therefore unreachable by any ASM operator at this source, and `ridge_gcv` is reachable only through mutation. Sources: `crates/jope-engine/assets/grammar/featmath-jain-v1.json` (`restriction.removed_ops`); `crates/feat-math/src/evolve.rs:29-39`, `:1480-1492`; `crates/jope-bench/src/panel/optimize.rs:254`; `crates/jope-bench/src/panel/methods.rs:60-89`. A token-compatible restriction is different from inventing a new vocabulary or silently snapping parameters to nearby bins.

2.9 Model lineages

A lineage is a chain of checkpoints sharing one architecture and parameter ancestry. A higher step number, a newer file or a shared vocabulary does not place a checkpoint in the accepted line. The table covers three lineage-B checkpoints and the planned experimental line, as of 2026-09-23 about 23:55 UTC.

Checkpoint or line	Architecture	Role and status	Identity and sources
Lineage B, step 60,000	Native tch `model.ot`: width 512, six layers, four heads, feed-forward 2048	The accepted regression model and the only identity the current consumer admits. Available.	State `8ac80c26…`, pinned with its model, parameter, configuration and vocabulary hashes (`crates/jope-policy/src/lib.rs:133-150`, `:161`; `crates/jope-policy/src/config.rs:289-325`); §2.8.1 Checkpoint binding and configuration
Lineage B, step 60,100	Same as step 60,000	Historical hardware continuation: the September 14 qualification run carried AdamW for 100 updates from step 60,000 on the unchanged corpus, with zero fresh programs, at world 1 × microbatch 256 × accumulation 8. It is neither a serving model nor a production continuation, and the accepted binding refuses any step other than 60,000.	Final state `745be4b5…`; §B.3.4.2 The RTX step 60,100 qualification run
Lineage B, step 69,000	Same as step 60,000	Endpoint of the last retained A100 block, steps 66,001–69,000. Unaccepted: an export receipt naming its state or model hash is refused before any other check.	State `58c4a1f6…`, model `f3694654…` (`crates/jope-policy/src/lib.rs:152-159`, `:249-253`; `crates/jope-policy/src/config.rs:177-182`); §B.3.4.1 A100 checkpoints and the accepted step 60,000
Experimental regression lineage v1 (256×2, new initialization)	Python `JopePolicyModelV3`: width 256, two layers, four heads, feed-forward 1024	The line the stopped training64 campaign would train, starting from a new initialization. No real campaign has run with it. Pending integration.	Stopped patch `595824ef…`; §8.4.1 What the pending campaign trains

The two architectures are not interchangeable. The Python path cannot load step 60,000, and the accepted loader rejects the smaller model's tensor shapes, so lineage v1 is a new line rather than a continuation of lineage B. Compare the two only through matched downstream fits under one protocol, never through loss values (§8.4.1 What the pending campaign trains). Training or exporting a lineage-v1 checkpoint does not move the accepted serving pointer (§8.4.3 Plan, run, resume, verify and export).

Historical lineage A and the legacy Python gen31 and round-1 runs are recorded in §B.1 Dated lineage and evidence coverage; the current accepted consumer admits none of their checkpoints.

Owner naming rules for lineage v1 (proposed):

- The new initialization is `M_init`. It is named `M0` only after it has actually trained on its first frozen corpus, `C0`, and that run has passed the pending campaign's verification, which includes positive optimizer updates, changed parameter digests, exact reload, an actual resumed optimizer state, increasing step counters and at least two committed passes (§8.4.3 Plan, run, resume, verify and export).

- Passes through one corpus are continuation, not generations: the pending plan's two passes through `C0` (`--passes 2`) do not make two generations. A new generation needs the LOOP-01 chain: a new eligible target, actual exposure, changed parameters, compatible reload and improved fitted search.
- The pending corpus request can name an optional Python initialization checkpoint (§8.4.2 Build the real corpus from admitted evidence); the campaign then uses a weight warm start, and otherwise a cold initialization. This manual retains cold initialization as the plan for lineage v1. Warm-start support is an unselected interface option, not a qualified lineage start; the owner has not ruled whether lineage v1 admits a warm start, and no prohibition is inferred. Before any admitted run, record which form `M_init` actually took.
- Step 60,100 remains historical hardware continuation and step 69,000 remains unaccepted; neither seeds lineage v1.

CHAPTER 3

How ASM works: data, encoding, model, GP, code, training and learning

IN THIS CHAPTER

3.1 Explain it like I'm 5 39
3.1.1 The recipe-writing robot 39
3.1.2 Checking that a win is real 39
3.1.3 What the robot cannot do yet 39
3.2 The stack at a glance 40
3.3 Deep technical walkthrough 41
3.3.1 Stage 1: rows, folds and what each stage may see 41
3.3.2 Stage 2: the 2,048-float context 41
3.3.3 Stage 3: the policy network 43
3.3.4 Stage 4: grammar, sampling and ranking 44
3.3.5 Stage 5: the bounded GP search 45
3.3.6 Stage 6: refit, the held fold and code emission 46
3.3.7 Stage 7: teaching and training 46
3.3.8 Stage 8: "reinforcement learning" — what the code actually does 47
3.4 Training runs and global metrics 48
3.4.1 Which numbers can be compared 48
3.4.2 A fixed-panel learning curve for the A100 lineage 48
3.4.3 Every step of the A100 run 50
3.4.4 Throughput and timing 51
3.4.5 Training versus validation in the legacy gen31 run 51
3.5 Measurement status and figure provenance 52

This chapter describes the whole ASM stack in one place. The first section explains it without jargon. The rest describes the code and the real measurements. Every number comes from source at the stated commit or from a retained file with a recorded SHA-256. When something has not been measured, this chapter says so and draws no curve for it.

Scope and snapshot.

- Source references are to `main` at `5de90b1cc0115e8938a8996138873cc8079520bd` unless marked otherwise.
- The one-dataset measurements come from two verified origin-proof journals, `68f56404` and `c5d11ffc`, retained in ASM custody. The canonical report is that of journal `68f56404`, written by the build of source `68f56404c3e199e06413bfd645d3fe0803d7b96f`. Several crates changed between that build and `main` (PR88).
- Numbers recomputed from those journals therefore describe that build. They are not a new run of `main`.
- No new training experiment, GPU window or sealed row was used to write this chapter. Source at the PR90 merge changes none of the core-code files cited here; source support and operational qualification remain separate.

3.1 Explain it like I'm 5

3.1.1 The recipe-writing robot

Imagine a robot chef that invents recipes. Here a *recipe* is a small computer program that turns a table of numbers into predictions, for example "clean the numbers, then draw the best straight line".

1. **You show it your ingredients.** The robot never gets the whole table. It gets a short *summary card*: 2,048 numbers that describe the columns, how they vary, and how the thing you want to predict behaves. The card is made only from rows the robot is allowed to learn from.
2. **It writes recipe ideas.** A neural network reads the card and writes recipes one word at a time from a fixed list of 401 words. A grammar checker stops it from writing nonsense. It keeps the first 64 different recipes that make sense and adds 16 random "wild" ones, 80 in all.
3. **A cooking contest picks the best.** Every different recipe is sent off to be really cooked and tasted. Failures and timeouts stay in the record but cannot win. Each recipe is trained on four practice folds and tested on the remaining fold, five times over. The best recipes are mixed and changed to make new ones (*mutation* and *crossover*), and the contest runs again. The network's opinion never decides the winner. Only the real tasting does.
4. **Writing the winner down is not finished yet.** Code writers can already turn some simpler recipes into Python, Rust, C++17, C++20 and JavaScript programs that re-train themselves. They cannot yet handle an ASM winner, and the separate fitted-state exporter supports only an allowlisted set of fitted models and requires numerical qualification.
5. **Winners can become lessons.** Later, a recipe that wins *fairly* becomes a teaching example for the network, so its next ideas can be better. That last step is built but has not yet been run on real data.

3.1.2 Checking that a win is real

A child who only does well on the one test they peeked at has not learned the subject. A child who does well in practice *and* on a new test probably has.

ASM's lift measurement uses the same idea (§9.1 Observed transferable lift: reporting contract, not yet measured). It is built but has not yet been run on any real data. Under it, a new way of searching earns credit only when it beats the old way in two places:

- in the practice rounds, meaning cross-validation on the training rows;
- on a separate round of rows it never used, the held fold.

Improvement on only one side earns no positive credit under this descriptive rule. The rule does not identify why the sides disagree. When both win, it earns the smaller of the two wins. §9.1 Observed transferable lift: reporting contract, not yet measured makes this exact.

3.1.3 What the robot cannot do yet

- **It has not yet taught itself anything new.** The step "winners become lessons, lessons make a better network" is built, but no admitted teacher exists and no new network has been trained with it.
- **These training paths do not use policy-gradient reinforcement learning.** The network learns by copying good recipes (imitation) and by learning to rank them. The contest does the exploring. Rewards can weight imitation targets, but there is no PPO/GRPO policy-gradient update in these paths.

- **The bounded origin proof covers one dataset.** It does not demonstrate that the network helps; absence of demonstrated improvement is not evidence of equivalence.

3.2 The stack at a glance

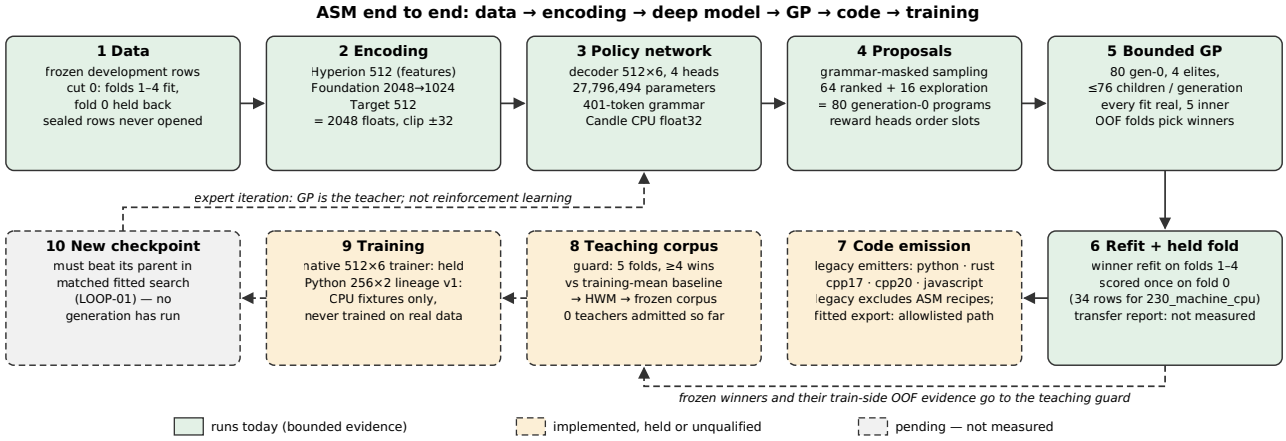


Figure 3.1 ASM end to end. Green boxes run today with bounded evidence, amber boxes are implemented but held or unqualified, and grey dashed boxes are pending. The generator checks selected architecture constants against source; status labels summarize the dated evidence in this chapter.

Stage	What happens	Main source	Status
1 Data	Frozen development rows. Cut 0 fits on outer folds 1–4 and holds fold 0 back. Sealed rows are never opened.	`crates/jope-pipeline/src/prepared_data.rs`, `search_plan.rs`	available
2 Encoding	Hyperion 512 + projected Foundation 1024 + Target 512 = 2,048 floats, clipped to ±32	`crates/jope-policy/src/context_producer.rs`	available; CPU outputs are labelled unqualified outside the admitted proof path
3 Policy network	Decoder-only transformer, 512 wide and 6 layers deep, 27,796,494 parameters, accepted step 60,000	`crates/jope-policy/src/model.rs`, `config.rs`	available (Candle CPU float32)
4 Proposals	Grammar-masked sampling; 64 ranked + 16 exploration programs	`crates/jope-policy/src/decode.rs`, `ranking.rs`, `seeds.rs`	available
5 Bounded GP	80 generation-0 candidates, 4 elites, up to 76 children per generation (capped by the evaluation limit; 16 in the origin proof), every distinct candidate submitted for a real fit	`crates/jope-pipeline/src/neural_search.rs`	available
6 Refit and held fold	Each arm's winner is refitted on the fitting rows and scored once on fold 0	`crates/jope-bench/src/panel/asm_campaign.rs`, `campaign.rs`	available

Stage	What happens	Main source	Status
7 Code emission	Legacy recipe emitters: Python and Rust, plus C++17, C++20 and JavaScript where coverage allows	`crates/jope-pipeline/src/emit.rs`	legacy recipe-refit emitters only; they cannot emit an ASM winner; separate allowlisted fitted-state export is implemented; each emitted model still requires qualification
8 Teaching corpus	Guard: 5 folds, at least 4 wins over the training-mean baseline, then HWM and a frozen corpus	`crates/jope-learning/src/eligibility.rs`	implemented; zero teachers admitted
9 Training	Native 512×6 trainer, and the Python 256×2 experimental lineage v1	`crates/jope-engine/src/training.rs`, `python/src/jope/policy/training_campaign.py`	native held; Python verified only on CPU fixtures
10 New checkpoint	Must beat its parent in a matched fitted search (LOOP-01)	§9.5 LOOP-01: the central regression milestone	not measured

3.3 Deep technical walkthrough

3.3.1 Stage 1: rows, folds and what each stage may see

A dataset in the frozen panel is split into five outer folds by group. The development partition file holds only development rows. Each row carries its outer fold, a group id, a row id and the original target string.

For cut 0:

- the **fitting rows** are outer folds 1–4;
- the **held rows** are outer fold 0.

For `230_machine_cpu` that is 135 fitting rows and 34 held rows. Their row-id hashes are bound by the frozen evaluation request (`fitting_rows_sha256`, `scoring_rows_sha256`). `SelectionPlan::validate` rejects any overlap between fitting and held rows or between their groups (`crates/jope-pipeline/src/search_plan.rs:68-149`).

Inside the fitting rows, selection uses five **inner folds**. `grouped_inner_folds` counts rows per group and sorts groups largest first, then by group id. It gives each group to the fold with the smallest (size, index) (`search_plan.rs:13-36`). Groups never cross inner folds. Only exact duplicate feature tuples and declared group columns form groups: eight of the ten regression panel datasets declare no group columns.

3.3.2 Stage 2: the 2,048-float context

The CPU producer builds the context from the fitting rows only (`crates/jope-policy/src/context_producer/input.rs:84-217`):

- numeric columns get training-median imputation and a population z-score;
- categories are indexed;
- the target is z-scored.

It then assembles three blocks. `context_producer.rs:189-271` builds the components, `:423-445` pools Hyperion, and `:357-421` standardizes, projects and assembles.

- **Hyperion 512.**
  - Features only, with no labels.
  - Each column is quantile-bounded and quantized.
  - A V7 sweep produces 128 values per row.
  - The rows are pooled per coordinate as mean, standard deviation, 10th and 90th percentile: 4 × 128.
  - Then `h = (hyp − μ_h)/σ_h`.
- **Foundation 1024.**
  - Features plus the training targets as in-context labels, so this block is **not** label-free.
  - It is 2,048 raw values.
  - It is projected with a frozen 1024×2048 matrix: `z = ((f − μ_f)·Pᵀ − μ_p)/σ_p`.
- **Target 512.**
  - Training targets min–max scaled to [0, 1].
  - Two frozen target windows with seed 17.
  - Then `t = (tgt − μ_t)/σ_t`.

The vector is `concat[h, z, t]`, clipped to ±32 (`context_producer.rs:416`). The recipe identity ends in `frozen-512+1024+512-clip32/v1`; the full string is at `context_producer.rs:127`. The policy configuration still names the first 1,536 values `dataset_dim` and the last 512 `target_dim` (`crates/jope-policy/src/config.rs:39-55`).

§2.7.1.2 Component dimensions and assembly has the qualification tolerances and the pinned asset hashes.

3.3.3 Stage 3: the policy network

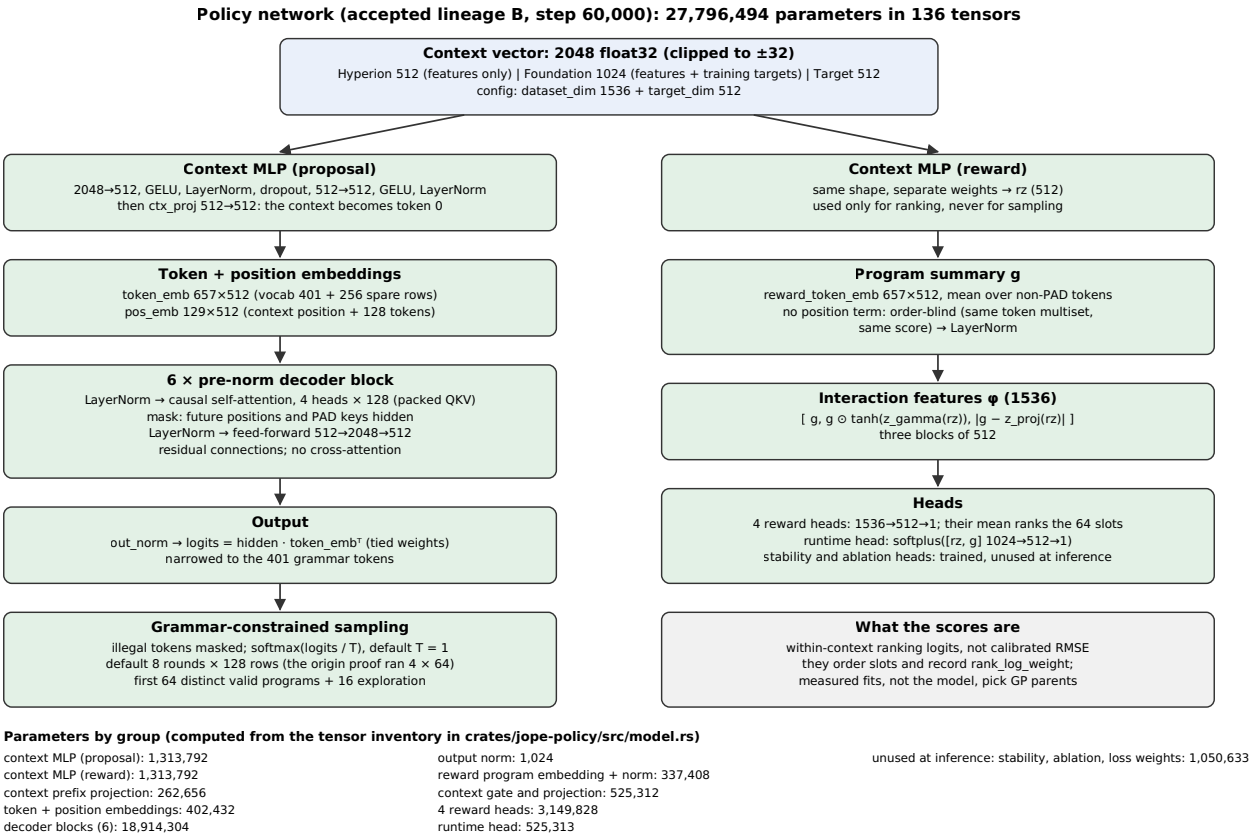


Figure 3.2 The accepted policy network. Shapes and parameter counts are computed from a mirror of the tensor inventory in crates/jope-policy/src/model.rs; selected source constants are checked during generation.

The accepted model is lineage B, step 60,000. It is one set of weights with two heads that answer different questions:

- a decoder that **writes** programs;
- a reward branch that **scores** programs for ordering.

Decoder (proposal).

1. The context passes through a two-layer MLP (2048→512→512, GELU and LayerNorm after each layer) and `ctx_proj`.
2. It becomes the token at position 0.
3. Program tokens are embedded with `token_emb`, which has 657 rows: 401 vocabulary rows plus 256 spare. A learned `pos_emb` with 129 rows is added.
4. Six pre-norm decoder blocks follow. Each has causal self-attention with 4 heads of width 128 (packed QKV projection, with future positions and PAD keys masked) and a 512→2048→512 feed-forward network. There is no cross-attention: the context influences every token only through the prefix position.
5. After `out_norm`, the logits are the hidden state times the *tied* token-embedding matrix, narrowed to the 401 real tokens.

Sources: `crates/jope-engine/src/neural.rs:406-461` for the reference forward pass, and `crates/jope-policy/src/model.rs:366-608` for the Candle port.

Reward branch (ranking).

1. A second context MLP of the same shape produces `rz`.
2. The program is summarised as `g`: the mean of a separate `reward_token_emb` over non-PAD tokens, followed by LayerNorm. It has **no position term**, so two programs with the same multiset of tokens get the same `g`.
3. The interaction features are `φ = [g, g ⊙ tanh(z_gamma(rz)), |g − z_proj(rz)|]`, 1,536 values.
4. Four reward heads (1536→512→1) each emit a score. Their mean orders the 64 neural slots and records `rank_log_weight`.
5. A runtime head, `softplus(MLP([rz, g]))`, estimates cost.
6. The stability head, the program-only ablation head and seven loss-weight scalars are stored but unused at inference.

Source: `neural.rs:489-531`.

Parameters. These are computed from a checked mirror of `expected_inventory` (`model.rs:40-96`): 136 tensors and 27,796,494 parameters.

Group	Parameters
Context MLP (proposal)	1,313,792
Context MLP (reward)	1,313,792
Context prefix projection	262,656
Token + position embeddings	402,432
Six decoder blocks	18,914,304
Output norm	1,024
Reward program embedding + norm	337,408
Context gate and projection	525,312
Four reward heads	3,149,828
Runtime head	525,313
Unused at inference (stability, ablation, loss weights)	1,050,633
Total	**27,796,494**

The Rust port runs in float32 on the CPU. Its numerical parity with the trained tch forward pass is not established, and its `cuda` feature has never been compiled (`crates/jope-policy/src/lib.rs:82-103`).

3.3.4 Stage 4: grammar, sampling and ranking

The vocabulary (`crates/jope-engine/assets/grammar/featmath.json`) has 401 tokens:

Category	Tokens
`PAD, BOS, EOS`	3
`ENGINE`	1
`TASK`	4

Category	Tokens
OP (31 transforms, 22 heads)	53
VAL (binned parameter values)	291
APPEND	3
EXPR_VAR, EXPR_CONST, EXPR_FUNC, EXPR_BINOP, EXPR_END	14, 16, 11, 4, 1

The product restriction `featmath-jain-v1` keeps the same table but allows fewer operators (§2.8.3 Accepted identities and the product restriction).

Sampling (`crates/jope-policy/src/decode.rs`):

1. Every row starts from `[BOS, ENGINE=featmath, TASK=regression]`.
2. At each step, a grammar automaton gives the legal next tokens. Illegal logits are masked, and the token is drawn from `softmax(logits/T)` with a ChaCha20 generator seeded per request.
3. The defaults are 8 rounds × 128 rows at T = 1 (`decode.rs:39-46`). The origin proof requested 4 rounds × 64 rows at T = 1: see `proposal.request.sampling_rounds` and `sampling_batch` in the neural arm's retained search receipt (journal `origin-proof-68f56404…/journal/asm-search/ec2b7ba9…/search-receipt.json`, SHA-256 `5eae7243b5442635eacb326e0a1ce0b7a4e43ba413ed68e281e1ff33dcd6e069`).
4. Sampling stops after the round in which `POOL_SIZE` = 64 distinct valid programs exist, and the first 64 in draw order are kept (`decode.rs:221`, `:457-459`). A row is valid if it decodes and `feat_math` parses it. Rows that do not reach `EOS` within 128 tokens are dropped.
5. A shortfall is filled with grammar-uniform programs, recorded as `neural_rejection_fallback` and never as neural. Then `EXPLORATION_SIZE` = 16 random exploration programs are added (`decode.rs:26-28`).

Ranking (`crates/jope-policy/src/ranking.rs:338-445`) scores the 64 pool programs with the mean of the four reward heads. It sorts them by score, breaking ties by program SHA-256. The scores are within-context logits, not calibrated losses. They order slots; they do not choose GP parents.

3.3.5 Stage 5: the bounded GP search

`neural_search.rs` runs two arms under the same declared budgets:

- the **neural** arm: 64 policy programs plus 16 exploration programs;
- the **generic** arm: 80 grammar samples.

Constants: `INITIAL_SIZE` = 80, `NEURAL_SLOTS` = 64, `ELITES` = 4 (`neural_search.rs:28-30`).

1. Every canonically distinct candidate is submitted for a real fit on the five inner folds under the execution pool's resource limits (at most 20 workers, 1 thread each, cgroup memory limits; `experiment.rs:58-97`). A canonical duplicate is recorded as failed without a second fit (`neural_search.rs:1192-1233`). A failed or deadline-cut fit gets no loss.
2. Its **selection loss** is the unweighted mean of its five inner-fold losses (`crates/jope-bench/src/panel/optimize_fit.rs:252`).
3. The four best by that loss are the parents.
4. Children alternate between mutation and crossover by attempt parity, and are deduplicated by canonical executable hash. A generation has up to 76 children: `min(76, max_evaluations − observed)` (`neural_search.rs:1718-1719`). The origin proof's cap of 96 allowed one wave of 16.

5. The arm normally stops at its evaluation limit or its original deadline, whichever comes first. It can also stop with `no_measured_parent` or `search_space_exhausted`, or in a resumable cancellation or admission-pending state (`neural_search.rs:1633`, `:1694`, `:1715`, `:1769`).
6. The winner is the minimum selection loss, with ties broken by program SHA-256 (`neural_search.rs:653-667`). It is frozen before any held-fold fit.

In the retained origin proof on `230_machine_cpu` (journal `68f56404`):

	Neural arm	Generic arm
Completed candidates	77 (62 in generation 0)	89 (76 in generation 0)
Stop reason	original deadline	evaluation limit
Winner	generation 1, crossover	generation 1, mutation

The neural winner descends from exploration candidates, not from a `neural_policy` proposal (§1.5.2 Canonical proof record and per-arm facts).

3.3.6 Stage 6: refit, the held fold and code emission

Each arm's winner is refitted on all 135 fitting rows and scored once on the 34 held rows (`asm_campaign.rs:197-250`). The refit sees the held rows' features only, and scoring happens in the parent process (`campaign.rs:279-289, :617`).

The legacy emitters (`crates/jope-pipeline/src/emit.rs`) write Python and Rust, and C++17, C++20 and JavaScript where their coverage gates admit the recipe. The emitted program contains `fit` and `predict` and refits from a training CSV at run time.

They cannot emit an executable ASM recipe. Every ASM recipe carries an inserted `(input …)` stage (`crates/jope-pipeline/src/synthesized_recipe.rs:34-47`), and the legacy planner rejects that stage (`crates/feat-math/src/spec.rs:383-389`, `:592`). A separate allowlisted fitted-state path is implemented in `fitted_export.rs` and `fitted_export_run.rs`, including synthesized worker artifacts through `FittedInferenceView::synthesized`. It packages supported saved state and qualifies emitted predictions; it does not make the legacy planner accept synthesized ASM recipes or establish general ASM-winner coverage. The historical deployment requirements are retained in §10.1 Deployment support: definition, priority and status.

3.3.7 Stage 7: teaching and training

Teaching.

- A candidate becomes a teacher only if the fixed guard admits it: five folds, at least four fold wins over the training-mean (or class-frequency) baseline, and a mean-loss gain above `1e-12 + 1e-6·|baseline|` (`crates/jope-learning/src/eligibility.rs`, `GuardPolicy::current`).
- It must then pass strict HWM admission into a frozen corpus (§7.2.2 The admission guard, label exposure and HWM identity).
- No teacher has been admitted under this guard.

The native objective, which produced step 60,000 (`crates/jope-engine/src/training.rs:1257-1339`):

```
L = seq_nll + 0.25·BT + 0.1·ListNet + 0.1·runtime_Huber + 0.05·ablation_BT
```

- `seq_nll` is grammar-masked token cross-entropy, with 0.05 smoothing inside the legal set and row weight `0.05 + percentile⁴`.

- `BT` is a weighted pairwise Bradley–Terry loss over the eight programs of each comparison.
- `ListNet` targets `softmax(percentile/0.25)`.
- The optimizer is AdamW at a constant learning rate of 1e-4, weight decay 0.01, gradients clipped to norm 1.0 after the all-reduce, with a global batch of 2,048.
- The trainer is held: `require_original_gpu_grant` refuses every call (`crates/jope-engine/src/task_contract.rs:88-92`).

The Python v3 objective (`python/src/jope/policy/losses.py`):

- Kendall–Gal uncertainty weights on `seq_nll` and `runtime_huber`, and fixed weights on the ranking, stability, ablation and routing terms.
- In the default trainer, the imitation loss becomes reward-weighted (RWR) after phase A. The experimental 256×2 campaign sets `phase_a_fraction=0.0`, so it applies RWR from the first step.
- The experimental lineage v1 (256×2) campaign in `training_campaign.py` has run only on CPU fixtures.

Continuation modes: cold initialization, weight warm start, explicit new-corpus transition, and exact interrupted resume (§8.2 The four continuation names and their commands).

3.3.8 Stage 8: "reinforcement learning" — what the code actually does

Mechanism	Present?	Where
Policy-gradient reinforcement learning (REINFORCE, PPO, GRPO)	**No.** "GRPO" appears only in a docstring.	`model_v3.py:8`
Expert iteration: policy proposes, GP searches as the expert, guard admits, retrain	Structure implemented; no real generation completed	`neural_search.rs`, `teaching.rs`, `eligibility.rs`, `training_campaign.py`
Reward-weighted regression on imitation targets	Implemented (Python)	`losses.py:387-442`
Rank-weighted behaviour cloning (`0.05 + percentile⁴`)	Implemented (native)	`neural.rs:534-577`; `training.rs:1288-1294`
Preference optimisation (DPO)	Functions written, never called	`neural.rs:578-698`
Tree search (MCTS)	Rejected for now	`expert_iteration_setup.py`

The repository's own assessment records `flywheel_effect=False` (`python/src/jope/tools/expert_iteration_setup.py:214`) and a neural-seed success rate of 0 of 140 (`:72-84`). The file itself marks that rate as not verified against its source report (`:83`). **Expert iteration, meaning imitation of search results, is the accurate name for this design.** Calling it reinforcement learning would overstate it.

3.4 Training runs and global metrics

3.4.1 Which numbers can be compared

A training loss describes the batch it was computed on. Two losses are comparable only when they measure the same population, objective and model family. This section follows three rules:

- **Comparable across checkpoints:** the fixed monitor. It has one panel (`fbff7a04`), one corpus (`11e60a43`) and 2,048 examples for the step-3,000 parent and every A100 block-end checkpoint from 6,000 to 69,000 (§3.4.2 A fixed-panel learning curve for the A100 lineage).
- **Comparable only within one block or one run:** per-step training losses, and adaptive validation on a panel that changes between blocks.
- **Not comparable at all:**
  - composite losses across objectives (native v3 and Python v3.2/v3.3 with learned weights; RTX additionally exposes `contextfree_bt` telemetry, which alone does not establish a changed objective);
  - any legacy metric across generations, whose training rows grow;
  - internal Elo against anything.

Metrics that were never logged are listed as such: per-step learning rate (it was constant), tokens per second, a reward-MSE, and any reinforcement-learning advantage.

3.4.2 A fixed-panel learning curve for the A100 lineage

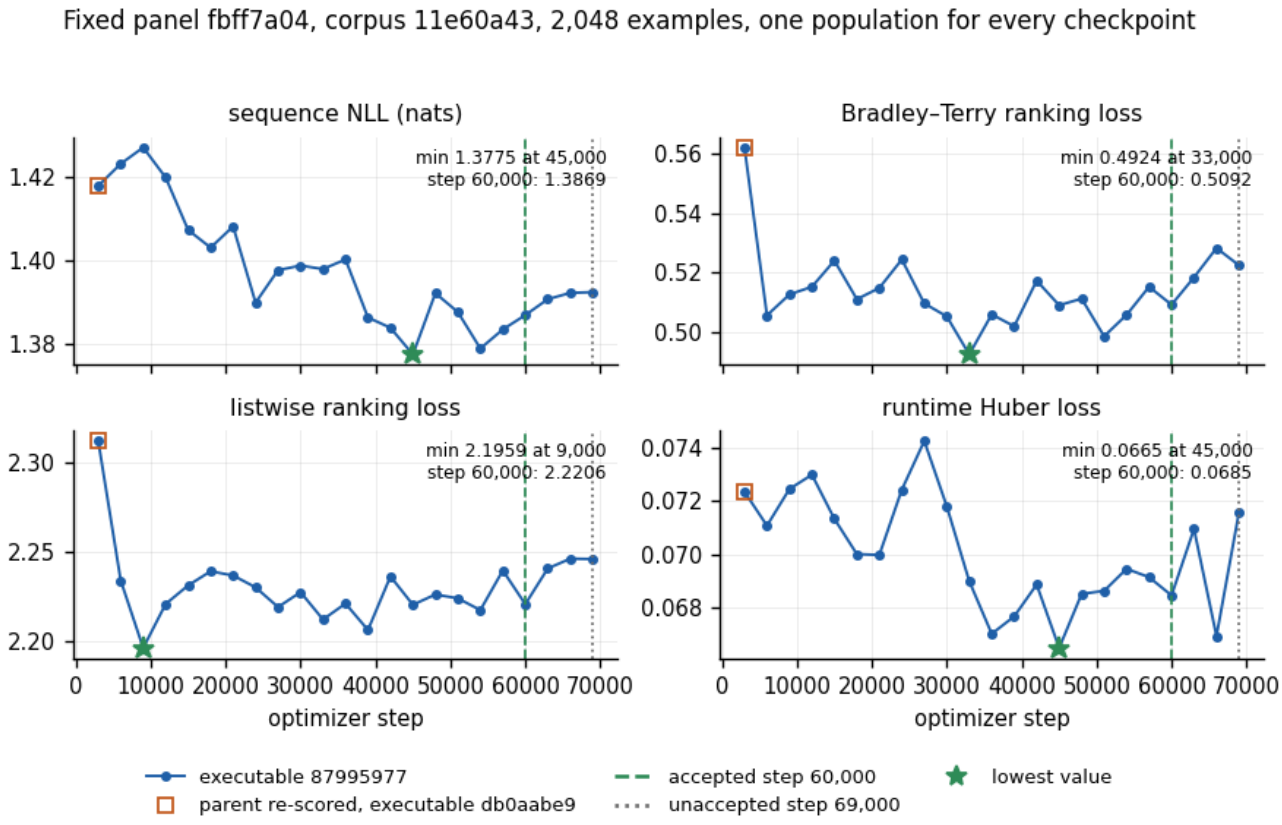


Figure 3.3 Fixed-panel validation of every A100 block-end checkpoint on one unchanged population (panel fbff7a04, corpus 11e60a43, 2,048 examples). The accepted step 60,000 is dashed, and the unaccepted step 69,000 is dotted. The step-3,000 parent was scored by two executables with identical values.

Source. The 24 records live under `/home/ubuntu/jain-split/jope-dime/parallel/recovery/eight-a100-20260912/learning-receipts/learning/monitoring/` (schema `parallel.frozen_monitoring.v1`). They cover:

- the parent at step 3,000, scored under executables `87995977…` and `db0aabe9…` with identical values;
- every block end from 6,000 to 69,000.

Each record states `training_selection_used: false`, `final_tests_opened: false`, `independent_evaluation: false` and `evaluation_bias: "frozen adaptive teaching validation panel"`. The extraction is `data/deep-stack/a100-fixed-monitor.csv`, with a SHA-256 manifest `data/deep-stack/extraction-manifest.json`.

Checkpoint	Sequence NLL	Bradley–Terry	Listwise	Runtime Huber
Parent, step 3,000	1.41778	0.56216	2.31246	0.07236
Lowest value on the panel	1.37754 (step 45,000)	0.49244 (33,000)	2.19595 (9,000)	0.06648 (45,000)
Accepted step 60,000	1.38693	0.50920	2.22060	0.06846
Unaccepted step 69,000	1.39241	0.52248	2.24601	0.07158

What this shows:

- **On one unchanged adaptive population, training lowered all four objective losses.** From the parent to step 60,000, sequence NLL fell by 0.0309 nats (2.2%) and Bradley–Terry ranking loss fell by 0.0530 (9.4%).
- **The minimum retained monitor values occurred before step 60,000.** Sequence NLL was lowest at step 45,000, and from there it rose by 0.0094 by step 60,000 and by 0.0149 by step 69,000. Bradley–Terry was lowest at step 33,000.
- **So the accepted checkpoint was not this panel's best**, and the later blocks drifted.
- **The limits.** The panel is adaptive teaching validation, not an independent test. The numbers measure how well the objective fits, not whether GP search improves.

This 24-record series was not in the earlier editions. They showed only the separate fixed comparison of the recovery checkpoint, cycle 25 and cycle 107 (§B.3.2 Fixed monitoring and actual promotion evidence). The statement in §B.3.3.3 Telemetry limits and the recovery boundary that the 78-checkpoint series has no unchanged population is still true of that series. The fixed monitor is a separate, unchanged population for the block-end checkpoints.

3.4.3 Every step of the A100 run

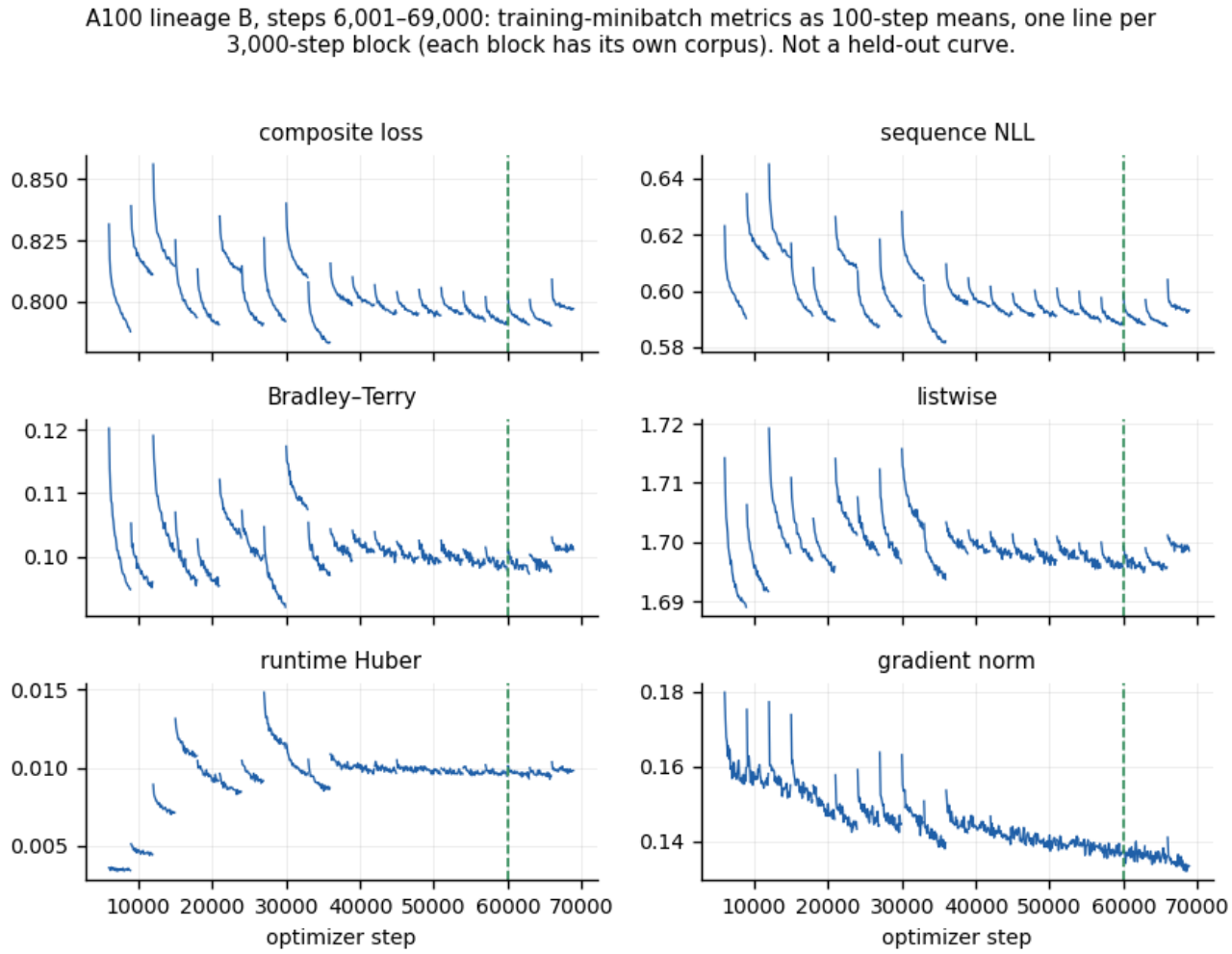


Figure 3.4 Training-minibatch metrics for all 63,000 retained A100 steps, as 100-step means, one line per 3,000-step block (each block has its own corpus). The dashed line marks the accepted step 60,000. These are training batches, not a held-out curve.

The six plotted metrics come from the retained 100-step bins (`data/per-step-history/a100-step-bins.csv`, 3,780 rows): composite loss, sequence NLL, Bradley–Terry, listwise, runtime Huber and gradient norm. §B.3.3 The complete A100 loss series: steps 6,001–69,000 describes how they were extracted.

- Training losses generally fall within each block. Composite loss increases at all 20 single-step corpus boundaries and at 19 of the 20 boundaries when compared using the plotted 100-step means.
- The gradient norm falls from a first 100-step mean of 0.1798 to a last of 0.1334. It spikes briefly at each block boundary, but unlike the losses it trends down across the whole run.

3.4.4 Throughput and timing

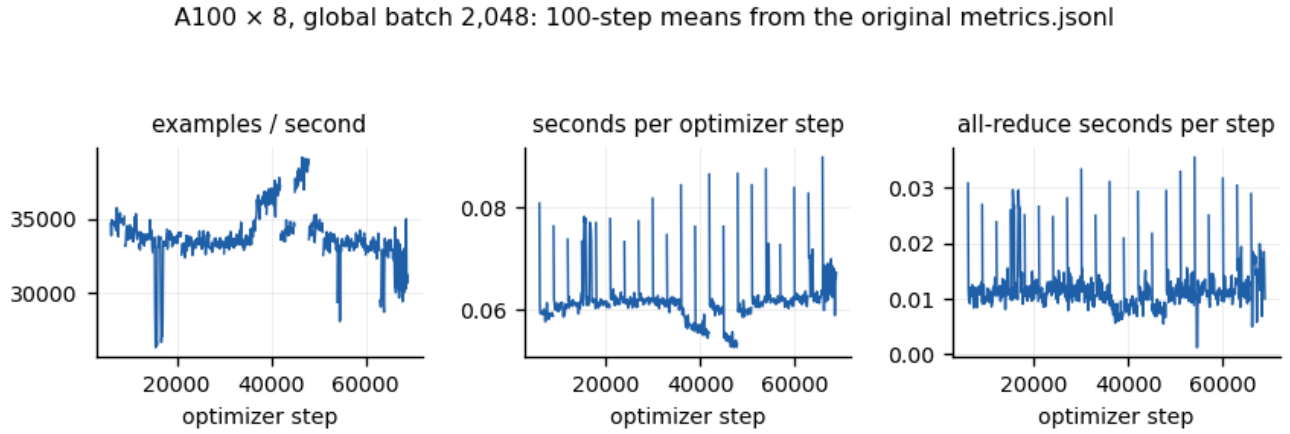


Figure 3.5 A100 × 8 throughput with global batch 2,048: examples per second, seconds per optimizer step, and all-reduce seconds per step, as 100-step means read from the 21 original metrics.jsonl logs.

The median of the 21 block medians gives these figures:

- 33,071 examples/s;
- 31,599 examples/s end to end;
- 0.0619 s per step;
- 0.0108 s of all-reduce per step, about 17% of the step.

The 100-step means range from 26,346 to 39,112 examples/s. The extraction (`data/deep-stack/a100-throughput-bins.csv`) records the SHA-256 of each source log. Tokens per second were never logged.

3.4.5 Training versus validation in the legacy gen31 run

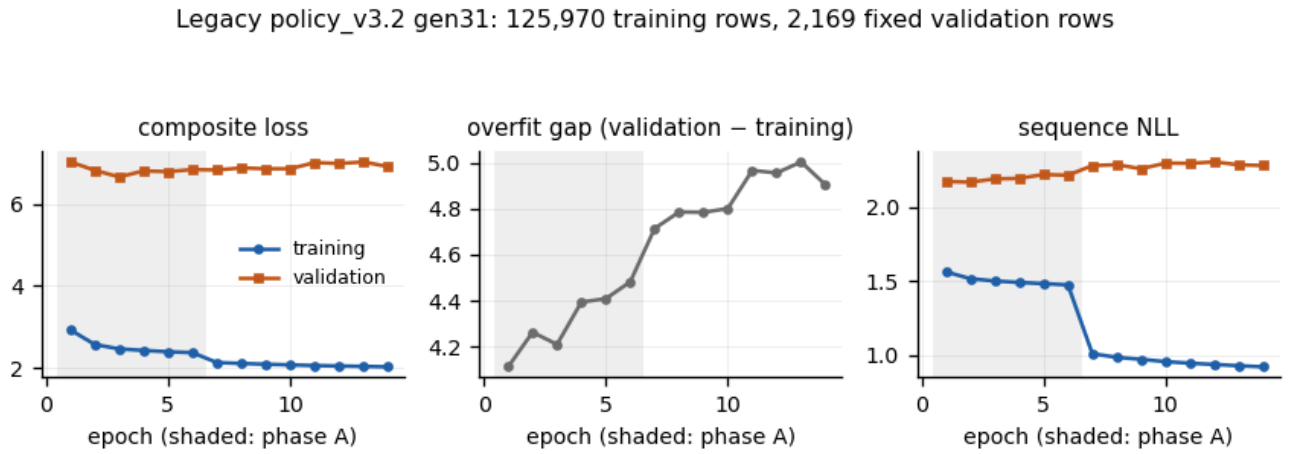


Figure 3.6 Legacy policy_v3.2 gen31: composite loss, the recorded overfit gap and sequence NLL on 125,970 training rows and 2,169 fixed validation rows. Phase A (epochs 1–6) is shaded.

gen31 is the clearest example in the retained data of training loss improving while validation does not:

- Training composite loss fell from 2.926 to 2.022.
- Validation composite loss stayed between 6.67 and 7.04.
- The recorded overfit gap grew from 4.11 to 4.90.

- Training sequence NLL dropped sharply at epoch 7, from 1.474 to 1.008. That is where phase B starts and reward weighting is applied to the imitation loss, so the drop reflects a **change of objective**, not a sudden gain.
- The 0.05-smoothed validation sequence-loss component (`val_loss_components.seq_nll`) rose from 2.175 to 2.283. This differs from the unsmoothed `val_seq_nll` reported in the historical chapter (2.055572 to 2.182772).
- Validation pairwise ranking accuracy stayed between 0.527 and 0.53850.

§B.2.1 Gen31: fitting the objectives versus using the context gives the context-advantage diagnostics.

3.5 Measurement status and figure provenance

The global KPI is **not yet measured**. The reporting contract in §9.1 Observed transferable lift: reporting contract, not yet measured uses independently repeated nested searches, the per-unit minimum of audit and outer gains, and equal family weighting. The adaptive panel and these historical curves cannot supply an untouched evaluation. No alternative score or statistical significance gate is introduced here.

Reproduce the new historical extraction with `python3 docs/internal/asm-manual/tools/extract_deep_stack_data.py --out OUTPUT`, then draw the six figures with `python3 docs/internal/asm-manual/tools/make_deep_stack_figures.py --repo REPO --training-data OUTPUT --out FIGURES`. The extraction manifest binds every original monitor record and timing log. The figure receipt binds the source files, retained numerical inputs and all output hashes, with per-plot coverage, units and interpretation limits. These are historical observations and source diagrams; none is a new training or fitted-search result.

CHAPTER 4

Installation, preparation, assets, workers and doctor

IN THIS CHAPTER

4.1 Scope and command surface 53
4.1.1 Product, checkout and CLI evidence 53
4.1.2 Command surfaces and the stopped integration order 54
4.2 Physical layout and identities 55
4.3 Build and install 55
4.3.1 Development build of the CLI 55
4.3.2 Private installer after integration 56
4.4 Prepare the panel and external runtimes 56
4.4.1 Prepare and verify the panel 56
4.4.2 Native roster and external comparator runtimes 57
4.5 Stored model assets and the active encoder distinction 57
4.5.1 Bundle manifests and declared asset sizes 57
4.5.2 Encoder identity pitfalls and provenance declarations 58
4.6 Bounded workers: the context worker and the policy producer 59
4.6.1 Context worker contract 59
4.6.2 Policy producer contract 59
4.7 Doctor checks and troubleshooting 60
4.7.1 Reading identities from the immutable CLI 60
4.7.2 Troubleshooting and safe recovery 60

4.1 Scope and command surface

4.1.1 Product, checkout and CLI evidence

Algorithm Synthesis Model (ASM) means **“learns to generate and optimize executable algorithms from data.”** The product is the private `veox-ai/JopeDime` repository. On this host its canonical checkout is `/home/ubuntu/JopeDime`; the similarly named research repository at `~/jain-split/jope-dime` is a different upstream.

This chapter is pinned to source `68f56404c3e199e06413bfd645d3fe0803d7b96f`, the reviewed PR83 head. PR83 merged it on 2026-09-23 at 16:34:47 UTC as main `a75e543d2166…` with the same Git tree (`176b75bc36d7…`); as of 2026-09-23 22:00 UTC no later change had merged to main. Sixteen actual version/help invocations against its immutable CLI completed successfully. The executable reports `jope 0.1.0`; its SHA256 is `eab2d5b1b05629020aab3d540d51eb24ba0db18ff0917da188f89a34b6dc5e2b`. The machine-readable evidence is `agent-maintainer-manual/cli-help-68.json` under the ASM custody root (`C` in §4.2 Physical layout and identities); the `68` in its name abbreviates source `68f56404`. A version string alone cannot distinguish later builds; use the full source, executable digest and build receipt.

The source inventory, a help page, an executed fixture, exact-head CI and a live operational qualification establish different things. This chapter records interfaces and operational requirements. It does not turn an unexecuted command example into measured capability. The maintainer-private

examples were executed at the same source: all fourteen commands passed. Their separate receipt and scope limits appear in §10.4.3 Measured execution of the private examples and `data/example-acceptance/`.

4.1.2 Command surfaces and the stopped integration order

Surface	Present in the immutable 68f56404 CLI	Remaining distinction
`doctor`	Yes	Static capability inventory; it does not inspect every runtime dependency or qualify a live campaign.
`benchmark verify-manifest`, `verify-inputs`, `methods`, `plan`, `run`, `report`	Yes	Planning performs zero fits. Run/report require exact admitted resources, inputs and journal identities.
`optimize capabilities`, `proof-plan`, `plan`, `run`, `report`	Yes, execution feature-gated	A compiled ASM feature is separate from qualified encoding, genuine neural-origin fits and successful evolution.
`fit`, `emit`, `search`, `pipeline run`	Yes	Legacy numerical regression/recipe paths; their outputs are not complete frozen-panel fitted inference exports.
`encode`, `champions`, `train-step`, `route`	Yes	`train-step` trains a local centroid router, not the bounded neural policy campaign.
`datasets`, `models`	Yes	Existing catalog/model tools; a fetchable file is not automatically admitted as an ASM input.
`train prepare-inputs`, `plan`, `run`, `resume`, `export`, `verify`	Pending combined training overlay	Includes native admission, actual updates/reload/continuation checks and experimental exports. See §8.4 The bounded training campaign chain.
`demo` and private CLI installer	Pending product overlay	Includes generic fitted demos and authenticated paired ASM demos; actual speed and parity must be measured.
`benchmark export-winner`, `qualify-export`	Pending winner add-on	Preserves the selected native state and ranking; backend qualification needs actual supervised execution.
`motifs mine`, `validate` under the umbrella CLI	Not present at 68f56404	Separate preparation exists; production eligible-State reading, mine command and umbrella integration were identified as remaining work.

The stopped integration order is combined **training64** (including regression consumption, accounting and coverage), then **product28**, then unchanged **warm-target** and **winner-export** add-ons. A later two-file regression proof-gate patch (`0e634858…` under `agent-training-proof-gate`, stopped on 2026-09-23 at 19:30 UTC) comes after them: generate the training, product and add-on replays before applying it, because the downstream guards pin every training postimage. The whole queue waits until the

manual's own publication lifecycle closes (§13.8.5 Updating the book). The add-on refresh is pinned under `agent-delivery-refresh-68f56404`; two strict replays produced identical nine-file postimages, with 95 canonical guards. These are custody patches, not installed behavior. Never apply the training coverage/accounting constituents a second time or copy an older loose installer postimage over its corrected form.

4.2 Physical layout and identities

Keep source, bulk inputs and evidence in separate locations. The examples below use these names; resolve them from actual receipts before execution:

```
R=/home/ubuntu/JopeDime
C=/home/ubuntu/jope-rescue-20260921/asm-integration-20260923
D=/home/ubuntu/.cache/jope/asm-private-30-v1
M="$R/benchmarks/panel.json"
# S: full reviewed source SHA; J: physical immutable CLI path.
# A: new durable private acceptance directory, outside Git and /tmp.
```

`JOPE_HOME` chooses the source/assets root; `JOPE_STATE` chooses mutable local state; `JOPE_REGISTRY` chooses the dataset registry. Those roots are resolved at call time. Pass explicit paths when producing reproducible evidence. Do not repurpose `HOME` or globally override `CARGO_TARGET_DIR`. A managed installation symlink is convenient for interactive use; a campaign pins the physical executable path and checksum so changing the symlink cannot silently replace its worker.

There are two different hashes in the CLI contracts. An artifact SHA256 hashes exact file bytes. A campaign's `plan_sha256` is its validated semantic identity, printed by planning. Do not substitute `sha256sum plan.json` for that semantic ID. Worker/build/source inventories have their own schemas and pins; preserve each rather than renaming one receipt into another type.

Campaign journals require absolute durable paths outside Git working trees and `/tmp`. Private directories are restricted by the front doors. Do not publish entire run directories: they can include raw development features, labels, recipes, private context, model state and provenance.

4.3 Build and install

4.3.1 Development build of the CLI

The current development build path builds the Rust CLI from the existing checkout:

```
cd "$R"
CARGO_INCREMENTAL=0 CARGO_PROFILE_DEV_DEBUG=line-tables-only \
  cargo build --locked --release -p jope-cli --features asm --bin jope
```

Allocate build/test resources through the repository's existing coordination before executing it. Use one reused `target/` per claimed checkout. The CLI's `asm` feature exposes the private Linux execution path; a default-feature build may show the command while reporting ASM execution unavailable. The ordinary CLI should not require the monolithic engine, Torch or CUDA. Separate context and policy workers and their authenticated model inputs are still necessary for a live neural run.

4.3.2 Private installer after integration

After the pending installer is integrated, prefer its measured source-build and checksum-install front doors:

```
python3 "$R/deploy/install-private.py" source \
  --checkout "$R" --source-sha "$S" --output "$A/cli-build" --jobs 4
python3 "$R/deploy/install-private.py" binary \
  --binary "$PRIVATE_BINARY" --sha256 "$BINARY_SHA256" \
  --source-sha "$S" --prefix "$A/install"
```

Read the binary path and digest from the actual source-build receipt. The source command checks the checkout before and after compilation, uses the same `asm` selection for dependency inventory and build, rejects unwanted engine/Torch/CUDA dependencies, and selects the executable from Cargo artifact messages. Its receipt includes the toolchain, dependency graph, features, source identity, compiler output, executable digest and measured cold duration. Compilation failures retain failure receipts.

The binary installer requires Linux x86_64, Python 3.10 or later and `readelf`. It validates the supplied checksum before executing the binary, checks architecture/glibc/dynamic dependencies, probes version and command help, and activates a digest-addressed private binary through a managed symlink. It does not download models or datasets. A caller-supplied checksum proves byte identity; retain independent source/review/build provenance as well. Cold build time, cold installation time, data preparation and warm demonstration time are separate measurements.

The installer's `jope.private-cli-source-build/v1` receipt does not replace a `jope.asm-cargo-build/v1` worker receipt or `jope.asm-worker-build-inventory/v1`. Build and retain the context/policy workers, source manifest, accepted assets and native context qualification through their owning workflow. The native canonicalizer used by experimental proposal admission must match the final source/build and physical executable bytes.

4.4 Prepare the panel and external runtimes

4.4.1 Prepare and verify the panel

The approved panel contains ten regression, ten binary and ten multiclass datasets. Its exact manifest SHA256 is `ba7dace4cd535687002cc95538dbc7f7aaf77ec973177c7159b3c6d22471eee0`. Reformatting the JSON changes its bytes and is refused even when its apparent meaning is unchanged. Do not replace a failed dataset with an easier fixture.

```
uv run --project "$R/python" --extra benchmark python "$R/benchmarks/prepare.py" \
  --manifest "$M" --output "$D"
"$J" benchmark verify-manifest --manifest "$M"
"$J" benchmark verify-inputs --manifest "$M" --datasets "$D"
"$J" benchmark methods
```

Preparation must preserve actual frozen row assignments, original labels, category strings, nulls and groups. `verify-inputs` verifies development bytes and schema, not just file existence. The benchmark does not open the sealed evaluation partition. An operational proof of one regression family retains the original thirty-dataset manifest but selects a separately declared proof scope; it does not modify the official panel.

4.4.2 Native roster and external comparator runtimes

The native roster has 17 regression recipes, five binary recipes and four multiclass recipes. Aliases do not create extra screening slots. The second GPR configuration is distinct because it has a distinct fitted implementation. Every task also includes four external adapters, each run at library defaults with seed 0 and one thread: XGBoost, LightGBM, CatBoost and a scikit-learn random forest. Native recipes are fixed before the dataset screen and the external adapters keep those defaults, so the campaign performs no per-dataset hyperparameter search for any non-ASM comparator (a head's own internal fitting, such as `ridge_gcv`'s generalized cross-validation, is part of its fixed recipe; §11.6 Minimum complete future campaign report), while each ASM arm searches within its declared allowance. The highest-rated methods on the historical benchmark-valid ladder, such as TabFM Ensemble, are not panel comparators (§C.2.3 SOTArena published and corrected ladders); panel wins against this roster would therefore not show that ASM beats those methods. Sources: `crates/jope-bench/src/panel/campaign_cli.rs:67`; `crates/jope-bench/adapters/sklearn_rf_adapter.py:2`; `crates/jope-bench/src/panel/selection_exposure.rs:91`; `crates/feat-math/src/heads.rs:65-67`. External library versions, entry points, environment and package bytes must be authenticated before admission.

```
python3 "$R/deploy/prepare-panel-runtime.py" \
  --binary "$J" --native-command benchmark \
  --build-receipt "$CLI_BUILD_RECEIPT" --source-sha "$S" \
  --python "$EXTERNAL_PYTHON" --site-packages "$EXTERNAL_SITE_PACKAGES" \
  --adapters "$R/crates/jope-bench/adapters" --output "$A/runtimes.json"
"$J" benchmark plan --manifest "$M" --datasets "$D" --source-sha "$S" \
  --runtimes "$A/runtimes.json" --output "$A/base.json"
```

The preparation helper checks actual dependencies/constructors and pins the runtime. It is not a completed fit comparison. Retain unavailable-dependency reasons rather than dropping methods. Prior inventories observed XGBoost 3.4.1, LightGBM 4.7.0, CatBoost 1.2.10 and scikit-learn 1.7.2 on this host; these historical metadata observations do not authenticate a later environment. A new package installation invalidates old package-byte inventory. Keep the external CPU environment separate from the dedicated CUDA training environment.

Omitting `--runtimes` from `benchmark plan` produces a verified Rust screening inventory with zero fits; it is not the complete admitted ASM/comparator campaign. The runtime-aware plan is the base for `optimize plan`.

4.5 Stored model assets and the active encoder distinction

4.5.1 Bundle manifests and declared asset sizes

The metadata-only S3 audit on 2026-09-23 fetched the three bundle manifests below and pinned their returned bytes, ETags and lengths. No model bodies were downloaded for this chapter. These are declared object sizes from the retrieved manifests; they do not measure resident memory, compute cost or a new numerical qualification.

Bundle asset	Bytes	Role in this chapter
`policy-step-60000/policy.safetensors`	111,199,088	Accepted policy export identity, consumed only with its checkpoint binding

Bundle asset	Bytes	Role in this chapter
`policy-step-60000/context_transforms.safetensors`	8,413,808	Accepted fixed context assembly transforms
`jope-prime-gen31-v2/model-bundle/hyperion_v5_arm_b.safetensors`	6,650,692	Exact Hyperion weights pinned by the current CPU context producer
`jope-prime-gen31-v2/starforge/starlight_core_regressor.safetensors`	233,183,532	Foundation component; manifest describes512 hidden features pooled into2048
`jope-prime-gen31-v2/model-bundle/target_encoder_fused_v1…safetensors`	78,615,176	Target component; manifest describes normalized64+64+384 fused representation
`hyperion-v7/model.safetensors`	246,456,988	Separate stored bundle, not the accepted context producer's pinned weights
`jope-prime-gen31-v2/model-bundle/jope_policy.safetensors`	108,846,812	Separate gen31 policy artifact; not step-60000

The three current model bundle manifests are `hyperion-v7`, `jope-prime-gen31-v2`, and `policy-step-60000`. Their existence and object sizes do not establish new training, inference parity or serving promotion. §1.8.2 The identity tuple gives their roles among the product identities, and §13.6 Release and installation are independent from model promotion gives the acceptance boundaries. Historical lineage-A material in learning storage cannot replace its missing canonical training-state chain.

4.5.2 Encoder identity pitfalls and provenance declarations

The storage name `hyperion-v7` must not be confused with the CPU implementation's `V7Model` Rust type. The actual current producer pins weight hash `88b9e834…`, which matches the v5 arm-B asset, with row width 128. The separate stored v7 manifest describes dimension512, eight heads,12 layers and six bands. Replacing the pinned asset based on its newer-looking name changes the encoder identity and invalidates compatibility.

Two identity pitfalls matter when maintaining this boundary. Hyperion `load_settings` reports a legacy 16-hex identifier, not a SHA-256 digest. The producer therefore authenticates manifest bytes before and after the path-based settings load, authenticates weights before and after `V7Model::load_cpu`, and requires embedding width 128. A legacy identifier must never be compared with or substituted for a cryptographic admission pin. Separately, ordinary `jope encode` resolves `HYPERION_V7_WEIGHTS` as a **bundle directory**, else `$JOPE_HOME/models/bundles/hyperion-v7` (relative to the working directory when unset). The default stored bundle is 512-dimensional and differs from the pinned v5-arm-B producer; both manifests use the loader string `version: v7`, so that string cannot establish checkpoint identity. `jope encode` is not the qualified 2048-value ASM context interface. Current documentation drift remains at `docs/models.md:105`, which supplies a file instead of the expected directory, and the default-v5 comment in `crates/hyperion/src/hyperion.rs:28`. Use exact asset hashes and the qualified context-worker flow.

The retrieved manifests also contain internal/redistribution/provenance declarations that vary by artifact. They are retained evidence about the bundle metadata, not proof that every object is approved for every distribution. The active private run's exact release/input inventory remains the authority for which files it used; S3 presence alone does not establish serving eligibility. Sources: `crates/jope-policy/src/context_producer.rs:41`; custody `metadata-receipts/veox-jopedime-models-use2--bundles--policy-step-60000--manifest.json`, `…--hyperion-v7--manifest.json`, and `…--jope-prime-gen31-v2--manifest.json`.

4.6 Bounded workers: the context worker and the policy producer

4.6.1 Context worker contract

In the bounded ASM flow, encoder weights are loaded inside `jope-context-worker` (`jope-policy`, feature `context-producer`). `--version` prints `jope-context-worker/1`. Its exact eight named arguments are:

```
jope-context-worker --mode produce|qualify-case --input ABSOLUTE_PATH --input-sha256 HEX64 --
assets ABSOLUTE_ENCODER_FILES_JSON --assets-sha256 HEX64 --source-sha40 HEX40 --producer-sha256
HEX64 --output ABSOLUTE_PATH
```

Extra arguments such as `--dataset-root` are refused. The child requires OMP, OPENBLAS, MKL, BLIS, NUMEXPR and RAYON thread limits of 1, `VECLIB_MAXIMUM_THREADS=1` and `CUDA_VISIBLE_DEVICES=-1`; it authenticates its own executable and the component closure before model loading. The pinned bundle manifest begins `5e7884ec…`; each inventoried file/notice is verified with a 512 MiB per-file limit. Outputs are create-new, mode 0600 and fsynced. Stdout records `jope-context-worker/1 <output_sha256>`. A failed `qualify-case` comparison retains `<stem>.unqualified.json`, exits nonzero and does not create the requested successful output.

For current `jope optimize`, input derives from `dataset.cut(0)` fitting rows and is retained at `<journal>/asm-context-inputs/<input_sha256>.json`. Admitted output is `<journal>/asm-contexts/<neural-arm-id>/context.f32` plus `bundle.json`; the vector contains 2,048 little-endian float32 values. Invoke this through the supervised planner/controller contract, not as an unsupervised shortcut.

4.6.2 Policy producer contract

`jope-policy-producer` is a separate pinned bounded job. Its `asm.private-policy-producer/v1` request binds worker source, prepared/training/context hashes, cut, accepted checkpoint state, policy/vocabulary hashes, three RNG seeds and sampling shape. It accepts only the pinned step-60000 weights (`4b4b5a40…44f1e`) and state, rounds 1–8, batch 1–128 and a finite positive temperature. Context input is exactly 8,192 bytes, finite and clipped within [-32,32]. It requires either a qualified CPU context bundle or an authenticated original `parallel.encoding.v1` receipt with the original LibTorch lineage, train-side partition and `final_test_targets_used:false`. Frozen transforms are pinned to `1b779c6f…6e22`.

Its output must be an absolute physical-parent path outside `/tmp`, created new with mode 0600. The final stdout JSON declares `numerical_backend:candle_cpu_float32` and `native_cuda_parity_qualified:false`. Reading the envelope proves consistency, not execution: consumption also requires the completed pinned worker/output receipts. Replacing accepted weights, transforms or vocabulary is not a free configuration substitution.

The producer shares the 30-second worker cap and original 120-second neural allowance; the accepted proof used a 3 GiB worker allocation. The pre-cache proof at source `55e9ed688faf` was killed at 30.004805335 seconds, before neural fits. With cached decoding, later measured producer wall times were 20.832133058 seconds at source `3fd7d5b2ebb6`, 20.651920435 at `c5d11ffca92c` and 20.486968041 at `68f56404c3e1`. These leave about 9.5 seconds of per-job headroom in the last measurement, not a portability guarantee. Sampling-shape or host changes need another bounded proof, and the fixture-only decoder-equivalence limits of §2.7.1.2 Component dimensions and assembly still apply.

Sources: `crates/jope-policy/src/context_producer/worker.rs:28, :188, :255`; `crates/jope-policy/src/producer.rs:16, :91`; `crates/jope-policy/src/bin/jope-policy-producer.rs:13, :78, :121, :172`; `crates/jope-bench/src/panel/optimize.rs:454, :705`; retained `proposal-execution.json` receipts.

4.7 Doctor checks and troubleshooting

4.7.1 Reading identities from the immutable CLI

These metadata commands do not train or load a checkpoint:

```
R=/home/ubuntu/JopeDime
C=/home/ubuntu/jope-rescue-20260921/asm-integration-20260923
J="$C/neural-integration/live-build-68f56404c3e199e06413bfd645d3fe0803d7b96f/bin/jope"
"$J" --version
"$J" doctor
"$J" optimize capabilities
git -C "$R" rev-parse HEAD
sha256sum "$J" "$R/Cargo.toml" "$R/python/pyproject.toml" "$R/benchmarks/panel.json"
```

Do not interpret `qualified: false` in capability metadata as a missing command, or compile-time availability as completed runtime qualification. The measured executable SHA is `eab2d5b1b05629020aab3d540d51eb24ba0db18ff0917da188f89a34b6dc5e2b`; a different executable needs its own receipt.

4.7.2 Troubleshooting and safe recovery

Symptom	Meaning and next action
`compiled: false` or ASM feature unavailable	Use the reviewed Linux CLI built with `asm`; then separately qualify its model inputs and workers.
Running binary differs from pinned native runtime	Resolve the installed physical binary, verify its hash and prepare a new pre-execution inventory/plan; do not alter a running plan.
Frozen manifest or development checksum differs	Stop admission, preserve both files and find the preparation/source mismatch. Do not regenerate folds or substitute datasets to make a check pass.
Missing external dependency/compiler	Retain unavailable status and inventory. Install/qualify before opening a new authorized experiment; never drop the method or replace a winner.
Context/reference/worker identity mismatch	Rebuild or requalify through the owning source-bound workflow; a copied success flag is insufficient.
Deadline expired	Recover/report the original window. A repeated command cannot renew it. Any new experiment needs its own explicit scope and resource declaration.
Started job lacks a finished receipt	Treat execution cost/status as unknown; let the shared pool perform terminal recovery/drainage. Do not blindly retry the child or delete its artifacts.
No complete five-fold winner	Keep incomplete/failed confirmations visible. Do not select a different recipe merely because it has an emitter.
Warm result exceeds 60 seconds	Preserve fitted artifacts, comparisons and the measured miss of the 60-second goal. Do not relabel elapsed time or omit preparation performed inside the warm invocation.

Symptom	Meaning and next action
S3 access stalls at instance metadata	On the host, select the configured credential-process profile such as `AWS_PROFILE=veox-jepson-prod`; never source the bare-token credential note files. Runners have no AWS credentials.
No eligible training families/no-signal admission	Preserve refusal; obtain real eligible data under the declared protocol before GPU training. Do not use development/test gain as a substitute for the train-side OOF admission rule.

CHAPTER 5

Data, splits, groups, families and permitted input boundaries

IN THIS CHAPTER

5.1 The frozen thirty-dataset panel: identity, totals, groups and immutability 62
5.2 Local preparation and panel storage 63
5.3 The broader S3 corpus catalog 63
5.3.1 Catalog composition by task 63
5.3.2 Licence metadata, anomalies and the corpus figure 64
5.4 The historical input closure and the meta-test question 65
5.4.1 Record split and the meta-test closure question 65
5.4.2 Audit the entire input closure before continuation 65
5.5 Family-level domains and the permitted input closure 65
5.5.1 Three family-level domains 66
5.5.2 Hashes, eligibility and the complete input closure 66
5.5.3 Datasets outside the frozen panel 67

Observation date: 2026-09-23T16:41:48.371336+00:00. Frozen source: `68f56404c3e199e06413bfd645d3fe0803d7b96f`, the reviewed PR83 head; `main` commit `a75e543d2166` has the same tree. These are observations at that source and time. They do not turn stored metadata, a source packet, or historical results into a current operational qualification.

Every number in this chapter comes from a pinned manifest, prepared development table, immutable report, committed historical snapshot, filesystem metadata, or authenticated AWS metadata request. The audit did not run a fit, load a model, open sealed target bodies, or execute a report command.

5.1 The frozen thirty-dataset panel: identity, totals, groups and immutability

Panel `asm-private-30-v1` uses ten regression, ten binary, and ten multiclass datasets from PMLB revision `7c1f4bdc00136dc2e55c87fa6b8ba6e8af6d1a68`. Manifest SHA-256: `ba7dace4cd535687002cc95538dbc7f7aaf77ec973177c7159b3c6d22471eee0`. It was selected before measuring ASM outcomes, excludes the historical favorable test-10 collection, and cannot replace a dataset because a method failed or lost.

There are **55,029 total rows**, **44,048 development rows**, and **10,981 sealed rows** according to the frozen assignments. Feature counts sum to 522 across datasets; this is not a union of unique semantic variables. Development data contains 1,612 missing feature cells in 302 rows. All thirty development, split-assignment and metadata file SHA-256 values match the frozen manifest. Every row ID is unique within its dataset, and every group stays wholly within one original development fold or sealed partition.

The panel is immutable once comparison begins. Its identity covers dataset metadata, original labels and types, row IDs, group membership, original development folds, sealed membership and prepared data hashes. Do not replace difficult datasets, reshuffle rows, map categories to arbitrary numeric order, drop missing rows or substitute a different target to make a task executable. Missing source subject or time identifiers remain limitations even when duplicate-feature groups are protected.

Known date columns and identical-feature groups are retained by the prepared split. Missing original subject/time identifiers remain a limitation. A grouped split cannot establish independence when the source never supplied those identifiers. Checkpoint exposure is unknown for these datasets; an unseen-family generalization claim is not supported. Pretrained-encoder exposure is also unknown, and the panel remains an adaptive development domain (§5.5.1 Three family-level domains).

5.2 Local preparation and panel storage

Local preparation: `/home/ubuntu/.cache/jope/asm-private-30-v1/`. `development/` contains the permitted development tables; `splits/` contains original row/group/fold assignments; `metadata/` contains pinned upstream YAML; `downloads/` contains the original compressed archives; `sealed/` contains withheld partitions. The audit opened development, assignment, and metadata files only. It inspected sizes and expected hashes of raw and sealed files without opening their bodies.

File class	Logical bytes	Content read in this audit
development	17,923,164	Yes; metadata/development statistics and SHA-256
metadata	82,681	Yes; metadata/development statistics and SHA-256
sealed	4,518,613	No; size and expected identity only
source_archive	2,075,982	No; size and expected identity only
split_assignments	9,812,165	Yes; metadata/development statistics and SHA-256

File sizes are logical bytes, not compressed-memory use or total disk allocation. The raw archive sizes sum to 2,075,982 B, matching the frozen declared download byte counts; content hashes of those raw archives were not revalidated here because the archives also contain sealed targets. The prepared sealed file hashes are declared frozen identities, not freshly verified content checks in this audit.

5.3 The broader S3 corpus catalog

5.3.1 Catalog composition by task

The pinned current catalog is `ab9fda8d2dea46067b70a42812e9d3c1d9dc6ba025780100df34377e81aa1120`: 3,859 distinct dataset hashes, all marked real, 6,208,051 declared rows. Shape metadata sums to 265,105 features and 4,628 categorical features; sums across datasets are not deduplicated semantic quantities. The catalog contains no task explicitly named multiclass; 322 entries are multi-label, which is a different task and must not be silently relabeled.

Catalog task	Datasets	Total declared rows	Training rows	Test rows metadata only	Feature-count range	Row-count range
binary	843	1,490,965	892,307	598,658	[1, 2000]	[10, 69624]
multi_label	322	786,436	555,756	230,680	[1, 20531]	[23, 50000]
regression	2,611	3,025,922	1,898,974	1,126,948	[0, 1688]	[10, 50000]
timeseries	83	904,728	775,519	129,209	[2, 86]	[81, 49999]

5.3.2 Licence metadata, anomalies and the corpus figure

Catalog discrepancy: the stored registry collection description says 1,416 unknown licenses. Counting the current catalog's actual license-status fields gives **3,755 unknown and 104 recorded**. This chapter uses the counted fields and retains the stale description as provenance. All 2 collections remain private: `registry-real-v1` has 3,859 members and `test-10` has 10; every listed member occurs in the catalog. No payload or training eligibility validation was performed simply by counting them.

Seven regression catalog entries declare zero features. These are retained anomalies, not automatically usable supervised tasks: `5ffbe1543f683e4d`, `66fea7ec7dffa02a`, `72235646c52be632`, `7d1497edfe70495d`, `d12486ba35015a09`, `e1c1ff8cdf4ee36f`, `f773853db5b3dc9c`. Corpus metadata lacks feature distributions, missingness, actual class balance and row-level dependence evidence for many entries; those statistics remain unknown here.

The broader catalog has 3,859 real datasets and 6,208,051 metadata-reported rows. It is a different collection from the frozen panel. Its `multi_label` task is not interchangeable with admitted multiclass classification. Per-payload statistics for the entire corpus were not recomputed.

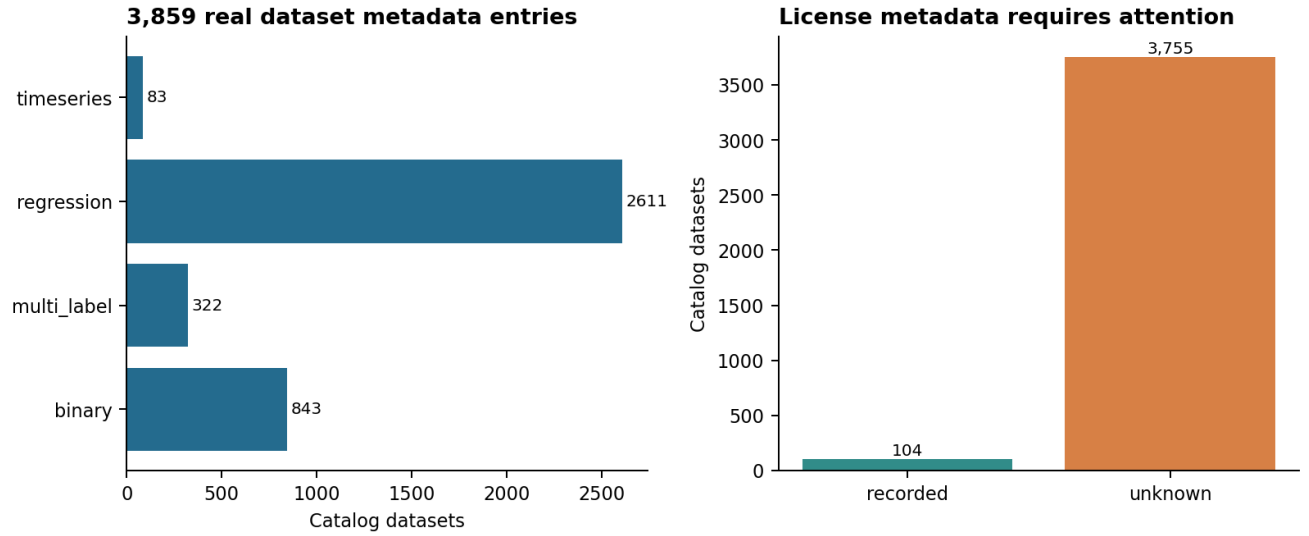


Figure 5.1 Corpus task counts and license metadata completeness

The catalog has 3,755 entries with unknown license metadata and 104 with a recorded license. The collection's older prose count of 1,416 unknown licenses is inconsistent with this current pinned catalog. This is a metadata quality gap; neither the chart nor a repository-level license resolves each dataset's provenance. See §D Dated data estate, recovery evidence and binary formats for exact storage bytes, prefixes and observation scope.

5.4 The historical input closure and the meta-test question

5.4.1 Record split and the meta-test closure question

The accepted corpus's record split is precise: 9,382,874 `meta_train` records in 6,836 shards and 831,458 `meta_validation` records in 1,041 shards. There are no `meta_test` record shards. The context index nevertheless contains 6,962 training, 1,054 validation and **878 meta-test context descriptors**. Current legacy `training.rs` stages every context in that index before sampling records. Consequently, withholding meta-test record labels is not a proof that sealed families were excluded from the entire historical encoder/trainer input closure. This is a concrete remaining closure audit, not a claim that this documentation audit opened sealed vectors or labels.

A separate external historical-inventory report also discloses parsing manifests for eight datasets marked `meta_test` **and hashing their source CSVs**; its shorter coordination entry mentioned manifests only. That report says all eight have lineage-A `test_exact` high-water marks. This audit did not reopen those paths or inspect their labels. The disclosure does not establish whether labels reached lineage-B training. Before reuse or continuation under the current policy, audit the complete encoder, context-staging, target-admission and sampler input closure, and resolve the old exposed-HWM lineage. Neither zero meta-test record shards nor an old qualification flag closes that question.

The accepted index SHA `1c840597…`, vocabulary `8d55fee4…` and context index `d77e856b…` bind the preserved checkpoint lineage. Its label policy is explicitly `reference_v1`. An old `qualification: true` flag means qualification under that historical implementation and policy. It does not retroactively satisfy current finite train-side OOF improvement over a trivial baseline, improvement on four of five folds, exact source closure and non-exposure requirements.

5.4.2 Audit the entire input closure before continuation

Before any future continuation, audit the **entire** input closure, including every context descriptor and staged tensor, not only training records. The retained snapshot 97 index lists 8,894 contexts, including 878 `meta_test` descriptors, while record rows are train/validation only. Current legacy training stages all indexed contexts before sampling. Absence of sealed record rows therefore does not prove strict sealed-family exclusion. Do not blindly continue that corpus under the new closure policy; preserve the metadata observation and resolve it without opening sealed payloads. See §5.4.1 Record split and the meta-test closure question and context staging source.

§5.4.1 Record split and the meta-test closure question records unresolved historical input closure: meta-test context descriptors were staged by the legacy trainer, while the external author disclosed meta-test manifest inspection and source-CSV hashing alongside old exposed high-water marks. Do not treat these findings as proof of training-label leakage or dismiss them because the packed record split excludes meta-test rows. Resolve the complete allowed-input closure and current teacher eligibility before continuing the old corpus under the new policy.

5.5 Family-level domains and the permitted input closure

This section states the owner's edition-2 rules for which data may reach teaching, training, selection and evaluation. They are requirements, not results. Each carries one status as of 2026-09-23 about 23:55 UTC: available, implemented but unqualified, pending integration or proposed.

5.5.1 Three family-level domains

Domains are assigned to dataset families, not to rows, files or split names. A split name alone cannot prove an unseen family (§9.7.1 Freezing the experiment and choosing families).

Domain	Permitted use	Status
Teacher/train	Train-side evidence may produce eligible teachers and policy-training rows	Pending integration: the stopped `training64` packet partitions admitted families into `train` and `development` sides only (pending `crates/jope-cli/src/train/corpus.rs:196-264`)
Adaptive development	Labels may guide screening, selection, confirmation and model monitoring; results are adaptive, not independent	Implemented but unqualified: every panel report at `a75e543d2` records `confirmation_role: "adaptive_development"` (`crates/jope-bench/src/panel/campaign.rs:894`); no full panel has run
Evaluation outside training and selection	Families whose labels informed no teaching, training, selection or monitoring	Proposed for this ASM path. The legacy engine assigns a sealed meta-test family partition and withholds its observations from corpus records, but the legacy trainer stages all indexed descriptors, including the 878 meta-test descriptors; that does not establish their exclusion from the worker input closure

The frozen panel `asm-private-30-v1` remains an adaptive development domain. Its development labels drive the screen, freeze and confirm procedure (§11.2.2 Adaptive development exposure), and a panel family that supplies teachers must additionally be reported as exposed (§9.7.1 Freezing the experiment and choosing families). Because domains are family-level, withholding the panel's sealed rows does not place its families in the third domain. Prior exposure is unknown in two separate places: the accepted policy checkpoint, which `optimize plan` records as `checkpoint_training_exposure: "unknown"` (`crates/jope-bench/src/panel/optimize_cli.rs:128`), and the fixed pretrained encoder assets that produce the proposal context (§2.7.1.2 Component dimensions and assembly). Pretrained-encoder exposure remains unknown; neither unknown may be reported as zero.

5.5.2 Hashes, eligibility and the complete input closure

A hash is not eligibility. A digest authenticates bytes; it does not decide whether those bytes may be used. The legacy trainer at `a75e543d2` checks every indexed context's bytes against its recorded `sha256` digest and expected size, then stages all of them on the device (`crates/jope-engine/src/training.rs:1187-1203`), so the 878 `meta_test` descriptors in the retained index would pass that integrity check (§5.4.1 Record split and the meta-test closure question). Eligibility comes from each input's admission record and family domain (§7.2.2 The admission guard, label exposure and HWM identity), never from a matching digest.

The permitted input closure covers everything a teaching or training worker can read, directly or transitively: records, context descriptors and tensors, fitted and frozen transforms, and all assets, including encoder and policy weights, vocabulary, admission manifests and checkpoints. Its parts have different status:

Closure requirement	Status as of 2026-09-23 about 23:55 UTC
Native projection of admitted contexts	Pending integration: the stopped `training64` preparation requires the requested contexts to cover exactly the admitted comparisons and writes only the selected row contexts (pending `crates/jope-cli/src/train/corpus.rs:572-575, :600-635`)
One authenticated inventory of every permitted worker read and its role	Proposed: the same packet's Python campaign opens the complete context index, encoder, vocabulary and admission file, and validates only the contexts that retained rows reference (pending `python/src/jope/policy/training_campaign.py:298-363`)
Rejection of unused forbidden descriptors before loading	Proposed: neither current nor pending source rejects extra context-index entries, and the legacy native route still stages every indexed context

An unused forbidden descriptor, such as a `meta_test` or evaluation-domain context that no admitted row references, must be rejected before any model, tensor or corpus is loaded; leaving it unsampled is not enough. The tests this requires do not exist yet: an injected unused forbidden context, an unlisted reachable artifact, invariance under mutation of an excluded artifact, and invariance of preprocessing and context under changed holdout labels. Source: the read-only boundary investigation `plan-execution-20260923/boundary-resume-siblings.md` in the ASM custody root.

5.5.3 Datasets outside the frozen panel

Arbitrary ingestion is separate from the frozen panel. Status: proposed. No arbitrary-dataset ingestion command exists at `a75e543d2`: the one-dataset ASM front door takes a dataset already present in the frozen panel (§6.5.1.1 Front door and safe metadata checks), and a fetchable catalog file is not automatically an admitted ASM input (§4.1.2 Command surfaces and the stopped integration order). A separate authenticated ingestion request must preserve the new dataset's labels, categories, missingness and groups and assign its family to a domain; it never edits the frozen manifest (§6.5.2 What is missing for a sustained, incumbent-seeded dataset attack). Results on an ingested dataset are not frozen-panel results, and frozen-panel results do not qualify an ingested dataset.

CHAPTER 6

Regression search, fitting, prediction and focused optimization

IN THIS CHAPTER

6.1 Proposal generation and ranking 68
6.2 Paired arms, generation zero and evolution 69
6.3 Dense-width refusals and the proposed feasibility preflight 69
6.4 Warm fitted demonstration after integration 70
6.4.1 Generic and ASM demonstration commands 71
6.4.2 The under-sixty-second target and historical timing 72
6.5 Focused single-dataset search 72
6.5.1 Commands that exist today for one dataset 72
6.5.1.1 Front door and safe metadata checks 73
6.5.1.2 Required inputs 73
6.5.1.3 Plan, run and report a focused proof 74
6.5.1.4 Other current search surfaces 75
6.5.2 What is missing for a sustained, incumbent-seeded dataset attack 75

6.1 Proposal generation and ranking

Accepted proposals are generated under grammar constraints, and the neural pool is the first 64 distinct valid programs in draw order; any shortfall after the bounded sampling rounds is filled with distinct grammar-uniform programs labelled `neural_rejection_fallback`. Ranking orders that pool by the mean of four reward heads, descending; exact ties use program SHA256 then text. Runtime estimates are reported without subtracting an undocumented runtime penalty. Ranking binds the checkpoint, context and vocabulary and records that scoring labels and fitted GP scores were not read. It does not select or reject programs: all 64 ranked programs and the 16 exploration programs enter generation zero. Sources: `crates/jope-policy/src/ranking.rs:22`; `crates/jope-policy/src/decode.rs:9-11, :212-225, :359`; `crates/jope-policy/src/seeds.rs:257-270`.

Each ranked policy-pool program carries `rank_log_weight`, the log-softmax of its four-head mean predicted reward over that pool. Historical files called this `policy_logprob`. It is a within-context ranking weight, not a sampling/selection probability or propensity; do not use it for importance weighting or off-policy correction. Exploration candidates carry no rank weight.

The four reward heads read a masked mean of the program's token embeddings, with no position information (`crates/jope-policy/src/model.rs:555-567`). Within one context, two programs with the same token multiset therefore receive the same rewards in exact arithmetic, so their relative order carries no learned signal; an exact tie falls to the program-hash tie-break. This does not remove token order from the autoregressive proposal decoder (§9.8 Distinguish routing, ranking and search recovery). The owner's rule for this limitation (status: proposed): first measure how often such collisions occur and the ranking regret they cause, and defer architecture changes, including adopting an order-aware reward input, so that the first causal training experiment keeps architecture, objective, corpus and sampler changes distinguishable.

6.2 Paired arms, generation zero and evolution

ASM compares a 64-slot policy pool plus 16 exploration generation-zero slots with 80 generic slots. The pool retains `neural_policy` versus `neural_rejection_fallback` origins explicitly. Both arms receive the same declared search and fit allowances, but encoding and proposal work consume the neural arm's own clock; fitting time can differ. A producer refusal, duplicate or failed candidate is an outcome to record. It cannot silently become another neural success or a free replacement arm. Actual inner-fold fits determine selection and offspring ancestry. A model's predicted reward never substitutes for measured loss.

Each arm starts with 80 generation-zero slots. In the neural arm, slots 0–63 form the ranked policy pool: distinct valid checkpoint samples are labelled `neural_policy`; any shortfall after the bounded sampling rounds is filled by distinct grammar-uniform programs labelled `neural_rejection_fallback`. Slots 64–79 are `random_exploration`. Only `neural_policy` roots and completed descendants with verified neural ancestry count as genuine neural evidence; fallback and exploration roots do not. In the origin proof at source `68f56404` all 64 pool slots were `neural_policy`, 48 completed all five inner folds, and no fallback slots were needed. Receipts retain initial-slot origin and descendant ancestry. Actual completed inner OOF losses select the best four parents. New generations alternate mutation and crossover attempts, canonicalize recipes, reject duplicate executable identities, and retain variation attempts in the journal. Source: `crates/jope-pipeline/src/neural_search.rs:27, :1380`.

Neither arm is seeded from the other, but at `a75e543d2` only the neural arm's initial population is authenticated against its producer. The neural arm's 80 generation-zero slots come only from the verified policy-producer artifact: the controller verifies the artifact against the worker request, requires exactly 80 slots and refuses any slot whose declared origin does not match its position (`crates/jope-pipeline/src/neural_search.rs:1066-1102`), and report recomputation replays that producer output against every generation-zero genome, origin and rank weight (`crates/jope-bench/src/panel/asm_campaign.rs:516-597`). The generic arm draws its 80 distinct candidates in process from its own grammar stream, seeded by the plan's `random_seed` (`neural_search.rs:1147-1176`). Resume and report replay check that population's slot count, `generic` origins and empty parents, but do not re-derive it from that stream: a resumed generic arm reloads its saved initial candidates (`neural_search.rs:1136-1146`), and the population check, by its own contract, does not authenticate proposal origins (`neural_search.rs:422-424`). Generation zero reads no candidate discovered by the other arm. Historical final searches differed: their arms shared incumbent, retained-teacher and exploration seed pools, and shared incumbents can create many ties (§9.8 Distinguish routing, ranking and search recovery). The owner's rule for future seeding (status: proposed; not yet met for the generic arm): initial populations must be authenticated, and a discovery cannot silently seed another arm. This manual proposes declaring any incumbent, retained teacher or another arm's discovery as an authenticated origin in the plan before execution.

6.3 Dense-width refusals and the proposed feasibility preflight

At `a75e543d2` the 512-column cap applies to generated dense width, not to total schema or raw input width (`MAX_GENERATED_DENSE_WIDTH`, `crates/feat-math/src/advanced_v3/feature_block.rs:6, :107-110`). No check runs before launch: the recipe validation that precedes fitting checks task, schema, syntax and stage admissibility but does not predict generated widths, so an over-wide recipe fails inside a launched fit. In the origin proof those refusals were 130 failed commands from 26 recipes, including 16 of 64 `neural_policy` generation-zero candidates (§1.5.4 Failures: dense-width refusals and the earlier failed reproducer).

A pre-launch dense-width feasibility check is **proposed**; nothing below is implemented. The owner's requirements (relayed 2026-09-23T22:27:11Z) and Codex's read-only source investigation (`plan-execution-20260923/feasibility-phase-accounting.md`, SHA-256 `0154e6b0…`) bound it:

- **Permitted fitting information only.** The check may read each inner fold's task, exact schema, class order and training-side fitting rows, never prediction rows or validation labels. One-hot and hybrid categorical widths depend on the fitting-side category layout, which the compiler deliberately refuses to guess; an undecidable case stays unknown and goes to the actual worker rather than being rejected.
- **Unchanged limits and clocks.** A rejected candidate keeps its slot, origin and ancestry, counts against the arm's evaluation allowance and earns no replacement draw. The check runs inside the existing arm allowance.
- **Separate reporting.** A preflight rejection is reported separately from a launched fit failure, with its own receipt and zero physical fit starts.
- **No unlimited free retries.** Resume neither retries a proven rejection nor changes the deadline.

The accounting hazard is specific. `PoolBackend::evaluate` records every job-preparation error as `Unavailable`, with zero physical fit starts and `retryable_inventory: true` (`crates/jope-pipeline/src/neural_search.rs:1608-1624`). A naive preflight that returned an error from job preparation would therefore mark a deterministically infeasible recipe as retryable and could repeat preflight work on resume. The investigation instead proposes a typed terminal outcome with accounting: all five folds are checked before any fold of the candidate launches, and the candidate keeps one observation with zero physical fit starts, no worker receipt, no out-of-fold loss and `retryable_inventory: false`. Recovery then verifies the retained preflight receipt without fitting. The closest existing precedent is the zero-fit terminal record for canonical duplicates (`evaluate_unique`, `neural_search.rs:884-925`).

6.4 Warm fitted demonstration after integration

Runbook summary (status as of 2026-09-23 about 23:55 UTC: **pending integration**):

Runbook part	Warm fitted demonstration
Purpose and status	Fit, save, reload and predict in one timed invocation: a generic mode with one registered recipe, and an authenticated ASM mode with paired arms on one frozen regression dataset. **Pending integration:** `demo` belongs to the stopped product overlay and is absent from the immutable `68f56404` CLI (§4.1.2 Command surfaces and the stopped integration order)
Prerequisites and resources	Generic mode: the allowance stated in §6.4.1 Generic and ASM demonstration commands. ASM mode: the reviewed CLI built with `asm`, the worker build inventory, qualified context and policy workers, the runtime inventory for the installed executable, and a resource declaration; the product guide's example declares 20 workers and CPUs, 64 GiB, zero swap, no GPU tasks, 60 seconds in total with 5 for reporting, and 30 seconds per fit
Manifest-creation command	Generic mode writes its own private plan inside its output directory. ASM mode uses `benchmark plan` and then `optimize proof-plan`, both present at `a75e543d2`. No `jope` command writes the warm resource declaration or the warm-only ASM configuration

Runbook part	Warm fitted demonstration
Execution	The `demo` commands in §6.4.1 Generic and ASM demonstration commands
Artifacts and success	Generic: the private plan, worker execution journal, `fitted.json`, `predictions.json`, `baseline.json`, `demo-result.json` and `invocation-receipt.json`. ASM: `asm-fitted.json`, `generic-fitted.json`, their predictions, `baseline.json`, the complete `origin-proof.json`, `asm-demo-result.json` and `invocation-receipt.json`. Success requires saved complete fitted state, unchanged predictions after reload and finite comparisons; ASM mode also requires two full equal arm allowances, genuine neural-origin fits and evolution, and both selected refits. `under_60_seconds` (and, with the warm-target add-on, `target_met`) is true only for a successful full invocation measured below 60 seconds
Failure and resume	Failed attempts keep their failure and execution evidence. The pending guide describes no resume of an interrupted demonstration: a new attempt uses a new output directory outside Git checkouts and `/tmp`, and cannot reuse an expired origin-proof journal. Missing or clipped arm time sets `matched_comparison: false` and fails the ASM demonstration
Limitations	One dataset: not a thirty-dataset result, a held-out quality claim or training evidence. Synthetic mode is labelled. Optional exports stay `emitted-unqualified` or explicitly unsupported. No speed claim exists until a run is measured

Source: pending `docs/private-install-demo.md:90-203` and `crates/jope-bench/src/panel/demo_asm.rs:373-459` in the product overlay replay (`agent-product-final-68f56404/replay-001/source-text` under the ASM custody root).

6.4.1 Generic and ASM demonstration commands

The pending generic demo fits one task-compatible registered recipe, saves complete private state, reloads it, verifies unchanged predictions and compares against a train-mean or train-prior baseline. It uses one CPU/worker, 2 GiB total, zero swap/GPU, 30 seconds per fit and 60 seconds total with five seconds reserved for reporting. Its synthetic mode is explicitly labelled; it is not evidence from a real panel dataset.

```
"$J" demo --dataset "$DATASET_ID" --manifest "$M" --datasets "$D" \
  --source-sha "$S" --output "$A/demo-real"
"$J" demo --synthetic multiclass --source-sha "$S" \
  --output "$A/demo-synthetic" --export python
```

The authenticated ASM route consumes a separately prepared single-regression proof plan:

```
"$J" demo --asm-plan "$WARM_PLAN" --asm-plan-sha256 "$WARM_PLAN_ID" \
  --manifest "$M" --datasets "$D" --output "$A/demo-asm" \
  --export python --export rust
```

6.4.2 The under-sixty-second target and historical timing

The warm-target add-on treats under 60 seconds as a declared, measured goal, not a promise and not a cap that prevents a required real fitted demonstration. Both complete equal arm allowances must fit within the declared original execution window. Unequal, missing or globally clipped arms fail the matched-comparison condition. An explicitly admitted longer allowance can produce a successful complete execution with `target_met: false`; it cannot claim under-60 acceptance. It does not change the official panel's 120-second arm defaults.

The retained proof at source `3fd7d5b2` started the policy about 22.1 seconds into its original window, spent about 20.8 seconds in policy production and started the first fit around 50.4 seconds. Those are historical measurements, not a forecast or a proof that equal 20-second warm arms work. A suggested longer configuration is a sizing proposal until explicitly admitted. Neither a generic substitute nor clipping the second arm solves this requirement.

6.5 Focused single-dataset search

Status as of 2026-09-23 about 23:55 UTC: the bounded one-dataset proof front door below is **available** at `a75e543d2` for a regression dataset already in the frozen panel; the sustained, incumbent-aware attack is **proposed** (§6.5.2 What is missing for a sustained, incumbent-seeded dataset attack).

6.5.1 Commands that exist today for one dataset

Runbook summary:

Runbook part	One-dataset focused proof
Purpose and status	Run one bounded paired neural and generic search on one frozen-panel regression dataset and recompute an authenticated development report. **Available** at `a75e543d2`; measured in the cited `68f56404` proof on `230_machine_cpu`; this is not a count of all earlier origin runs (§1.5.2 Canonical proof record and per-arm facts)
Prerequisites and resources	The inputs in §6.5.1.2 Required inputs; no exclusive full-CPU CI running; a configuration and resource declaration within the implementation caps (an arm of at most 120 seconds, 80–1000 evaluations, at most 30 seconds per fit). The measured proof declared 20 workers and CPUs, 64 GiB, zero swap, no GPU tasks and a 600-second window including 30 seconds for reporting
Manifest-creation command	`benchmark plan` writes the base (comparator) plan (§4.4.2 Native roster and external comparator runtimes); `optimize proof-plan` writes the focused plan and prints its semantic `plan_sha256`. No `jope optimize` subcommand writes the complete `jope.asm-configuration/v1` configuration or the resource declaration: the origin proof's configuration came from the custody helper `agent-neural/prepare-origin-config.py` (receipt `configuration-preparation.json` beside that proof), which is not a product command. The frozen panel manifest itself is fixed, not generated
Execution	`optimize run` (§6.5.1.3 Plan, run and report a focused proof); it refuses a running binary that differs from the pinned native runtime

Runbook part	One-dataset focused proof
Artifacts and success	`plan.json`, `plan.selection.json` and `plan-receipt.json`; the journal (window, campaign and per-arm records, started and finished worker receipts); the report returned by `run` and recomputed by `optimize report` without fitting. Success is an authenticated report of both arms inside the original window, not a neural win
Failure and resume	The same `run` resumes the journal within its original window; an expired window cannot be renewed. Failed and refused fits, such as dense-width refusals (§6.3 Dense-width refusals and the proposed feasibility preflight), remain recorded outcomes
Limitations	One regression dataset already in the frozen panel; no `--incumbent` input; no arbitrary-dataset ingestion; development-hold scores only; the dense-width cap is checked only inside launched fits

Sources: `crates/jope-bench/src/panel/optimize_cli.rs:48-89` and `crates/jope-bench/src/panel/campaign_cli.rs:130-208` at `a75e543d2`; custody `neural-integration/origin-proof-68f56404…/configuration-preparation.json`.

6.5.1.1 Front door and safe metadata checks

The current private Linux `asm` build supports a **separately bounded operational proof on one frozen regression dataset**. This is the closest current neural search front door to focused optimization. It is not an arbitrary-dataset ingestion command, a daemon, or an incumbent-seeded high-water-mark attack.

The following metadata commands are safe preparation and were checked against the immutable reviewed 68 CLI:

```
"$JOPE" optimize capabilities
"$JOPE" optimize proof-plan --help
"$JOPE" optimize run --help
"$JOPE" optimize report --help
```

`compiled:true` means support was compiled, not that assets, context qualification or training have passed. Use a binary whose build inventory, source manifest and executable hashes match the configuration. The implementation at `a75e543d2` caps an arm at 120 seconds and its candidate allowance at 80–1000 evaluations (`crates/jope-bench/src/panel/optimize.rs:132-133`). Per-fit time remains at most 30 seconds (`crates/jope-bench/src/panel/origin_proof.rs:86`). Editing a JSON number beyond those caps does not extend the implementation.

6.5.1.2 Required inputs

Variable	Meaning
`JOPE`	Authenticated Linux CLI built with `asm`; not an unrelated binary found on `PATH`
`BASE_PLAN` / `BASE_PLAN_ID`	Verified comparator plan and its emitted semantic identity
`ASM_CONFIG` / `ASM_CONFIG_SHA`	Complete qualified ASM configuration and its file SHA256
`FOCUS_RESOURCES` / `FOCUS_RESOURCES_SHA`	Explicit resource-window declaration and its file SHA256

Variable	Meaning
`PANEL` / `PREPARED`	Unchanged frozen panel manifest and verified prepared development data
`FOCUS_DIR`	New durable private output directory given as an absolute path, outside Git working trees and `/tmp`; `optimize run` refuses a relative journal path
`DATASET`	One regression dataset already present in the frozen panel, such as `230_machine_cpu`

The base plan and complete configuration are prepared as described in §4.4.2 Native roster and external comparator runtimes and §2.7.1.3 Qualification and admission. They include real runtime/worker inventories, checkpoint/vocabulary/transform assets and context qualification. Hand-writing a short configuration with invented hashes cannot replace those artifacts. The resource declaration includes workers, CPUs, memory, zero swap, GPU task allowance, threads per worker, total/reporting seconds and the per-fit cap.

6.5.1.3 Plan, run and report a focused proof

Prepare a plan before admitting numerical work:

```
mkdir -m 700 -- "$FOCUS_DIR"
"$JOPE" optimize proof-plan \
  --base-plan "$BASE_PLAN" --base-plan-sha256 "$BASE_PLAN_ID" \
  --configuration "$ASM_CONFIG" --configuration-sha256 "$ASM_CONFIG_SHA" \
  --dataset "$DATASET" \
  --resources "$FOCUS_RESOURCES" --resources-sha256 "$FOCUS_RESOURCES_SHA" \
  --output "$FOCUS_DIR/plan.json" > "$FOCUS_DIR/plan-receipt.json"

FOCUS_PLAN_ID=$(jq -er '.plan_sha256' "$FOCUS_DIR/plan-receipt.json")
```

Planning reports `execution_budget_started:false`. After the declared allocation is available and exclusive full-CPU CI has finished, the execution command is:

```
"$JOPE" optimize run \
  --plan "$FOCUS_DIR/plan.json" --plan-sha256 "$FOCUS_PLAN_ID" \
  --manifest "$PANEL" --datasets "$PREPARED" \
  --journal "$FOCUS_DIR/journal"
```

The same `run` command resumes that journal within its original window. Do not change directories, timestamps or identities to disguise an expired run as a resume. Recompute the report without fitting:

```
"$JOPE" optimize report \
  --plan "$FOCUS_DIR/plan.json" --plan-sha256 "$FOCUS_PLAN_ID" \
  --manifest "$PANEL" --datasets "$PREPARED" \
  --journal "$FOCUS_DIR/journal"
```

The command audit for this chapter executed only metadata/help operations. These example execution commands were not run as a new campaign while writing the manual. The separately measured origin proof at source `68f56404` (§1.5.2 Canonical proof record and per-arm facts) remains the actual numerical example.

6.5.1.4 Other current search surfaces

`jope search --input "$FIXTURE" --json` scores a tiny ridge-family candidate bank. It is useful for its documented fixture scope; it is not sustained neural GP exploration.

`jope pipeline run` accepts a catalog dataset or dataset directory, cuts, seeds and a budget, and can emit several languages. Its `--policy auto` and `skip` report a skip, while `require` fails closed. Its legacy budget description explicitly allows an in-flight fit to overrun. Therefore neither increasing its attempt count nor choosing `--policy require` supplies the current bounded neural learning path. See §10.4.2 Example B: actual legacy pipeline/search shape for the actual legacy pipeline example and its receipt.

6.5.2 What is missing for a sustained, incumbent-seeded dataset attack

There is no current `--incumbent` input on `optimize proof-plan`, and no shipped `jope optimize attack` command. A finite, wave-by-wave attack interface is a proposed follow-on. Documenting a plausible command string would not make it available.

The owner's rules for that follow-on (relayed 2026-09-23T22:27:11Z; status: proposed): focused optimization is finite and incumbent-aware and runs inside one total window declared before searching. Before the first wave it freezes the exact rows, groups, evaluator, vocabulary and numerical identity, the authenticated initial population, the exploration and wave rules, and the atomic update rule. A tie keeps the incumbent, and a race with a concurrent incumbent change forces a re-decision rather than an overwrite. Ingesting an arbitrary dataset is a separate, authenticated request, never an edit of the frozen thirty-dataset panel.

An implementation should reuse the current worker pool, selection plan and journals, and integrate the pending eligibility overlay (TEACH-01/TEACH-02) only after it is qualified, with these specific additions:

Addition	Required behavior
Frozen focus request	Authenticate one dataset, task, exact fold/group/row identity, evaluator, vocabulary, numerical policy and declared total window before searching
Eligible incumbent reader	Read the incumbent under the complete comparison identity; refuse a stale, incompatible or unauthenticated record
Incumbent-aware initialization	Authenticate the complete initial population; include the incumbent and bounded mutations/macros with explicit, declared origins; preserve an exploration fraction and canonical duplicate accounting
Wave accounting	Reuse the original global deadline and journal; each bounded wave consumes remaining time rather than resetting it
Re-evaluation and admission	Use complete finite train-side OOF evidence; meet the required trivial-baseline/4-of-5-fold gate, then compare against the incumbent under the same identity
Atomic improvement	Strict improvement moves the pointer only after durable evidence; ties keep the incumbent; concurrent pointer changes trigger re-decision
Useful teaching delta	Export newly eligible programs and justified comparison records with original context/labels/exposure, rather than copying a winner's reported score

Addition	Required behavior
Honest completion	Report best retained incumbent, all waves and costs, improvement/stagnation, failures and unmet backend support even when no new winner appears

The original acceptance requires four-of-five-fold improvement over a trivial baseline. A proposed extra four-of-five-fold comparison against the incumbent is a separate robustness policy, not a shipped requirement or retrospective reason to reject old evidence. Freeze that policy before running a focused extension. The current HWM strict-improvement rule also does not by itself prove the new teaching eligibility contract.

For a dataset outside the frozen panel, first implement or use a separate authenticated ingestion/preparation request with the same label, category, missingness and group guarantees. Do not edit the frozen panel's manifest (asm-private-30-v1) to add it. Repeated comparisons use development labels; sealed evaluation remains outside search, teaching and motif mining.

CHAPTER 7

Teacher admission, HWM identities and corpus updates

IN THIS CHAPTER

7.1 A dataset is not one training example 77
7.1.1 Accounting units 77
7.1.2 Context vectors and program targets 78
7.1.3 The learning chain 78
7.2 How a fitted recipe becomes an eligible teacher 79
7.2.1 The preregistered freeze and train-side inner split 79
7.2.2 The admission guard, label exposure and HWM identity 80
7.3 From optimization to reusable knowledge 80
7.4 Present teacher readiness and the smallest real path 81
7.5 Prepare a teaching campaign before it starts 81
7.5.1 Final build and the teaching configuration 82
7.5.2 Plan, run and recover a teaching campaign 83
7.6 Corpus updates: reuse, device caches and loading 84
7.6.1 On-disk shard reuse 84
7.6.2 The two device caches in the native trainer 84
7.6.3 Identifier memory and whole-corpus cache invalidation 85
7.6.4 Python training input and the pending bounded campaign 85
7.7 Concrete corpus improvements, with acceptance criteria 86

7.1 A dataset is not one training example

7.1.1 Accounting units

A training context describes a particular dataset, split and encoding. Many executable programs can share that context. Each program supplies a separate target sequence and measured supervision. The useful accounting units are therefore:

Unit	Meaning	Why the distinction matters
Dataset/family	A problem and its related datasets	Family partitions control generalization; row counts alone do not identify independence.
Cut/context	A dataset split plus prepared-data and encoder identities	The same dataset can have many contexts. Reusing a context must preserve its actual split and fitting rows.
Program observation	A measured evaluation of a concrete executable recipe	Failed, duplicated, incompatible or exposed observations need not become training examples.

Unit	Meaning	Why the distinction matters
Admitted target record	A representable program and its supervision for one context. Under the pending contract, a **teacher** is such a record that passed the `jope.teaching-guard/v1` admission (§7.2.2 The admission guard, label exposure and HWM identity) and entered a frozen corpus	Repeated observations of the same program/cut do not automatically earn repeated votes.
Corpus snapshot	An immutable collection of target records and identities	Its record count is not the number of optimizer updates or unique datasets.
Presentation	A sampled training example used in an update	Sampling revisits records. Millions of presentations do not mean millions of newly discovered targets.

7.1.2 Context vectors and program targets

The legacy Rust context is 2,048 little-endian float32 values, or 8,192 bytes. The model receives a 1,536-value dataset lane and a separate 512-value target-statistics lane. The historical encoder recipe described the dataset lane as Hyperion 512 plus Foundation/PCA 1,024. Despite its name, the dataset lane is not label-free: in the current context producer the 1,024 Foundation values are computed from the features together with the standardized training targets, and only the 512 Hyperion values use the features alone (§2.7.1.2 Component dimensions and assembly). These vectors are shared by context identity; they are not repeated inside every 532-byte target record. The program target is a grammar-token sequence, with the legal-token set for each step, context ID, score-derived value, percentile and log-runtime supervision.

A concrete metadata binding in the accepted step-60000 corpus connects dataset `031201c6895f724b`, family `rdatasets-agridat-kalamkar-potato-uniformity`, cut 0 and `meta_train` to a prepared view and context vector. Its context SHA begins `6f5fdd79…`, prepared-view SHA `811feabd…`, and content-addressed shard contains 24,231 records. The full paths and hashes are in `accepted-corpus-binding.json`. This audit verified that metadata relation without opening the encoded vector or target records.

7.1.3 The learning chain

The learning chain is:

1. Pin the raw dataset, family partition and actual fitting/development row assignments.
2. Fit preprocessing and encoding only within the permitted input closure; pin their bytes and versions. The closure covers records, contexts, transforms and every asset, and a forbidden descriptor must be rejected before loading even when no admitted row uses it. At the retained PR83/stopped-overlay snapshot, that rejection is **proposed**: neither source rejects extra context-index entries, and the legacy trainer at `a75e543d2` stages every indexed context after only a digest and size check (§5.5.2 Hashes, eligibility and the complete input closure).
3. Generate a concrete executable program, fit it and retain its outcome, measurement, resource use and evaluator identity.
4. Apply the declared admission policy; reject failed, incompatible, exposed or unsupported targets.
5. Tokenize the exact program and retain original observations beside the training representation.
6. Sample the admitted record, update parameters, and prove checkpoint reload/continuation.
7. Evaluate the new policy through actual proposals and fits under the comparison protocol.

A file at step 5 does not prove steps 6–7. Conversely, a lower optimizer loss does not establish better executable proposals.

The chain yields four distinct claims, and the owner requires them to stay separate (relayed 2026-09-23T22:27:11Z). Each needs its own evidence; none implies the next:

Claim	Established by	Not established by
Eligible teacher	A record that passed the preregistered `jope.teaching-guard/v1` admission under its complete comparison identity (step 4)	A content hash, a saved recipe, a completed run or a historical HWM entry
New discovery	A program absent from the authenticated initial seed population of the search that found it, with its discovering arm, generation and first measurement retained (§9.7.2 What to retain for each cycle)	A carried incumbent, a re-measured seed or a duplicate of an existing record
Actually consumed information	Sampler and optimizer receipts showing the record sampled in committed optimizer updates (step 6); changed parameters are a separate LOOP-01 link (§9.5 LOOP-01: the central regression milestone)	Corpus membership or a manifest row
Improved policy	A matched comparison of parent and challenger checkpoints with actually fitted proposals under the same protocol and budget (step 7; §9.6 The closure ledger)	A lower training loss, a new checkpoint file or the teacher's own development score

For the new ASM path none of the four is established yet: zero teachers have been admitted under the new contract (§7.4 Present teacher readiness and the smallest real path), and the pending campaign records no initial-population or discovery identity. Reporting the four separately is a **proposed** rule.

7.2 How a fitted recipe becomes an eligible teacher

7.2.1 The preregistered freeze and train-side inner split

The pending eligibility path freezes independent facts before optimization: original dataset and prepared-data identity, ordered rows and groups, target role, exact inner-fold assignment, evaluator and vocabulary identities, numerical policy, baseline and worker/resource pins. This freeze cannot be created retrospectively from a winning candidate's own output. A historical successful fit is not equivalent to preregistered teaching evidence.

The train-side inner split uses five whole-group folds. Groups are assigned deterministically by largest size first, with lexical tie-breaking and balancing. No group's rows are split simply to fill a fold. The cut's held-out development fold remains outside the search's inner fitting/selection closure.

7.2.2 The admission guard, label exposure and HWM identity

Each candidate must produce complete authenticated OOF predictions for all five inner folds. Scores are recomputed from retained predictions, exact targets and ordered class probabilities. The trivial baseline is fitted from each fold's training rows: target mean for regression, class frequencies for classification. The guard uses mean fold loss and admits only if:

- all five fold losses and baseline losses are finite;
- the mean candidate loss is below the mean baseline loss by more than `1e-12 + 1e-6 * abs(mean_baseline_loss)`;
- at least four of five folds each improve by more than `1e-12 + 1e-6 * abs(that fold's baseline_loss)`;
- actual worker requests, process outcomes, output hashes, fitted artifacts, seeds, source, resources and deadlines match the original freeze.

The criteria are intentionally stronger than “a run completed” or “the score field is positive.” Constant/no-signal predictors, missing folds, reordered rows, changed class order, stale scores, unauthenticated prediction blobs and unknown execution accounting cannot provide teaching authority. A failed admission is retained as evidence rather than discarded. A hash is not eligibility: a content digest proves that bytes are unchanged, not that the record passed this guard under the right comparison identity with permitted labels. In the pending overlay, verifying a retained admission report re-derives the qualification from the learning state (`eligibility::revalidate_qualification`) and refuses any mismatch, rather than trusting a stored `qualified` flag (pending `crates/jope-bench/src/panel/teaching.rs:730-736`).

Optimization-visible labels initially called `test` become development labels, with a durable record of that exposure. `sealed` and `meta_test` inputs are refused for optimization/teaching. The HWM identity includes the full comparison context, so an improvement under a different evaluator, split, vocabulary, preprocessing or numerical protocol cannot displace an unrelated incumbent. These details belong to the pending eligibility/HWM correction, not a claim that every historical HWM record already meets this rule.

Sources: pending `crates/jope-learning/src/eligibility.rs:170`, `:253`, `:277`, `:427`, `:700`, `:1023`; `crates/jope-learning/src/eligibility/inner.rs`; `crates/jope-learning/src/record.rs`; `crates/jope-pipeline/src/search_plan.rs`.

7.3 From optimization to reusable knowledge

The pending strict HWM key includes task, dataset hash, cut identity, evaluator, vocabulary and protocol. Scores from different identities cannot overwrite each other or supply training feedback. A recipe must show finite train-side out-of-fold improvement over a trivial baseline and improve on four of five folds before teaching admission. No-signal data should be refused by this gate. An optimization-visible test label is development evidence even if its filename says otherwise.

Typed motifs can reuse eligible program fragments and macros while retaining provenance. The prepared motif library does not yet establish the production eligible-state reader, `motifs mine` integration or operational reuse. A new operator requires compatibility tests and an ablation under the unchanged comparison protocol. Repeatedly selecting the best development result can improve development performance without proving improved sealed generalization.

7.4 Present teacher readiness and the smallest real path

The metadata readiness audit verified 28 source-family evidence files and the retained original process/context receipts. The inspected historical origin runs at `55e9ed688faf` and `3fd7d5b2ebb6` had no preregistered teaching freeze or vocabulary pins; they cannot be upgraded after the fact. The operational origin proof at `68f56404` (the reviewed PR83 head, merged on 2026-09-23 as main `a75e543d2` with the identical tree) also was not a teaching campaign. There is therefore no currently demonstrated eligible real corpus for the new ASM admission-and-training proof. This does not negate the historical training corpus or the optimizer work that produced the retained model.

The prospective small pair selected from frozen provenance/size, before observing losses, is:

Dataset	Evidenced family	Cut 0 fitting rows (development fold 0 held out)
`230_machine_cpu`	UCI computer-hardware relative-performance source	135
`712_chscase_geyser1`	Chatterjee casebook geyser source	140

Both must independently produce eligible recipes. The minimum of ten completed inner fits is only the five-fold×two-family floor for one successful recipe per family. It is not a replacement for the specified matched search and does not promise that either family will improve.

If exactly these two families qualify, seed 7 assigns `230_machine_cpu` to the policy-validation families and `712_chscase_geyser1` to the policy-training families under the current partition rule, leaving a single training family. If more families qualify, use the entire admitted set and let the native family partition determine membership. Do not omit inconvenient eligible comparisons, force the two-family allocation, or reuse the frozen panel's (`asm-private-30-v1`) deadline for a separate trial. Historical checkpoint exposure remains a separately reported unknown; family provenance alone does not prove zero prior exposure. Keep three family-level domains distinct: teacher and training families; adaptive development families, where the frozen thirty-dataset panel remains; and evaluation families outside both training and selection. Exposure of the pretrained encoder components to any family also remains unknown.

Both prospective families are small regression datasets of the frozen panel itself (169 and 172 development rows, §A.1.1 Per-dataset summary); no real binary or multiclass family is prospective. With exactly these two families, the first training campaign is a **mechanics qualification**: it can show optimizer updates, reload, continuation and consumption of a new checkpoint, but it cannot test whether training improves proposals, and its export must not be compared with step 60,000 as a flywheel challenger. That campaign trains experimental regression lineage v1 (256×2, new initialization), a width-256, two-layer model family separate from the accepted 512×6 step-60000 lineage (§8.4.1 What the pending campaign trains). If either dataset is used to manufacture teachers, later panel evaluations on it are labelled exposed/adaptive (§9.7.1 Freezing the experiment and choosing families); see also OD-8 in §14.1.4 Proposed owner decisions and external dependencies.

Source evidence: custody `agent-training-readiness/readiness-v3.json`, SHA256 `9197900d4b696811909c4ba4e779e6c5d488569dd9c360165cc55fd43bf8c735`. Its metadata inspection does not itself admit training or re-run numerical qualification.

7.5 Prepare a teaching campaign before it starts

Runbook summary (status as of 2026-09-23 about 23:55 UTC: **pending integration**):

Runbook part	Teaching campaign
Purpose and status	Preregister train-side teaching freezes, run a bounded search, and admit each arm's selected recipe into the learning state only if it passes `jope.teaching-guard/v1`. **Pending integration:** the preregistration and admission code belongs to the stopped training64 overlay, and the configuration at main `a75e543d2` has no teaching member. Zero teachers have been admitted
Prerequisites and resources	The final reviewed build and its receipts (§7.5.1 Final build and the teaching configuration); a `teaching` member whose `state_root` and `prepared_root` are absolute, whose state root lies outside `/tmp` and outside Git checkouts, whose panel file is the approved frozen manifest and whose label role maps to development; a separately declared, approved resource allocation; no exclusive CPU CI running
Manifest-creation command	`benchmark plan` writes the comparator plan; `optimize plan` or `optimize proof-plan` writes the ASM plan and, in the pending overlay, preregisters the `jope.inner-teaching-registry/v1` freezes before the final experiment identity. No `jope optimize` subcommand, at `a75e543d2` or in the pending overlay, writes the complete configuration or its `teaching` member
Execution	`optimize run` (§7.5.2 Plan, run and recover a teaching campaign). Admission runs inside the controller before outer-refit scheduling and consumes completed receipts only; it starts no fit and no clock
Artifacts and success	In the journal, a `jope.inner-teaching-admission/v1` report at `teaching-admission/<key>.json` when every selected arm's entry qualified or the registration was unavailable, otherwise a `teaching-inventory/<key>-<hash>.json` record. Success is at least one qualified record per family, re-verified from the retained evidence
Failure and resume	`optimize run` resumes the original journal within its window. A non-terminal admission failure is inventory that revalidation may retry within the original experiment; a failed admission is retained, never deleted
Limitations	The two prospective families are frozen-panel regression datasets, so the first campaign is a mechanics qualification (§7.4 Present teacher readiness and the smallest real path); admission considers only each arm's selected recipe; the real admission path has not run

Sources: pending `crates/jope-bench/src/panel/teaching.rs:48-67`, `:123-125`, `:537-623`, `:754-756` and `crates/jope-bench/src/panel/optimize_cli.rs:13-53` in the training overlay replay (`agent-training-integration/refresh-68f56404/replay-001/source-text` under the ASM custody root).

7.5.1 Final build and the teaching configuration

The final reviewed build must supply its CLI, worker inventory, source manifest, runtime manifests, component assets and qualification receipts. Record the actual built source SHA; do not substitute the latest checkout hash for the binary's source. Recheck that no exclusive CPU CI is running before opening an authorized CPU campaign.

In the pending overlay, the `teaching` member of the complete ASM configuration has this shape (the configuration at main `a75e543d2` has no teaching member):

```
{
  "state_root": "/durable/private/learning-state",
  "prepared_root": "/durable/private/prepared-panel",
  "panel": {"path": "/source/benchmarks/panel.json", "sha256": "EXACT_PANEL_FILE_SHA256"},
  "original_label_role": "development"
}
```

The capitalized value is a placeholder, not a valid digest. The complete configuration also requires exact source/build/runtime/checkpoint/vocabulary/context fields; the small object above cannot be passed as the whole configuration. Planning preregisters the teaching freezes before final experiment identity, so worker evidence can bind them.

7.5.2 Plan, run and recover a teaching campaign

The following shell variables must already name authenticated artifacts. Commands are interface examples and allocate no budget merely by appearing here:

```
"$JOPE_FINAL_CLI" benchmark plan \
  --manifest "$JOPE_PANEL" --datasets "$JOPE_PREPARED" \
  --source-sha "$JOPE_FINAL_SOURCE" --runtimes "$JOPE_RUNTIMES" \
  --output "$JOPE_CAMPAIGN/comparator-plan.json"

"$JOPE_FINAL_CLI" optimize plan \
  --base-plan "$JOPE_CAMPAIGN/comparator-plan.json" \
  --base-plan-sha256 "$JOPE_BASE_PLAN_ID" \
  --configuration "$JOPE_TEACHING_CONFIGURATION" \
  --configuration-sha256 "$JOPE_CONFIGURATION_FILE_SHA" \
  --output "$JOPE_CAMPAIGN/asm-plan.json"
```

Use the semantic plan identity emitted by native planning for `--base-plan-sha256` and `--plan-sha256`. A JSON file's byte hash and its semantic identity need not match. Configuration/resource arguments explicitly refer to their required file hashes.

For a separately admitted small proof, substitute this planning command. `proof-plan` selects one regression dataset from the verified frozen base panel, replaces the resources with the explicitly pinned proof resource manifest, and preregisters teaching only for that selected dataset:

```
"$JOPE_FINAL_CLI" optimize proof-plan \
  --base-plan "$JOPE_CAMPAIGN/comparator-plan.json" \
  --base-plan-sha256 "$JOPE_BASE_PLAN_ID" \
  --configuration "$JOPE_TEACHING_CONFIGURATION" \
  --configuration-sha256 "$JOPE_CONFIGURATION_FILE_SHA" \
  --dataset "$JOPE_SELECTED_DATASET" \
  --resources "$JOPE_PROOF_RESOURCES" \
  --resources-sha256 "$JOPE_PROOF_RESOURCES_SHA" \
  --output "$JOPE_CAMPAIGN/asm-plan.json"
```

A second family requires its own actual preregistered proof plan and journal, with its separately declared approved resource allocation. Both may point at the same learning state and frozen30 metadata. Neither proof changes or spends an unstarted official panel's immutable window. Proposed example durations in earlier queue documents were not resource admissions.

Run and recover using the same immutable plan and original journal:

```
"$JOPE_FINAL_CLI" optimize run \
  --plan "$JOPE_CAMPAIGN/asm-plan.json" --plan-sha256 "$JOPE_ASM_PLAN_ID" \
  --manifest "$JOPE_PANEL" --datasets "$JOPE_PREPARED" \
  --journal "$JOPE_CAMPAIGN/journal"

"$JOPE_FINAL_CLI" optimize report \
  --plan "$JOPE_CAMPAIGN/asm-plan.json" --plan-sha256 "$JOPE_ASM_PLAN_ID" \
  --manifest "$JOPE_PANEL" --datasets "$JOPE_PREPARED" \
  --journal "$JOPE_CAMPAIGN/journal"
```

Sources: `crates/jope-bench/src/panel/optimize_cli.rs:13` and `crates/jope-bench/src/panel/origin_proof.rs` (the `optimize plan`, `proof-plan`, `run` and `report` front door is present at main `a75e543d2`, where it makes no teaching call; the pending overlay adds the `teaching::preregister` call to `optimize plan` and `proof-plan` and the new `crates/jope-bench/src/panel/teaching.rs` that defines it); current `crates/jope-bench/src/panel/campaign_cli.rs:18`.

7.6 Corpus updates: reuse, device caches and loading

7.6.1 On-disk shard reuse

The on-disk teaching store can reuse unchanged verified shards **within an unchanged store binding**. In reviewed source `68f56404` (merged as main `a75e543d2` with the identical tree), a group's key includes binding, context, programs and lineage. The binding includes twelve semantic source files and the SHA256 of `/proc/self/exe`. A changed packer executable digest therefore rejects the old store binding and changes every group key, requiring a new store/full disk repack even if the logical records are unchanged. A byte-identical reproducible rebuild does not change that digest. This disk invalidation is separate from GPU cache invalidation. New ledger contributions mark affected groups dirty; changed or reordered historical source inputs require reconciliation or a new store rather than silently editing history. Publication uses immutable artifacts and atomic replacement of the published index. Tokenization caches are bounded and exact; invalid exact parameters remain rejected.

Snapshot 97's historical `packing-timing.json` records **7,859 reused groups and 18 rebuilt groups**, from 51 new ledgers. Attribute these figures to its legacy writer: the retained receipt has the four-field legacy format, whereas that source writes seven fields and has the stronger executable/semantic/lineage binding. The old reuse observation is not a qualification of the current writer. That is 99.7715% reuse by group count, not a measured 99.7715% reduction in bytes, CPU or latency. The September 11 snapshot 421 audit recorded 7,863 reused and 14 rebuilt groups. Neither receipt by itself measures end-to-end pack time or memory. The accepted snapshot's 7,877 record files all exist at the declared lengths in this metadata audit, totaling 5,434,024,624 bytes; their label sidecars add 3,227,289,461 bytes. Sidecars preserve auditability but are not free storage.

7.6.2 The two device caches in the native trainer

The native trainer implementation maintains two different device caches. Its current direct dispatch remains held behind the original-GPU-grant requirement described in §8.2.4.2 The dispatch hold, resource incompatibility and valid topologies; the following source behavior is not permission to launch it:

Cache	Reuse identity	Behavior when identity changes
Context vectors	Context-index SHA	Release prior tensor; read/hash every indexed 2,048-float context and stage the new table

Cache	Reuse identity	Behavior when identity changes
Token/legal/context-ID tensors	Whole corpus SHA	Release prior tensors; traverse **every** target record and rebuild the full tables

Tokens and legal IDs are each stored on the device as 16-bit signed tensors with all unsigned bits preserved; after gathering a minibatch, the code converts IDs to int64 and masks with `0xffff`. Context IDs are int64. The legal mask table is shared and boolean. For each minibatch, the sequence length is trimmed to the longest selected record, rounded up to a multiple of eight and capped at 128. Thus the implementation already avoids recomputing grammar masks per optimizer step and reduces unnecessary sequence work within a batch.

7.6.3 Identifier memory and whole-corpus cache invalidation

For 10,214,332 records, token-plus-legal identifiers alone are `N × 128 × 4 = 5,229,737,984` bytes. Context IDs add `N × 8 = 81,714,656` bytes. The 8,894 shared float32 contexts would occupy 72,859,648 bytes. These are layout calculations, not measured GPU peaks; they exclude the legal mask table, model, optimizer, activations, batch conversions, allocator overhead and host staging. The emitted `resident_identifier_bytes` counter covers only the token/legal identifier term, not complete device use.

A newly published corpus SHA invalidates the entire token/legal/context-ID cache even when most disk groups were reused under an unchanged binding. The historical 7,859-of-7,877 receipt illustrates disk reuse under the legacy writer; a changed current packer executable additionally forces full disk repacking as described above. There is no qualified incremental device-shard cache in this path. Disk-level incremental packing therefore does not guarantee low-latency continual training. Repeated trainer processes also cannot rely on process-local tensor residency. This is a concrete optimization opportunity to measure before changing the format.

7.6.4 Python training input and the pending bounded campaign

The current Python `train_v3.py` path accepts JSONL `TrainingRow` objects or an in-memory list. It materializes the input rows, partitions them before grammar/reward preparation, and caches each train/validation row's token IDs and sparse per-position legal-ID lists once before the epoch loop. It still rebuilds a dense boolean legal mask of shape `[batch, sequence − 1, vocabulary size]` on the host for every batch before moving it to the device. It also supports a vocabulary-bound persistent row-artifact cache. It is inaccurate to say that Python reparses or retokenizes every example on every optimizer step.

However, the pending `training64` campaign's `admitted_rows` path still reads JSONL into memory and writes a per-attempt `corpus.jsonl`, then invokes `train_policy_v3_from_jsonl`. That stopped source packet does not add a reader for the Rust `teaching_shards.v1` packed store. Its stricter context/task/admission checks must be integrated and qualified before any real category training. The existence of an efficient historical Rust corpus does not prove that the pending Python route can ingest ten million examples within the current twenty-minute category window or 64 GiB host limit. No such performance run was authorized or performed for this chapter.

7.7 Concrete corpus improvements, with acceptance criteria

The useful next change is evidence-led rather than another format by name. These are proposals (status: **proposed**), not implemented delivery claims. The owner's order (relayed 2026-09-23T22:27:11Z) is: profile the phases; then a verified Python packed-reader bridge; then immutable shard reuse with bounded staging; then compression; and a narrower record layout only if measurements justify it. Every step preserves float32 supervision, the legal-set mappings and the 128-token grammar limit, and proves record, batch, loss, gradient, update and resume parity (§D.7.4.2 Compression, the proposed 180-byte layout and label precision).

1. **Measure the actual preparation path.** Retain separate wall time and peak host/device memory for metadata parsing, admission validation, tokenization, hashing/sealing, shard loading, host construction and device transfer. Measure cold and reused cases on the same admitted corpus identity. Include controller/setup cost; do not equate one identifier-array counter with total memory or CPU.
2. **Give Python a verified packed-reader bridge if those measurements justify it.** Preserve exact tokens, legal sets, float semantics, task/class routing, contexts, splits, original observations and vocabulary/admission pins. Prove JSONL/packed equivalence on admitted fixtures, corruption and changed-input rejection, exact recipe round trips, reload and continuation. A converter that copies old `qualification: true` fields without rechecking the current teacher contract is not sufficient.
3. **Then reuse immutable shards with bounded staging.** Key reusable material by content digest and explicit vocabulary/grammar/context identities; measure only changed shards transferred, and stage them through a bounded, measured host buffer rather than rebuilding the whole table. Global legal-set remapping, context IDs, balanced sampling, frozen labels and optimizer/sampler resume must remain reproducible. Appending raw old IDs into a new global table is unsafe without that mapping contract.
4. **Then evaluate compression.** Current u16 IDs and shared contexts already save space. The accepted-corpus audit (§D.7.4 What the accepted-corpus packing audit measured) gives externally measured compressed byte counts. Compressed archive size is not a loader-memory result; preserve bounded random access, decompressed and stored-byte hashes, corruption rejection and deterministic sampling before adopting it.
5. **Adopt a narrower layout only if measured.** The same audit gives a measured padding opportunity. Length bucketing or a narrower layout still requires parity and measured GPU throughput. Keep the 180-byte record and binary-index estimates labelled as unqualified proposals.
6. **Close the evidence chain before promoting the result.** Prove an eligible new target entered the corpus, was actually sampled, changed parameters, survived reload and improved later bounded executable-search behavior. Keep resource deadlines and sealed boundaries unchanged.

Keep exactly one learning-store writer until an atomic server-side pointer update is qualified; this manual records no such qualification. The on-disk packer already takes an exclusive, non-blocking `writer.lock` on its store directory and refuses a second concurrent packer (`crates/jope-engine/src/teaching_store.rs:377-385` at `a75e543d2`), but that local lock does not coordinate writers of a remote learning store. Backup, replication and retention operations on learning storage remain separately authorized; a corpus update must not start them.

Relevant source contracts are `crates/jope-engine/src/corpus.rs`, `teaching_store.rs`, `sealed_storage.rs` and `training.rs`; Python uses `python/src/jope/schema.py` and `policy/train_v3.py`. Existing source tests cover byte-identical reuse, changed inputs, frozen labels, original-precision midranks, interruption and

immutable verified records. This documentation audit inspected those tests but did not run them. `source-pins.json` binds the inspected source and pending overlay; `HANDOFF.json` binds the chapter and its data.

CHAPTER 8

Training, continuation, export and compatibility

IN THIS CHAPTER

8.1 What can be claimed now 88
8.2 The four continuation names and their commands 89
8.2.1 Names, runbooks and status 89
8.2.2 Rules for exact interrupted resume 90
8.2.3 Variables used in the continuation examples 91
8.2.4 Native lineage B: the held command and its blockers 92
8.2.4.1 The implemented command shape 93
8.2.4.2 The dispatch hold, resource incompatibility and valid topologies 93
8.2.5 Current Python training and a concrete continuation call 94
8.2.5.1 Weight warm start with the existing CLI 94
8.2.5.2 Exact optimizer continuation through the Python API 95
8.2.5.3 The companion continuation helper and the resume artifact 95
8.2.5.4 Limits of the current Python route 96
8.3 Evidence and identity before a bounded campaign 96
8.4 The bounded training campaign chain 97
8.4.1 What the pending campaign trains 98
8.4.2 Build the real corpus from admitted evidence 98
8.4.3 Plan, run, resume, verify and export 99
8.4.4 The pending bounded extension path: sequence and limits 100
8.4.5 Proposed sibling fresh and replay branches 101
8.4.6 Training input packing and the learning store 101
8.5 Consuming a newly trained checkpoint 101
8.5.1 The experimental.regression configuration entry 101
8.5.2 Compatibility checks and fail-closed rules 102
8.5.3 Proving consumption in a new operational proof 103
8.6 Remaining acceptance queue and maintenance risks 104

8.1 What can be claimed now

At source `68f56404c3e199e06413bfd645d3fe0803d7b96f`, the latest bounded regression origin proof has actual neural proposals, fitted generation-zero candidates, fitted descendants and a successfully authenticated recovered report. The newly authored training path is still a stopped follow-on. Its guarded 64-file patch is `595824ef6eb8e15872be058668503cedad1dd9a5ff98ac4700c6f713251d384c`; manifest `b86b550d7ee39fd93d5202accd5f77c975b7bcb4fd8f8e7706c69c5f279294a2` binds every original packet, overlap resolution and current source/API guard.

The follow-on passed custody replay guards, source syntax, pure metadata checks and feature-graph inspection. That evidence does not establish Rust type checking, a GPU optimizer run, a real newly trained checkpoint, or a successful live trained-checkpoint consumption loop. The demonstrated teacher count under this pending ASM admission contract remains zero in the inspected evidence. Historical lineage-B teaching corpora and its accepted trained checkpoint are separate retained assets, described in §D.6 Retained corpus sizes and §B.3.4 Carried state, accepted step 60,000 and RTX step

60,100; they have not been retroactively certified under the new admission contract. Source support, fixture tests, exact-head CI, live qualification and serving promotion must remain separate status fields.

All commands below involving `jope train` or experimental consumer configuration describe the **pending overlay's implemented interface**. They are a maintainer runbook for integration and authorized acceptance, not claims that these commands were executed at main `a75e543d2` (PR83). Preserve this status notice when extracting examples.

Historical interface status as of 2026-09-23 about 23:55 UTC. The implementation snapshot is unchanged: reviewed `68f56404` merged as main `a75e543d2`. At that observation, main had advanced to `bdfc95c63` (PR85), whose changes since `a75e543d2` touch only documentation, `README.md` and the coordination ledger. Training64 remains a stopped patch outside main; no training GPU window has been admitted and no training clock has started. Each runbook in this chapter carries exactly one status label per route: **available**, **implemented but unqualified**, **pending integration** or **proposed**. The owner's edition-2 design requirements appear here as rules or proposals, not as achieved results.

8.2 The four continuation names and their commands

8.2.1 Names, runbooks and status

Every way this chapter starts or continues training has exactly one of four names. Report a run by its name rather than by a command flag, and never describe one kind of run with another's name. §14.2.1 The continuation interfaces that exist today lists the same interfaces at a glance; this chapter remains the operational runbook.

Name	Definition	Runbooks in this chapter, with status
Cold initialization	New parameters from the declared architecture and seed, with a fresh optimizer; no parent weights or optimizer state	Native `jope-parallel train` with neither `--initialize` nor `--resume`, which records initialization kind `fresh`: implemented but unqualified (dispatch held). `jope-py train-policy` without `--init-from`: implemented but unqualified. The pending campaign's first pass when its inputs name no initialization checkpoint: pending integration.
Weight warm start	Parent weights only; the optimizer starts fresh	Native `--initialize` with no replay policy (recorded as fresh FP32 AdamW, with the new corpus starting at step zero) or with a policy whose `optimizer` is `reset`: implemented but unqualified (held). `jope-py train-policy --init-from` in §8.2.5.1 Weight warm start with the existing CLI: implemented but unqualified. The pending campaign's first pass from an initialization checkpoint: pending integration.

Name	Definition	Runbooks in this chapter, with status
Explicit new-corpus transition	Parent weights **and** optimizer state carried onto a different corpus, with the optimizer carry and the sampler initialization both declared	Native `--initialize` with a verified `parallel.replay_policy.v2` policy whose `optimizer` is `carry`; the run records "carried FP32 AdamW with bias-correction step" and a lineage-bound sampler counter whose local block step starts at zero: implemented but unqualified (held). No authenticated Python new-corpus transition exists in this snapshot: the pending trainer's strict input provenance rejects a changed corpus on resume, while the existing Python API can restore optimizer state without binding corpus identity. Sibling branches built on this transition are proposed (§8.4.5 Proposed sibling fresh and replay branches).
Exact interrupted resume	The original run continues from its own saved state, with its original inputs and original deadline; no allowance is renewed	Native `--resume` in §8.2.4.1 The implemented command shape: implemented but unqualified (held). The Python API and its companion helper in §8.2.5.2 Exact optimizer continuation through the Python API: implemented but unqualified. Pending `train resume` in §8.4.3 Plan, run, resume, verify and export: pending integration.

Two passes through one corpus are continuation, not two generations: the pending campaign's second pass continues the same run on the same corpus inside the same window (§8.4.3 Plan, run, resume, verify and export).

Historical instances. Lineage B's A100 recovery blocks each trained on a distinct corpus while carrying FP32 AdamW, the historical form of a new-corpus transition. Their accepted 512x6 step 60,000 remains the serving model and cannot be exactly resumed, because its state records no `execution_partition`. RTX step 60,100 is a historical hardware continuation: carried AdamW on the unchanged corpus with zero fresh programs, not a production continuation or a promotion. Step 69,000 remains unaccepted. See §B.3.4 Carried state, accepted step 60,000 and RTX step 60,100.

8.2.2 Rules for exact interrupted resume

These are owner requirements, **proposed**: neither PR83 main nor the stopped training64 overlay implements both of them.

1. **Reject optimizer fallback before any update.** An exact resume whose optimizer state is missing or marked incompatible must stop before its first update. It must never continue with a fresh optimizer under the resume name.
2. **Prove state parity.** A deterministic run that is interrupted and resumed must reach the same parameters, optimizer moments, scheduler, RNG, sampler position and counters as the same run left uninterrupted.

Present behavior against these rules:

- The Python API starts a fresh optimizer, scheduler and scaler when a resume artifact marks its optimizer state incompatible (main `train_v3.py:1919–1930`; pending `train_v3.py:2095–2111`). The companion helper refuses such an artifact before it calls the trainer, so it meets rule 1 for its own narrow case; it provides no parity proof.
- The pending campaign checks `optimizer_state_resumed` only after a pass has trained (pending `training_campaign.py:771`), so a fallback can consume updates and window time before it is rejected.
- Resume artifacts are written at epoch end without a minibatch cursor, and the campaign commits whole passes: an interrupted pass restarts from the last committed head inside the original window (pending `training_campaign.py:799`, `:1065`). That re-runs the pass; it is not an update-level continuation.
- Native `--resume` checks the execution partition, sampler, objectives, corpus, world, global batch and seed against the checkpoint's recorded state and optimizer metadata before CUDA (`crates/jope-engine/src/training.rs:931`, `:968–977`), and repeats the partition, sampler and objective checks after CUDA initialization (`:1023–1027`, `:1106–1110`). This manual has no parity evidence for it, and its dispatch is held.
- No Python test in that snapshot compares interrupted and uninterrupted trajectories: the pending `test_policy_training_contract.py:126–196` checks only changed parameters, increasing optimizer counters, compatible reload and contract rejection. Two native checks at `a75e543d2` do compare restored against uninterrupted state: the CPU unit test `cpu_optimizer_restore_completes_original_window_identically` (`crates/jope-engine/src/neural.rs:1359`) shows that a saved and reloaded AdamW finishes a three-step window with parameters and moments identical to the uninterrupted optimizer, and the driver `python/src/jope/tools/qualify_training_recovery.py` compares the final parameter hash of an eight-rank run with an injected rank failure against an uninterrupted run. Neither compares scheduler, RNG or sampler state, neither exercises the Python route, and this manual cites no receipt from that driver.

The proposed follow-up is an explicit strict-resume mode that rejects before updates; update-boundary checkpoints that carry the sampler permutation and cursor, counters and all RNG state; and deterministic equality tests of interrupted against uninterrupted runs. Warm-start behavior keeps its own name. Source: Codex's read-only analysis `boundary-resume-siblings.md` in custody `plan-execution-20260923`.

8.2.3 Variables used in the continuation examples

The paths below are operator-supplied placeholders, not newly admitted inputs. Resolve them from the compatible original run inventory; use absolute paths and preserve its hashes, resource limits and original time window. The native command remains held. The pending commands require reviewed integration and the admission checks described below.

Variables	Required meaning
`JOPE_PARALLEL_NEURAL`, `NATIVE_ROOT`	Source-bound native executable built with `neural`, and its authenticated native data/run root; these paths do not release the dispatch hold.
`QUALIFIED_NATIVE_CORPUS`, `VERIFIED_NATIVE_PARENT`, `VERIFIED_CARRY_POLICY`	Compatible corpus directory with verified allowed input closure, original native checkpoint directory, and validated optimizer-carry policy JSON. Do not substitute a Python checkpoint or blindly reuse the legacy corpus.

Variables	Required meaning
`NATIVE_RUN`	Output directory for the declared native run; an interrupted resume retains that same directory, inputs and original allowance.
`PYTHON_ENV`, `TRAINING_PYTHON`	Verified Python environment prefix and the matching Python executable, respectively.
`PYTHON_CORPUS`, `TRUSTED_PYTHON_POLICY`	Compatible admitted Python JSONL corpus and trusted weight artifact for the warm-start example; warm-start weights alone do not restore optimizer state.
`PYTHON_RESUME`, `ORIGINAL_SETTINGS`	Trusted `PrimePolicyTrainerResume.v1` artifact file and companion JSON containing its original compatible settings and audited input inventory. These are separate from the native checkpoint directory.
`PYTHON_CORPUS_SHA256`, `PYTHON_RESUME_SHA256`, `ORIGINAL_SETTINGS_SHA256`	Expected file SHA256 values from the original authenticated input inventory, checked before the helper imports Torch.
`NEW_RUN`	New private output directory for the Python example; the continuation helper requires it not to exist and refuses it under `/tmp` or inside a Git working tree. This new output location does not renew a resource or time allowance.
`FINAL_JOPE`, `FINAL_SOURCE_SHA`	Future source-bound CLI and full source commit after the pending training integration and its required checks; not an executable built from the current main `a75e543d2`.
`LEARNING_STATE`, `CORPUS_REQUEST`, `CORPUS_REQUEST_SHA256`, `CORPUS_OUTPUT`	Admitted learning state, exact corpus-preparation request and its file SHA256, and private prepared-input output directory, as specified in §8.4.2 Build the real corpus from admitted evidence.
`TRAINING_STATE`, `TRAINING_EXPORT`	Original supervised training state directory and private experimental export output directory; resume reuses `TRAINING_STATE`.

8.2.4 Native lineage B: the held command and its blockers

Runbook part	Native lineage-B continuation
Purpose and status	Continue lineage B natively: a weight warm start or an explicit new-corpus transition with `--initialize`, an exact interrupted resume with `--resume`. **Implemented but unqualified**: every direct dispatch stops at the dispatch hold.
Prerequisites and resources	A source-bound `neural` build; a compatible corpus with verified allowed input closure; a verified parent checkpoint and carry policy. The controller's fresh-telemetry check requires at least 128 GiB available RAM and 50 GiB free disk, more than the approved 64 GiB limit.
Manifest-creation command	None is qualified in this manual. Main contains a corpus publisher (`jope-parallel build-corpus`) and code that writes `parallel.replay_policy.v2` policies, but none has been qualified for this runbook. Supply an already verified corpus and policy, never a fabricated one.

Runbook part	Native lineage-B continuation
Execution	The commands in §8.2.4.1 The implemented command shape, which are not to be dispatched under the present allocation.
Artifacts and success	Checkpoint directories whose `state.json` binds model, optimizer, configuration, corpus and vocabulary hashes, plus the run's `initialization.json`. No success receipt exists for this held route.
Failure and resume	`--resume` applies only to an interrupted original run with the identical window and inputs. A completed run refuses resume, and accepted step 60,000 cannot be resumed (§8.2.2 Rules for exact interrupted resume).
Limitations	The dispatch hold, the resource gate and the world-8 partition of the accepted lineage (§8.2.4.2 The dispatch hold, resource incompatibility and valid topologies).

8.2.4.1 The implemented command shape

This is the exact implemented command shape, **not an instruction to dispatch it under the present allocation**:

```
"$JOPE_PARALLEL_NEURAL" --root "$NATIVE_ROOT" train \
  --corpus "$QUALIFIED_NATIVE_CORPUS" --output "$NATIVE_RUN" \
  --world 1 --microbatch 64 --steps 100 --checkpoint-steps 50 \
  --initialize "$VERIFIED_NATIVE_PARENT" --replay-policy "$VERIFIED_CARRY_POLICY"

# Only an interrupted original run, with identical original window and inputs:
"$JOPE_PARALLEL_NEURAL" --root "$NATIVE_ROOT" train \
  --corpus "$QUALIFIED_NATIVE_CORPUS" --output "$NATIVE_RUN" \
  --world 1 --microbatch 64 --steps 100 --checkpoint-steps 50 \
  --resume --replay-policy "$VERIFIED_CARRY_POLICY"
```

Here `--steps 100` declares the same increment for the whole original run; resume does not authorize another hundred steps. A completed run explicitly refuses resume. `--initialize` alone starts fresh FP32 AdamW and resets the new-corpus sampler; preserving optimizer ancestry requires the verified carry policy and compatible objective/partition evidence. Do not fabricate that policy JSON or relabel an incompatible recovered checkpoint. Corpus, vocabulary, model, optimizer, config, sampler, partition and deadline checks occur before CUDA. In this chapter's names, the first command is a weight warm start when the policy's `optimizer` is `reset`, and an explicit new-corpus transition when it is `carry` and the corpus differs from the parent's; the second command is an exact interrupted resume.

8.2.4.2 The dispatch hold, resource incompatibility and valid topologies

The current direct controller calls `require_original_gpu_grant`, which **always returns an error**: “original native GPU grant verifier unavailable; numerical dispatch is held.” Environment variables cannot release this gate. A separate typed native funding-admission API exists, but this is not a working direct-trainer bypass. Even after a future proper dispatch integration, the controller's fresh-telemetry check requires at least 128 GiB available RAM and 50 GiB free disk, incompatible with the current approved 64 GiB limit. Valid normal microbatches are 64, 128 or 256, and global batch is 2048. With world 1/microbatch 64, accumulation is 32. The retained step 60100 hardware qualification actually used world 1, microbatch 256 and accumulation 8, carrying AdamW from 60000 to 60100 on the unchanged corpus with zero fresh program presentations. Its 100-step update is genuine optimizer work; it is not production continuation or promotion. The ADR's microbatch 32 example is inaccurate for that retained run and is not a currently accepted CLI topology.

Sources: native command parser, controller admission and step window, dispatch hold, checkpoint checks and warm start.

8.2.5 Current Python training and a concrete continuation call

8.2.5.1 Weight warm start with the existing CLI

For a compatible, already admitted Python corpus, the existing CLI can launch a weight warm start:

```
"$PYTHON_ENV/bin/jope-py" train-policy \
  --corpus "$PYTHON_CORPUS" --checkpoint "$NEW_RUN/policy.pt" \
  --init-from "$TRUSTED_PYTHON_POLICY" --epochs 1 --batch-size 256 \
  --lr 0.0005 --device cuda:0 --seed 7 \
  --d-model 256 --n-layers 2 --ffn-dim 1024 --n-heads 4 \
  --metrics-jsonl "$NEW_RUN/metrics.jsonl" \
  --progress-jsonl "$NEW_RUN/progress.json"
```

This example uses the stated 256-width architecture; it is suitable only when the intended experiment explicitly uses that architecture. Warm starts may shape-filter or prefix-copy weights. The script's default model, architecture and vocabulary must not be assumed to reproduce the accepted native step 60000 model. The CLI command is source-checked but was not executed for this chapter.

Runbook part	Python weight warm start
Purpose and status	Start a new Python optimization run from compatible weights: a weight warm start. **Implemented but unqualified**: the CLI exists at `a75e543d2` and was source-checked, not executed, for this chapter.
Prerequisites and resources	A verified Python environment, an admitted compatible JSONL corpus, a trusted weight artifact of the stated architecture, and one CUDA device inside a separately admitted resource/time supervisor.
Manifest-creation command	No manifest-creation command exists at `a75e543d2` for an admitted corpus: the admission contract and its exporter, `train prepare-inputs`, are pending integration (§8.4.2 Build the real corpus from admitted evidence).
Execution	The command above.
Artifacts and success	`policy.pt`, `policy.imitation.pt`, `metrics.jsonl` and `progress.json` under `$NEW_RUN`. With `--epochs 1` and the default `--phase-a-fraction 0.4`, the single epoch is phase A, so the trainer also writes the phase-A imitation checkpoint `policy.imitation.pt` beside `policy.pt`. A completed run shows an optimization run only, not eligibility, optimizer continuation or improvement.
Failure and resume	The CLI exposes no `--resume-from`. A failed warm start is a failed run; repeating it is a new run in a new directory, without a renewed allowance.
Limitations	Shape-filtered or prefix-copied weights, defaults that do not reproduce the accepted model, and the device fallback in §8.2.5.4 Limits of the current Python route.

8.2.5.2 Exact optimizer continuation through the Python API

For **actual optimizer continuation**, call the existing API from a private maintainer script. The following is the essential code; `settings.json` must contain the original compatible `kwargs`, and all referenced inputs must be hash-pinned in the operator's original run inventory. The complete companion `continue-python-policy.py` includes those byte-hash checks, exclusive output creation, explicit GPU preflight, refusal of distributed or non-current resume modes, and refusal of an `--output` under `/tmp` or inside a Git working tree (the path itself or any parent directory containing a `.git` entry). It is an illustrative API caller, not a new installed product command or a qualified supervisor.

```
from jope.policy.train_v3 import train_policy_v3_from_jsonl

result = train_policy_v3_from_jsonl(
    corpus=verified_corpus_path,
    checkpoint=new_output / "policy.pt",
    resume_from=verified_resume_path,
    resume_checkpoint=new_output / "resume.pt",
    metrics_jsonl=new_output / "metrics.jsonl",
    progress_jsonl=new_output / "progress.json",
    epochs=1,                         # one additional epoch, not target epoch 1
    checkpoint_state="current",     # keep export and optimizer state aligned
    allow_train_val_fallback=False,
    amp=False,
    autotune_batch_size=False,
    device="cuda:0",
    **original_compatible_settings,
)
```

8.2.5.3 The companion continuation helper and the resume artifact

A concrete invocation of the companion is below. Without the final switch it verifies the supplied byte inventories and prints the call without importing Torch. Fill settings from the original run; do not copy arbitrary architecture, routing or vocabulary choices into a resume. An `audited_input_files` list does not itself prove closure completeness or teacher eligibility.

```
MANUAL_EXAMPLES="$R/docs/internal/asm-manual/examples"
"$TRAINING_PYTHON" "$MANUAL_EXAMPLES/continue-python-policy.py" \
  --settings "$ORIGINAL_SETTINGS" --settings-sha256 "$ORIGINAL_SETTINGS_SHA256" \
  --corpus "$PYTHON_CORPUS" --corpus-sha256 "$PYTHON_CORPUS_SHA256" \
  --resume "$PYTHON_RESUME" --resume-sha256 "$PYTHON_RESUME_SHA256" \
  --output "$NEW_RUN"
# Numerical execution, only inside the separately admitted supervisor:
# append --execute-diagnostic to that same command.
```

The settings object uses the illustrative schema `manual.python-resume-settings/v1`, an `audited_input_files` array of `{path, sha256}`, and a `kwargs` object containing exactly: `batch_size`, `lr`, `seed`, `phase_a_fraction`, `checkpoint_metric`, `greedy_eval_datasets`, `d_model`, `n_layers`, `ffn_dim`, `n_heads`, `prime_vocab`, and `routing_classes`. These values must come from the compatible original configuration. This helper is deliberately limited to ordinary current-state resumes: staged schedules, anti-overfit experiments and distributed training require their own original configuration and proof. Only syntax/API and metadata-path checks were performed for this new example; no model was loaded or trained.

Use the trusted `PrimePolicyTrainerResume.v1` artifact, not `model.ot`, a safetensors export, or a selected `policy.pt` without optimizer state. Existing current-state resume loads model state strictly, verifies action tokens and routing classes, restores optimizer/scheduler/RNG, and appends epochs after `global_epoch`. A best/EMA selected checkpoint may carry an explicit optimizer-incompatible marker and

cause a fresh optimizer; that is not accepted as exact continuation by the companion example. Sources: restore and corpus partition, strict model restore, optimizer compatibility, resume artifact contents.

Runbook part	Python exact interrupted resume (companion helper)
Purpose and status	Continue an ordinary current-state Python run through the existing API from its epoch-end resume artifact. This is an exact interrupted resume only when the settings, corpus and resume artifact are the original run's and its original window is still open; the helper cannot check the window. **Implemented but unqualified**: only syntax/API and metadata-path checks were performed.
Prerequisites and resources	The trusted resume artifact, original settings, corpus and audited input closure, each with its expected SHA256; one visible RTX 3090; a separately admitted resource/time supervisor.
Manifest-creation command	No manifest-creation command exists: the operator writes `settings.json` in the illustrative `manual.python-resume-settings/v1` schema from the original run's configuration.
Execution	The helper without `--execute-diagnostic` (metadata only, no Torch import), then the same command with it inside the supervisor.
Artifacts and success	`policy.pt`, `resume.pt`, `metrics.jsonl`, `progress.json` and `api-result.json` in the new output directory. The result is scoped as a diagnostic API call, not product acceptance.
Failure and resume	Refuses changed input digests, missing or unknown settings, a resume artifact that is not an ordinary current-state resume with compatible optimizer state, more than one process, anything other than one visible RTX 3090, an existing output directory, and an output under `/tmp` or inside a Git working tree. A retry needs a new output directory and keeps the original window.
Limitations	No interrupted-versus-uninterrupted parity proof and no in-epoch cursor (§8.2.2 Rules for exact interrupted resume). It fixes `amp=False` and compares only the four architecture fields with the artifact, so it does not detect a changed AMP or batch-size regime.

8.2.5.4 Limits of the current Python route

This current Python route has material limits. CUDA requests can silently fall back to CPU in the underlying code, so the companion refuses absent CUDA before calling it. Current task-routing/zero-update and resume-proof fixes belong to the pending training overlay. The loader can discard minority embedding widths with a warning; a qualified input inventory should reject such a mixed corpus earlier. Existing Python training alone does not provide the shared cgroup resource contract, original campaign deadline, teaching admission, parameter-change proof or serving promotion. Execute any diagnostic caller only inside a separately admitted resource/time supervisor, not as a way to bypass the pending product gates. See current device fallback and mixed-width handling.

8.3 Evidence and identity before a bounded campaign

A training run has at least four separate identities:

Identity	What it binds	Why it is separate
Learning state	Eligible HWM records and teaching evidence	Accumulated data authority is distinct from a single training attempt
Training plan	Task/category inputs, source, runtime, numerics, seed, passes and resource policy	Changing it invalidates an existing run's compatibility
Execution window	Actual first admission and immutable deadline	Resume must use elapsed budget rather than restart the clock
Checkpoint/export	Parameters, architecture, vocabulary, task, optimizer ancestry and completed proof	A file's existence is not evidence of an update or eligibility

Use durable directories outside Git working trees and outside `/tmp`. Keep corpus and model assets out of Git. The native front door inventories the actual Python interpreter, preserves its virtual-environment invocation path, captures environment and installed runtime identity, and supervises workers rather than treating a bare Python subprocess as resource isolation.

The training runtime inventory hashes installed dependency bytes, excluding generated bytecode, and records all installed distributions rather than only a short library allowlist. Version strings alone do not protect against modified installed files. The full source commit and source-package hashes are part of compatibility. An incompatible environment should be rebuilt/replanned before admission, not silently accepted during resume.

Sources: pending `crates/jope-cli/src/train.rs:1`; `python/src/jope/policy/training_campaign.py:120`.

8.4 The bounded training campaign chain

Runbook part	Bounded training campaign chain
Purpose and status	Train, verify and export bounded experimental checkpoints, one per planned category: experimental regression lineage v1 (256×2) from admitted real regression evidence, plus explicitly synthetic classifier fixtures, with a smaller architecture, only where real classification admission fails. **Pending integration**: the training64 patch is outside main and has never run on a GPU.
Prerequisites and resources	Reviewed integration with required checks at the final head, a source-bound `jope` build, fresh real teacher admission covering at least two independent families, and an admitted training window. Resources: one RTX 3090 and one GPU-hour in total, at most 20 minutes per category, 20 CPUs, 64 GiB and zero swap.
Manifest-creation command	`train prepare-inputs` writes the inputs manifest (`inputs.json`) from a `jope.real-training-corpus-request/v1` request, and `train plan` freezes the plan without starting a clock. No command, at `a75e543d2` or in the pending overlay, writes the request itself; its comparison proofs must come from actual admission (§8.4.2 Build the real corpus from admitted evidence).
Execution	`train run`, or `train resume` after an interruption inside the original windows (§8.4.3 Plan, run, resume, verify and export).

Runbook part	Bounded training campaign chain
Artifacts and success	Per-category attempts, receipts and heads in the state directory. `train verify` succeeds only with positive updates, changed parameters, exact reload, resumed optimizer state and at least two committed passes; `train export` then writes a private experimental bundle with a file inventory.
Failure and resume	An exhausted window is recorded as `budget_exhausted`. Resume keeps the plan, the state and the original global and category deadlines; an interrupted pass restarts from the last committed head (§8.2.2 Rules for exact interrupted resume).
Limitations	Real corpus preparation supports regression only. With two prospective families it is a mechanics qualification, not a flywheel test. Input is JSONL without a packed reader, there are no sibling fresh/replay branches, and export never moves the serving pointer.

8.4.1 What the pending campaign trains

The pending training campaign uses the existing Python `JopePolicyModelV3` with task-routed reward/stability behavior. Real admitted training uses width 256, two layers, four heads and feed-forward 1024; synthetic qualification fixtures use width 8, one layer, two heads and feed-forward 16. Both consume the explicit context schema. Training uses one-epoch passes, learning rate 5e-4, batch 256 for admitted rows or 2 for fixtures, AMP disabled and automatic batch-size tuning disabled. These are the bounded campaign's settings, not the accepted step-60000 model's architecture (width 512, six layers, feed-forward 2048; §2.8.1 Checkpoint binding and configuration). The campaign therefore produces a **new, smaller model family**, not a continuation of step 60,000: its optional initialization is a Python policy payload, and its checkpoint cannot be loaded in place of the accepted model, whose loader rejects incorrectly shaped tensors. Compare it with step 60,000 only through matched downstream fits under the same protocol, never through loss values.

Naming. The regression line that this campaign would start is called **experimental regression lineage v1 (256×2, new initialization)**. It is pending integration and has not been trained. The planned initial model, M_init, uses new parameters from the declared architecture and seed, with no parent weights. The campaign also exposes an optional weight-warm-start interface, but this plan has not selected it; its shape-filtered loading is not proof of full checkpoint compatibility. M_init is named M0 only after it has actually trained on the lineage's first admitted corpus, C0, and that run has passed verification; until then no M0 exists. The line stays separate from the accepted 512×6 step 60,000 of lineage B: it is neither its successor nor a replacement for it.

8.4.2 Build the real corpus from admitted evidence

`train prepare-inputs` takes a request, not hand-written training examples. The request schema binds requested tasks, seed, family registry, vocabulary, encoder and optional initialization checkpoint. Its `contexts` map has one independently pinned proof for every eligible comparison. Each proof includes producer/source, original input, qualification/reference manifest, vector/artifact, frozen transforms, component bundle, actual context execution and eight case executions.

The exporter reopens the original exact fitting input, verifies feature schema/values/labels and row order, verifies complete context admission, and checks each recipe against exact executable vocabulary bins. It refuses parameter snapping, wrong-task heads and fabricated vectors. It then writes corpus rows with 1536 dataset plus 512 target dimensions, family-disjoint train/development metadata and retained source/context identities.

```
"$JOPE_FINAL_CLI" train prepare-inputs \
  --learning-state "$JOPE_LEARNING_STATE" \
  --request "$JOPE_CORPUS_REQUEST" --request-sha256 "$JOPE_CORPUS_REQUEST_SHA" \
  --output "$JOPE_CORPUS_OUTPUT"
```

Do not author a request using invented eligible comparison IDs: these must come from actual successful admission. The exported request requires all eligible requested comparisons. At least two independent families are necessary for a real train/development partition. Source: pending `crates/jope-cli/src/train/corpus.rs:43`, `:195`, `:362`, `:485`.

8.4.3 Plan, run, resume, verify and export

The intended real all-category path preserves real admitted regression while allowing explicitly labeled synthetic fallback only for classification categories whose real admission fails:

```
"$JOPE_FINAL_CLI" train plan \
  --state "$JOPE_TRAINING_STATE" --python "$JOPE_TRAINING_PYTHON" \
  --inputs "$JOPE_CORPUS_OUTPUT/inputs.json" \
  --task all-admitted --device cuda:0 --source-sha "$JOPE_FINAL_SOURCE" \
  --learning-state "$JOPE_LEARNING_STATE" --seed 7 --passes 2

"$JOPE_FINAL_CLI" train run --state "$JOPE_TRAINING_STATE"
"$JOPE_FINAL_CLI" train verify --state "$JOPE_TRAINING_STATE"
"$JOPE_FINAL_CLI" train export \
  --state "$JOPE_TRAINING_STATE" --output "$JOPE_TRAINING_EXPORT"
```

If interrupted before the original deadlines and while all plan/runtime/input pins still agree, continue with:

```
"$JOPE_FINAL_CLI" train resume --state "$JOPE_TRAINING_STATE"
```

Do not use the top-level `--synthetic-fixture` switch in place of missing real regression evidence. For deliberately small mechanical CPU qualification only, the explicit interface is `train plan --synthetic-fixture --device cpu`, with the same source/runtime/state requirements. Such a result stays synthetic and does not satisfy the real-data task.

The planner records requested, admitted, unavailable and fallback categories. With `all-admitted`, a missing real regression category can remain explicitly unavailable while classifier fixtures proceed; that is partial coverage and cannot be reported as completed real-regression training. Current real-classification corpus admission is not qualified. Binary/multiclass fixture fallback retains its own synthetic provenance and task labels.

Verification must establish positive optimizer updates, changed parameter digests, exact reload, an actual resumed optimizer state and increasing step counters, at least two committed passes, complete planned category coverage where claimed, and compatible device/source/runtime/vocabulary/input identities. The worker reloads both `policy.pt` and `resume.pt` and requires equal parameter proof. Each pass belongs to the original category/global windows and contributes an immutable execution/receipt chain. A zero-update checkpoint or interrupted partial output cannot become a successful pass. Both

planned passes read one corpus inside one category window, so the second pass is continuation within one generation, not a second LOOP-01 generation. It supplies the optimizer-continuation proof: verification refuses a category head with fewer than two committed passes (pending `training_campaign.py:1210`). Verification establishes training mechanics only. An eligible teacher, a new discovery, information actually consumed by the optimizer and an improved policy remain four distinct claims, each needing its own evidence.

Export first performs verification, then writes the exact plan, verification and category artifacts into a private output bundle with a file inventory. It creates experimental training artifacts; it does not update the accepted serving pointer. Sources: pending `python/src/jope/policy/training_campaign.py:744`, `:780`, `:1043`, `:1203`, `:1303`.

8.4.4 The pending bounded extension path: sequence and limits

The stopped training64 patch is `595824ef6eb8e15872be058668503cedad1dd9a5ff98ac4700c6f713251d384c`, with manifest `b86b550d7ee39fd93d5202accd5f77c975b7bcb4fd8f8e7706c69c5f279294a2`. It has not been integrated into main (`a75e543d2`, PR83) or qualified by a fresh real GPU run. Its exact command contract is described in §8.4.3 Plan, run, resume, verify and export; pending files cannot honestly be linked as existing GitHub files at main.

After reviewed integration, source-bound build/tests, fresh real two-family teacher admission, and verified corpus preparation, the operational sequence is:

```
"$FINAL_JOPE" train prepare-inputs \
  --learning-state "$LEARNING_STATE" \
  --request "$CORPUS_REQUEST" --request-sha256 "$CORPUS_REQUEST_SHA256" \
  --output "$CORPUS_OUTPUT"
"$FINAL_JOPE" train plan --state "$TRAINING_STATE" \
  --python "$TRAINING_PYTHON" --inputs "$CORPUS_OUTPUT/inputs.json" \
  --task all-admitted --device cuda:0 --source-sha "$FINAL_SOURCE_SHA" \
  --learning-state "$LEARNING_STATE" --seed 7 --passes 2
"$FINAL_JOPE" train run --state "$TRAINING_STATE"
# If interrupted within the original allowance, retain the same plan/state:
"$FINAL_JOPE" train resume --state "$TRAINING_STATE"
"$FINAL_JOPE" train verify --state "$TRAINING_STATE"
"$FINAL_JOPE" train export --state "$TRAINING_STATE" --output "$TRAINING_EXPORT"
```

Run and resume are alternatives according to state; do not blindly execute resume after a completed run. The whole campaign permits one RTX 3090 and one GPU-hour, at most 20 minutes per category, 20 CPUs, 64 GiB and zero swap. Two passes share each category window; they do not receive fresh time allowances. At least two real admitted families are necessary before claiming the real regression training path; with only the two prospective panel families of §7.4 Present teacher readiness and the smallest real path, the campaign is a mechanics qualification, not a flywheel test. Classification fixtures may demonstrate labeled mechanics when real admission fails, but remain synthetic. Unknown or unavailable categories are reported rather than filled with invented success.

Verification must establish positive updates, changed parameters, reload equality, actual optimizer continuation, full input/runtime/source/device/vocabulary compatibility and persisted accounting. Export is experimental. The pending `experimental.regression`, `experimental.binary` and `experimental.multiclass` consumer entries bind task-specific exported checkpoints; a later real fit must prove the new checkpoint generated and fitted candidates. None of these commands updates the accepted serving pointer. Product28, warm-target and winner-export follow training64; they do not fix teacher eligibility by themselves.

8.4.5 Proposed sibling fresh and replay branches

Status: proposed. No implementation exists: the pending campaign advances one mutable head per category over one pinned corpus. It has no immutable sibling parent, branch identifier, committed-update target, 25/75 sampler allocation or per-update fresh-exposure record (pending `training_campaign.py:799, :1065`). The owner's design for one controlled round is:

- Two sibling branches, F (fresh) and R (replay only), start from one immutable parent model **and** optimizer state and share the original deadline; neither receives a new window.
- Both commit the same number of updates and batch slots. F samples 25% fresh and 75% replay examples; R samples replay only.
- Each records measured fresh exposure, duplicate and unique families, non-padding tokens and GPU cost.
- Neither branch is promoted to serving.

Codex's read-only analysis (`boundary-resume-siblings.md` in custody `plan-execution-20260923`) proposes three further details: each branch also records its consumed example identities; the unchanged parent is kept as a comparison artifact; and moving onto the new corpus is an explicit new-corpus transition that declares optimizer carry and sampler initialization, not a resume.

The first causal experiment keeps architecture, objective, corpus and sampler changes distinguishable, so it does not change the architecture or the reward representation alongside these branches. Within the existing 20-minute regression GPU cap, the two controlled LOOP-01 rounds may remain unfunded or inconclusive. An unfunded round, or one with insufficient data, yields mechanical proof and descriptive results, and its causal outcome is reported as inconclusive, not as success. §9.5 LOOP-01: the central regression milestone defines LOOP-01.

8.4.6 Training input packing and the learning store

Status: proposed. The pending campaign still reads JSONL into memory and writes a per-attempt `corpus.jsonl`; no packed-reader scalability qualification exists (§7.6.4 Python training input and the pending bounded campaign). Packing work follows phase profiling, in this order:

1. Profile the preparation phases, as in §7.7 Concrete corpus improvements, with acceptance criteria.
2. Add a verified Python packed-reader bridge.
3. Reuse immutable shards, with bounded staging.
4. Compress.
5. Adopt a narrower record layout only if measurement justifies it.

Every step preserves float32 supervision, the legal-set mappings and the 128-token grammar, and must show record, batch, loss, gradient, update and resume parity (§D.7.4.2 Compression, the proposed 180-byte layout and label precision). Keep one learning-store writer until an atomic server-side pointer update is qualified. Backup, replication and retention operations remain separately authorized; nothing in this chapter authorizes them (§D.4.2 Recorded custody risks and disk capacity).

8.5 Consuming a newly trained checkpoint

8.5.1 The experimental.regression configuration entry

The pending final regression consumer is selected by the `experimental.regression` entry of the complete `jope.asm-configuration/v1` object. This entry is an object with:

Field	Required meaning
`schema`	`jope.experimental-python-configuration/v1`
`task`	`regression`, `binary`, or `multiclass`, matching its configuration key
`runtime`	Full pinned PythonRuntime object, including program, manifest, source/package files and runtime digest
`export_manifest`	Exact exported training manifest artifact
`checkpoint`	Exact task's exported `policy.pt` artifact
`vocabulary`	Exact frozen action vocabulary artifact
`canonicalizer` / `canonicalizer_build`	Native executable and actual successful build evidence
`canonicalizer_features`	Exact feature set from Cargo artifact messages
`sampling`	Bounded count, temperature, top-p and maximum sequence length
`generic_draws`	Declared bounded generic bank allowance

Artifacts use `{path, sha256}` with absolute paths and real file hashes. Runtime is not merely a Python executable/version: source files and installed package files must be inventoried. The native canonicalizer must have a successful build receipt whose source and source-manifest pins agree with the consumer. Do not copy an accepted checkpoint receipt into this export field.

8.5.2 Compatibility checks and fail-closed rules

Regression additionally requires the export to identify real admitted regression training, the same source commit, and the exact native encoder identity. The consumer admits the complete native context and independently verifies architecture, task, vocabulary, checkpoint and proof. Classification uses its separate context lineage. Missing architecture fields, missing/extra model state, altered vocabulary, source changes, changed encoder assets or a different training task fail closed. No explicit experimental selection silently falls back to the accepted model.

The export records the complete architecture, vocabulary, task, source, input identities, model state and training proof. The pending experimental consumer requires all architecture fields explicitly; the general-purpose Python loader's backward-compatible defaults cannot reinterpret a saved experimental checkpoint. It compares the reconstructed model's architecture and full state to the export. Sources: pending `python/src/jope/policy/training_campaign.py:780`, `python/src/jope/policy/experimental_proposal.py`, `crates/jope-pipeline/src/experimental_producer.rs`.

The final regression follow-on closes a specific gap: `Configuration.experimental.regression` can explicitly select a newly trained regression export. Omission keeps the accepted baseline. This operational regression route requires **real admitted** training, the same source commit and the exact native encoder identity used by training. It first admits the qualified native2048 regression context, then wraps those exact values in the separate `asm.experimental-regression-context/v1` lineage with zero classes and `policy_compatible:false`. It does not promote the new checkpoint to accepted serving or weaken the accepted hard pins.

Binary and multiclass use their separate experimental context path; a synthetic classifier fixture can qualify mechanical training/consumption behavior only when explicitly reported as synthetic. Synthetic regression training is permitted as a CPU test but cannot be substituted into the operational real-regression consumer. There is no silent accepted-policy fallback after an explicitly chosen experimental checkpoint fails compatibility.

8.5.3 Proving consumption in a new operational proof

After building a new configuration around the verified export, preregister a separately authorized operational proof using the same `optimize proof-plan` front door. Its report must bind the new checkpoint hash to actual generation-zero neural fits, retain fitted artifacts/predictions and, when evolution is claimed, show completed descendants with neural ancestry. Recover the report from the same original journal and deadline. Only after those facts are demonstrated should the final official panel configuration be frozen around that checkpoint.

Source: pending `crates/jope-bench/src/panel/experimental_proposal.rs:36`, `crates/jope-pipeline/src/experimental_producer.rs:36`, `python/src/jope/policy/experimental_proposal.py`; overlay documentation `docs/experimental-regression-consumption.md`.

Runbook part	Consumption of a newly trained checkpoint
Purpose and status	Show that a verified experimental export generated and fitted real candidates. **Pending integration**: the `experimental.regression` consumer exists only in the stopped overlay.
Prerequisites and resources	A verified export, a complete configuration whose `experimental.regression` entry pins it (§8.5.1 The experimental.regression configuration entry), and a separately authorized operational proof with its own resources.
Manifest-creation command	`optimize proof-plan` preregisters the proof; it exists at `a75e543d2`, without the experimental consumer. No command writes the configuration entry: in the pending source the schema string occurs only as the consumer's expected value, and the overlay documentation has the operator bind it.
Execution	The same front door's run and report steps, recovered from the original journal and deadline.
Artifacts and success	A report that binds the checkpoint hash to actual generation-zero neural fits, with retained fitted artifacts and predictions, and completed descendants with neural ancestry when evolution is claimed.
Failure and resume	Compatibility failures fail closed, with no fallback to the accepted model; recovery uses the original journal and deadline.
Limitations	Consumption is not improvement, and nothing here moves the serving pointer.

8.6 Remaining acceptance queue and maintenance risks

The required next integration/acceptance work is sequential: apply guarded training64 after PR83 lifecycle completion; compile and run all required tests at the final head; admit fresh genuine teachers; materialize and verify the corpus; execute the one approved GPU hour; verify/export; consume the new checkpoint in actual fits; qualify fitted portability and warm/cold delivery; execute the unchanged30 panel and report all outcomes. The product chapters cover the remaining deployment stages in detail.

LOOP-01 is the central regression product milestone (§9.5 LOOP-01: the central regression milestone): a new eligible target, actual exposure, changed parameters, compatible reload, improved fitted search, then another new generation. No LOOP-01 generation has run. Once integrated and qualified, the pending training chain in this chapter can supply changed parameters and compatible reload; both links are pending integration in that snapshot. The stopped version cannot supply actual exposure, because it keeps no per-update fresh-exposure record (§8.4.5 Proposed sibling fresh and replay branches), so that link remains proposed. Improved fitted search and a further generation also need the consumption proof above and a preregistered decision. Under the owner's proposed rule, the loop stops on no eligible discoveries, incompatibility, no material gain, insufficient evidence or an exhausted budget; the best checkpoint is preserved, and serving is never promoted automatically.

Maintain explicit checks for changed dependency bytes, an unsupported GPU/device, absent families, zero updates, wrong task heads, changed vocabulary bins, missing continuation, missing native component receipts, stale code hashes, counterfeit fitted outputs, incomplete process-group cleanup and partially written journals. The experimental route is intentionally separate from accepted serving. Any future promotion mechanism must add its own independent evaluation and pointer discipline rather than rename an experimental export.

The current source-only regression follow-on has an independent review with no confirmed production/API blocker. Seven pure Python checks passed; the three actual-model regression/binary/multiclass cases were explicitly deselected because custody work was barred from model execution. The next required CI/runtime acceptance must execute those cases and verify their collected inventory. This chapter records the gap rather than claiming it closed.

CHAPTER 9

Measuring repeated learning and deciding improvement

IN THIS CHAPTER

9.1 Observed transferable lift: reporting contract, not yet measured 105
9.1.1 What the two measurements mean 105
9.1.2 Why this needs a new frozen evaluation 106
9.1.3 What the score does and does not establish 106
9.1.4 Reading the real historical plots alongside this KPI 107
9.2 Preserved historical chapter snapshot 107
9.3 What “better and better” must mean 107
9.4 What the historical training and target evidence establishes 107
9.5 LOOP-01: the central regression milestone 108
9.5.1 Lineages, continuation names and generations 109
9.5.2 Two controlled rounds within the existing GPU cap 110
9.6 The closure ledger 110
9.7 A minimal experiment that can establish learning 111
9.7.1 Freezing the experiment and choosing families 111
9.7.2 What to retain for each cycle 112
9.7.3 The proposed preregistered regression decision 112
9.7.3.1 Family-normalized gain 113
9.7.3.2 Round thresholds 113
9.7.3.3 Baselines, stopping and reporting 113
9.8 Distinguish routing, ranking and search recovery 113
9.9 Conditioning diagnostics before the first causal experiment 114
9.10 Specialization without starving the general learner 115
9.11 A KPI ledger with defined denominators 115
9.11.1 KPI definitions 115
9.11.2 Win counting and a proposed acceptance record 117
9.12 What is still needed to demonstrate the learning flywheel 118

9.1 Observed transferable lift: reporting contract, not yet measured

Current global KPI: not yet measured. Merged PR90 adds private zero-fit planning and retained-prediction reporting. This publication contains no new training campaign or untouched nested evaluation. Existing acceptance thresholds, LOOP-01 definitions and serving decisions remain unchanged. The older proposal sections below retain their dated scope.

9.1.1 What the two measurements mean

For each dataset, outer fold and seed, compare the challenger with its parent. The audit component averages five gains, each measured by an independently repeated complete search using only that audit fold’s training complement, with inner selection confined there, before scoring its held-out audit rows. The outer component measures a separate complete search selected and fitted using the full

outer training partition. Each gain is `(parent RMSE − challenger RMSE) / training-mean baseline RMSE`; the baseline is fitted only on that score's permitted training rows. Average the five separately normalized audit gains, not their raw RMSEs divided by one pooled denominator.

Take `q = min(audit gain, outer gain)` **before aggregation**. Average folds/seeds within each dataset, datasets within each family, then families equally. Display `100 × result`, in percentage points of baseline RMSE. A holdout-only gain receives no positive contribution; audit-only improvement also receives none. For units `(audit, outer) = (0.3, −0.1)` and `(−0.1, 0.3)`, the contribution remains −0.1, rather than becoming +0.1 by taking the minimum after pooling. These numbers illustrate the definition and are not experimental results.

The headline is accompanied by both gains, parent/challenger absolute skill against the trivial predictor, worst-family degradation, three-way sign agreement, planned/valid coverage, family distributions and an audit-versus-outer scatter plot. If relative improvement exists without positive mean absolute skill on both components, it says **no absolute skill demonstrated**. Generic-search and fresh-versus-replay controls remain separate contrasts.

9.1.2 Why this needs a new frozen evaluation

Five outer folds and five training-side audit folds require **30 complete searches per arm/dataset/seed**, each with its own inner selection and final refit. Different folds may select different recipes. Re-scoring a previously selected winner, or selecting from an adaptively generated pool after the fact, does not satisfy this contract.

The planner freezes checkpoints, procedure, controls, families, ordered rows/groups, folds, seeds, assets, source and resource estimates. Insufficient groups/resources remain explicit failures. Every planned prediction or terminal failure is committed before any audit or outer evaluation score is released. Inner candidate-selection scores remain available within their permitted search. Missing comparisons, contaminated exposure, invalid identities and zero/nonfinite baselines block a complete global score; they never disappear from the denominator.

The existing thirty-dataset panel is adaptive development. It cannot become untouched by changing its label in a manifest. Checkpoint and encoder exposure are tracked separately; unknown historical exposure prevents an unseen-family claim. A trusted external release receipt must attest the admitted executor's isolation and chronology. Hash verification checks bytes and declared dependencies; this reporting implementation does not establish the truth of an executor's isolation claims or admit that future executor.

9.1.3 What the score does and does not establish

This is a conservative descriptive score, not a confidence bound or proof of causality. Folds and seeds share data and are not independent statistical samples. Released outcomes become exposed development evidence when used to design subsequent work. The selection-bias problem is described by Cawley and Talbot, fold-dependence by Bengio and Grandvalet, and adaptive reuse by Dwork et al.. These references motivate the precautions; they do not prove a special statistical guarantee for this minimum score.

The CPU tests exercise arithmetic, evidence validation, coverage and a deterministic toy selection boundary. Changing its own scoring labels leaves the fixture's frozen recipe and predictions unchanged. That test is not production executor qualification. See the private contract `docs/internal/transfer-measurement.md` and merged source `7f2b2bf860e4b1ac8815c8da55e507143570a905`.

9.1.4 Reading the real historical plots alongside this KPI

The figures in §B Historical training, checkpoint lineage and original loss trajectories retain the original observations and their hashes: 63,000 A100 steps (6001–69000, not invented steps 1–6000), the separate 100-step RTX continuation, fixed-monitor comparisons and available conditional ranking/ regret diagnostics. Training corpus, objective, lineage and validation-population boundaries stay visible. The fixed monitor is adaptive teaching evidence; decreasing training loss or ranking regret cannot substitute for the unmeasured nested transfer KPI. Raw histories remain intact; plotted bins are declared within-corpus summaries, with no interpolation across missing history.

9.2 Preserved historical chapter snapshot

The following sections retain the original dated source narrative and evidence. The current-source additions above supersede implementation-status descriptions where explicitly stated.

9.3 What “better and better” must mean

A genuine flywheel joins four measured improvements. GP finds a better executable teacher for an unchanged comparison. Training actually consumes that new information. The policy becomes better at proposing or ranking programs for the relevant dataset-and-split context. Finally, the next bounded GP search improves because of that policy. Each arrow needs its own evidence; a growing corpus, a saved checkpoint or a completed search cannot stand in for the whole chain.

Four claims therefore stay distinct, and each needs its own evidence: an **eligible teacher** (a target admitted under its full comparison identity), a **new discovery** (a program absent from the authenticated initial population), **actually consumed information** (the optimizer sampled it in committed updates) and an **improved policy** (the trained checkpoint beats its parent in a matched downstream fitted comparison under the same budgets). None implies the next. Initial populations must be authenticated, and a discovery made in one arm must not silently seed another arm.

The comparison identity is essential. A target belongs to a task, dataset content, exact split and groups, evaluator, vocabulary and numerical protocol. If any changes, the score is a new experiment rather than a better version of the old target. Context identity also binds the fitted preprocessing, encoder assets and allowed fitting rows. Two vectors with the same width need not represent the same problem.

An incumbent can improve monotonically under a fixed development objective while real generalization gets worse through repeated selection. We therefore need both a target-quality ratchet on development data and independent evidence that the learned search transfers to other families. The latter is the north star (proposed; owner ratification pending; defined in §1.3.1 The proposed north star and matched comparisons). Repeatedly attacking one dataset is useful specialization; it is not evidence of broad transfer.

9.4 What the historical training and target evidence establishes

ASM means **Algorithm Synthesis Model: learns to generate and optimize executable algorithms from data**. The repository has real historical neural training and real retained GP teaching corpora. It also has substantial uncompleted work before a newly bounded ASM learning campaign can establish useful improvement. These facts must be stated together. “Zero freshly

admitted teachers under the new ASM admission guard" describes the new campaign's inspected readiness; it does **not** mean that JopeDime has never trained on real data, that the accepted policy is untrained, or that its historical ten-million-record corpora are synthetic.

This chapter reconstructs progress from original retained metrics and checkpoint metadata without loading model tensors, reopening sealed targets, or starting training. It separates three kinds of evidence: loss measurements, changed training state, and improved executable algorithms. Losses measure how the model fits its training objectives. Checkpoint lineage records which parameters and optimizer state were produced. Only a matched downstream fit/search comparison can establish that conditioning on a dataset and split produces better algorithms. A completed optimizer run does not imply the third result.

The retained evidence shows both substantial historical training corpora and some improving historical GP targets. It does **not** show a newly qualified ASM teaching loop, a current thirty-dataset result, or a causal advantage from neural training. Those are different claims with different evidence requirements.

9.5 LOOP-01: the central regression milestone

LOOP-01 is the central regression product milestone. One LOOP-01 generation is the complete chain:

new eligible target → actual exposure → changed parameters → compatible reload → improved fitted search → another new generation

Each arrow is a separate claim with its own evidence. A new eligible target passes the admission guard under its full comparison identity and is absent from the authenticated initial population. Actual exposure is measured in committed optimizer updates, not inferred from corpus membership. Changed parameters are shown by parameter digests before and after training. A compatible reload loads the exported checkpoint in the consumer with its architecture, vocabulary, encoder and source identities intact. An improved fitted search is judged by the preregistered rule in §9.7.3 The proposed preregistered regression decision. Another new generation means that search yields new eligible targets for the next round. Two passes through one corpus are continuation, not two generations.

Status of each link as of 2026-09-23 at about 23:55 UTC:

Link	Mechanism	Status	Evidence now
New eligible target	Five-fold admission guard and strict HWM identity in the pending eligibility source (§7.2.2 The admission guard, label exposure and HWM identity)	pending integration	Zero teachers admitted under the new contract
Actual exposure	Per-update fresh-exposure record and initial-population/discovery identity	proposed	Not measured; the stopped training campaign records neither

Link	Mechanism	Status	Evidence now
Changed parameters	Campaign verification of positive updates and changed parameter digests (§8.4.3 Plan, run, resume, verify and export)	pending integration	Not measured for a new lineage. The retained step 60,100 run did genuine optimizer work on the unchanged corpus with zero fresh program presentations: continuation, not a generation
Compatible reload	Worker reload of `policy.pt` and `resume.pt` with equal parameter proof, then the fail-closed `experimental.regression` consumer (§8.5.2 Compatibility checks and fail-closed rules)	pending integration	Not measured
Improved fitted search	Consumption of the export in a new operational proof, judged by the proposed decision rule	proposed	Not measured
Another new generation	A further teaching campaign over searches conditioned on the reloaded checkpoint	proposed	Not measured

The current one-dataset origin proof, which closes context → proposal → fit → evolution for the accepted model, is available evidence; it is not a LOOP-01 link. No LOOP-01 generation has run.

9.5.1 Lineages, continuation names and generations

The accepted **512×6 step-60,000** model of lineage B (width 512, six layers) remains the serving model and a report baseline. The pending campaign would start a separate line, **experimental regression lineage v1 (256×2, new initialization)**, planned and not yet trained: the width-256, two-layer Python model described in §8.4.1 What the pending campaign trains. Its newly initialized model, `M_init`, is called `M0` only after it has actually trained on the first admitted corpus, `C0`. Step 60,100 is historical hardware continuation of step 60,000, and step 69,000 remains unaccepted (§2.8.1 Checkpoint binding and configuration); neither is a LOOP-01 generation. The two lines are compared only through matched downstream fits, never through loss values.

Every training attempt must declare one of four continuation names (proposed naming): **cold initialization**; **weight warm start**; **explicit new-corpus transition**, which declares whether optimizer state is carried and how the sampler is initialized; and **exact interrupted resume**, which keeps the original state, inputs and deadline. Exact resume must reject an optimizer fallback before any update and prove that interrupted and uninterrupted runs reach the same state. Both requirements are proposed: the stopped campaign checks optimizer resumption only after training and has no interrupted-versus-uninterrupted parity test. A new generation needs a new corpus containing new eligible targets; a second pass or an exact resume on the same corpus continues the current generation.

9.5.2 Two controlled rounds within the existing GPU cap

LOOP-01 calls for two linked controlled rounds. Each round (proposed) trains two sibling branches from one immutable parent model and optimizer state, under the round's original deadline:

- **F (fresh):** samples 25% fresh and 75% replay;
- **R (replay-only):** samples replay only.

Both branches commit the same number of updates and batch slots. For each branch, record the measured fresh exposure, duplicate and unique families, non-padding tokens and GPU cost; keep the unchanged parent as a comparison artifact. Neither branch is promoted to serving. The stopped training campaign has no sibling branches, immutable sibling parent or 25/75 sampler allocation.

The existing allocation gives the regression category at most 20 minutes of GPU time inside the one-GPU-hour campaign (§12.2.2 Resource policy and the training clock), and LOOP-01 grants no further allocation. The two controlled rounds may therefore remain unfunded, or inconclusive within that cap. Either outcome is reported as mechanical proof and descriptive results, with the causal question marked inconclusive.

9.6 The closure ledger

Link in the loop	Existing evidence	Remaining proof	Roadmap connection
Dataset and split → valid context	Current regression producer checks original rows, preprocessing, assets and eight reference cases	Classifier context/ checkpoint consumption and newly trained context compatibility	NEURAL / TRAIN items
Context → real initial GP candidates	Current 80-slot path (64-slot policy pool plus 16 exploration, with explicit rejection fallback) has authenticated genuine `neural_policy` fits	Show the newly trained checkpoint produced the measured candidates	NEURAL / TRAIN items
GP evaluation → improved target	Current bounded search fits and evolves candidates; historical champion and corpus records exist	New admission needs complete finite OOF evidence, baseline improvement on at least 4/5 folds and the full identity	TEACH items
Improved target → frozen teaching corpus	Historical binary teaching stores and corpora survive; pending exporter authenticates inputs	Execute the new eligible-state export with complete admitted-family coverage; record fresh versus repeated examples	TEACH / TRAIN items
Corpus → parameter update	Historical optimizer logs and accepted step-60000 exist	New bounded run must prove positive updates, changed parameters, reload and true optimizer continuation	TRAIN items

Link in the loop	Existing evidence	Remaining proof	Roadmap connection
Parameter update → better policy	Historical imitation/ranking diagnostics exist	Compare parent and challenger on unchanged diagnostic inputs; then measure fitted-program benefit under matched budgets	NEURAL / TRAIN items
Better policy → better next search	Current accepted model reaches actual evolution on one dataset	Run the exported challenger in a subsequent search and retain a causally interpretable comparison	NEURAL / PANEL items
Improved search → next useful training generation	Historical controller and target chains exist	Complete at least two linked new cycles (flywheel turns, that is LOOP-01 generations as defined in §9.5 LOOP-01: the central regression milestone, not training blocks or passes) with eligible new targets, actual exposure and measured downstream outcomes	End-to-end flywheel qualification
Qualified checkpoint → serving	Product is bound to recovered accepted step-60000	Design and qualify the missing experimental-to-serving promotion contract, then require the new checkpoint to pass it; training-pointer movement alone does not move the serving pointer	FUTURE-01: deferred; no executable acceptance command yet

The prefixes above refer to the detailed roadmap's existing workstreams; they are not new executable command names. The current one-dataset proof closes context→proposal→fit→evolution. It does not close the new teaching→training→improved-search cycle. The historical training chapter separately reports what earlier controllers actually achieved, including unavailable or inconclusive promotion evidence.

9.7 A minimal experiment that can establish learning

9.7.1 Freezing the experiment and choosing families

Freeze the experiment before opening the run window: source and binaries, exact parent checkpoint, corpus membership, encoder and vocabulary, objective weights, optimizer carry policy, sampler, random streams, fit/search budgets and comparison metrics. Keep the accepted parent available as a control. A new objective, new encoding or incompatible optimizer state starts a new lineage; it must not inherit a misleading uninterrupted-step counter.

Use eligible teachers from independently identified training families. Keep family-disjoint development diagnostics for the model. When a dataset was used to manufacture teachers, label subsequent evaluations on that dataset as exposed/adaptive. The pending path's two prospective teaching families, `230_machine_cpu` and `712_chscase_geyser1`, are both frozen-panel regression datasets (§7.4 Present teacher readiness and the smallest real path); if they supply teachers, later panel reports must label those two datasets exposed/adaptive. Unknown historical checkpoint exposure remains unknown. A split name alone cannot prove an unseen family. Assign families, at family level, to three domains:

teacher/train, adaptive development, and evaluation outside training and selection. The frozen thirty-dataset panel remains adaptive development, and the pretrained encoders' exposure to any family remains unknown. A hash authenticates bytes; it does not make an input eligible. The input closure covers records, contexts, transforms and all assets, and unused forbidden descriptors must be rejected before loading. These are proposed requirements: the stopped training packet validates only the contexts that its retained rows reference, does not reject extra context-index entries, and partitions families only into train and development domains.

9.7.2 What to retain for each cycle

For each cycle, retain all of the following:

1. **Target creation:** incumbent and challenger recipe hashes, all five candidate and baseline OOF losses, complete fit outcomes and exact comparison identity. Authenticate the complete initial seed population. Distinguish a newly discovered program absent from that seed set from a carried incumbent, a neural seed and its evolved descendants. Retain discovering arm, program hash, generation, context, parent/ancestry, first measurement and later shared-seed exposure. Count each distinct dataset/cut/program discovery once before reporting per-arm reuse. Record rejected, tied, failed and unavailable candidates too.
2. **Corpus delta:** previous/new corpus hashes; added, improved, removed and unchanged comparison/program identities; distinct contexts and families; rejection reasons. Distinguish logical rows from unique programs and unique contexts.
3. **Actual optimizer exposure:** how often newly admitted programs and contexts appeared in committed steps, their weights, sample repetition, padding fraction and fresh/broad sampling fractions. New rows in a manifest are insufficient if the sampler never reaches them.
4. **Training continuity:** initialization parent, parameter hashes before/after, optimizer initialization versus restored moments, local and cumulative committed steps, effective batch and actual clock/resource use. A failed attempt's compute stays in accounting.
5. **Policy diagnostics:** component losses on a fixed, identified monitor; valid-program rate, uniqueness, conditional ranking and proposal quality. Report corpus/monitor changes as boundaries rather than drawing a continuous evaluation curve through them.
6. **Next-search outcome:** execute both parent and challenger proposals as actual fits under the same candidate/compute allowance, plus the generic control. Retain neural-origin and ancestry counts, failure rates, selected program identities and the downstream losses.
7. **Decision:** accept, reject or inconclusive with the exact gate and original evidence; for regression rounds, apply the proposed rule in §9.7.3 The proposed preregistered regression decision. Preserve the old serving pointer unless promotion succeeds.

Two linked successful cycles are a practical demonstration of repeatability. They do not prove unlimited improvement. Plateaus, search-space ceilings, noisy estimates, sample reuse and model capacity remain real constraints. Report the improvement per cycle and cost per eligible improvement; do not promise a monotonic performance curve on unseen data.

9.7.3 The proposed preregistered regression decision

This is the owner's proposed decision rule for LOOP-01 regression rounds. It is a proposal, not an achievement: no round has been judged by it. Preregister it, together with the monitor families, seeds, folds and trivial predictor, before any challenger is compared, and do not change it after seeing results.

9.7.3.1 Family-normalized gain

For each monitor family, the gain is `g = (L_control − L_challenger) / L_trivial`.

`L_control` is the comparison model's loss: the parent for the main test, and the replay-only branch for the fresh-versus-replay test. `L_challenger` is the challenger's loss, and `L_trivial` is the loss of the preregistered trivial predictor on the same family. Positive `g` means the challenger is better. Average over the declared seeds and folds within each family, so that each family contributes one value. The preregistration states explicitly how a family with zero or nonfinite `L_trivial` is treated; no hidden epsilon is added and no such family is dropped silently.

9.7.3.2 Round thresholds

A round shows a gain only if all of the following hold:

- at least 10 independent monitor families, each with complete comparisons;
- at least 9/10 of the families gain more than 1% against the parent;
- the mean gain across families exceeds 1%;
- no family is worse than −5%;
- the fresh branch beats the replay-only branch in at least 9/10 of the families, with a positive mean gain.

Round 2 uses monitor families separate from those of round 1.

9.7.3.3 Baselines, stopping and reporting

Generic search and the accepted step-60,000 model remain report baselines. A gain over the parent is a different claim from an advantage over generic search or over the product's accepted model. Stop the loop when there are no eligible discoveries, on an incompatibility, when there is no material gain, when the evidence is insufficient, or when the budget is exhausted. Preserve the best checkpoint and never promote to serving automatically. An unfunded round, or one with insufficient data, yields mechanical proof and descriptive results; its causal conclusion is "inconclusive", never an invented success.

9.8 Distinguish routing, ranking and search recovery

The learned route is a conditional distribution over executable program tokens given the dataset-and-target context. It is richer than selecting a fixed named estimator. The same model also supplies learned ranking signals. Its reward representation mean-pools independent token embeddings, a bag of actions, with no positional term: at `a75e543d2` both the Python model (`python/src/jope/policy/model_v3.py:399-404`) and the product's Rust port (`crates/jope-policy/src/model.rs:555-567`) compute a padding-masked mean. It has no learned representation of token order: for the same context, programs with the same token multiset have the same pooled representation in exact arithmetic. This limits ranking distinctions between programs whose execution differs only through token order; it does not remove order from the autoregressive proposal decoder. See §2.8.2 Decoder and reward architecture; §9.9 Conditioning diagnostics before the first causal experiment proposes measuring its collisions before any architecture change.

Measure the policy before GP recovery as well as after it. A long search can find the same winner despite poor neural proposals, while a short search may reveal useful initialization that disappears at a larger budget. Shared incumbents can create many ties. Equal time caps do not imply equal completed fit counts, especially when candidate runtimes differ.

The admitted historical evidence in §C.1 Did GP targets get better? illustrates this limit: at 1,000 completed evaluations, 3,268 paired neural/generic final-search observations contain 24 lower neural scores, 17 higher scores and 3,227 exact ties. Their geometric mean neural/generic RMSE is 1.0001869094. They repeat families and generations, consume unequal reported wall time and measure adaptive final winners. The historical arms also share incumbent, retained-teacher and exploration seed pools; a winner can seed every arm in the next generation. In the separate first-to-last table, 64 lower arm groups collapse to 23 distinct dataset/cut pairs out of 55, with identical available-arm final scores in 22 of those 23. Those are dependent carried-forward outcomes, not 64 independent discoveries or wins. They neither establish initial routing quality nor show that later training caused better search. A larger external extract is kept outside this claim where its rows have not joined the admitted inventory.

Useful controls answer different questions: generic grammar sampling tests end-to-end benefit over the non-neural search; the previous accepted checkpoint tests whether training helped; shuffled contexts test conditional information; and random ranking on the same candidate pool tests ranking utility. These additional controls are proposed experiments, not extra runs hidden inside the approved panel. Declare their budget and scope before execution.

9.9 Conditioning diagnostics before the first causal experiment

These diagnostics ask whether the policy actually uses its dataset-and-split context. All are proposed and none has been measured. This manual proposes running them before the first causal LOOP-01 experiment, and keep every diagnostic substitution outside teacher admission: a program proposed or ranked under a substituted context never becomes a teacher.

Diagnostic	Question	Comparison	Status
Boundary correctness	Is the context built only from permitted fitting information, within an input closure that covers the exact records, contexts, transforms and assets?	Unused forbidden descriptors are rejected before loading	proposed
Functional sensitivity	Do proposals and ranking scores change when the context changes?	The true context against substituted contexts	proposed
Useful ranking	Does the learned ranking order candidates better than simpler signals?	Donor-context, program-only and random baselines on one fixed, already-fitted candidate bank	proposed

The current regression producer already checks original rows, preprocessing, assets and eight reference cases (§9.6 The closure ledger). The stopped training campaign, however, does not reject extra context-index entries or unlisted transitive artifacts, so boundary correctness is not yet established for training inputs.

The reward path's order blindness is measured before it is changed. Both implementations mean-pool token embeddings without a positional term (§9.8 Distinguish routing, ranking and search recovery). First measure how often distinct programs in one context collide on the same pooled representation, and the ranking regret those collisions cause. Defer architecture changes, so that the first causal experiment keeps architecture, objective, corpus and sampler changes distinguishable.

Dense-width feasibility checks must use only permitted fitting information, keep the original proposal limits and clocks, and report a preflight rejection separately from a launched fit failure; they grant no unlimited free retries. A naive preflight must not rely on the retryable `Unavailable` outcome that a job-preparation error receives at `a75e543d2` (`crates/jope-pipeline/src/neural_search.rs:1608-1623`); it needs a terminal outcome with its own accounting. The source investigation is custody `plan-execution-20260923/feasibility-phase-accounting.md`, SHA256 `0154e6b0074d6b3c7100abf28568abb41a998b6ed21730237d58649f31cc9f18`. This check is proposed source work.

9.10 Specialization without starving the general learner

One difficult dataset can produce many near-duplicate teachers. Count unique canonical programs, meaningful improvements and distinct contexts, not only serialized rows. Retain broader replay and family balance so a successful focused attack does not make the policy forget other problems. §D.7.2 Packing semantics, legal sets and label policies describes how to share repeated contexts and immutable shards without losing original provenance.

Record the fresh-program exposure actually sampled. A nominal 50% fresh-comparison schedule need not yield 50% fresh program slots when each comparison also contains an incumbent and old comparators. Prefer measured exposure, a frozen baseline checkpoint, component-loss diagnostics and actual downstream fitting over assumptions based on the sampler's name.

If the target ratchet moves but fixed-monitor policy quality deteriorates, investigate the objective, label consistency, sampler and comparator support before simply training longer. If policy metrics improve but fitted searches do not, inspect proposal diversity, candidate runtimes, grammar coverage and GP recovery. If neither improves, report the plateau and preserve the best verified incumbent. The system earns a flywheel claim through this evidence, not through an endlessly running process.

9.11 A KPI ledger with defined denominators

9.11.1 KPI definitions

These are proposed reporting definitions for acceptance, not a claim that every current report already emits every field. Preserve the fixed protocol where it already specifies a quantity. Additions must be preregistered before comparing challengers; do not change a definition after seeing results.

KPI	Definition and denominator	Evidence and present gap
Task loss delta	For a valid matched pair, `loss_reference − loss_challenger`; positive means better. Regression loss is RMSE; binary loss is `1−AUROC`; multiclass is ordered probability-matrix log loss. Report both generic and previous-checkpoint references separately.	Exact predictions, rows/classes, evaluator, numerical policy and resource receipts. Never average raw RMSE with log loss across tasks.
Strict wins / planned comparisons	Count valid matched improvements over all preregistered comparisons, including failed/unavailable comparisons in the denominator. Report ties, losses, one-sided failures and two-sided failures separately.	Complete planned inventory. Failure is not a fabricated numerical loss; paired mean delta is separately labeled as complete-pair-only with coverage.

KPI	Definition and denominator	Evidence and present gap
PANEL-WINS (proposed definition; owner ratification pending)	Per task, the number of the 10 frozen-panel datasets (asm-private-30-v1, protocol asm-panel-v1) on which `asm.neural`'s confirmed five-fold mean development loss is strictly below the minimum over all eligible frozen non-ASM comparators (every `rust.*` and `external.*` recipe). `asm.generic` is an ASM control reported in its own column. An incomplete, failed or unconfirmed dataset is not won. Reported as (regression, binary, multiclass); the owner's goal of ten per task, (10, 10, 10), is a 30/30 sweep.	Not computed: no win-counting function exists in `crates/jope-bench/src` at `a75e543d2`, and the panel has not run. The statistic is adaptive and favours `asm.*`: the ASM recipe is selected on cut-0 fitting rows that the five-fold mean also scores, while `rust.*` and `external.*` recipes are fixed before the dataset screen (`crates/jope-bench/src/panel/selection_exposure.rs:91`; §11.2.2 Adaptive development exposure). At source `68f56404` binary and multiclass have no neural arm (§2.7.2 Experimental classification context).
Family generalization	Per-task mean of family means under the preregistered split, with cluster uncertainty at family level when sample size permits	Frozen family provenance and prior-exposure evidence. No current frozen-panel (asm-private-30-v1) unseen-family claim; historical unknown exposure stays unknown.
Relative improvement	`(Lref−Lnew)/Lref` only for finite, strictly positive reference loss; report absolute delta when zero	No hidden epsilon convention or cross-task aggregate introduced after results.
Family-normalized gain (LOOP-01; proposed)	Per monitor family, `(L_control − L_challenger)/L_trivial`, averaged over the declared seeds and folds within that family; the denominator is the preregistered monitor families, each with complete comparisons	Not measured. A zero or nonfinite `L_trivial` needs an explicitly preregistered disposition (§9.7.3 The proposed preregistered regression decision).
Teaching admission	Eligible comparisons / all requested comparisons, and eligible independent families / all requested families	Preregistered five-fold train-side OOF evidence, finite baseline improvement and improvement on four of five folds. Newly demonstrated ASM-qualified count currently zero; historical corpus counts are separate.
Genuine neural consumption	Completed `neural_policy` generation-zero candidates / actual `neural_policy` slots; also report completed genuine neural candidates /64 policy-pool slots, fallback slots and completions, and exploration completions /16. Completed descendants require verified neural ancestry.	Authenticated proposal→request→started→finished→artifact chains. A recipe file, neural tag, generic fit or started-only attempt is insufficient.
Validity and fit coverage	Canonical task-compatible proposals / all proposed draws; completed fits / all scheduled eligible fits	Distinguish grammar refusal, dependency unavailability, fit error, timeout, cancellation, never-started and unknown. State whether duplicate candidates are deduplicated.

KPI	Definition and denominator	Evidence and present gap
Optimizer progress	Updates this run, cumulative optimizer step, start/end parameter digests, committed passes, and exact resumed-state identity	Batch/epoch metrics and checkpoint proofs. Positive epoch count or a checkpoint filename is insufficient; training loss denominator and objective must match before plotting together.
Latency	Cold build/install/preparation separately; warm end-to-end wall time and declared target `<60s`; actual per-arm planned/available/executed time	Original monotonic/absolute windows and launch receipts. A completed >60s demo can be execution-complete with target missed.
Resource and cost efficiency	Total known creation CPU/GPU/wall and monetary/token costs divided by authenticated distinct complete fitted artifacts; also cost per distinct reusable recipe	Distinct means declared hash identity, not proven functional uniqueness. Whole-unit counters overlap worker sums; do not add them. Unknown overhead, price or tokens remain null with reason.
Fitted portability	Qualified exact selected fitted predictor/backend pairs / all requested selected predictor/backend pairs	Actual emitted code execution and prediction parity, in order Python/Rust then C++17/C++20 then JS. Unsupported pairs stay in the inventory and never replace the winner. Current five-language toy parity is recipe refitting only.
Reproducibility	Successfully authenticated independent report/reload replays / all requested replays	Same source, model, input, plan, protocol and artifact identities. Stale outputs, missing rows or changed class order are failures.
Promotion	Accepted serving-pointer advance with independent matched evaluation, exact checkpoint identity and controlled pointer update	Separate from training-pointer advance and experimental export. No new promotion is claimed.

9.11.2 Win counting and a proposed acceptance record

Define a dataset-level comparison using the protocol's frozen fold summary; retain fold-level results separately. Keep separate win columns for generic search, the previous checkpoint, and the best eligible frozen non-ASM comparator; the last column is PANEL-WINS as proposed in §9.11.1 KPI definitions. A win against one reference is not automatically a win against the others. For the owner's goal of ten wins per task on the 30-dataset panel, each reference's denominator remains ten per task and all thirty outcomes are listed, including unavailable ones; meeting it in every task is a 30/30 sweep. It is a declared goal, not a promise. As of 2026-09-23 there is no completed full-panel result establishing those counts.

A minimal additional acceptance record could use the following explicit nulls. This is a **proposed reporting shape**, not an existing on-disk schema:

```
{
  "definition_status": "proposed_operator_reporting",
  "source_commit": null,
  "checkpoint_state_sha256": null,
  "reference_checkpoint_state_sha256": null,
  "corpus_sha256": null,
  "encoder_identity_sha256": null,
```

```
  "vocabulary_sha256": null,
  "panel_manifest_sha256": null,
  "protocol": null,
  "original_deadline_unix_ms": null,
  "planned_comparisons": 30,
  "valid_matched_pairs": null,
  "strict_wins": null,
  "eligible_teacher_families": null,
  "optimizer_updates_this_run": null,
  "exact_fitted_backend_pairs_qualified": null,
  "creation_cost": {"usd": null, "tokens": null, "unknown_reason": "not measured"},
  "serving_promoted": false
}
```


Retain the receipt path and content hash for each populated quantity, plus its unit, scope, numerator, denominator, exclusions and measurement time. Compare the supplied one-hour/ten-million-token scenario with actual token and cost records if those become available; a token budget is not evidence of tokens consumed, nor does it determine GPU time or dollars. Historical totals, current implementation counts and newly executed acceptance must remain separate series.

9.12 What is still needed to demonstrate the learning flywheel

The desired loop is concrete: a dataset and split produce verified encodings; conditioned policy proposals become real generation-zero fits; bounded GP/evolution produces eligible improved programs; the full comparison identity admits those programs as teaching targets; optimizer updates produce a new checkpoint; matched downstream fits show whether that new checkpoint proposes and searches better algorithms. Only then can a promotion change the serving pointer. §7.1 A dataset is not one training example and §D.7 Binary format: the exact packed teaching record trace the historical input-to-target representation, and §C.1 Did GP targets get better? separately reports observed adaptive target improvements in repeated searches with matching recorded comparison fields, including improvements in generic and shuffled arms. Those observations are largely carried-forward development scores from seed pools the arms share; they do not establish neural causality. §9.3 What “better and better” must mean defines the remaining current acceptance work.

For every cycle, preserve both sides of the feedback: target improvement under the same task/dataset/split/evaluator/vocabulary/protocol identity, and policy improvement in fitted performance, failure rate and cost under a frozen comparison. Report cold/warm preparation and unique fitted predictors separately from recipe counts. Keep a context-free control, generic proposal control and unchanged accepted policy. A lower teacher-forced NLL can accompany memorization, degraded ranking, or no better executable search, as the historical measurements demonstrate.

The missing historical evidence is specific: full intermediate trajectories for the older 108-block capture (distinct from the complete later 63,000-step series); an exact run-source binding and event timestamp for gen31; per-term gradient diagnostics; a common independent evaluation series across changed corpora; and a fully joined sequence of qualified target improvements and serving promotions. For the new bounded ASM campaign, corpus admission, actual optimizer update/reload/resume proof, trained-export consumption and matched downstream improvement remain acceptance work. Keep these gaps visible rather than replacing them with a smooth synthetic curve or treating old training as if it had never happened. This chapter contains no performance plot. Every LOOP-01 quantity, including per-round family-normalized gain, the fresh-versus-replay difference and cycle-over-cycle improvement, is not measured. A future figure of any of them is drawn only from receipts, and every unmeasured series says "not measured" rather than showing an illustrative curve.

To establish a closed improvement loop rather than these separate observations, join: original search/fit completion → exact program and comparison identity → admission event and immutable target record → corpus membership → actual sampler exposure and optimizer continuation → later policy proposal/fit under the same comparison protocol. The historical logs in §B.1 Dated lineage and evidence coverage show training activity; the target receipts here show some improving, largely carried-forward development scores. That complete causal/eligibility join is still missing.

CHAPTER 10

Complete fitted inference and language support

IN THIS CHAPTER

10.1 Deployment support: definition, priority and status 120
10.2 Language interfaces at the frozen source 121
10.3 Complete registered-recipe matrix 122
10.3.1 Coverage by recipe and backend 122
10.3.2 State restrictions beyond the head table 124
10.3.3 External winners 124
10.4 Private worked examples and their measured execution 125
10.4.1 Example A: a tiny private recipe, fit and predictions 125
10.4.1.1 The synthetic fixture 125
10.4.1.2 Fit and emit the recipe 126
10.4.1.3 Run and compare the emitted programs 126
10.4.2 Example B: actual legacy pipeline/search shape 127
10.4.3 Measured execution of the private examples 127
10.4.3.1 What passed and what was measured 127
10.4.3.2 Resources, accounting and scope limits 128
10.5 Example C: exact fitted winner export after pending integration 128
10.5.1 Export commands and winner selection 129
10.5.2 Tool inventory, qualification order and isolation 130
10.5.3 Backend numerical policies, public metadata and the export clock 130
10.6 Future work and acceptance boundaries 131

At source `68f56404c3e199e06413bfd645d3fe0803d7b96f`, merged unchanged as main `a75e543d2` (same source tree), the legacy `jope emit` and `jope pipeline run --emit` paths produce type 3. `jope fit` computes a fit and reports scores but does not provide a saved fitted-artifact output flag. The pending product/winner packets, which are not part of merged main, expose type 4 from complete benchmark/demo fitted state. Do not advertise a legacy recipe file as the exact fitted winner.

This chapter and the supplied tiny fixture are **maintainer-private**. The DSL, private recipe manifest, discovery metadata and qualification references do not belong in a public inference bundle. The public allowlist described below is deliberately narrower than the manual's working directory.

10.1 Deployment support: definition, priority and status

The owner's rule for this chapter, relayed on 2026-09-23 at 22:27:11 UTC, sets what may be called deployment support. It is a requirement for export work, not a description of the current source:

- **A complete fitted pipeline.** A deployable artifact carries the whole fitted pipeline of the selected model: its ordered transforms, fitted dictionaries, target mappings, classes and their order, and its numerical policy.
- **The actual winner, without refit.** The backend executes the selected winner from its saved fitted state. Refitting a recipe, even the same recipe on the same rows, is not deployment support.

- **Unsupported stays visible.** When a backend cannot represent the winner, that backend's result is "winner + unsupported". The winner is never replaced by another recipe that happens to export.
- **Priority.** Python and Rust first, then C++17 and C++20, then JavaScript.
- **Inside the original window.** Export work uses the original campaign window and stops before its reporting reserve (§10.5.3 Backend numerical policies, public metadata and the export clock).

By this rule, no route at merged main `a75e543d2` provides deployment support yet. Status as of 2026-09-23, about 23:55 UTC:

Route	What it executes	Status
Legacy recipe emitters (`jope emit`, `jope pipeline run --emit`)	Generated source that refits a recipe from a training CSV. None of the 26 registered benchmark recipes can be emitted (§10.3.1 Coverage by recipe and backend).	**available** for recipe refit only, which the fourteen-command private example run qualified on its fixtures (§10.4.3 Measured execution of the private examples). This is not deployment support.
Private fitted C++17 library (`emit_fitted_cpp17`)	Scalar-state C++ generated from a saved `BoundModel`, without refit	**implemented but unqualified** as a deployment route. At `a75e543d2` only the research crate `crates/jope-campaigns` calls it; the `jope` CLI does not. Its admission record, `crates/jope-campaigns/manifests/advanced-v3/fitted-cpp-v3-admission.v1.json`, is marked `test_adaptive: true` and `benchmark_valid: false`.
Public fitted bundles for Python/Rust, C++17/C++20 and JavaScript, driven by `benchmark export-winner` and `benchmark qualify-export`	The selected winner's saved fitted state, lowered for the supported subset, without refit	**pending integration**. These exist only as stopped custody patch views, absent from merged main (§10.5 Example C: exact fitted winner export after pending integration). No winner has been exported or qualified.
CUDA inference	Nothing yet	**proposed**, and deferred until the backends above qualify.

10.2 Language interfaces at the frozen source

Interface	Input and output	Coverage and operational meaning
Native Rust `feat_math`/advanced-v3	Typed recipe or authenticated `BoundModel`; numeric or raw-table prediction boundary	The benchmark uses the raw-table boundary and preserves complete private fitted state. The native roster is task-specific.
Legacy Python emitter	Generated single-file Python with `Model` fit/predict and CSV CLI	Numerical recipe execution; host `libm`. Supports the emitter's legacy program set, not every advanced-v3 state.
Legacy Rust emitter	Generated standard-library Rust with fit/predict and CSV CLI	Numerical recipe execution; source emitted with Python as a pair. Compiler/runtime and fixture qualification remain required.

Interface	Input and output	Coverage and operational meaning
Legacy C++ emitter	One C++ source, C ABI, numeric input, fit/predict; external CSV driver for examples	Ridge/ElasticNet heads and the emitter's admitted transform subset. C++17 and C++20 use this same source with different standard flags.
Legacy JavaScript emitter	ES2020 numeric fit/predict API; external CSV driver	Ridge/ElasticNet heads and its admitted transform subset. `Math` may differ from native platform libm.
Existing private fitted C++ library	`supports_fitted_cpp17`, `emit_fitted_cpp17` over `BoundModel`	Broader scalar-state emitter exists at `68f56404` and merged main `a75e543d2`. Its source gate is not the pending public bundle gate or proof of winner parity. Multiclass probability matrices are refused by its scalar ABI.
Pending public fitted Python/Rust	`predict_raw` over ordered raw rows with string/null cells	Complete selected-state lowering for the supported subset below; no model refit.
Pending public fitted C++17/C++20	Same generated C++ inference core plus `codec.json` and Python `ctypes` raw-table boundary	Public gate currently restricts this broader emitter to regression ridge and binary logit ridge. Inference therefore needs a Python 3 `ctypes` host beside the compiled library (the pending packet names a Python 3.10 `ctypes` typed-table host); it is not a self-contained C++ raw-table parser.
Pending public fitted JavaScript	`predictRaw(rows)` over raw rows; Node/CommonJS qualification harness	Linear and multiclass subset below; a declared ECMAScript numerical policy and tolerance are used.
CUDA	No qualified public fitted-export route in this delivery sequence	Deferred until Python/Rust, the one C++ emitter under both standards and JavaScript are qualified. A training GPU does not establish CUDA inference parity.

Source anchors: `crates/jope-cli/src/kernel.rs`, `crates/jope-pipeline/src/emit.rs`, `crates/feat-math/src/{transpile,emit_py,emit_rs,emit_cpp,emit_js}.rs`, `crates/feat-math/src/advanced_v3/fitted_cpp.rs`, and the pending `advanced_v3/fitted_portable.rs` plus `crates/jope-bench/src/panel/inference.rs`, which are absent from merged main `a75e543d2` and exist only as stopped, unintegrated custody patch views (§E. 4.2 Pending source references).

10.3 Complete registered-recipe matrix

10.3.1 Coverage by recipe and backend

The table covers all 26 task-compatible native benchmark recipes. **Yes means the source admits the recipe/head under the additional state restrictions; it is not a measured parity result.** Every row has native Rust fit/save/reload support in the benchmark contract. Public fitted columns come from the pending product packet, which is not in merged main `a75e543d2`; they are unverified against merged source and require actual selected-state qualification. The private C++ column describes the head gate that exists at `a75e543d2` only. Legacy `jope emit` has no column because it can

emit none of the 26 recipes: each carries an `input` stage (`crates/jope-bench/src/panel/methods.rs:20-21`, `:50`), which the legacy planner rejects (`crates/feat-math/src/spec.rs:383-389`, `:592`). External adapters are covered in §10.3.3 External winners.

Task	Registered method ID	Private C++ scalar head gate	Public Python/ Rust fitted	Public C++17/20 fitted	Public JavaScript fitted
Regression	`rust.ridge`	Yes	Yes	Yes	Yes
Regression	`rust.ridge_gcv`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.elasticnet`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.histgbm`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.histgbm2`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.ksmooth`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.ensemble`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.krr`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.gpr`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.hurdle2`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.stack`	Yes	Unsupported	Unsupported	Unsupported
Regression	`rust.tab_boost`	Yes	Yes	Unsupported	Unsupported
Regression	`rust.extra_trees`	Yes	Yes	Unsupported	Unsupported
Regression	`rust.gam_boost`	Yes	Yes	Unsupported	Unsupported
Regression	`rust.pls`	Yes	Yes	Unsupported	Unsupported
Regression	`rust.portfolio` (`core_v1`, `preset=core`)	Yes, members checked	Unsupported	Unsupported	Unsupported
Regression	`rust.gpr.rbf_linear`	Yes	Unsupported	Unsupported	Unsupported
Binary	`rust.logit_ridge`	Yes	Yes	Yes	Yes
Binary	`rust.tab_boost`	Yes	Yes	Unsupported	Unsupported
Binary	`rust.extra_trees`	Yes	Yes	Unsupported	Unsupported
Binary	`rust.gam_boost`	Yes	Yes	Unsupported	Unsupported
Binary	`rust.portfolio`	Yes, members checked	Yes	Unsupported	Unsupported
Multiclass	`rust.multiclass_ridge`	Unsupported matrix ABI	Yes	Unsupported	Yes
Multiclass	`rust.multiclass_ovr_logistic`	Unsupported matrix ABI	Yes	Unsupported	Yes
Multiclass	`rust.multiclass_diagonal_lda`	Unsupported matrix ABI	Yes	Unsupported	Yes

Task	Registered method ID	Private C++ scalar head gate	Public Python/ Rust fitted	Public C++17/20 fitted	Public JavaScript fitted
Multiclass	`rust.multiclass_nearest_centroid`	Unsupported matrix ABI	Yes	Unsupported	Yes

This yields static public coverage of 14/26 recipes for Python/Rust, 2/26 for each C++ standard and 6/26 for JavaScript. The existing private C++ head gate admits 22/26 (every regression and binary recipe); it is neither the public bundle gate nor a parity result. Those counts are not percentages of datasets, winners or actual successful parity runs. Python/Rust's twelve unsupported registered recipes are legacy regression heads outside the public fitted lowering. Generated programs can introduce additional unsupported state even when their final head appears in a supported row.

The registry records aliases separately: regression ridge has the historical alias `logistic`, histogram boosting has `gbm`, the ensemble has `ensemble_ridge_gbm`, and binary logit ridge has `logistic`. Aliases do not consume extra fit slots. `gpr.rbf_linear` remains a distinct configuration. Do not infer task compatibility from an ambiguous alias; use task plus exact registered recipe identity.

Source anchors: `crates/jope-bench/src/panel/methods.rs::rust_methods`, `advanced_v3/fitted_cpp.rs::{supports_fitted_cpp17,check_head}`, pending `advanced_v3/fitted_portable.rs::{fitted_portable_unsupported,fitted_javascript_unsupported,fitted_public_cpp_unsupported}` and `advanced_v3/fitted_portable/scalar.rs::supports`.

10.3.2 State restrictions beyond the head table

The pending public lowering validates the full model binding and fitted input dimensions. It admits IID rows and fitted IID row order only. Historical state, other model forms, target mappings, nonempty general transform lists and calibration are currently unsupported. Categorical encoding must be the admitted fitted one-hot mode. Missing-aware state is limited to the supported tabular-boosting and extra-tree implementations. Invalid coefficients, schema bindings or dimensions are errors, not legitimate unsupported fallbacks.

The benchmark's registered recipes use numeric median imputation, categorical one-hot encoding, missing indicators, `min_count=1` and `max_onehot=256`. Their fitted dictionaries and imputation statistics are part of the state. A deployment must submit columns in the recorded order and use the recorded raw string/null representation. An unseen category must follow the fitted input policy; neither the host nor caller should invent a new category mapping. Binary probability outputs include the two original class names and order; multiclass outputs include every original class column.

The existing private C++ library has broader checks for target mappings, transforms, calibration and ordered/raw input than the public fitted gate. Its broader admission must not be used to skip public privacy, raw-boundary or numerical qualification. The selected public subset is a deliberate current boundary, not evidence that every other native model is intrinsically unexportable.

10.3.3 External winners

XGBoost, LightGBM, CatBoost and scikit-learn are executable benchmark adapters for the admitted tasks. Their native fitted artifact retains the estimator pickle and preprocessing/reload evidence. The pending public winner-export helper reports standalone Python, Rust, C++17, C++20 and JavaScript export of that external library state as **typed unsupported**. It preserves the external winner, all losses and native fitted artifact. Installing a library in Python does not make its pickle a dependency-free Python inference bundle.

10.4 Private worked examples and their measured execution

Examples A and B are runbooks that can be executed at merged main. Their runbook summary:

Runbook part	Examples A and B
Purpose and status	Show a legacy recipe fit, five-language recipe emission with refit and a bit comparison (A), and the shape of the legacy pipeline search (B). **available**: all fourteen packet commands passed on 23 September 2026 (§10.4.3 Measured execution of the private examples).
Prerequisites and resources	A checksum-verified immutable CLI (`J`), the reviewed source checkout (`R`) for the drivers and helpers, `python3`, `rustc`, `g++` and `node`, and a new durable private output directory (`A`) outside Git and `/tmp`. Numerical work needs its own allocation; the executed run declared one CPU, 4 GiB memory, no swap or GPU, no network and 300 seconds in total.
Manifest-creation command	None. The packet `examples/commands.json` is a committed, hand-prepared file, and no committed runner executes it; the 23 September run used a private controller retained in custody. `examples/execution-receipt.json` points from the packet to that run's receipts.
Execution	The listings below, or the argument arrays of `examples/commands.json` with their placeholders filled in.
Artifacts and success	For A: `fit-report.json`, `emit-report.json`, five `pred-*.txt` files, `predictions.bits` and `recipe-parity.json`. For B: `pipeline-result.json` and the pipeline's `receipt.json`. Success means every command exits zero, the validator finds all five prediction files with exactly eight finite, bit-identical values, and receipt verification passes.
Failure and resume	A nonzero exit is a failure; keep its stderr. The packet has no resume step: rerun it into a new output directory.
Limitations	Recipe refits on one synthetic fixture and one legacy pipeline fixture only. No saved-fitted-state export, neural ASM, panel or training qualification.

10.4.1 Example A: a tiny private recipe, fit and predictions

10.4.1.1 The synthetic fixture

The supplied `examples/gp-synthetic.json` contains 24 explicitly synthetic rows with two integer, non-null features and target `3*x0 − 2*x1 + 5`: 16 training rows, four validation rows and four test rows. It is a maintainer example, not one of the frozen thirty datasets or the official ten generated fixtures. `train.csv` contains the same sixteen training rows as headerless `x0,x1,target`; `predict.csv` contains the eight validation/test feature rows with targets removed.

One training row is `-12,-5,-21`; the corresponding typed JSON row is:

```
{"id":0,"features":
[-12,-5],"target":-21,"split":"train","group":null,"timestamp":null,"series":null}
```

10.4.1.2 Fit and emit the recipe

Set `R` to the repository root, `E` to `$R/docs/internal/asm-manual/examples`, `J` to a checksum-verified immutable CLI and `A` to a new durable private output directory. Execute numerical work only after its allocation is available. Executed on 23 September 2026: all fourteen packet commands of Examples A and B passed (§10.4.3 Measured execution of the private examples). The listings are reproducible instructions, not receipts; `mkdir` and the two `cat` concatenations are setup steps outside the fourteen:

```
mkdir "$A"
"$J" fit --input "$E/gp-synthetic.json" \
  --genome '(pipe X (standardize) (ridge))' --json > "$A/fit-report.json"
"$J" emit --genome '(pipe X (standardize) (ridge))' \
  --lang all --output "$A/recipe" --json > "$A/emit-report.json"
```

`fit-report.json` records training/holdout R^2 for this numeric legacy path. The holdout prefers validation rows; the fallback is test rows if validation cannot be loaded. Do not use this convenience path to establish the frozen-panel leakage or classification protocol. It does not persist a complete fitted artifact. `emit-report.json` lists language source sizes, paths and hashes, and each emitted program below fits its own model from `train.csv`.

10.4.1.3 Run and compare the emitted programs

```
python3 "$A/recipe/model.py" "$E/train.csv" "$E/predict.csv" "$A/recipe/pred-python.txt"
rustc -O "$A/recipe/model.rs" -o "$A/recipe/model-rs"
"$A/recipe/model-rs" "$E/train.csv" "$E/predict.csv" "$A/recipe/pred-rust.txt"

cat "$A/recipe/model.cpp" "$R/deploy/ci/emitters/driver.cpp" > "$A/recipe/program.cpp"
g++ -std=c++17 -O3 -ffp-contract=off -fno-fast-math \
  "$A/recipe/program.cpp" -o "$A/recipe/model-cpp17"
"$A/recipe/model-cpp17" "$E/train.csv" "$E/predict.csv" "$A/recipe/pred-cpp17.txt"
g++ -std=c++20 -O3 -ffp-contract=off -fno-fast-math \
  "$A/recipe/program.cpp" -o "$A/recipe/model-cpp20"
"$A/recipe/model-cpp20" "$E/train.csv" "$E/predict.csv" "$A/recipe/pred-cpp20.txt"

cat "$A/recipe/model.js" "$R/deploy/ci/emitters/driver.js" > "$A/recipe/program.js"
node "$A/recipe/program.js" "$E/train.csv" "$E/predict.csv" "$A/recipe/pred-js.txt"
python3 "$R/deploy/ci/emitters/compare-bits.py" "$A/recipe" > "$A/predictions.bits"
python3 "$E/validate-outputs.py" --recipe-output "$A/recipe" > "$A/recipe-parity.json"
```

The supplied `validate-outputs.py` requires all five prediction files to exist, each with eight finite values, and checks their complete bit patterns before producing a success receipt. The helper compares every existing `pred-*.txt`; by itself it does not enforce that the required five files were produced. Record each command's nonzero exit as failure and retain its stderr. A comparison of only one surviving file cannot qualify the language matrix. The observed exact bit agreement on this ridge fixture does not imply exact JavaScript agreement for arbitrary transcendental operations.

`examples/commands.json` supplies the exact argument arrays and placeholders for a maintainer runner. Compiler and interpreter versions, source hashes, fixture hashes, commands, exit codes and wall/CPU measurements belong in the execution receipt. Preparation and execution receipts are retained separately: `commands.json` keeps its original `prepared, not executed` status field as a historical record, and the execution is recorded in §10.4.3 Measured execution of the private examples. The packet is not edited after execution. Instead, `examples/execution-receipt.json` points from it to the execution: it names the packet and its SHA-256, the fourteen-command run of 23 September 2026, and the SHA-256 of each execution receipt under `data/example-acceptance/`.

10.4.2 Example B: actual legacy pipeline/search shape

The existing pipeline takes a locally cached catalog dataset directory with `metadata.json` or `manifest.json` and verified files. A bare CSV and the synthetic JSON above are not that catalog format. Use the committed regression fixture for a reproducible offline command:

```
"$J" pipeline run \
  --dataset "$R/datasets/test/pmlb-229-pwlinear" \
  --budget evaluations=4,seconds=60 --seed 7 --rng-seed 7 --cuts 1 \
  --emit rust,python,cpp17,cpp20,js --policy skip \
  --output "$A/pipeline" --json > "$A/pipeline-result.json"
"$J" pipeline verify-receipt --receipt "$A/pipeline/receipt.json" --json
```

At this budget the command measures four legacy candidates on one prepared cut: generation zero's bootstrap genome `(pipe X (ridge))` plus three grammar-random draws. No evolution runs: generation zero has `POPULATION = 12` slots, and offspring are bred only in later generations, which need an evaluation budget above twelve (the default budget is `evaluations=64,seconds=300`). The command then selects the lowest mean validation RMSE, emits supported recipe sources and writes a pipeline receipt. Sources: `crates/jope-pipeline/src/search.rs:75-80, :466-484`; `crates/jope-pipeline/src/select.rs:18`; `crates/jope-pipeline/src/cli.rs:38`. `--policy skip` explicitly records the absent neural proposal stage; `auto` also records a skip and `require` fails closed in this legacy route. It is not the qualified neural ASM arm.

The legacy budget is per cut, combines exploration/comparison attempts and active seconds, and is nonpreemptive for an in-flight fit. Multiple cuts can consume multiple per-cut allowances. It is not the twenty-worker panel's hard isolated 30-second fit and original global deadline contract. Preserve the recorded `deadline_overrun` and stage counts rather than applying official panel claims to this command.

The emitted winner may not be covered by C++ or JavaScript. The receipt retains exclusions and their reasons; it never replaces the selected recipe solely to make all requested languages emit. Reusing the same C++ source under `-std=c++17` and `-std=c++20` establishes the intended single-emitter structure. The maintained `deploy/ci/emitters.sh` and `deploy/ci/e2e-smoke.sh` provide larger qualification workflows; running them builds and executes models and requires its own resource allocation.

10.4.3 Measured execution of the private examples

10.4.3.1 What passed and what was measured

On 23 September 2026, all **14 unchanged example commands passed**, with zero failed, unavailable, timed-out or unstarted commands. The original command packet and static-only preparation receipts are retained unchanged as historical inputs; this execution is separate evidence.

The immutable CLI came from reviewed source `68f56404c3e199e06413bfd645d3fe0803d7b96f`; merged main `a75e543d2166017cb0139e0a251d51d9a0c98e08` has the identical source tree `176b75bc36d76c7ce4eaeb7234677ec013468889`. The executable SHA256 was `eab2d5b1b05629020aab3d540d51eb24ba0db18ff0917da188f89a34b6dc5e2b`. The original fourteen-command packet SHA256 was `f34771938934d1360cbe054689393198a53929479ba119f5a0faf2b0da7a170a`.

The private synthetic fixture contains **24 rows: 16 training, four validation and four test**. The recipe `(pipe X (standardize) (ridge))` achieved training R^2 **0.9995230651** and validation R^2 **0.9918641493**. Its emitted Python, Rust, C++17, C++20 and JavaScript programs each fitted from the same training CSV and produced **eight finite predictions with identical IEEE-754 bits**. A separate validator required all five output files and all eight rows, so a missing backend could not pass by omission.

The packet also ran the separately identified committed **PMLB 229 pwlinear** legacy pipeline fixture (`1cb97b1d7314d43d`). It completed four successful candidate measurements in one generation with no failed measurements (`stopped_by: evaluations`), selected the bootstrap genome `(pipe X (ridge))`, reported validation RMSE **0.0775685574** and R^2 **0.8151398564**, and passed receipt verification. Its policy stage was explicitly skipped. This fixture is not the synthetic twenty-four-row example and this invocation is not the official thirty-dataset campaign.

10.4.3.2 Resources, accounting and scope limits

Measurement	Recorded value
Controller elapsed time	10.407138 seconds
Launcher elapsed time	10.506140 seconds
Exact systemd unit CPU	9.382049 CPU-seconds
Fourteen-command child CPU subtotal	2.952963 CPU-seconds
Python	3.12.3
Rust	1.97.1
G++	13.3.0
Node.js	26.1.0

The unit CPU includes overlapping command/controller work; **do not add it to those subtotals**. The separately granted allocation declared one CPU, 4 GiB memory, zero swap/GPU, network denial and 300 seconds total, with 30-second numerical-command and 60-second compiler limits. The shared exclusive CPU lock was held and released; all command process groups drained. CPU affinity `[0]` and the dedicated unit cgroup were observed. A live property readback raced unit collection and returned unloaded defaults, so independent readback of memory/swap/quota/network/device limits is **not established**. The unit reported 5,332,992 bytes peak memory and zero swap, while cumulative reaped-child maximum RSS was 168,808 KiB; retain these different accounting domains without claiming a reconciled total peak.

This result qualifies the documented **recipe refits and legacy pipeline example**. It does not qualify exact saved-fitted-state export, neural ASM consumption, an ASM under-sixty-second warm demonstration, GPU training or the full panel. No Cargo build or download ran. Full command outputs, tool/input/source/output hashes and the original unit journal are retained in the private `agent-maintainer-manual/examples-run/` evidence directory.

Evidence pins: `HANDOFF.json` SHA256 `a05e8a0e9f9430b0c71ff23c57e5e29532a61bbe118daac3fe358b45f2423346`; `audit.json` SHA256 `363fbe62a706b44030205da78411243ba206ad2366d546a277babe966b802c91`; `result.json` SHA256 `71395a038feeef768bd6c69bb43dcabbfced20333d989cd1161c07213f4f68d6`. Byte-identical copies of these three files are in `data/example-acceptance/`, and `examples/execution-receipt.json` records the same pins beside the packet's hash.

10.5 Example C: exact fitted winner export after pending integration

This runbook is the route intended to meet the rule of §10.1 Deployment support: definition, priority and status. At the PR83 snapshot, its two export commands, `benchmark export-winner` and `benchmark qualify-export`, existed only in stopped custody patch views; `benchmark report` existed at `a75e543d2`, but

the `fitted_inference` section through which it reports export results existed only in those views. PR88 later integrated these interfaces and passed CPU fixture checks (§1.8.1 Implementation snapshot and this manual's own identity); real campaign winner export remains unqualified. Its runbook summary:

Runbook part	Example C: exact fitted winner export
Purpose and status	Export the selected winner's saved fitted state to Python, Rust, C++17, C++20 and JavaScript, and qualify each backend against the native predictions, without refit. **pending integration**: `benchmark export-winner` and `benchmark qualify-export` are absent from merged main `a75e543d2`, and the pending view compiles them only with the `asm` feature on Linux.
Prerequisites and resources	The same integrated immutable installed CLI, approved plan and plan ID, manifest, development root and journal as the campaign, with complete confirmation; a pinned tools file (§10.5.2 Tool inventory, qualification order and isolation); and the original campaign's remaining allowance. Each job has one thread and at most the original 30-second fit limit, inside the campaign's existing supervisor and worker cgroup.
Manifest-creation command	None of its own: the export consumes the campaign's existing plan, manifest and journal. No manifest-creation command exists at `a75e543d2`, or in the pending export view, for the tools file; it must be prepared from the installed bytes, in the campaign `Runtime` schema.
Execution	The three commands of §10.5.1 Export commands and winner selection, with backends qualified in the order Python, Rust, C++17, C++20, JavaScript.
Artifacts and success	The semantic prepared identity, the private `prepared.json`, and a public bundle root with each validated bundle and its public manifest. A backend succeeds only through an executed qualification that meets its numerical policy; `all_backends_qualified` is true only when all five pass.
Failure and resume	A compiler absence, timeout, crash or parity failure blocks the later backends, and a failed comparison stays `failed_parity`. Started work without a finished receipt stays unknown until shared terminal recovery. Repeating the command cannot give an expired campaign a new allowance, and report recovery is read-only.
Limitations	Static public coverage is 14 of 26 recipes for Python/Rust, 2 for each C++ standard and 6 for JavaScript (§10.3.1 Coverage by recipe and backend). External-library winners are typed unsupported. Fold-zero parity does not prove the other four folds. No separate post-panel validation stage exists.

10.5.1 Export commands and winner selection

Use the final integrated installed CLI, original admitted plan and journal, unchanged panel/development inputs and exact selected recipe. The pending front door, absent from merged main `a75e543d2`, accepts a dataset and an existing fitted fold, rather than a replacement recipe:

```
"$J" benchmark export-winner \
  --plan "$PLAN" --plan-sha256 "$PLAN_ID" --manifest "$M" --datasets "$D" \
```

```
  --journal "$JOURNAL" --dataset "$DATASET_ID" --fold 0
"$J" benchmark qualify-export \
  --prepared "$PREPARED_EXPORT" --prepared-sha256 "$PREPARED_EXPORT_ID" \
  --tools "$TOOLS" --tools-sha256 "$TOOLS_FILE_SHA256"
"$J" benchmark report \
  --plan "$PLAN" --plan-sha256 "$PLAN_ID" --manifest "$M" --datasets "$D" \
  --journal "$JOURNAL"
```

The first command returns the semantic prepared identity, private `prepared.json` and public bundle root. It chooses the smallest complete five-fold mean loss, breaking ties by method identity. Incomplete confirmation has no final winner. Choose another existing fold only when that exact fitted state is the intended qualification target; fold-zero parity does not prove the other four states.

10.5.2 Tool inventory, qualification order and isolation

The tools file uses the existing private `Runtime` schema keyed by command name, normally `python3`, `rustc`, `g++` and `node`. Each entry pins the physical executable and SHA256, exact version, empty argument prefix, required toolchain/package artifacts, admitted environment and unavailable reason or null. Use the real compiler executable, not an unpinned selector such as a mutable rustup shim. Tool inventory must be prepared from actual installed bytes; an illustrative JSON snippet with invented digests is not admissible.

Qualification order is Python, Rust, the same C++ emitter under C++17 then C++20, then JavaScript. Unsupported settles that backend's inventory without replacing the winner. A compiler absence, crash, timeout or parity failure prevents silently proceeding as though that backend qualified. The complete probability matrix, row order and class order are compared, not just the winning class or a rounded score.

The child receives generated inference files and unlabeled raw prediction rows. Expected predictions, original scoring labels and dataset-root access stay with the controller. Each job uses the existing bounded worker supervisor. Report recovery checks source/tool/input/artifact/command/receipt identities and runs no compiler or predictor. Started work without a finished receipt remains unknown until shared terminal recovery; it is not a successful retry.

10.5.3 Backend numerical policies, public metadata and the export clock

Backend	Public files	Pinned numerical comparison
Python	`model.py`	Exact IEEE-754 bits, host platform libm.
Rust	`model.rs`	Exact IEEE-754 bits, host platform libm.
C++17 / C++20	`model.cpp`, `codec.json`, `boundary.py`	Exact IEEE-754 bits through the typed Python/ctypes boundary; no fast-math/FMA contraction drift.
JavaScript	`model.js`	Absolute/relative `1e-12` policy with explicit ECMAScript Math identity.

Necessary public metadata records the task, ordered columns/classes, fitted raw-cell/input policy, emitter/template hashes, runtime, numerical policy and file checksums. The initial bundle is `emitted-unqualified`. `all_backends_qualified` becomes true only after five actual successful qualifications. An operation can complete with explicit unsupported outcomes while that all-backends flag remains false.

These jobs use the original campaign's remaining execution allowance; they do not open a new export clock. For the official two-hour panel, no new export job may start after original `t0 + 6900 seconds`, and the last five minutes remain reporting reserve. If confirmation consumes the window, preserve unattempted export statuses. A future separately bounded post-panel validation stage would need its own explicit identity and cost accounting; the current front door does not implement one.

10.6 Future work and acceptance boundaries

Priority gaps are actual full-state parity for supported winners, broader fitted-state lowering, a typed multiclass C++ boundary, qualified external-library export where feasible, and only later CUDA. Each extension needs exact native-state replay, missing/unknown-category tests, numerical policy fixed before scoring, stale-evidence refusal and public-bundle canaries. New operators additionally need ablation and compatibility evidence; adding an emitter branch alone does not establish reusable learning value.

Two existing emitter options are not exposed by any command. The legacy emitters' default math policy, `platform-libm-v1`, lowers transcendentals to the host math library, so exact parity does not extend to JavaScript or CUDA. The reserved `portable-scalar-kernel-v1` policy makes the Python/Rust pair fail closed until every emitted transcendental is routed through the portable kernel (`crates/feat-math/src/transpile.rs:51-80`, `:176-187`), while the C++ and JavaScript emitters accept it and substitute portable math helpers (`crates/feat-math/src/emit_cpp.rs:48-52`, `:72-81`; `crates/feat-math/src/emit_js.rs:37-45`); only in-source tests select it. `EmitOptions.fast_runtime` exists, but neither `jope emit` nor `jope pipeline run --emit` exposes it (`crates/jope-cli/src/kernel.rs:78-81`; `crates/jope-pipeline/src/emit.rs:6-7`, `:122`).

Retain native winners even when every requested language is unsupported. Report the full loss/ranking and export gap together. Performance claims need measured creation/prediction time on the actual artifact and hardware; binary size, source-line count, emission throughput or a passed static support function cannot substitute for those measurements.

CHAPTER 11

Benchmark execution, ranking, Elo and interpretation

IN THIS CHAPTER

11.1 Three evidence collections with different purposes 132
11.2 Frozen-panel execution contract 133
11.2.1 The screen, freeze and confirm procedure 133
11.2.2 Adaptive development exposure 133
11.3 Running the frozen-panel campaign: plan, execute and report 134
11.3.1 Purpose, status and prerequisites 134
11.3.2 Preregistered arms and per-task scope 135
11.3.3 Execution, artifacts and success 135
11.3.4 Failure, resume and limitations 136
11.4 Metrics and class order 137
11.5 What Elo means here 137
11.5.1 The Bradley–Terry model, tie rule and defaults 137
11.5.2 Interpreting a rating 138
11.5.3 How the retained Elo scale is constructed 138
11.6 Minimum complete future campaign report 138

11.1 Three evidence collections with different purposes

The frozen ASM panel, the historical same-cut benchmark and the imported SOTArena ladders have different inputs and protocols. Keep their figures and identities separate. A strong historical Elo is not a result for the frozen thirty-dataset ASM panel, which has not yet been run. A rank computed on development labels is not a sealed-test estimate.

Collection	Inputs and purpose	Current interpretation
Frozen `asm-private-30-v1`	Ten regression, ten binary and ten multiclass datasets; original frozen row assignments	Current implementation target and adaptive development domain; one regression operational proof, full panel unrun
Historical same-cut v1/v2	Ten committed regression datasets; all methods share each cut; screening and deepening	Reproducible historical comparisons; v1 and v2 are separate snapshots
Historical tabular-regression-v2	Imported score inventory and corrected benchmark-valid/public-development views	Historical ladder with its own coverage and validity rules; not a current ASM qualification

The plot source tables retain method, dataset, snapshot, coverage and source hashes. Missing cells stay missing. The book does not manufacture thirty current outcomes from the historical collections. Historical method names are identifiers; the word “jope” in an identifier does not prove the method is this exact product checkpoint or executable.

11.2 Frozen-panel execution contract

11.2.1 The screen, freeze and confirm procedure

For each dataset, screen every applicable native method, configured external adapter and admitted search arm on development fold zero. Freeze the top five recipe identities and confirm those recipes across all five original development folds. Additionally confirm `asm.neural` if it missed the top five; the current code forces only `asm.neural`, not `asm.generic`, into this extra confirmation (`crates/jope-bench/src/panel/campaign.rs:779-783`). A configuration such as `gpr.rbf_linear` is distinct; a spelling alias does not justify an extra independent fit. The recipe is frozen, while the fitted state for each fold is learned from that fold's permitted training rows.

For each dataset, applicable methods screen on development fold zero. The top five recipes freeze before confirmation across all five development folds. `asm.neural` is also confirmed when it misses that top five. Report every original outcome. Selection can use adaptive development folds; these are not untouched external generalization estimates. Preserve the explicit selection-exposure metadata, and keep sealed evaluation untouched.

11.2.2 Adaptive development exposure

This screen/freeze/confirm design uses development data adaptively. The selected recipe can already have learned from rows that occur in a later confirmation fold. Fold-zero labels influence screening. The implementation records these intersections; the five-fold summary remains adaptive development evidence. Sealed rows and sealed meta-test families stay outside learning, optimization and this documentation audit. The word “test” in an optimization-visible file does not make it a valid test set. Under the owner's family-level domain rule, the frozen panel remains an adaptive development domain even after a complete campaign (§5.5.1 Three family-level domains).

The frozen protocol screens methods on development fold 0, freezes the top five recipes, and confirms those recipes on the same five development folds; `asm.neural` is additionally confirmed if it missed the top five. The synthesized recipe was learned from the outer fold 0 fitting set, containing development folds 1–4. Thus those later confirmation folds are exposed to recipe learning. Every frozen recipe's fold 0 labels influenced screen selection. The report derives row intersections and records them per recipe/fold; it labels the selected five-fold summary adaptive development and leaves sealed evaluation unopened. This is the chosen protocol, not an unbiased outer generalization estimate. Source: `crates/jope-bench/src/panel/selection_exposure.rs:1`.

The exposure is not symmetric, so the adaptive bias has a direction: it favours `asm.*` over fixed-configuration comparators. A synthesized `asm.*` recipe is recorded with the learning rule `inner_selection_on_cut_zero_training_rows`: it was chosen by a search of up to 1,000 evaluations per arm on the fold 0 fitting rows, which are development folds 1–4. Every `rust.*` and `external.*` recipe is recorded as `fixed_before_current_dataset_screen`. All methods share only the fold 0 screen selection. The five-fold mean therefore compares an optimistic ASM estimate with comparator estimates exposed only through that screen, and the difference can grow as search pressure grows; an apparent panel win can arise from selection alone. Sources: `crates/jope-bench/src/panel/selection_exposure.rs:71, :91-92`; `crates/jope-bench/src/panel/asm_campaign.rs:37`; `crates/jope-bench/src/panel/optimize.rs:132-133`. A nested variant, with an independent ASM search inside each outer fold, is not implemented; requiring it before any external claim is a review proposal pending owner decision.

11.3 Running the frozen-panel campaign: plan, execute and report

This runbook has seven parts: purpose and status, prerequisites and resources, manifest creation, execution, artifacts and success, failure and resume, and limitations. Status labels are as of 2026-09-23 about 23:55 UTC.

11.3.1 Purpose, status and prerequisites

Purpose. Run the unchanged thirty-dataset protocol once, as roadmap item PANEL-01 (§14.1.3 Compact task index): screen every applicable method on development fold zero, freeze five recipes, confirm them on all five development folds, and report adaptive development evidence for every dataset.

Status: implemented but unqualified. `optimize plan`, `benchmark run` and `benchmark report` exist in the immutable CLI at `68f56404`, which has the same tree as main `a75e543d2` (§4.1.2 Command surfaces and the stopped integration order). No full-panel campaign has run. ASM execution so far is limited to separately bounded one-dataset regression proofs (§1.5.2 Canonical proof record and per-arm facts). A campaign with a newly trained policy is pending integration: at `a75e543d2` the configuration validator accepts only the accepted step-60000 policy bytes (`crates/jope-bench/src/panel/optimize.rs:161-165`), and PANEL-01 waits for the final trained configuration.

Prerequisites and resources.

Prerequisite	Requirement
CLI `$J`	Linux build with the `asm` feature. `run` refuses a binary whose path or SHA-256 differs from the plan's pinned native runtime (`crates/jope-bench/src/panel/campaign_cli.rs:175-184`).
Panel `$M` and data `$D`	Unchanged manifest `ba7dace4…` and verified prepared development data (§4.4.1 Prepare and verify the panel)
Base plan and `BASE_PLAN_ID`	Runtime-aware comparator plan and its printed `plan_sha256`. `optimize plan` refuses a base plan whose scope is not `comparator-qualification` (`crates/jope-bench/src/panel/optimize_cli.rs:91-94`).
ASM configuration and its file SHA-256	Complete `jope.asm-configuration/v1` file. It pins a `jope.asm-worker-build-inventory/v1` release manifest, source manifest, checkpoint receipt, policy weights, vocabulary, frozen transforms, policy and encoder runtimes, encoder assets and optional context qualification (`crates/jope-bench/src/panel/optimize.rs:99-122`).
Journal	Absolute durable path outside `/tmp` and Git working trees; `run` creates it with mode 0700 (`crates/jope-bench/src/panel/campaign_cli.rs:130-147`).
Resources	Fixed by the base plan: 20 workers and 20 CPUs, 64 GiB, zero swap, at most one GPU task, one thread per worker, 30 seconds per fit, 7,200 seconds total including a 300-second reporting reserve (`crates/jope-bench/src/panel/campaign_cli.rs:83-93`). Do not overlap exclusive full-CPU CI (§12.2.3 Shared compute).

Manifest creation. The panel manifest is committed and frozen: it is verified, never created (§4.4.1 Prepare and verify the panel). Prepared data, the runtime inventory and the base plan have creation commands in §4.4 Prepare the panel and external runtimes, and `optimize plan` below creates the campaign plan. No command at `a75e543d2` creates the ASM configuration or its `jope.asm-worker-build-inventory/v1` release manifest: the committed source only reads and verifies both schemas (`crates/jope-bench/src/panel/optimize.rs:29, :102`; `crates/jope-bench/src/panel/worker_build.rs:9, :81`). Assemble them from real build, asset and qualification receipts; a short hand-written configuration with invented hashes cannot replace those artifacts (§6.5.1.2 Required inputs).

11.3.2 Preregistered arms and per-task scope

Prepare the qualified configuration before opening an execution window. It binds the actual context producer, source/worker/model inputs, numerical qualification and policy configuration. The standard comparison preregisters a 64-slot policy pool plus 16 exploration candidates against 80 generic candidates, with equal declared arm allowances. Policy-pool shortfalls carry `neural_rejection_fallback`; only `neural_policy` slots count as genuine checkpoint proposals. Generation-zero neural slots must actually fit; a proposal file or generic-only result cannot demonstrate neural consumption.

That preregistration applies in full only to regression at the pinned source `68f56404`. The accepted policy is regression-only, so binary and multiclass datasets have no neural arm, and their generic arm searches ridge heads only (`logit_ridge` or `multiclass_ridge` with at most four transforms; see §2.7.2 Experimental classification context):

Task	Frozen datasets	Neural arm	Generic arm	Non-ASM comparators
Regression	10	64 policy slots plus 16 exploration candidates	80 grammar-uniform candidates	17 native recipes plus 4 external engines
Binary	10	Unavailable	80 `logit_ridge` candidates	5 native recipes plus 4 external engines
Multiclass	10	Unavailable	80 `multiclass_ridge` candidates	4 native recipes plus 4 external engines

At this source, a better policy therefore cannot change binary or multiclass results. Sources: `crates/jope-bench/src/panel/optimize.rs:34, :35, :493`; `crates/jope-pipeline/src/neural_search.rs:28`.

11.3.3 Execution, artifacts and success

Plan first. Planning performs zero fits and reports `execution_budget_started: false` (`crates/jope-bench/src/panel/optimize_cli.rs:126`). Capture the printed semantic plan ID rather than hashing the file (§4.2 Physical layout and identities), then execute and report:

```
"$J" optimize plan \
  --base-plan "$A/base.json" --base-plan-sha256 "$BASE_PLAN_ID" \
  --configuration "$ASM_CONFIGURATION" \
  --configuration-sha256 "$ASM_CONFIGURATION_FILE_SHA256" \
  --output "$A/panel.json" > "$A/panel-plan-receipt.json"
PANEL_PLAN_ID=$(jq -er '.plan_sha256' "$A/panel-plan-receipt.json")

"$J" benchmark run --plan "$A/panel.json" --plan-sha256 "$PANEL_PLAN_ID" \
  --manifest "$M" --datasets "$D" --journal "$A/panel-journal"
```

```
"$J" benchmark report --plan "$A/panel.json" --plan-sha256 "$PANEL_PLAN_ID" \
  --manifest "$M" --datasets "$D" --journal "$A/panel-journal"
```

Artifacts. `optimize plan` prints `plan_sha256`, `scope: "asm-panel"`, `fits: 0`, `neural_slots: 64`, `exploration_slots: 16`, `generic_slots: 80` and `checkpoint_training_exposure: "unknown"` (`crates/jope-bench/src/panel/optimize_cli.rs:126-129`). `benchmark run` writes the retained plan as immutable `campaign.json` in the journal (`crates/jope-bench/src/panel/campaign.rs:661`) and, when the campaign completes, an immutable report `reports/<report SHA-256>.json` with schema `jope.panel-report/v2` (`campaign.rs:890-900`). The base plan declares `finished`, `observations`, `frozen` and `reports` as expected artifacts (`crates/jope-bench/src/panel/campaign_cli.rs:94-99`). `benchmark run` returns `ok: true` only when its supervised launch exits with status 0; once `campaign.json` exists, its output also carries the read-only recovery report (`campaign_cli.rs:199-208`). Retain the evidence listed in §12.4 Artifacts: what evidence to retain.

Success. Success is a `jope.panel-report/v2` whose dataset list covers all thirty datasets, each with screen outcomes, frozen recipes, confirmation observations, recipe summaries and selection exposure (`campaign.rs:869-872`), and which meets the minimum complete report in §11.6 Minimum complete future campaign report. Every report states `sealed_inputs_opened: false`, `optimization_visible_labels: "development"`, `confirmation_role: "adaptive_development"` and `five_fold_summary_is_unbiased_generalization_estimate: false` (`campaign.rs:893-894`). A successful campaign is therefore complete adaptive development evidence, not a sealed-test or unseen-family result.

11.3.4 Failure, resume and limitations

`optimize run` and `optimize report` delegate to the same campaign contracts. Repeating run with the same plan and journal resumes the original authenticated state and absolute deadline. A restart does not grant another two hours. Do not repair journals by editing receipt fields or copying successes from another experiment. Read-only report recovery authenticates retained artifacts and recomputes scores; it does not fit another predictor.

Failure and non-final outcomes. When bounded execution cannot be admitted, the supervised campaign reports `jope.panel-admission/v2` with `ok: false`, and no selection or scientific result is frozen (`crates/jope-bench/src/panel/campaign.rs:728-733`). Resumable inventory is reported as `jope.asm-partial/v1` for the paired arms (`campaign.rs:754-756`) or as `jope.panel-partial/v2` with `ok: false` for confirmation (`campaign.rs:839-841`). Resume each by repeating `benchmark run` with the same plan, plan ID and journal inside the original window; stop reasons and recovery phases are in §12.3.1 Reading stop reasons and resuming. A read-only `benchmark report` refuses a journal that belongs to another plan and returns `jope.panel-recovery-report/v2` with `fits_started: 0` and `clock_started: false` (`campaign.rs: 974-978, :1216-1222`).

Limitations. Both panel report constructors at `a75e543d2` set `complete_asm_demonstration: false` (`campaign.rs:895, :1222`). Binary and multiclass datasets have no neural arm at this source. The five-fold summary is adaptive development evidence whose exposure favours `asm.*` (§11.2.2 Adaptive development exposure); comparators are fixed default configurations (§11.6 Minimum complete future campaign report); and the panel's policy and pretrained-encoder exposure are unknown (§5.5.1 Three family-level domains). Selected-winner export and a newly trained policy are pending integration (§4.1.2 Command surfaces and the stopped integration order).

11.4 Metrics and class order

Task	Primary reported metric	Smaller-is-better search loss	Required prediction shape
Regression	RMSE	RMSE	One finite real prediction per row
Binary classification	AUROC	`1 − AUROC`	Probabilities with original two-class order
Multiclass classification	Log loss	Log loss	One probability for every original class on every row

RMSE is the square root of the mean squared prediction error. It has the target's units, so a raw RMSE on machine CPU cannot be averaged meaningfully with an unrelated dataset's RMSE without an explicitly declared normalization. AUROC measures score ordering between positive and negative examples; the positive class and class order must be fixed by the task contract. A one-class evaluation fold cannot produce a meaningful AUROC. Multiclass log loss averages the negative logarithm of the probability assigned to the true class. Integer class predictions alone cannot supply that probability-matrix metric.

Probability shape, finiteness, class names and order belong to the authenticated prediction boundary. A backend that produces the same numbers in a different class order has failed parity. A backend with only scalar output cannot be advertised as supporting a multiclass winner. Any numerical clipping, normalization, accumulation, tie or tolerance rule must be fixed before scoring and recorded with the evaluator; changing the rule afterward changes comparison identity.

Regression is scored by RMSE. Binary classification uses AUROC, with the frozen second class as positive and loss `1 − AUROC`. Multiclass uses the full probability matrix and original class order for log loss. Predictions must be finite; class order and row count must match; probability rows are validated without silently renormalizing. The pinned log-loss epsilon is `1e-15`, and probability row-sum tolerance is `1e-6`.

Regression uses RMSE; binary uses AUROC with minimization loss `1−AUROC`; multiclass uses probability-matrix log loss with the frozen class ordering. Features retain numeric/categorical type and missing cells; original labels are not converted to manufacture a task. Screen all applicable methods on original fold zero, freeze five recipes, confirm across all five development folds, and confirm `asm.neural` additionally if it missed the five. These optimization-visible folds remain development evidence. Sealed evaluation is separate.

The historical same-cut command has an older contract. It constructs its own hashed cuts, uses regression R^2, binary AUROC and multiclass macro-F1 for screening, and applies its recorded robust-fitness summary during deepening. Its regression ladder uses RMSE; its classification ladder can maximize AUROC or macro-F1. Those historical definitions must not be silently copied into the frozen ASM panel's multiclass log-loss contract. Source: `docs/same-cut-bench.md` and `crates/jope-bench/src/eval.rs`; current panel code lives under `crates/jope-bench/src/panel/`.

11.5 What Elo means here

11.5.1 The Bradley–Terry model, tie rule and defaults

The arena implementation fits a regularized Bradley–Terry model on an Elo scale. For two ratings A and B, its modeled pairwise probability is:

`P(A beats B) = 1 / (1 + 10 ** ((rating_B − rating_A) / 400))`.

Each dataset with both eligible scores supplies one equally weighted comparison for that method pair. The observed outcome is 1 for a win, 0 for a loss and 0.5 for a tie. Metric direction is explicit. The relative tie condition is `abs(a − b) <= 1e-12 * max(abs(a), abs(b), 1)`. A missing or ineligible score supplies no invented numerical loss.

The default implementation starts and recenters ratings at 1500, runs at most 250 diagonal-Newton iterations, applies L2 regularization of `1e-4`, clips each iteration's update to ±40 Elo points and stops when the largest update is below `1e-8`. The ±40 setting is an iteration clamp, not an online match-by-match chess update. Source: `crates/jope-arena/src/elo.rs`.

Two source paths preserve historical arithmetic: one accumulates individual sorted dataset/pair matches in the original Python order, and the reporter aggregates each pair's sufficient statistics. They are equivalent in exact arithmetic but can differ slightly in floating-point accumulation. Reproduction must identify the path and snapshot, not merely say “Elo.”

11.5.2 Interpreting a rating

The rating is relative to the included method universe, evidence eligibility and overlap graph. A high rating on a handful of datasets has different coverage from the same rating on every dataset. Report dataset coverage, full-coverage rank, average rank, wins, pending cells and protocol beside Elo. Adding methods, removing invalid rows or changing overlap can change ratings without any predictor changing. Elo does not measure the magnitude of an RMSE improvement, runtime, memory, statistical certainty or future unseen-data performance. No confidence interval or significance claim is implied by a plotted point.

11.5.3 How the retained Elo scale is constructed

`crates/jope-arena/src/elo.rs` fits regularized Bradley–Terry ratings. Each dataset contributes one equal-weight comparison for each pair of methods with available scores. The preferred direction is taken from the metric; RMSE comparisons minimize. A tie has outcome 0.5 when `|a−b| ≤ 1e−12 × max(|a|,|b|,1)`; otherwise the better score receives 1 and the other 0. Missing cells create no comparison and must not become invented defeats.

The expected pairwise outcome is `P(A beats B)=1/(1+10^((rating_B−rating_A)/400))`. The committed implementation centers at 1500, uses an L2 regularization coefficient of 1e−4, caps diagonal-Newton iterations at 250, clamps each iteration's rating update to ±40, and stops when the largest update falls below 1e−8. It preserves sorted dataset/method-pair accumulation and the historical numerical summation policy for reproduction. This describes the source algorithm; the audit did not rerun it or infer convergence where the snapshot does not report it.

A rating difference summarizes outcomes within one comparison pool. It does not encode an absolute RMSE difference, probability of unseen-dataset superiority, confidence interval, or universal ranking across tasks. A method observed only on its screened favorable subset needs its coverage displayed beside its rating. The historical `afm.lite-race.v1.resampled` row describes that search procedure; it does not prove the current neural ASM model.

11.6 Minimum complete future campaign report

For every one of the thirty immutable datasets, retain the input and cut identities; applicable, skipped and unavailable methods with reasons; screen outcomes; frozen finalists; all confirmation-fold outcomes; `asm.neural`'s extra confirmation when needed; original and fitted recipe identities; class

order; selected loss and exposure labels; resource limits and original deadlines; failures and cancellations; actual fit/predict counts; and retained winner/backend outcomes. Report all losses and gaps, not only wins. A deadline-exhausted dataset remains in the denominator with an explicit incomplete outcome.

The owner's target of ten wins per task category is not yet a countable definition. The estimate that counts, the eligible comparator set, whether `asm.generic` is a comparator or an ASM control, the tie rule and whether an incomplete or unconfirmed dataset can count as a win are not fixed, and no code at merged main `a75e543d2` counts wins against the best non-ASM comparator. A review proposal, pending owner ratification and not adopted, defines PANEL-WINS(asm-panel-v1) as, per task, the number of the ten frozen datasets on which `asm.neural`'s confirmed five-fold mean development loss is strictly below the minimum over all eligible frozen non-ASM comparators (every `rust.*` and `external.*` recipe). Under that proposal, `asm.generic` is reported in its own column as an ASM control, ties and incomplete, failed or unconfirmed datasets count as not won, and the result is reported as (regression, binary, multiclass). The separate win columns are described in §9.11.2 Win counting and a proposed acceptance record. With exactly ten frozen datasets per task, ten wins per category would be a 30/30 sweep. It remains a measured target; the current evidence does not promise that result. The full-panel report, newly trained policy consumption, selected fitted-winner parity and serving promotion remain separate acceptance milestones.

Panel comparators are fixed configurations. Each native `rust.*` recipe keeps its registered configuration, and the four external engines (XGBoost, LightGBM, CatBoost and scikit-learn) run as `external.{engine}.default`; the campaign performs no per-dataset hyperparameter search for any comparator (a head's own internal fitting, such as `ridge_gcv`'s generalized cross-validation, is part of its fixed recipe). Each ASM arm, by contrast, searches for up to 120 seconds and up to 1,000 evaluations. Sources: `crates/jope-bench/src/panel/methods.rs:60-63`, `crates/jope-bench/src/panel/campaign_cli.rs:67-72`, `crates/jope-bench/src/panel/optimize.rs:132-133`. The strongest historical benchmark-valid method, TabFM Ensemble (rank 1, Elo 1934.9), is not a panel comparator; on that historical ladder the `jope-dime-regressor` entry is rank 31 at Elo 1687.0 (§C.2.3.2 Why the two historical ladders differ). Ten panel wins per category against these default comparators would therefore not show that ASM beats the strongest known tabular methods. Tuned or foundation-model comparator tiers for a future `asm-panel-v2`, never retroactive to v1, are a review proposal pending owner decision.

Classification search is also narrower than its comparators. At merged main `a75e543d2`, binary and multiclass datasets have no neural arm, because the accepted policy is regression-only, and their `asm.generic` search samples only ridge heads (`logit_ridge` for binary, `multiclass_ridge` for multiclass). Their comparators are five binary or four multiclass native recipes plus the four external engines. Sources: `crates/jope-bench/src/panel/optimize.rs:35, :492-493`; `crates/jope-bench/src/panel/methods.rs:90-104`; §2.7.2 Experimental classification context.

CHAPTER 12

Scheduling, cancellation, recovery and accounting

IN THIS CHAPTER

12.1 Arm order and clocks 140
12.2 Resource limits, shared compute and supervision 140
12.2.1 Twenty bounded workers: resource limits and supervision 140
12.2.2 Resource policy and the training clock 141
12.2.3 Shared compute 142
12.3 Cancellation, resume and recovery 142
12.3.1 Reading stop reasons and resuming 142
12.3.2 Interrupted attempts and proof interpretation 142
12.3.3 Recovery phases 143
12.4 Artifacts: what evidence to retain 143
12.5 Cost accounting 144
12.5.1 Counting predictors, recipes, commands and export cost 144
12.5.2 Worked example: CPU, wall time and per-predictor cost of the one-dataset proof 145
12.5.3 The one-hour, ten-million-token reference scenario 145

12.1 Arm order and clocks

The arm order is fixed: neural first, then generic. Fresh context encoding and policy production run inside the neural arm clock, and each arm is clipped to the original execution cutoff. In the origin proof at source `68f56404` the producer gate opened 21.094 seconds after the campaign start and its exit was observed at 41.581 seconds; its measured wall time was 20.486968041 seconds. The neural cutoff then lay 78.613 seconds ahead, before allowing for post-exit verification/setup, so this is remaining arm time rather than guaranteed fit execution time. The neural arm stopped at `original_deadline`. The generic arm began 0.553 seconds after that cutoff, retained its full declared 120-second allowance and stopped at `evaluation_limit`. Equal declared allowances are not equal fit-only time. Sources: `crates/jope-pipeline/src/neural_search.rs:654`, `:701`, `:729`, `:772`; `crates/jope-pipeline/src/execution.rs:592`; the accepted proposal execution and recovered report.

12.2 Resource limits, shared compute and supervision

12.2.1 Twenty bounded workers: resource limits and supervision

Constraint	Official panel policy
Worker/CPU count	Twenty isolated workers shared by search and benchmarking; at most twenty CPUs.
Thread count	One thread per worker, including external libraries.
Memory and swap	64 GiB total; zero swap. The shared policy reserves 4 GiB for the controller and caps each fit at 3 GiB.

Constraint	Official panel policy
GPU capacity	At most one GPU task; the approved training hardware is one RTX 3090. GPU training has its own separate total hour.
Fit and search time	At most 30 seconds per fit, normally 120 seconds per search arm.
Panel wall time	7,200 seconds total, including 300 seconds reserved for reporting. No new fit/qualification starts after the original execution cutoff at 6,900 seconds.
Coexistence	Do not overlap the experiment with exclusive full-CPU CI.

The CLI uses the shared Linux systemd/cgroup-v2 supervisor and process-group cleanup. Missing host capability is unavailable execution. Do not put an additional outer allocation lock around a command that already acquires the shared allocation. A child crash, timeout, cancellation, launch failure and unavailable dependency have different recorded states; unknown work/cost is not zero.

The default admitted panel allocation is twenty CPUs/workers, 64 GiB memory, zero swap, one RTX 3090 when GPU work is explicitly allocated, thirty seconds per fit and 120 seconds per search arm. The full panel receives two hours total, including a five-minute reporting reserve. Training receives one GPU-hour total and twenty minutes per category. Do not overlap numerical campaigns with exclusive full-CPU CI. Resume continues the original campaign deadline and resource envelope.

12.2.2 Resource policy and the training clock

Operation	Required policy
Official frozen30 panel	20 workers/CPUs, 64GiB, zero swap, at most one GPU task, 30 seconds per fit, 7200 seconds total including300-second reporting reserve
Paired search	64 policy-pool slots (true neural or explicit fallback)+16 exploration versus 80 generic; default 120 seconds per arm; same declared allowance and grammar/variation identity
Separate origin/teacher qualification	Explicitly preregistered resource manifest and separate journal; no inferred extra allowance and no reset of another campaign
Training	One visible RTX3090, one campaign of at most3600 seconds, at most1200 seconds per category, one active category worker, CPU allowance20,64GiB,zero swap
Training numerical runtime	One Torch thread, FP32 storage, CUDA TF32 enabled by declared policy, AMP disabled; explicit experimental lineage
CPU training	Explicit synthetic qualification only; never an automatic fallback from requested CUDA
Full-CPU CI	Exclusive; no overlapping campaign/training resource allocation

The training device check requires exactly one visible CUDA device, an RTX3090 name and a usable GPU UUID. It records requested/resolved device identity. Resume requires the same identity. A missing GPU is an explicit failure; it must not become a successful CPU run under a CUDA claim.

Native planning does not open the training clock. First supervised admission creates the global window. Each category receives at most20 minutes, capped by the remaining global deadline minus its five-second reporting reserve. All passes, validation, reloads and checkpoint work must fit the original window. A pass is one supervised worker attempt: one call of `train_policy_v3_from_jsonl` with `epochs=1` and batch size 256 (2 for synthetic fixtures); every later pass resumes from the previous pass's `resume.pt`. Two passes are required for continuation evidence; they do not receive two category budgets. A fresh directory or modified plan is a new campaign, not a legitimate way to finish an expired approved campaign.

Sources: current `crates/jope-bench/src/panel/campaign_cli.rs:51`; pending `python/src/jope/policy/training_campaign.py:164`, `:704`, `:720`, `:1043`; `crates/jope-cli/src/train.rs:31`.

12.2.3 Shared compute

Fast CI runs on xbabe1. The canonical Ubuntu machine is local xbabe2 for full CPU work. The user's SSH alias `xbabe2` points elsewhere; use the runbook's local form for this machine.

The exclusive full runner and numerical campaigns share a host lock. Drain owned listeners and verify no active full job before numerical allocation. Do not seize the host by stopping another job. Record allocation ownership and restore the runner after numerical work releases it. Keep full CPU CI separate from the one-GPU-hour training campaign and the frozen panel.

Missing cgroups or dependencies cause explicit failures/unavailable outcomes; they do not authorize an unbounded fallback. Worker control readbacks, not prose, establish the limits actually enforced.

12.3 Cancellation, resume and recovery

12.3.1 Reading stop reasons and resuming

Final search reasons include `evaluation_limit`, `original_deadline`, `search_space_exhausted` and `no_measured_parent`. `admission_pending_original_window` and `cancelled_original_window` are non-final: no winner may be frozen while either holds. The proof front door can return `ok:false`, `state:pending_original_window`. Resume the same plan and journal, preserving its original arm and campaign deadlines; resume never creates another allowance. Before cutoff, unresolved never-admitted inventory may be retried; at/after cutoff it can only be finalized under the existing evidence rules.

A setup failure before gate release becomes terminal `budget_exhausted` if its original job cutoff has been reached; setup consumption stays unknown. Before cutoff, ordinary pre-gate failure is `unavailable` and cancellation is `cancelled`, subject to the retryable-inventory rules. Interrupted started-only attempts are excluded from that category, as described in §12.3.2 Interrupted attempts and proof interpretation. The earlier proof at source `3fd7d5b2` retained exit 1 after 245.93 seconds because an expired pre-gate attempt incorrectly remained retryable; its unchanged journal is evidence of the failure, not permission to reset the clock. Sources: `crates/jope-pipeline/src/neural_search.rs:1308`, `:1380`, `:1457`, `:845`; `crates/jope-pipeline/src/execution.rs:1209`; `crates/jope-bench/src/panel/origin_proof.rs:194`.

12.3.2 Interrupted attempts and proof interpretation

An attempt with a `started/` record and no `finished/` record is never rerun, and the pool does not recover its original result or final counters. After its process group is safely drained, the pool persists terminal `cancelled` with reason `controller_interrupted: original attempt not retried`; CPU, wall time and exit accounting remain unknown. If recovery cannot drain the group, no recovered terminal record is persisted and further admission is blocked (`recovery_not_drained`). Such attempts are excluded from

retryable never-admitted inventory. Separately, a normally watched attempt persists its observed terminal outcome before potentially slow cleanup; that rule does not reconstruct interrupted accounting. The search uses immutable observed journals and refuses changed completed evidence. At cutoff, only proven never-admitted inventory may normalize; the per-fold replay digest preserves ordered real attempts and output identities. Both live PoolBackend feedback and recovered FitAdapter scoring compute the same shared digest.

A regression caught during acceptance illustrates the distinction: the proof at source `d0f5748b` finished 964 worker commands (834 completed, 130 failed), but its recovered report failed because the two feedback constructors disagreed about replay evidence. Those fit counts did not justify declaring the report passed. The `68f56404` correction restored one shared digest computation, followed by a fresh bounded proof and recovered report. The `d0f5748b` records were retained unchanged.

Never extend an expired original deadline, mutate a failed result into a success, reinterpret an unknown cost as zero, or cite syntax parsing as model execution. A report can honestly say “source implementation prepared; CPU fixture passed; live training pending.” That is more useful than collapsing all three into one completion flag.

12.3.3 Recovery phases

Identify whether failure occurred before admission, in setup, after the gate, during cleanup, in controller scoring or during report generation.

- A pre-admission dependency refusal may be retried only inside the original window.
- Setup can consume CPU without starting a fit. Unknown setup cost stays unknown.
- A started identity is durable. Controller death does not authorize silently repeating it.
- Completed folds inside a failed candidate still require artifact authentication.
- Original-cutoff inventory settles without extending the clock.
- A report can fail after successful fits. Retain both; do not call the whole operation successful.
- Saving a terminal outcome does not release cleanup authority before descendants drain.

Resume with the same plan and journal. A changed source, dataset, budget or protocol requires a separately identified experiment with the old evidence retained, not a relabelled continuation.

12.4 Artifacts: what evidence to retain

Evidence	What to retain and inspect
Source/build/install	Full commit, source manifest, Cargo messages, exact features/toolchain, dependency graph, binary and worker hashes, physical paths, cold timings and exits.
Plan and configuration	Frozen panel identity, all runtime/model/encoder pins, resource limits, semantic plan ID, seeds, numerical policy and original window.
Worker execution	Started/finished union, exact job/input identity, output inventory, exits/signals, timeouts, process-group drainage, memory/CPU counters where known.
Scientific results	All screen/confirmation losses, frozen recipe set, original row/class order, baseline, selection exposure and unsupported/unavailable outcomes.

Evidence	What to retain and inspect
Fitted predictor	Complete private state, fitted preprocessing, recipe, native reload proof, prediction hashes and exact artifact identity.
Export qualification	Exact winner/fold/state, public bundle, toolchain/input/class/policy hashes, actual commands and parity result. See §10.5 Example C: exact fitted winner export after pending integration.

Retain source revision and input manifest; exact command, directory, exit and timestamps; executable/runtime/features/environment; data/split/evaluator/vocabulary/protocol identities; original limits/start/deadline/cutoff; collected/executed/failed/skipped/excluded/unavailable inventories; artifact hashes and worker receipts; report replay; limitations and next owner.

Verify a source manifest against the committed blobs it claims. The current bytes at `target/release/jope` need not be the bytes used by an earlier result. Preserve immutable executable copies with their build receipts before another build replaces the usual path.

Current ASM custody is outside Git at `/home/ubuntu/jope-rescue-20260921/asm-integration-20260923/`. This host location is not a portable runtime default.

12.5 Cost accounting

12.5.1 Counting predictors, recipes, commands and export cost

A recipe may be evaluated on multiple folds or re-used in later fits. Therefore proposal count, recipe count, unique fitted predictors, launched commands and physical fit starts are different denominators. A cache hit does not become another new predictor; a partial/unknown attempt does not become a zero-cost fit. §12.5.2 Worked example: CPU, wall time and per-predictor cost of the one-dataset proof applies this accounting to the current proof.

Report elapsed wall time, summed worker wall time, worker CPU time, aggregate supervisor CPU, GPU time if measured, launched commands, successful fits, failures, unique fitted predictors and unique reusable recipes separately. Aggregate supervisor CPU contains worker CPU; adding them double-counts. Parallel worker durations do not equal elapsed wall time. Deduplicated/cache reuse is useful but is not another physical fitted predictor.

Count a unique fitted predictor by authenticated complete fitted-artifact identity, and a reusable recipe by the declared recipe identity. This does not establish behavioral uniqueness. Fit failures, policy/context work and preparation still incur cost. A worker CPU subtotal is not automatically total creation CPU. Validated systemd unit CPU can include setup, controller and children; it overlaps worker CPU and must not be added to it. Unknown counters stay null with a reason.

In the pending ASM export accounting, `fitted_export_commands` is a subset of global worker commands. Comparator-only accounting requires its separate export cost to be added. Export preparation is measured separately; final receipt publication and other unmeasured overhead remain explicit. Retain the original panel cutoff: backend qualification after a campaign expires is unattempted through the existing front door. A distinct post-panel bounded validation stage is not implemented and cannot be claimed inside the two-hour panel allowance.

12.5.2 Worked example: CPU, wall time and per-predictor cost of the one-dataset proof

Exact supervisor CPU was **270.889403 CPU-seconds**; summed worker CPU was **181.220773 CPU-seconds**. The supervisor aggregate contains worker use: **do not add them**. Summed worker wall durations were **202.266164543 seconds**; concurrent durations are not elapsed campaign time.

Amortized over 832 unique predictors, these observations yield approximately 0.217813 worker CPU-seconds, 0.325588 aggregate CPU-seconds and 0.287432 run wall-seconds per predictor. Over 164 recipes, they yield approximately 1.105005, 1.651765 and 1.458190 seconds. These are campaign accounting ratios, not isolated latency predictions for arbitrary models.

The current one-dataset proof completed 964 worker commands: 834 completed and 130 failed. It counted 832 unique fitted predictors and 164 recipes, plus two final refits with 34 development-hold predictions each. Its run took 239.143121970 seconds; independent report recovery took 18.718997325 seconds. These values are measured at the qualified source and protocol, not projected full-panel throughput. The CPU totals are in the proof record above and the amortized denominators in the table below.

The supervisor figure is the exact `CPU_USAGE_NSEC=270889403000` event from local PID 1 systemd, linked to the precise supervisor unit, source, launch receipt and original experiment window. It includes controller/setup threads and child workers and excludes the outer launcher, immutable build and external preparation. It overlaps the worker CPU total; **never add them**. The rounded systemd stderr message is only diagnostic. No exact controller/setup overhead is derived by subtracting a rounded total. Isolated controller CPU, GPU seconds, token usage and currency cost remain unknown in the report.

Denominator	Worker CPU seconds	Supervisor aggregate CPU seconds	Amortized run wall seconds
One unique fitted predictor	0.2178134291	0.3255882248	0.2874316370
One unique recipe	1.1050047134	1.6517646524	1.4581897681

These are accounting ratios for one retained run, not extrapolated throughput or equal-quality claims. A recipe and a fitted predictor differ: the same recipe can create distinct fitted artifacts on separate folds. Unique artifact bytes do not prove behavioral uniqueness. Every failed fit remains in the inventory; failed or unavailable commands are not silently counted as zero cost.

12.5.3 The one-hour, ten-million-token reference scenario

GPU work, tokens, money, isolated controller CPU, downloads and installation were not measured by this proof. The supplied one-hour/ten-million-token scenario provides a 3,600-second contextual reference: this proof's run wall time is approximately 6.64% of that duration. This is not a matched-workload speedup. No scenario hardware allocation, predictor count, token trace or cost ledger exists here, so ten million tokens cannot be converted into a measured productivity or currency comparison.

The supplied one-hour/ten-million-token scenario has 3,600 seconds and 10,000,000 tokens by stipulation. This run used 239.143 seconds of wall time but has no measured token or monetary receipt. A dollar comparison, token savings claim, or one-hour quality extrapolation would require actual token usage, contemporaneous input/output/cache rates and an equivalent task/quality protocol. None is invented here.

The supplied one-hour/ten-million-token scenario is a contextual scenario only. Its workload, model, hardware, predictor inventory and token/cost traces are unavailable. This proof used approximately 6.64% of an hour for its run, but that fraction is not a matched speedup or token-efficiency result. Dollar cost and token usage remain unmeasured.

CHAPTER 13

Maintenance, privacy, release and preservation

IN THIS CHAPTER

13.1 Questions that should remain answerable after a maintainer handoff 147
13.2 Repository authority and coordination 147
13.3 Worktrees and builds 148
13.4 Coverage and reviewed lifecycle 148
13.4.1 Coverage inventories and CI lanes 148
13.4.2 Writer and reviewer identities 148
13.4.3 Freeze, review and merge 149
13.4.4 Main-guard and full-main scheduling 149
13.5 Privacy boundaries: the private manual and public fitted bundles 149
13.6 Release and installation are independent from model promotion 150
13.7 Preservation and lane closure 150
13.8 Rebuilding and updating this manual 150
13.8.1 Rebuilding the book 151
13.8.2 What the renderer reads, checks and writes 152
13.8.3 Regenerating the training and target figures 153
13.8.4 The publication record and its three identities 153
13.8.5 Updating the book 154

13.1 Questions that should remain answerable after a maintainer handoff

Which reviewed source and runtime produced this artifact? Which exact rows and original labels were available? Was this a recipe, a checkpoint or a saved fitted predictor? Which score selected it, under what comparison identity? Which failures, missing dependencies and cancelled jobs were retained? Did any nominal test labels influence selection? Did resume preserve the original deadline? Does the export predict from saved state, and which backend actually passed? Did training perform positive updates and change parameters? Can another maintainer locate and authenticate the receipt without reading credentials or sealed data?

The roadmap (§14.1.3 Compact task index) translates the unanswered parts into named acceptance items. Unknowns stay explicit until the corresponding evidence exists.

13.2 Repository authority and coordination

The canonical product checkout is `/home/ubuntu/JopeDime`, upstream `veox-ai/JopeDime`. `/home/ubuntu/jain-split/jope-dime` is a different research repository; `/home/ubuntu/jope-dime` may be its symlink. Similar names are not interchangeable source, checkpoint or promotion authorities.

Before source changes, inspect the open PR queue, worktrees, branch/full HEAD, dirty state and latest `AGENT_CHAT.md`. Under `flock` on `AGENT_CHAT.md.lock`, reread overlapping claims and append checkout, branch, full HEAD, owner, paths and verification. The append lock does not permit two writers in one checkout. An idle session or old claim is not a stopped-head handoff.

Keep one writer per checkout and one implementation PR queue. Preserve pending work and authorship. A resumed campaign retains its resource limits and deadline.

Repository operation remains serialized: read the current queue/ledger, claim paths under `AGENT_CHAT.md.lock`, keep one writer per checkout, and preserve pending work. Changes require exact-head CI, independent eligible review/native approval, guarded rebase merge and verification of remote `main` plus `main-guard`. A changed head invalidates an earlier head's approval. Preserve evidence before deleting closed lanes or build directories; `target/` can contain irreplaceable receipts.

13.3 Worktrees and builds

Extra copies belong under `<repo>/.agent/worktrees/`, at most four lanes. `.claude/worktrees` stays a symlink into that shared root. Verify it before worktree operations and keep `.agent/` locally excluded from Git status.

Each lane reuses its own `target/`. Before Cargo:

```
export CARGO_INCREMENTAL=0
export CARGO_PROFILE_DEV_DEBUG=line-tables-only
export CARGO_BUILD_JOBS=4
```

Do not set `CARGO_TARGET_DIR` globally or create a new target per task. CI selects persistent PR and trusted-main storage separately. Resolve binaries from Cargo artifact messages: guessing a hashed test executable can select an old binary built with different features.

Never put source, worktrees or builds in `/tmp`. A target directory can contain evidence. Cargo markers do not prove that every descendant is disposable.

13.4 Coverage and reviewed lifecycle

13.4.1 Coverage inventories and CI lanes

Read `deploy/ci/coverage.md`, mapping and generated inventory together. Discovery counts packages, features and families; execution receipts count what one invocation actually collected and ran. Invocations overlap. Path exclusions, ignored tests and unavailable dependencies do not become passing tests.

Fast CI has eight lanes plus aggregate. Full-main diagnostics cover additional families and have recorded failures/unavailable capabilities. A fast gate does not establish all-feature, GPU or live-S3 qualification. Regenerate inventories and scores through their owning proof scripts; never hand-edit generated evidence.

13.4.2 Writer and reviewer identities

The established writer is `jepsontaylor`; the designated independent Claude reviewer uses `neverhuman`. Verify them for every PR. The reviewer must differ from the opener and every associated commit author and committer. Use the full paginated inventory and record unassociated commits.

```
/home/ubuntu/.local/bin/gh-role writer api user --jq .login
/home/ubuntu/.local/bin/gh-role eligible reviewer veox-ai/JopeDime PR_NUMBER
```

The helper prevents identity mistakes but cannot prove independence. Changing credentials does not turn a writer into a reviewer. Never print tokens or credential-bearing environments.

13.4.3 Freeze, review and merge

Freeze the full head; dispatch fresh manual CI; retain source SHA, run ID, attempt and dispatch time. Verify current-attempt jobs/aggregate and nonzero meaningful coverage. Supply the independent reviewer with diff, acceptance commands and evidence. A native approval must explicitly bind the final head and remain undismissed. Changed source repeats checks and review.

```
/home/ubuntu/.local/bin/gh-role writer pr merge PR_NUMBER \
  --rebase --match-head-commit FULL_REVIEWED_SHA
```

Immediately recheck head, eligibility, CI and approval before merging. Afterwards verify `merged_at`, fetch remote main, compare the reviewed tree and verify main-guard. Preserve any ledger tail or other dirty work before restoring clean verified main. Protection enforcement has been procedural; a protection API 403 does not prove a preventive branch lock. Main-guard detects unreviewed pushes after they occur.

13.4.4 Main-guard and full-main scheduling

Main-guard and the full-main diagnostic share the single xbabe2 `jope-full-cpu` slot and run serially. A main push queues both; full-main has a 120-minute timeout, so main-guard can wait behind it. Read back job/run states and retain the exact source/run/attempt before changing the queue. In the PR83 lifecycle the owner-authorized full-main diagnostic was cancelled while still queued, before any job started, and the owned full timer was restored to allow main-guard to run. That is a recorded unexecuted diagnostic, not passing coverage or general permission to cancel running work. Follow the current allocation owner and runner runbook; never stop another running job to accelerate a merge. Sources: `.github/workflows/main-guard.yml:17`, `.github/workflows/full-main.yml:24`, `docs/runbooks/runner.md:15`, `:131`; the PR83 lifecycle cancellation receipt.

13.5 Privacy boundaries: the private manual and public fitted bundles

This is a maintainer document for the private product repository. Its explicitly requested GP examples and internal operational paths explain the implementation; they do not expand the allowlist for public inference bundles.

This is a maintainer document containing internal model and discovery design. The user's request authorizes this explanation and the companion private GP examples. It does not enlarge the allowlist for distributed inference artifacts or authorize publishing private weights, discovery histories, vocabulary assets or training corpora. Hashes identify dependencies; they are not the dependencies themselves.

Allowlisted public fitted bundles contain inference code, necessary fitted input dictionaries, class/schema metadata and their hashes. They exclude the private recipe, discovery history and scoring references. The private prepared plan and worker directory remain private. Canary/text-policy checks supplement the explicit file allowlist; passing static checks does not replace actual complete-prediction parity.

13.6 Release and installation are independent from model promotion

The current tag workflow runs for `v*`, checks tag = workspace = Python version, builds a Linux binary, writes build provenance and checksums, builds the Python wheel, and records model/dataset references by hash. The committed build command selects default CLI features; it does **not** include `--features asm`. Therefore the existence of this workflow does not prove that a released binary can execute the private ASM path. No newly published release is asserted here. The current private source build in §4.3.1 Development build of the CLI explicitly selects `asm`.

Release workflow and version consistency are source evidence. A future reviewed version bump can use `just bump X.Y.Z`; that recipe edits manifests and the lockfile and invokes Cargo. It is a developer mutation recipe, not a command executed for this manual or a decision to release any particular version. Exact bump implementation.

The checksum-verifying private binary installer and CLI-only source-build helper remain product28 follow-ons. Their concrete commands and cold/warm separation are in §4.3 Build and install. Installing a new CLI does not admit new data, train a model, or promote a checkpoint. Likewise, `models pull` or a storage manifest does not change the accepted policy pins.

13.7 Preservation and lane closure

Before deletion inspect `git status`, `git stash list`, `git log --all --not --remotes`, and local versus remote tags. A branch-only log misses commits elsewhere.

Preserve unique source/evidence for the tree being removed, restore independently into the reusable bare verification directory without alternates, and verify the exact commit, tree and connectivity. A valid bundle checksum does not prove the desired detached HEAD was included.

After preservation and a fresh consumer check, remove a closed lane with `git worktree remove`. Indefinite redundant copies are not an alternative to the required reclamation step. Unknown contents remain protected. Follow `deploy/maintenance/README.md`; its reporting timer proposes candidates and never authorizes deletion. Backup, replication and retention operations remain separately authorized: closing a lane does not authorize them, and neither does this manual.

13.8 Rebuilding and updating this manual

Rebuilding the book is itself a runbook. Its summary:

Runbook part	Rebuilding this manual
Purpose and status	Render both volumes from the committed Markdown, figures and data. **available**: the committed `render-receipt.json` records both edition-2 volumes, their input and output hashes and warnings; the receipt is written only after every renderer gate passes, and `publication.json` declares the build timestamp. The original PR87 build passed every renderer gate with no warnings. Each build is identified by its own `render-receipt.json`.
Prerequisites and resources	Python 3.12 with the direct versions in `tools/requirements-docs.txt`, Pango/Fontconfig, DejaVu fonts and Poppler; local CPU only. The build opens no network resource, model, dataset or sealed label.

Runbook part	Rebuilding this manual
Manifest-creation command	None. `book.json`, `publication.json` and `aliases.json` are reviewed inputs edited directly; no committed command generates them.
Execution	The figure scripts when their data changed, then the renderer, then `tools/verify_publication.py` (§13.8.1 Rebuilding the book, §13.8.3 Regenerating the training and target figures, §13.8.4 The publication record and its three identities).
Artifacts and success	Two PDFs under `docs/`, `manual.html`, `MANUAL.md`, `structure.json` and `render-receipt.json`. Success is a zero exit from the renderer with every gate passed and a zero exit from `tools/verify_publication.py`, followed by inspection of the rendered pages and extracted text.
Failure and resume	A failed gate exits nonzero and can leave a new PDF beside the previous build's `render-receipt.json`, which the renderer does not remove and which no longer describes it; such a PDF is not a build, and `verify_publication.py` rejects it as stale. Fix the input and rerun from the start: there is no partial resume.
Limitations	PDF bytes are not reproducible bit for bit; the renderer does not check that a figure is newer than its data; it runs no model.

13.8.1 Rebuilding the book

The 2026-09-23 builds used Python 3.12 and the observed direct versions in `tools/requirements-docs.txt`. That file is not a transitive lock or a verified cold installer. A fresh environment also needs working Pango/Fontconfig, DejaVu fonts, and Poppler's `pdfinfo`, `pdftotext` and `pdftoppm`. The render receipt records exact Python/library versions and font-file hashes. The following environment-creation recipe is provided for rebuilding; it was not executed as a fresh installation during this document audit:

```
MANUAL_ENV=/home/ubuntu/.venvs/jope-manual
python3 -m venv "$MANUAL_ENV"
"$MANUAL_ENV/bin/python" -m pip install \
  -r docs/internal/asm-manual/tools/requirements-docs.txt
. "$MANUAL_ENV/bin/activate"
```

From the repository root, render the audited retained inputs with these local-only commands:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/
make_inventory_figures.py
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/make_figures.py
python3 docs/internal/asm-manual/tools/build_manual.py
pdfinfo docs/asm-maintainer-manual.pdf
pdfinfo docs/asm-evidence-reference.pdf
pdftotext -layout docs/asm-maintainer-manual.pdf /path/to/durable/review/volume-1.txt
pdftotext -layout docs/asm-evidence-reference.pdf /path/to/durable/review/volume-2.txt
```

The two figure scripts above do not regenerate the training and target figures. A complete rebuild also runs the four scripts of §13.8.3 Regenerating the training and target figures before `build_manual.py`, because the renderer does not check that a figure is newer than its data. `make_figures.py` still defines a fixed host path to the ASM custody root (`tools/make_figures.py:39-41`), but never uses it.

Rendering uses Python Markdown, BeautifulSoup and WeasyPrint with system DejaVu fonts; their observed versions and font hashes are retained in the render receipt. Plotting uses Matplotlib and NumPy. The renderer rejects missing/nonlocal images and failed resource fetches, and checks that inputs did not change during rendering. It does not run models, fit predictors, fetch datasets or open sealed labels. Inspect the resulting pages and extracted text before treating a successful render as a complete document.

Byte-identical PDF rebuilding is not promised. `SOURCE_DATE_EPOCH` fixes font-subset timestamps, but the observed WeasyPrint 68.1 tagged-table identifiers contain process-specific values. Preserved first-edition repeat renders had identical normalized logical objects and decoded non-container streams after accounting for those identifiers, while their PDF hashes differed. Keep each output with its own render receipt and verify input hashes, content and layout; do not treat a changed PDF hash alone as changed measurements. This observation concerns the renderer, not numerical predictor reproducibility.

13.8.2 What the renderer reads, checks and writes

`tools/build_manual.py` takes the layout from `book.json` (schema `asm-manual.book/v1`):

- **Chapters and appendices.** `chapters` is a flat list of section files in reading order, numbered from 1 when rendered. `appendices` lists files whose letters must run A, B, C and onwards. The renderer refuses a file listed twice, a file in `sections/` that `book.json` omits, and a listed file that does not exist.
- **Volumes.** An appendix with a `volume` key is published in that volume's PDF. Edition 2 publishes chapters 1 to 13 as Volume 1, `docs/asm-maintainer-manual.pdf`, and appendices A to E as Volume 2, `docs/asm-evidence-reference.pdf`. After chapter 13, Volume 1 carries a signpost that keeps each appendix's letter, title and sections and links into Volume 2. It is followed by a legacy destination index, which gives every edition-1 destination now in Volume 2 a named landing linked to its new heading. Volume 2 has its own title page and two-level contents.
- **Position-independent anchors.** Section files are named by topic, not by position. A heading's anchor is `c-<file stem>--<GitHub heading slug>`, and a chapter title's is `c-<file stem>`, so moving a chapter does not change its anchors. Write a cross-reference as a Markdown link whose target is `file.md#slug`, or `#slug` within one file. The renderer fills a link whose text is exactly `§` with the target's number and title, and refuses any internal link it cannot resolve.
- **Aliases.** The required `aliases.json` maps all 266 published older destinations, such as edition 1's `chapter-NN` and `chapter-NN-M` anchors, to current headings. Each alias becomes an additional anchor on its heading. Destinations moved to Volume 2 also retain a named landing in the original Volume-1 PDF, with an explicit link to the exact semantic target. An alias that names no heading, or that repeats an existing heading anchor, is refused.
- **Strict heading check.** Each file must begin with exactly one `#` title. Headings go at most four levels deep and may not skip a level. Headings under one parent need distinct titles; a heading followed directly by a heading of the same or a shallower level, or by nothing, is refused as empty; and a heading may not have exactly one subdivision. The chapter title counts as a parent, so a file with exactly one section is refused; a file with no sections at all is only a warning. Headings are numbered by the renderer, never by hand, and heading-like lines inside fenced code are not treated as headings.
- **Size gate.** Each PDF must be at most 5 MiB (5,242,880 bytes), the repository's per-file cap. A larger volume stops the build.

- **Pinned inputs.** Every Markdown, JSON, JSON Lines, CSV, Python, CSS, PNG, SVG, property-list, gzip and text file under `docs/internal/asm-manual/`, except the generated outputs, is hashed together with `docs/asm-maintainer-status.plist` and the landing page `docs/asm-maintainer-manual.md`.
- **Outputs.** Besides the PDFs: `manual.html` (Volume 1), the combined `MANUAL.md`, `structure.json` with every numbered heading, figure and alias, and `render-receipt.json` (schema `jope.manual-render/v2`). The receipt's `volumes` list gives each PDF's name, page count, SHA-256 and bytes; its top-level `pages`, `pdf_sha256` and `pdf_bytes` describe Volume 1 only. It also records heading counts per level, the alias count, warnings, the Python, library and font identities and every input hash, and it is written only after every gate has passed.

13.8.3 Regenerating the training and target figures

Regenerate the training and target figures before the PDF after changing an audited source CSV:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/
make_training_figures.py
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/
make_target_figures.py
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/
make_legacy_training_figure.py
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 docs/internal/asm-manual/tools/
make_per_step_figure.py
python3 docs/internal/asm-manual/tools/build_manual.py
```

These commands render retained observations; they do not train a model or reconstruct unavailable metrics. The original-history verifier needs the separately preserved source files named in each CSV. Plot rebuilding uses the committed CSVs. No missing source or changed hash may be converted into a successful historical verification.

13.8.4 The publication record and its three identities

Three identities describe a copy of this book. They change independently, and none implies the others:

Identity	What it names	Rule and edition-2 value
Implementation snapshot	The reviewed application source that the book documents	Reviewed `68f56404c3e199e06413bfd645d3fe0803d7b96f`, merged 2026-09-23T16:34:47Z as main `a75e543d2166017cb0139e0a251d51d9a0c98e08`, tree `176b75bc36d76c7ce4eaeb7234677ec013468889` (§1.8.1 Implementation snapshot and this manual's own identity). The publication manifest and covers pin this same application snapshot. Later documentation and implementation deliveries have separate dated observations.
Qualification observations	Each measurement the book reports	Each keeps its own observation time and receipt, such as the fourteen-command example run of 23 September 2026 (§10.4.3 Measured execution of the private examples). Rebuilding the book repeats none of them.

Identity	What it names	Rule and edition-2 value
Document build	The edition, its pinned inputs and its PDFs	Recorded by the build itself: the input hashes and each volume's PDF hash, bytes and pages in `render-receipt.json`.

The renderer requires `docs/internal/asm-manual/publication.json` and prints it in the front matter as the publication record, with one subsection per identity. It validates the schema, UTC timestamps, implementation identity format and both PDF filenames, and refuses undeclared PDFs under `docs/`. The snapshot subsection gives source, main, merge time, tree and an optional note. The observations form a table of observation, observed UTC time and receipt. The document-build subsection states the edition and its date, `SOURCE_DATE_EPOCH`, the number of pinned inputs and a SHA-256 digest of their path and hash list. PDF hashes stay in `render-receipt.json`, because a PDF cannot contain its own hash. `publication.json` is a reviewed input edited directly and is itself pinned: the renderer derives none of it from Git and refuses to build without it. The declared `build_utc` sets `SOURCE_DATE_EPOCH`. The cover's edition line (edition, date and the manifest's implementation main) derives from the manifest, and its count of primary acceptance items derives from `data/roadmap.json`; the renderer computes no per-status counts. Run `python3 docs/internal/asm-manual/tools/verify_publication.py` after rendering to reject stale or missing input/output hashes, size violations and inconsistent task priorities or statuses across Markdown, JSON and plist.

13.8.5 Updating the book

Refresh inventories and receipts, then section Markdown, figures, checklist, `publication.json` and both PDFs. Keep a rendering input manifest. A checkpoint does not update performance figures by implication; merged training source does not prove actual learning. Historical Elo remains historical after regeneration.

A manual change lands through the same reviewed lifecycle as source (§13.4.3 Freeze, review and merge): fresh exact-head CI and an independent non-contributor native review of the final head. Every committed file, including the rendered PDF, must stay within the repository's 5 MiB (5,242,880-byte) file cap (`deploy/ci/check.sh:46`), and because the rebase merge lands every commit of the branch, the cap applies to intermediate commits as well. The earlier publication, PR84 (published head `e06c0551`), was closed as superseded for that reason: the independent non-contributor review 5295763455, a COMMENTED review at `e06c0551`, found two files above the cap in three ancestor commits. Its clean-history successor, commit `2022017a8` on `docs/asm-maintainer-manual-final`, is a single commit on top of main `a75e543d2166…` that carries a 203-page PDF (`69418c48…`). It was published as PR85 and merged at 2026-09-23T22:33:36Z as main `bdfc95c63d905c647e41175e79dc30cdc8600287`, after independent native approval 5297559032 at that exact head and full CI run 35927921851; the following main-guard run 35929010658 passed. This second edition restructures and corrects that published edition. The original authoring commits `7b9398b76` and `bda953f62` are preserved; their changes and the unfinished content pass were integrated on `docs/asm-manual-edition2-integration`, retaining the separately merged PR86 corrections. Edition 2 merged through PR87 at 2026-09-24T02:58:00Z as main `02c0bef6c81ab17e034daea308c1a0b5a7940467`, after exact-head CI 35948653955 attempt 1 and independent native approval 5299139743; main-guard 35949456287 passed. Later revisions need their own exact-head CI and independent native approval; local rendering alone does not establish publication.

Before delivery check local links, source identities, totals, metric directions, captions, capability status and rendered pages. Extract PDF text and inspect dense tables, examples and figures. Preserve unavailable statistics and measurements explicitly rather than filling them with zero.

CHAPTER 14

Prioritized implementation plan and extension interfaces

IN THIS CHAPTER

14.1 Prioritized implementation plan 155
14.1.1 Purpose and scope 155
14.1.2 Critical order 156
14.1.3 Compact task index 156
14.1.4 Proposed owner decisions and external dependencies 159
14.1.5 Stopping and completion rules 162
14.2 Extension interfaces 162
14.2.1 The continuation interfaces that exist today 162
14.2.2 Code to extend 163
14.2.3 Extending this version without losing comparability 164

14.1 Prioritized implementation plan

14.1.1 Purpose and scope

This checklist is the requested property-list of remaining work. `roadmap.json` is its machine-readable source, with stable IDs, priorities, evidence levels, dependencies, commands, required receipts, budgets and source references. It describes source 68f56404 and the stopped training64, product28, warm-target and winner-export follow-on patches (§E.8 Names and glossary). PR83 is merged: remote main `a75e543d2166017cb0139e0a251d51d9a0c98e08` has the reviewed source tree, and main-guard 35889731910 attempt 1 succeeded. That implementation snapshot is unchanged as of 2026-09-23 about 23:55 UTC. Edition 1 of this manual merged later, through PR85 at 22:33:36 UTC, as main `bdfc95c63d905c647e41175e79dc30cdc8600287`, after independent native approval 5297559032 and exact-head CI 35927921851; main-guard 35929010658 passed. That merge changed only documentation files (under `docs/`, plus `README.md` and `AGENT_CHAT.md`). The documentation-correction PR86 (head `0079ceddbaba739d736e80f6fe9b999955c4ad3a`, tree `01ecd51afc77e753ee4fe1f3fbe3c130803bbf43`) merged at 2026-09-24T00:04:12Z as main `2b375ef11fc9298ffa67d4d03e0bb83053201802`; main-guard 35936673729 passed. Edition 2 merged through PR87 at 2026-09-24T02:58:00Z as main `02c0bef6c81ab17e034daea308c1a0b5a7940467`, after exact-head CI 35948653955 attempt 1 and independent native approval 5299139743; main-guard 35949456287 passed; revisions repeat that lifecycle (§1.8.1 Implementation snapshot and this manual's own identity). Clean canonical main was verified before the documentation branch. LIFE-01 records the completed readback; later source work still needs its own lifecycle.

P0 blocks the specified downstream proof or integration; it does not mean every item is a newly discovered defect. P1 is required for the full requested demonstration or manual. P2 is explicit subsequent work. Commands are unexecuted acceptance templates unless their named evidence says otherwise. Variables must resolve to actual final-source artifacts and emitted identities. An empty command list records a missing truthful interface rather than inventing one.

There are 43 primary items. Five of them (DOC-05, DOC-06, PACK-01, FOCUS-01 and LOOP-01) respond to the user's training-history, continuation, flywheel, focused-search and packing questions. At the owner's direction for edition 2, LOOP-01 is the central regression product milestone, and this roadmap therefore lists it as P1 (§14.1.2 Critical order). Documentation work does not wait for implementation; implementation and numerical follow-ons preserve the original budgets and frozen protocol.

14.1.2 Critical order

Verified reviewed main → guarded source integration → actual tests, host qualification and final packaging → fresh qualified contexts and preregistered real teachers → all-eligible family-disjoint corpus → one GPU-hour training with continuation → explicit new-checkpoint consumption → real fitted warm demonstration → unchanged thirty-dataset campaign, selected-state backend qualification inside its remaining window, and report → eligible motif reuse.

LOOP-01 is the central regression product milestone (P1, not yet qualified). Its chain is: new eligible target → actual exposure → changed parameters → compatible reload → improved fitted search → another new generation. It builds on the order above: an eligible corpus (TEACH-04), a verified continuation and export (TRAIN-04) and live consumption of the newly trained checkpoint (NEURAL-03). Two passes through one corpus are continuation, not two generations. The accepted 512×6 step-60000 model stays separate from experimental regression lineage v1 (256×2, new initialization) (§8.4.1 What the pending campaign trains). LOOP-01's two controlled rounds may remain unfunded or inconclusive within the existing 20-minute regression GPU cap; no new allocation is implied. The comparison of sibling fresh and replay-only branches (25/75 fresh/replay versus replay-only, at equal committed updates) and the preregistered decision rule are proposals, not achieved results (§9 Measuring repeated learning and deciding improvement).

Library parity fixtures precede the full panel. Actual selected-winner exports require complete confirmations and must still fit the original execution window. They cannot consume its reporting reserve or silently run after expiry. Documentation can proceed alongside source preparation; actual numerical examples need an allocated execution window.

Do not spend the GPU hour until the regression consumer and its actual fixture tests are ready. Do not freeze the final official panel configuration until its checkpoint is chosen and consumed. Separate proof allowances suggested in older plans are proposals, not new budget authorization.

14.1.3 Compact task index

ID	Priority	Stage	Workstream	Current state	Dependencies
LIFE-01	P0	0	coordination	complete with retained readback	None
DOC-01	P1	0	manual	canonical private manual assembled	LIFE-01
DOC-02	P1	0	manual examples	complete at named recipe/ refit scope	DOC-01

ID	Priority	Stage	Workstream	Current state	Dependencies
DOC-03	P1	0	manual figures	figures rendered; local PDF inspection completed; publication pending	DOC-01
DOC-04	P1	0	manual quality	rendered; final publication gates pending	DOC-01, DOC-02, DOC-03
INT-01	P0	0	serial integration	stopped unmerged packets	LIFE-01
INT-02	P0	0	integration qualification	pending	INT-01
BENCH-01	P1	1	frozen benchmark foundation	implemented; retain during integration	INT-02
BENCH-02	P1	1	external comparators	refresh at final environment	INT-02
EXEC-01	P1	2	bounded workers	implemented; CI coverage gap retained	INT-02
EXEC-02	P1	2	fault regression tests	test gap: drain-and-fixtures-F2	INT-01
EXEC-03	P1	2	receipt replay regression tests	test gap: resume-fix-F2	INT-01
EXEC-04	P1	2	cutoff semantics regression	test gap: regressions-F2	INT-01
EXEC-05	P1	2	accounting and unknown outcomes	pending execution	INT-02
NEURAL-01	P0	3	context qualification	requires final-source binding	INT-02
TEACH-01	P0	6	HWM comparison identity	not runtime qualified	INT-02
TEACH-02	P0	6	label exposure readers	residual eligibility-B6	TEACH-01
TEACH-03	P0	3	real teacher collection	blocked on fresh admitted evidence	BENCH-01, BENCH-02, NEURAL-01, TEACH-01, TEACH-02
TEACH-04	P0	3	real corpus materialization	blocked on two real families	TEACH-03
TRAIN-01	P0	3	actual trainer tests	residual training-F3	INT-02

ID	Priority	Stage	Workstream	Current state	Dependencies
TRAIN-02	P0	3	native supervised training	host proof pending	TRAIN-01
TRAIN-03	P0	3	bounded real GPU training	blocked on admitted data and consumption readiness	TEACH-04, TRAIN-02, NEURAL-02
TRAIN-04	P0	3	checkpoint continuation and export	requires actual campaign	TRAIN-03
NEURAL-02	P0	3	trained-checkpoint consumer readiness	compile/runtime fixture pending	INT-02, TRAIN-01
NEURAL-03	P0	3	live newly trained consumption	newly trained consumption pending	TRAIN-04, NEURAL-01
INFER-01	P0	4	fitted state and privacy	qualification pending	INT-02
INFER-02	P1	4	exact selected-winner portability	actual export/ parity unrun	INFER-01, PANEL-01
INSTALL-01	P1	5	private installation	cold/live path pending	INT-02
INSTALL-02	P1	5	final input/ runtime preparation	pending final source	INSTALL-01, BENCH-01, BENCH-02
DEMO-01	P1	5	measured warm fitted demonstration	real trained warm proof pending	NEURAL-03, INSTALL-02
PANEL-01	P1	5	unchanged thirty-dataset campaign	pending final trained configuration	BENCH-01, BENCH-02, NEURAL-03, DEMO-01, INFER-01, INSTALL-02
REPORT-01	P1	5	complete measured report	full campaign report pending	PANEL-01, EXEC-05
MOTIF-01	P1	6	eligible reusable fragments	mining front door incomplete	TEACH-01, TEACH-02, TEACH-03
MOTIF-02	P1	6	reuse validation	live reuse pending	MOTIF-01
MOTIF-03	P2	6	new operators	future conditional work	MOTIF-02
IP-01	P1	0	public privacy boundaries	residual D-IP4 and future PR text work	INT-01

ID	Priority	Stage	Workstream	Current state	Dependencies
MAINT-01	P2	0	broad diagnostic qualification	not closed by fast CI	LIFE-01
FUTURE-01	P2	4	CUDA inference and serving promotion	deferred	INFER-02, REPORT-01
DOC-05	P1	0	real training and target history	Original training and target values verified; final composed-document inspection required	DOC-01
DOC-06	P1	0	versions, KPIs and continuation runbook	Current and pending interfaces source-audited; independent chapter review complete	DOC-01
PACK-01	P2	3	packed training consumer and incremental loading	Legacy binary store exists; pending Python category route still materializes JSONL	TEACH-04, TRAIN-01
FOCUS-01	P2	6	sustained single-dataset improvement	Current bounded proof exists; no incumbent-seeded attack front door	TEACH-01, TEACH-02, TEACH-03
LOOP-01	P1	3	central regression product milestone: repeatable measured learning	New complete improved-target-to-improved-search cycle not yet qualified	NEURAL-03, TEACH-04, TRAIN-04

14.1.4 Proposed owner decisions and external dependencies

The property list is self-contained: every proposed dependency resolves to a primary item, a coordinated proposal, or the registry below. The original contributor proposal is preserved as `data/coordinated-roadmap-proposal.json`, SHA256 `334ae97446ef29963bce47582f1550c7d369f0b329ef143c128685991422ffd4`. Its recommendations are contributor suggestions, not fresh user instructions. Already explicit user decisions require no reconfirmation.

Reference	Meaning	Current disposition
OD-1	Confirm the name ASM in writing (reported by Codex 16:07Z)	ASM naming is already explicit user direction; no reconfirmation needed. Any protocol rename remains a separately versioned change.

Reference	Meaning	Current disposition
OD-2	Confirm HWM train-side OOF admission and the test-to-development relabel with burn receipts (audit D3)	Train-side OOF admission and exposed-test-as-development treatment are already explicit user direction; no reconfirmation needed.
OD-3	D-IP1: ban 'tree' in emitted code	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-4	D-IP2: extra_trees naming	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-5	D-IP3: publication boundary	This private manual is explicitly requested and authorized. Broader public-release boundary changes remain separate; this item does not block the manual.
OD-6	D-IP4: PR 83 identifiers	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-7	B6 fit fence and data rows	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-8	Teachers from panel families 230_machine_cpu and 712_chscase_geyser1	Prospective small families already selected from provenance; exposure labels require actual evidence. Unknown must not become known-seen solely from this recommendation.
OD-9	W2 scale-out CPU (about 4 h)	Proposed additional compute outside the current allocation; no new hours or deadline are granted.
OD-10	3,755 unknown-licence catalog datasets	Unresolved data-use proposal. Unknown license metadata does not establish permission for any proposed use.
OD-11	Lineage-B corpora	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-12	Multiclass data (the catalog has none)	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-13	Sealed one-shot evaluation	Outside current execution scope: sealed evaluation remains untouched.

Reference	Meaning	Current disposition
OD-14	Cost scenario constants	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-15	CUDA timing	Unresolved contributor proposal; not a newly imposed approval prerequisite and not execution authorization.
OD-16	Second GPU hour	Proposed additional compute outside the current allocation; no new hours or deadline are granted.
OD-17	The proposed 300 s proof windows	Per-proof resource files must identify actual admission and remaining allowance; an example duration does not create a new budget.
product28 merged	Product28 stopped overlay has completed its own reviewed merge; part of INT-01, currently pending.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.
product28 on main	Same product28 reviewed-main prerequisite; currently pending.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.
training64 merged (eligibility API)	Training64 eligibility interface has completed its own reviewed merge; part of INT-01, currently pending.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.
freeze passed	Final implementation/runtime configuration and required acceptance are frozen; documentation freeze alone does not satisfy this.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.
compute window	A concrete finite resource allocation is admitted; no new allowance is implied.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.
DATA-01 for part b	The metadata inventory proposal DATA-01 is complete for the catalog-generation part of DOCFIX-01.	External prerequisite, explicitly unresolved unless a separate retained receipt closes it.

The dispositions above are the roadmap data's own and are not edited here. Their state between the PR86 merge (2026-09-24T00:04:12Z) and the PR88 merge (2026-09-24T05:11:50Z):

- training64 and product28 remain stopped patches outside `main`, so the rows “product28 merged”, “product28 on main” and “training64 merged (eligibility API)” stay open.
- OD-8: both proposed teacher families, `230_machine_cpu` and `712_chscase_geyser1`, are regression datasets of the frozen panel. If they are used to manufacture teachers, later evaluations on them must be labelled exposed/adaptive (§9.7.1 Freezing the experiment and choosing families).
- DATA-01: Claude's metadata-only packet was released at 19:30:23 UTC (manifest `SHA256SUMS`, `4bebee5a…`, 148 entries), but its sealed-name gate failed on one raw file. A later clarification (`RELEASE-CLARIFICATION.md`, `380f3402…`, with manifest `SHA256SUMS.release-2`, `a4b0c436…`) limits the release to eight aggregate files and keeps `raw/`, `scratch/` and the full HANDOFF in custody. Codex imported only an aggregate subset (stopped subset `558548be…`): the summary `e4dbe075…` and the release pointer `366fa000…`, present here as `data/data-estate-extension-summary.json` and `data/data-`

`estate-extension-release.json`. No retained receipt closes “DATA-01 for part b”. The DOCFIX-01 correction landed through the separate documentation PR86 (head `0079ceddb`), merged as `2b375ef11` at 2026-09-24T00:04:12Z.

14.1.5 Stopping and completion rules

An unsuccessful prerequisite remains a recorded refusal or failure. Do not substitute synthetic real-data teachers, another winner, easier datasets or a new deadline. Source completion does not close a runtime task. A plan, help output or checkpoint file is not the corresponding live receipt.

Keep four claims distinct: an eligible teacher, a new discovery, information actually consumed in training, and an improved policy. Evidence for one does not establish another, and a hash alone does not establish eligibility.

For LOOP-01 the owner’s stop rules are proposed: stop on no eligible discoveries, an incompatibility, no material gain, insufficient evidence or an exhausted budget; preserve the best checkpoint; never promote serving automatically. If its rounds are unfunded or the data insufficient, report the mechanical proof and descriptive results, and record the causal verdict as “inconclusive”, not as success. Gain over the parent checkpoint is a separate claim from advantage over generic search or the product; generic search and the accepted step-60000 model remain report baselines (§9 Measuring repeated learning and deciding improvement).

Full delivery requires all thirty outcomes, losses and gaps, backend limitations, measured unique-predictor/recipe costs, honest training category scope and actual newly trained consumption. Ten wins per task and an under-sixty-second warm result remain declared goals, not promises. No win-counting rule is ratified: §E.8 Names and glossary gives the proposed PANEL-WINS definition, pending owner ratification, and §9.11.2 Win counting and a proposed acceptance record the separate win columns. The one-hour/ten-million-token comparison must retain unknown hardware, workload, token and currency quantities.

The document gate requires one readable PDF plus matching section Markdown, reproducible figure inputs, source/lifecycle references and this maintained checklist. The maintainer-private examples are authorized documentation; their existence does not expand public inference bundle contents.

14.2 Extension interfaces

14.2.1 The continuation interfaces that exist today

The training chapter is the authoritative continuation runbook: its per-mode procedures, variables and limits govern (§8 Training, continuation, export and compatibility). This table is only an at-a-glance reference. Its status labels repeat that chapter's: available, implemented but unqualified, pending integration and proposed. An interface that is present at main `a75e543d2` but has no execution receipt is implemented but unqualified, not available, and no label by itself qualifies a campaign.

Interface	Status	Current behavior	Appropriate interpretation
`jope train-step --local`	implemented but unqualified	Fits a centroid router from local labels	Small router training; not the deep policy model.
`jope learn ...`	proposed	Not exposed by the umbrella parser at `68f56404` (main `a75e543d2`)	ADR operating intent; do not present as a working command.

Interface	Status	Current behavior	Appropriate interpretation
`jope-parallel train ...` with `neural`	implemented but unqualified (dispatch held)	Native regression trainer syntax exists; direct numerical dispatch is held	Retained native continuation machinery, not presently usable through this direct entry point.
`jope-py train-policy --init-from ...`	implemented but unqualified	Python policy weight warm start; CLI exposes no `--resume-from`	A new optimization run initialized from compatible weights, not exact optimizer continuation.
`train_policy_v3_from_jsonl(..., resume_from=..., resume_checkpoint=...)`	implemented but unqualified (Python API)	Existing Python API restores model, optimizer, scheduler, scaler and RNG subject to compatibility	A real continuation API, requiring a trusted matching Python resume artifact; not an accepted-native-checkpoint conversion.
Pending `jope train plan/run/resume/verify/export`	pending integration	Supervised, task-routed, bounded training proof and experimental export	Preferred integration target for the approved one-GPU-hour demonstration; still unintegrated and unexecuted.

Today no executable route produces a successor of the accepted step-60000 serving model. Exact native `--resume` of step 60,000 is refused: its `state.json` has no `execution_partition`, and resume requires the original partition (`crates/jope-engine/src/training.rs:1106`). Every native `jope-parallel train` dispatch, including `--initialize` with a carried optimizer, stops at the dispatch hold (§8.2.4.2 The dispatch hold, resource incompatibility and valid topologies); a `parallel.checkpoint_continuation.v1` carry must in addition reproduce the parent run's recorded world/microbatch partition (`crates/jope-engine/src/checkpoint_recovery.rs:191`), and the accepted lineage ran at world 8. `jope-py train-policy --init-from` loads a Python policy payload and copies only shape-compatible tensors (`python/src/jope/policy/train_v3.py:1635`, `:1702`); it is not a loader for the native 512-wide, six-layer model (§2.8.1 Checkpoint binding and configuration). The Python resume API accepts only a `PrimePolicyTrainerResume.v1` artifact (`train_v3.py:1457`), a schema the native trainer does not write. The pending training64 campaign trains a new, smaller Python model family, experimental regression lineage v1 (256×2, new initialization), not a continuation (§8.4.1 What the pending campaign trains). Under the owner's naming rule, its initial model (M_init) is named M0 only after it has actually trained on the first corpus (C0). The RTX step 60,100 run is a historical hardware-qualification continuation, and the later A100 step 69,000 remains unaccepted; neither is a serving successor.

Sources: current umbrella commands, local train-step dispatch, Python CLI flags, Python API.

14.2.2 Code to extend

These are the narrow extension points at reviewed source `68f56404`, whose tree is identical to merged main `a75e543d2`. Follow the exact-revision links; line numbers refer to that snapshot.

File	Responsibility for the focused extension
optimize_cli.rs	Add the request/command only after defining its schema and budget semantics

File	Responsibility for the focused extension
origin_proof.rs	Existing single-frozen-regression plan and resource binding; current dataset restriction is explicit
optimize.rs	Configuration, source/assets, search allowance and fit adapter integration
search_plan.rs	Exact train-side five-fold selection closure and original rows/groups
neural_search.rs	Initial population, measured ancestry, mutation/crossover, duplicate handling and original-arm resume
campaign_cli.rs	Immutable plan and private journal handling, execution/report front doors
hwm.rs	Existing HWM admission and immutable event history; stricter identity/eligibility changes remain a pending overlay
Pending `jope-learning/src/eligibility.rs` and `jope-cli/src/train/corpus.rs`	New authenticated OOF admission and eligible context-to-program corpus export; integrate and qualify before using them as authority

Required behavioral tests include an improving incumbent, a tie that retains the prior pointer, a worse challenger, a wrong evaluator/split/vocabulary, missing folds, changed categories/class order, unavailable dependencies, cancelled partial waves, concurrent incumbent changes, an exhausted resume window and an attempted sealed/burned-partition admission. Actual successful improvement and successful no-improvement completion are separate acceptance cases.

The owner’s rules for this extension are proposed, not implemented. Focused optimization is finite and incumbent-aware: one total window; exact identity for rows, groups, evaluator, vocabulary and numerics; a declared initial population, exploration and wave schedule; and an atomic incumbent update. A tie keeps the incumbent, and a race with a concurrent incumbent change requires a fresh decision. Ingesting an arbitrary new dataset is a separate interface from frozen30 (§6.5.2 What is missing for a sustained, incumbent-seeded dataset attack).

14.2.3 Extending this version without losing comparability

A new dataset requires a content/split/group identity, family and exposure classification, train-only preparation, task-correct scoring, and a retained unavailable outcome if any prerequisite fails. A changed encoder requires component/assembly qualification and a new input identity, even when it still outputs 2048 values. A vocabulary change requires explicit token compatibility, executable canonicalization and checkpoint migration decisions. A new learned policy requires optimizer/reload/continuation proof followed by actual downstream generation-and-fit evaluation. A new operator additionally requires typed grammar support, task applicability, ablation and fitted-state/export coverage. A faster scheduler or emitter requires unchanged input/loss semantics and actual cancellation/resume/parity evidence.

Do not close this loop by editing version strings, declaring an old snapshot newly eligible, treating a warmer loss curve as promotion, or retagging a recipe-refit example as a fitted export. Use the source interfaces that exist, integrate the bounded missing path through the serial reviewed queue, and retain every remaining acceptance gate in §9.6 The closure ledger.

VOLUME 2

Evidence reference

These appendices are published as `docs/asm-evidence-reference.pdf`, with the same numbering and anchors; the entries link into that file. Volume 2 holds the complete historical records, per-dataset statistics, inventories, binary formats, task catalogue, source and receipt indexes and glossary that the fourteen chapters of Volume 1 summarize and cite.

Appendix A All thirty datasets and all 522 feature records

A.1 All thirty datasets at a glance Volume 2
A.2 Regression datasets Volume 2
A.3 Binary classification datasets Volume 2
A.4 Multiclass classification datasets Volume 2
A.5 Explicitly omitted or unknown statistics Volume 2

Appendix B Historical training, checkpoint lineage and original loss trajectories

B.1 Dated lineage and evidence coverage Volume 2
B.2 Legacy Python training: gen31 and round 1 Volume 2
B.3 Native training: the 108-block audit and the A100 recovery lineage Volume 2
B.4 Current publication regeneration and provenance Volume 2

Appendix C Historical GP targets, rankings, Elo and attribution limits

C.1 Did GP targets get better? Volume 2
C.2 Existing historical rankings and Elo Volume 2

Appendix D Dated data estate, recovery evidence and binary formats

D.1 Configured product stores and local roots Volume 2
D.2 Additional archive, replica and custody metadata Volume 2
D.3 Additional host data roots Volume 2
D.4 Recorded copy and retention gaps Volume 2
D.5 Recovery evidence: what is preserved, and what was lost Volume 2
D.6 Retained corpus sizes Volume 2
D.7 Binary format: the exact packed teaching record Volume 2

Appendix E Full task catalogue, source/schema index, receipts and glossary

E.1 Primary acceptance items by stage Volume 2
E.2 Coordinated future backlog: 24 proposals Volume 2
E.3 Planned execution and review ownership Volume 2
E.4 Source index Volume 2
E.5 Machine-readable companions Volume 2
E.6 Receipts Volume 2
E.7 Reproduction boundaries Volume 2
E.8 Names and glossary Volume 2

Legacy destination index

Published edition-1 links to evidence now land here. Follow the entry to Volume 2.

- chapter-03-12 → E.4.1 Current implementation references
- chapter-06-5 → C.2.2 Ranking figures: interpretation contract
- chapter-07-10 → C.2.1 Retained snapshots and plot data
- chapter-07-4 → D.1.1 Access method, store defaults and path conventions
- chapter-07-5 → D.2.1 Additional and replica buckets
- chapter-07-6 → D.3.1 Root inventory and totals
- chapter-07-7 → D.4.1 Material without an established remote copy
- chapter-08 → A.1.2 Frozen dataset coverage
- chapter-08-1 → A.1.2 Frozen dataset coverage
- chapter-08-2 → A.1.2 Frozen dataset coverage
- chapter-08-4 → C.2.2 Ranking figures: interpretation contract
- chapter-08-5 → C.2.3.1 Historical benchmark-valid Elo
- chapter-08-6 → C.2.3.2 Why the two historical ladders differ
- chapter-08-7 → C.2.4.1 Same-cut v1 and v2 snapshots with screen and deep Elo
- chapter-08-8 → C.2.4.2 Historical relative loss and per-dataset rank
- chapter-08-9 → C.2.5 Historical production time and the supplied reference scenario
- chapter-10-10 → E.1.1.8 INT-01 — serial integration
- chapter-10-11 → E.1.1.9 INT-02 — integration qualification
- chapter-10-12 → E.1.2.1 BENCH-01 — frozen benchmark foundation
- chapter-10-13 → E.1.2.2 BENCH-02 — external comparators
- chapter-10-14 → E.1.3.1 EXEC-01 — bounded workers
- chapter-10-15 → E.1.3.2 EXEC-02 — fault regression tests
- chapter-10-16 → E.1.3.3 EXEC-03 — receipt replay regression tests
- chapter-10-17 → E.1.3.4 EXEC-04 — cutoff semantics regression
- chapter-10-18 → E.1.3.5 EXEC-05 — accounting and unknown outcomes
- chapter-10-19 → E.1.4.1 NEURAL-01 — context qualification
- chapter-10-20 → E.1.7.1 TEACH-01 — HWM comparison identity
- chapter-10-21 → E.1.7.2 TEACH-02 — label exposure readers
- chapter-10-22 → E.1.4.2 TEACH-03 — real teacher collection
- chapter-10-23 → E.1.4.3 TEACH-04 — real corpus materialization
- chapter-10-24 → E.1.4.4 TRAIN-01 — actual trainer tests
- chapter-10-25 → E.1.4.5 TRAIN-02 — native supervised training
- chapter-10-26 → E.1.4.6 TRAIN-03 — bounded real GPU training
- chapter-10-27 → E.1.4.7 TRAIN-04 — checkpoint continuation and export
- chapter-10-28 → E.1.4.8 NEURAL-02 — trained-checkpoint consumer readiness

- chapter-10-29 → E.1.4.9 NEURAL-03 — live newly trained consumption
- chapter-10-3 → E.1.1.1 LIFE-01 — coordination
- chapter-10-30 → E.1.5.1 INFER-01 — fitted state and privacy
- chapter-10-31 → E.1.5.2 INFER-02 — exact selected-winner portability
- chapter-10-32 → E.1.6.1 INSTALL-01 — private installation
- chapter-10-33 → E.1.6.2 INSTALL-02 — final input/runtime preparation
- chapter-10-34 → E.1.6.3 DEMO-01 — measured warm fitted demonstration
- chapter-10-35 → E.1.6.4 PANEL-01 — unchanged thirty-dataset campaign
- chapter-10-36 → E.1.6.5 REPORT-01 — complete measured report
- chapter-10-37 → E.1.7.3 MOTIF-01 — eligible reusable fragments
- chapter-10-38 → E.1.7.4 MOTIF-02 — reuse validation
- chapter-10-39 → E.1.7.5 MOTIF-03 — new operators
- chapter-10-4 → E.1.1.1 LIFE-01 — coordination
- chapter-10-40 → E.1.1.10 IP-01 — public privacy boundaries
- chapter-10-41 → E.1.1.11 MAINT-01 — broad diagnostic qualification
- chapter-10-42 → E.1.5.3 FUTURE-01 — CUDA inference and serving promotion
- chapter-10-44 → E.1.1.6 DOC-05 — real training and target history
- chapter-10-45 → E.1.1.6 DOC-05 — real training and target history
- chapter-10-46 → E.1.1.7 DOC-06 — versions, KPIs and continuation runbook
- chapter-10-47 → E.1.4.10 PACK-01 — packed training consumer and incremental loading
- chapter-10-48 → E.1.7.6 FOCUS-01 — sustained single-dataset improvement
- chapter-10-49 → E.1.4.11 LOOP-01 — repeatable measured learning flywheel
- chapter-10-5 → E.1.1.1 LIFE-01 — coordination
- chapter-10-50 → E.2 Coordinated future backlog: 24 proposals
- chapter-10-51 → E.2.1.1 Proposal HANDOFF-01
- chapter-10-52 → E.2.1.2 Proposal HYG-01
- chapter-10-53 → E.2.1.3 Proposal HELLO-01
- chapter-10-54 → E.2.1.4 Proposal B6-01
- chapter-10-55 → E.2.2.1 Proposal IP-Q1
- chapter-10-56 → E.2.2.1 Proposal IP-Q1
- chapter-10-57 → E.2.2.3 Proposal IP-Q3
- chapter-10-58 → E.2.2.4 Proposal IP-Q4
- chapter-10-59 → E.2.2.3 Proposal IP-Q3
- chapter-10-6 → E.1.1.2 DOC-01 — manual
- chapter-10-60 → E.2.2.1 Proposal IP-Q1
- chapter-10-61 → E.2.1.1 Proposal HANDOFF-01
- chapter-10-62 → E.2.1.5 Proposal DOCFIX-01
- chapter-10-63 → E.2.3.2 Proposal DATA-02
- chapter-10-64 → E.2.3.3 Proposal DATA-03
- chapter-10-65 → E.2.3.4 Proposal TEACH-05

- chapter-10-66 → E.2.3.5 Proposal TRAIN-05
- chapter-10-67 → E.2.3.6 Proposal HWM-01
- chapter-10-68 → E.2.4.1 Proposal SEALED-01
- chapter-10-69 → E.2.4.2 Proposal EMIT-01
- chapter-10-7 → E.1.1.3 DOC-02 — manual examples
- chapter-10-70 → E.2.4.3 Proposal INFER-03
- chapter-10-71 → E.2.4.4 Proposal EMIT-03
- chapter-10-72 → E.2.4.5 Proposal MOTIF-00
- chapter-10-73 → E.2.4.6 Proposal COST-01
- chapter-10-74 → E.2.1.6 Proposal LAYOUT-01
- chapter-10-76 → E.3 Planned execution and review ownership
- chapter-10-8 → E.1.1.4 DOC-03 — manual figures
- chapter-10-9 → E.1.1.5 DOC-04 — manual quality
- chapter-11-2 → E.6.1 Important local receipts
- chapter-11-3 → E.7 Reproduction boundaries
- chapter-12 → A All thirty datasets and all 522 feature records
- chapter-12-1 → A.2.1 197_cpu_act
- chapter-12-10 → A.2.10 503_wind
- chapter-12-11 → A.3.1 breast_cancer_wisconsin_diagnostic
- chapter-12-12 → A.3.2.1 Summary and feature statistics
- chapter-12-13 → A.3.3.1 Summary and feature statistics
- chapter-12-14 → A.3.4 haberman
- chapter-12-15 → A.3.5 ionosphere
- chapter-12-16 → A.3.6.1 Summary, identities and class counts
- chapter-12-17 → A.3.7.1 Summary, identities and class counts
- chapter-12-18 → A.3.8 phoneme
- chapter-12-19 → A.3.9.1 Summary and feature statistics
- chapter-12-2 → A.2.2 225_puma8NH
- chapter-12-20 → A.3.10.1 Summary and feature statistics
- chapter-12-21 → A.4.1 iris
- chapter-12-22 → A.4.2 wine_recognition
- chapter-12-23 → A.4.3.1 Summary and feature statistics
- chapter-12-24 → A.4.4.1 Summary and feature statistics
- chapter-12-25 → A.4.5.1 Summary, identities and class counts
- chapter-12-26 → A.4.6 segmentation
- chapter-12-27 → A.4.7 vehicle
- chapter-12-28 → A.4.8.1 Summary, identities and class counts
- chapter-12-29 → A.4.9 penguins
- chapter-12-3 → A.2.3 230_machine_cpu
- chapter-12-30 → A.4.10 tae

- chapter-12-31 → A.5 Explicitly omitted or unknown statistics
- chapter-12-4 → A.2.4 522_pm10
- chapter-12-5 → A.2.5 529_pollen
- chapter-12-6 → A.2.6 579_fri_c0_250_5
- chapter-12-7 → A.2.7.1 Summary and feature statistics
- chapter-12-8 → A.2.8 690_visualizing_galaxy
- chapter-12-9 → A.2.9 712_chscase_geyser1
- chapter-13 → E.4.1 Current implementation references
- chapter-13-1 → E.4.1 Current implementation references
- chapter-13-2 → E.4.2 Pending source references
- chapter-13-3 → E.5.1 Companion files
- chapter-13-4 → E.5.2 Lossless evidence packaging
- chapter-13-6 → E.5.3.1 Companion inventory
- chapter-14-10 → E.5.3.2 Reusing the evidence
- chapter-14-2 → B.2.1.1 Epoch losses and the routing warning
- chapter-14-3 → B.2.1.1 Epoch losses and the routing warning
- chapter-14-4 → B.2.2.1 Round-1 epochs and the gate receipt
- chapter-14-5 → B.3.1.1 Native architecture and objective
- chapter-14-6 → B.3.2 Fixed monitoring and actual promotion evidence
- chapter-14-7 → B.3.3.1 Source and extraction
- chapter-14-8 → B.3.4.1 A100 checkpoints and the accepted step 60,000
- chapter-15-2 → D.5.1 Lineages A and B and the staging store
- chapter-15-3 → D.6.1 Six retained corpora
- chapter-15-4 → C.1.1 Carried-forward scores and the admitted receipt inventory
- chapter-15-5 → C.1.4 Paired final-search outcomes within the admitted development inventory
- chapter-15-6 → D.7.1 Record layout
- chapter-15-7 → D.7.4.1 Occupancy, padding and index size